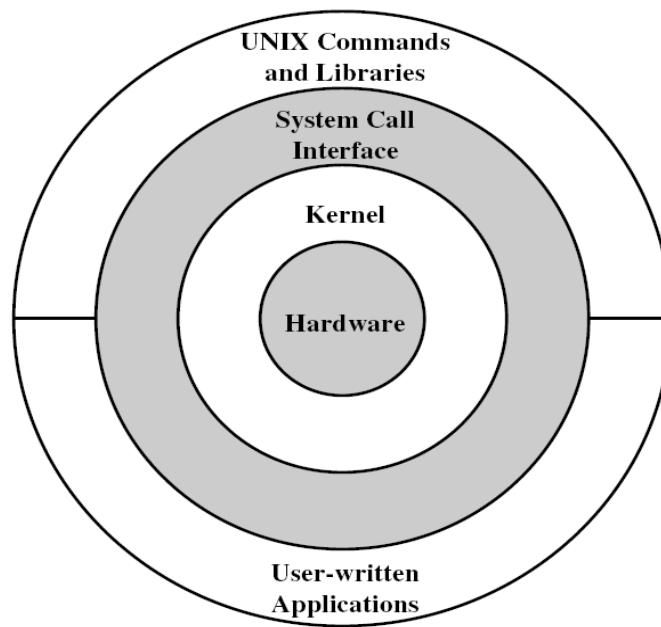


ΤΕΙ ΗΠΕΙΡΟΥ
ΣΧΟΛΗ ΔΙΟΙΚΗΣΗΣ ΚΑΙ ΟΙΚΟΝΟΜΙΑΣ
ΤΜΗΜΑ ΤΗΛΕΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΔΙΟΙΚΗΣΗΣ

ΛΕΙΤΟΥΡΓΙΚΑ ΣΥΣΤΗΜΑΤΑ Η/Υ



ΠΑΝΑΓΙΩΤΗΣ ΧΑΤΖΗΔΟΥΚΑΣ

ΑΡΤΑ 2007

ΠΙΝΑΚΑΣ ΠΕΡΙΕΧΟΜΕΝΩΝ

Κεφάλαιο 1 – Εισαγωγή.....	1
1.1 Ορισμός λειτουργικού συστήματος.....	1
1.2 Εξέλιξη λειτουργικών συστημάτων	3
1.3. Ιστορική Εξέλιξη των ΛΣ	4
Ερωτήσεις Επανάληψης	10
Ασκήσεις.....	10
Κεφάλαιο 2 – Σκοποί Λειτουργικών Συστημάτων	13
2.1. Σκοποί και Λειτουργίες των ΛΣ.....	13
2.2 Σημαντικά σημεία εξέλιξης των ΛΣ	17
2.3 Χαρακτηριστικά σύγχρονων λειτουργικών συστημάτων	22
Ερωτήσεις Επανάληψης	22
Ασκήσεις.....	22
Κεφάλαιο 3 – Διεργασίες.....	25
3. 1. Εισαγωγή.....	25
3. 2. Καταστάσεις διεργασίας	25
3.3 Διαγράμματα καταστάσεων.....	26
3.4. Μπλοκ ελέγχου διεργασίας	29
3.5. Υπηρεσίες ΛΣ για διαχείριση διεργασιών.....	31
Ερωτήσεις Επανάληψης	33
Ασκήσεις.....	33
Κεφάλαιο 4 – Αρχιτεκτονικές ΛΣ.....	35
4. 1. Μονολιθικά συστήματα.....	35
4. 2. Στρωματοποιημένη αρχιτεκτονική	36
4. 3. Αρχιτεκτονική μικροπυρήνα	36
4.4. Νήματα - Πολυνημάτωση (multithreading).....	37
4.5. Συστήματα πολυπεξεργασίας	40
4.6. Παράλληλα συστήματα	40
4.7. Συστήματα πραγματικού χρόνου	41
4.8. Κατανεμημένα συστήματα.....	41
Ερωτήσεις Επανάληψης	42
Ασκήσεις.....	42
Κεφάλαιο 5 – Αμοιβαίος Αποκλεισμός	45
5. 1. Εισαγωγή	45
5.2. Κρίσιμα τμήματα (Critical Sections)	45
5.3. Υλοποίηση του αμοιβαίου αποκλεισμού.....	47
Ερωτήσεις Επανάληψης	52
Ασκήσεις.....	52
Κεφάλαιο 6 – Αδιέξοδο.....	55
6.1. Εισαγωγή	55
6.2. Γράφοι εκχώρησης πόρων	58
6.3. Συνθήκες αδιεξόδου.....	59
6.4. Προσεγγίσεις αντιμετώπισης αδιεξόδου.....	61
6.5. Πρόβλημα συνδαιτημόνων φιλοσόφων.....	66
Ερωτήσεις Επανάληψης	67
Ασκήσεις.....	67

Κεφάλαιο 7 – Διαχείριση Μνήμης	69
7.1. Εισαγωγή	69
7.2. Βασική διαχείριση μνήμης	70
7.3. Μνήμη και πολυπρογραμματισμός	71
7.4. Τμηματοποίηση σταθερού μεγέθους	72
7.5 Δυναμική τμηματοποίηση	74
7.6 Εναλλαγή (swapping)	75
Ερωτήσεις Επανάληψης	76
Ασκήσεις	76
Κεφάλαιο 8 – Ιδεατή Μνήμη	79
8.1. Εισαγωγή	79
8.2. Ιδεατές και πραγματικές διευθύνσεις	79
8.3. Λογική οργάνωση	79
8.4. Τμηματοποίηση ιδεατής μνήμης	80
Ερωτήσεις Επανάληψης	83
Ασκήσεις	83
Κεφάλαιο 9 – Δρομολόγηση Διεργασιών	85
9.1. Εισαγωγή	85
9.2. Κριτήρια αποτίμησης της απόδοσης	85
9.3. Κριτήρια βελτιστοποίησης	86
9.4. Τύποι δρομολόγησης του επεξεργαστή	86
9.5. Πολιτικές δρομολόγησης	88
9.6. Αλγόριθμοι Δρομολόγησης	88
9.7. Σύγκριση αλγορίθμων δρομολόγησης	103
Ερωτήσεις Επανάληψης	104
Ασκήσεις	105
Κεφάλαιο 10 – Το Λειτουργικό Σύστημα Unix	107
10.1 Εισαγωγή	107
10.2 Λογαριασμοί χρηστών	107
10.3 Είσοδος στο σύστημα	108
10.4 Το σύστημα αρχείων	109
10.5 Ονόματα διαδρομών (Pathnames)	110
10.6. Η εντολή ls	111
10.7. Μετακίνηση στο σύστημα αρχείων του UNIX	112
10.8 Διαχείριση αρχείων	113
10.9 Χαρακτηριστικά αρχείων (File attributes)	113
10.9 Άλλες εντολές	115
10.10 Ανακατεύθυνση εισόδου / εξόδου (Input / Output Redirection)	115
10.11 Πολυπρογραμματισμός στο UNIX	116
Ερωτήσεις Επανάληψης	117
Ασκήσεις	117
Βιβλιογραφία	119
Λύσεις Ασκήσεων	119

Κεφάλαιο 1 – Εισαγωγή

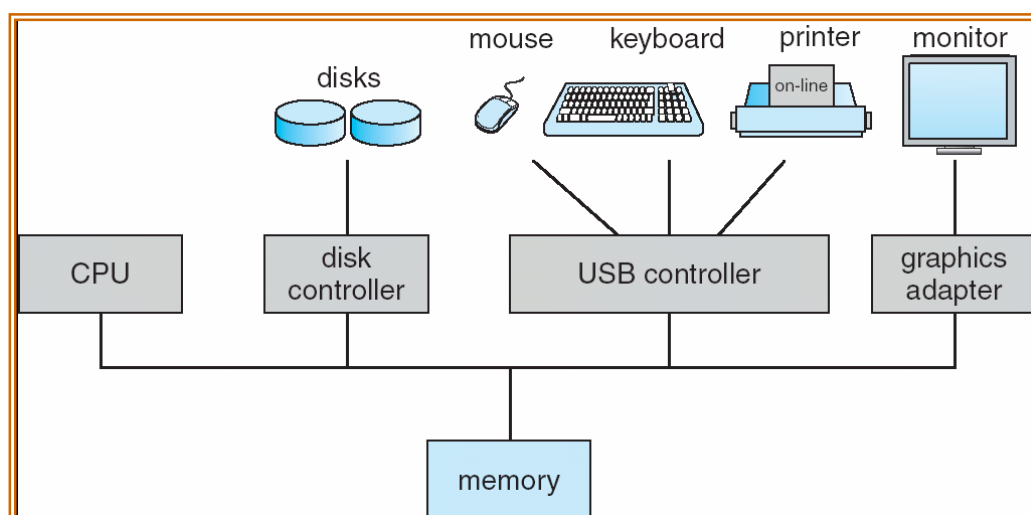
1. Ορισμός λειτουργικού συστήματος
2. Εξέλιξη λειτουργικών συστημάτων
3. Ιστορική εξέλιξη των λειτουργικών συστημάτων

1.1 Ορισμός λειτουργικού συστήματος

Ως Λειτουργικό Σύστημα (ΛΣ) μπορεί να θεωρηθεί ένα πρόγραμμα που λειτουργεί ως ενδιάμεσος μεταξύ των χρηστών των Υπολογιστικών Συστημάτων και του υλικού του Υπολογιστικού Συστήματος (ΥΣ). Βασικοί στόχοι ενός Λειτουργικού Συστήματος είναι η εκτέλεση προγραμμάτων των χρηστών, η ευκολία χρήσης του υπολογιστικού συστήματος, η χρήση του υλικού και των περιφερειακών του υπολογιστικού συστήματος με αποτελεσματικό και αποδοτικό τρόπο, καθώς και η προστασία των προγραμμάτων και των δεδομένων των διαφόρων χρηστών του υπολογιστικού συστήματος.

Τα βασικά στοιχεία που αποτελούν ένα υπολογιστικό σύστημα είναι τα ακόλουθα:

- Το υλικό (hardware), που παρέχει τους βασικούς υπολογιστικούς πόρους, όπως είναι ο επεξεργαστής, η μνήμη, οι συσκευές εισόδου/εξόδου (I/O devices, κλπ. (Σχήμα 1)
- Το λειτουργικό σύστημα, το οποίο ελέγχει και συντονίζει τη χρήση του υλικού μεταξύ των διαφόρων προγραμμάτων εφαρμογών των διαφόρων χρηστών.
- Τα προγράμματα εφαρμογών, που καθορίζουν τους τρόπους με τους οποίους χρησιμοποιούνται οι πόροι για την επίλυση των υπολογιστικών προβλημάτων των χρηστών (π.χ. μεταγλωττιστές, συστήματα βάσεων δεδομένων, προγράμματα επιχειρήσεων), και τέλος
- Οι χρήστες, στους οποίους περιλαμβάνονται οι άνθρωποι, τα μηχανήματα αλλά και άλλοι υπολογιστές.



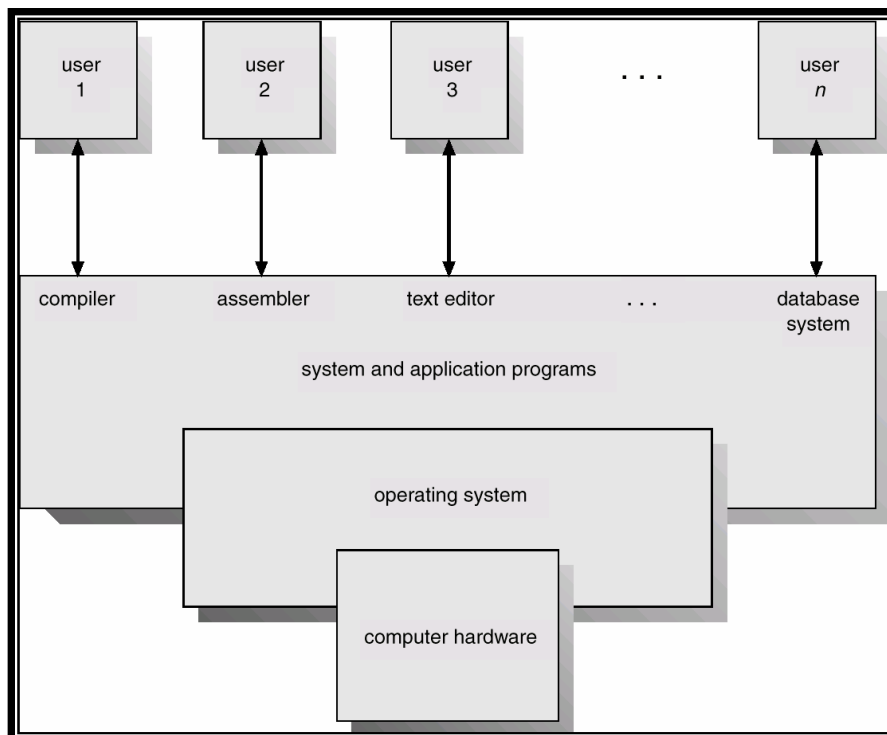
Σχήμα 1.1. Υλικό ενός τυπικού υπολογιστικού συστήματος

Η έννοια του Λειτουργικού συστήματος μπορεί να οριστεί σύμφωνα με τις ακόλουθες προσεγγίσεις:

1. Το Λειτουργικό Σύστημα μπορεί να θεωρηθεί ως μία εκτεταμένη μηχανή (extended ή virtual machine). Στην περίπτωση αυτή αποτελεί το πρόγραμμα που

κρύβει από το χρήστη / προγραμματιστή την αλήθεια για το υλικό. Για παράδειγμα, το λειτουργικό σύστημα παρουσιάζει μια απλή και εύχρηστη απεικόνιση από ονόματα και λειτουργίες που σχετίζονται με το χειρισμό αρχείων και καταλόγων. Επίσης παρουσιάζει κατάλληλα τη διαθέσιμη φυσική μνήμη στα προγράμματα των χρηστών και αναλαμβάνει τη διαχείριση διακοπών (interrupt handling).

2. Το Λειτουργικό Σύστημα αποτελεί ένα διαχειριστή για την ανάθεση πόρων (resource allocation). Αποτελεί το πρόγραμμα που αναλαμβάνει να μοιράσει τους πόρους του συστήματος ανάμεσα στις διάφορες εφαρμογές. Παράδειγμα αποτελεί η χρήση κοινών εκτυπωτών, όπου θα πρέπει το ΛΣ να παρέχει έναν τρόπο για την ορθή και με συγκεκριμένη σειρά εκτύπωση των δεδομένων όλων των χρηστών, που χρησιμοποιούν ταυτόχρονα τον εκτυπωτή. Ένα άλλο παράδειγμα είναι η διαχείριση και προστασία της μνήμης, ιδιαίτερα σε συστήματα που εξυπηρετούν ταυτόχρονα πολλούς χρήστες.
3. **Top down view:** Προσεγγίζοντας το λειτουργικό σύστημα από την κορυφή (χρήστης) προς τη βάση (υλικό υπολογιστικού συστήματος), ο ρόλος του είναι να παρέχει στα προγράμματα εύκολη και αποδοτική επικοινωνία με τους διάφορους πόρους του υπολογιστικού συστήματος.
4. **Bottom up view:** Προσεγγίζοντας το λειτουργικό σύστημα από τη βάση προς την κορυφή, μέριμνα του λειτουργικού συστήματος είναι να παρέχει μια συστηματοποιημένη και ελεγχόμενη κατανομή των επεξεργαστών, των μνημών, και των άλλων συσκευών εισόδου / εξόδου, ανάμεσα στα διάφορα προγράμματα-πελάτες που ανταγωνίζονται μεταξύ τους για να τα χρησιμοποιήσουν.



Σχήμα 1.2. Θεώρηση των στοιχείων ενός υπολογιστικού συστήματος

Συμπερασματικά, μπορούμε να πούμε ότι το Λειτουργικό Σύστημα είναι υπεύθυνο για τη διαχείριση της ανάθεσης πόρων (resource allocator), αποτελεί ένα πρόγραμμα ελέγχου το οποίο ελέγχει την εκτέλεση των προγραμμάτων χρηστών και τη λειτουργία

των συσκευών εισόδου / εξόδου, ενώ ο πυρήνας του (kernel) αποτελεί το μόνο πρόγραμμα που τρέχει συνέχεια, όσο βρίσκεται σε λειτουργία το υπολογιστικό σύστημα ενώ όλα τα υπόλοιπα στοιχεία θεωρούνται επιπρόσθετες υπηρεσίες και προγράμματα εφαρμογών.

Τα Λειτουργικά Συστήματα έχουν μια σχέση «εξάρτησης» με την αρχιτεκτονική των υπολογιστικών συστημάτων στα οποία εκτελούνται. Οι εξελίξεις στο υλικό των υπολογιστικών συστημάτων έκανε δυνατή την παροχή επιπλέον λειτουργιών προς τα προγράμματα των χρηστών. Οι λειτουργίες αυτές υλοποιούνταν με την ανακάλυψη και την εξέλιξη των λειτουργικών συστημάτων.

Κατά την αρχική τους σχεδίαση, στα λειτουργικά συστήματα υπήρχε μόνο μια διεργασία που εκτελούνταν κάθε φορά (π.χ. DOS). Αντίθετα, στα σύγχρονα λειτουργικά συστήματα υπάρχει η έννοια του πολυπρογραμματισμού, που είναι «ο διαμερισμός της μνήμης σε διάφορα τμήματα, ώστε κάθε διαφορετική εργασία να καταλαμβάνει διαφορετικό τμήμα» (Tanenbaum, 1993). Το λειτουργικό σύστημα μπορεί να φορτώσει περισσότερες από μια εργασίες ταυτόχρονα στην κυρίως μνήμη του υπολογιστικού συστήματος ενώ κάθε χρονική στιγμή, υπάρχουν πολλές διεργασίες που είναι φορτωμένες και εκτελούνται από το σύστημα.

Τα βασικά χαρακτηριστικά των λειτουργικών συστημάτων που σχετίζονται με τον πολυπρογραμματισμό είναι τα ακόλουθα:

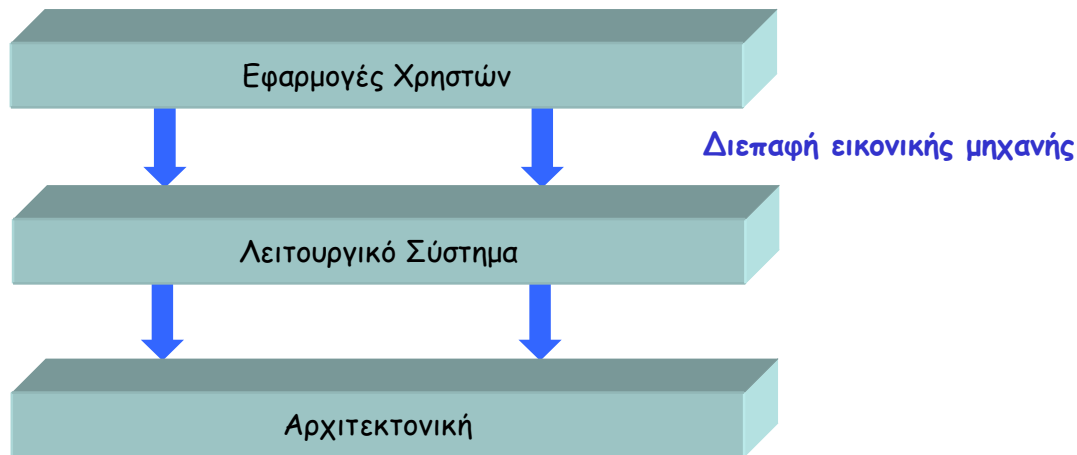
- Διαχείριση μνήμης: το λειτουργικό σύστημα πρέπει να είναι σε θέση να χωρίσει τη μνήμη σε τμήματα (ένα για κάθε διεργασία) και να προστατεύσει το τμήμα κάθε διεργασίας από (ηθελημένες ή αθέλητες) παρεμβολές των υπολοίπων διεργασιών.
- Διαχείριση διεργασιών: το λειτουργικό σύστημα πρέπει να είναι σε θέση να επιλέξει ποιες διεργασίες θα αποκτήσουν χώρο στη μνήμη. Υπάρχει η θεώρηση ότι η μνήμη δεν επαρκεί για να «στεγάσει» όλες τις διεργασίες ταυτόχρονα και ότι μόνο μια διεργασία που είναι στη μνήμη μπορεί να τρέξει.
- Χρονοπρογραμματισμός της κεντρικής μονάδας επεξεργασίας: το λειτουργικό σύστημα πρέπει να επιλέξει μεταξύ των διεργασιών που έχουν χώρο στη μνήμη κάποια για να τρέξει.
- Ανάθεση πόρων με τέτοιο τρόπο ώστε να μην «επηρεάζεται» η μια διεργασία από την εκτέλεση άλλων διεργασιών.

Ο πολυπρογραμματισμός έχει ιδιαίτερη σημασία στα συστήματα καταμερισμού χρόνου. Δηλαδή, αφορά κυρίως τα συστήματα με τερματικά και χρήστες που αλληλεπιδρούν (interactive) με ένα υπολογιστικό σύστημα. Η ιδέα είναι να γίνεται καταμερισμός της κεντρικής μονάδας επεξεργασίας στις εργασίες που περιμένουν εξυπηρέτηση, για την προσφορά γρήγορης εξυπηρέτησης σε έναν αριθμό από χρήστες.

1.2 Εξέλιξη λειτουργικών συστημάτων

Το λειτουργικό σύστημα αποτελεί τη διεπαφή μεταξύ των εφαρμογών των χρηστών και του υλικού, υλοποιώντας ένα είδος ιδεατής μηχανής (virtual machine) που είναι ευκολότερη στον προγραμματισμό σε σχέση με το υλικό.

Ένα λειτουργικό σύστημα είναι ένα πρόγραμμα που ελέγχει την εκτέλεση των προγραμμάτων εφαρμογών και ενεργεί ως ενδιάμεσος μεταξύ εφαρμογών και υλικού μέρους του υπολογιστή. Αποτελεί το λογισμικό που τοποθετείται στην κορυφή του υλικού ώστε να διαχειρίζεται όλα τα συστατικά μέρη του συστήματος και να προσφέρει στο χρήστη τη διεπαφή (interface) ή μια ιδεατή μηχανή που είναι πιο εύκολη στην κατανόηση και τον προγραμματισμό.



Σχήμα 1.3. Το λειτουργικό σύστημα ως διεπαφή εικονικής μηχανής

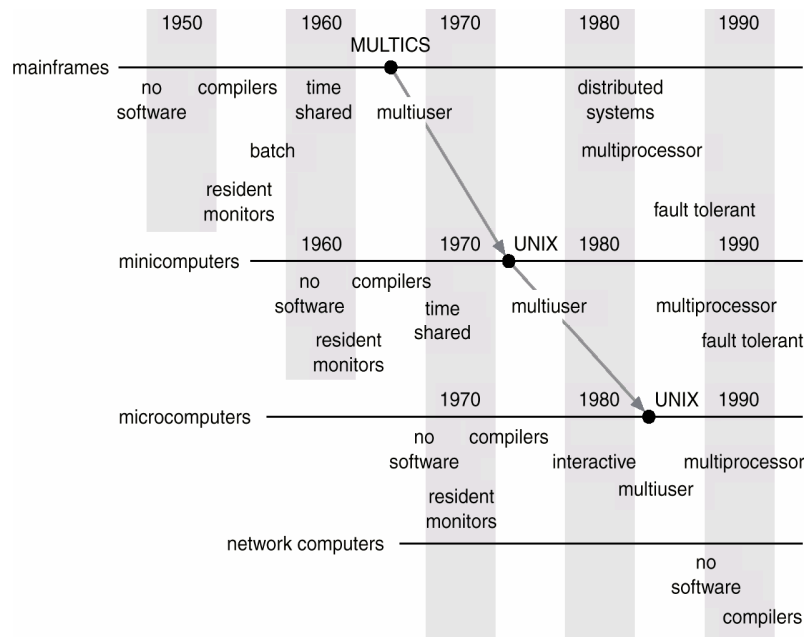
Το λειτουργικό σύστημα καθορίζει ένα πλαίσιο για τους χρήστες και τα προγράμματά τους ώστε να συνυπάρχουν, να συνεργάζονται και να λειτουργούν ταυτόχρονα και αποδοτικά, υποστηρίζοντας:

- ταυτόχρονη εκτέλεση και αλληλεπίδραση πολλών προγραμμάτων των χρηστών
- διαμοιραζόμενες εφαρμογές που καλύπτουν συνήθεις απαιτούμενες διευκολύνσεις
- μηχανισμούς διαμοίρασης και συνδυασμού συστατικών λογισμικού
- πολιτικές για ασφαλή και δίκαιη διαμοίραση των πόρων
 - φυσικών πόρων (π.χ. χρόνος της CPU και χώρος αποθήκευσης)
 - λογικών πόρων (π.χ. αρχεία δεδομένων, προγράμματα)

1.3. Ιστορική Εξέλιξη των ΛΣ

1.3.1 Πρώτο στάδιο εξέλιξης

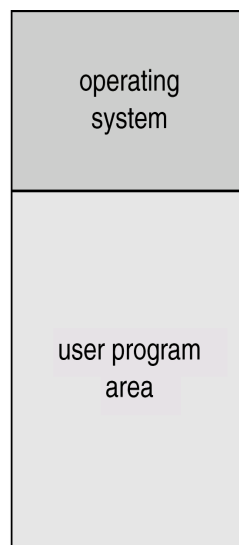
Το πρώτο στάδιο εξέλιξης στην ιστορία των λειτουργικών συστημάτων εκτείνεται από τις αρχές της δεκαετίας 1940 έως τα μέσα της δεκαετίας 1950 και περιλαμβάνει την σειριακή επεξεργασία. Τα βασικά προβλήματα του σταδίου αυτού αφορούν στη δρομολόγηση εργασιών – προγραμμάτων λόγω της σπατάλης σε υπολογιστικό χρόνο, αφού τα προγράμματα που έμεναν στη μέση της εκτέλεσής του λόγω του μεγέθους του μπλοκ χρόνου, αλλά και στο χρόνο εγκατάστασης αφού ένα μοναδικό πρόγραμμα, η εργασία, φόρτωνε τον μεταγλωττιστή και το πρόγραμμα σε γλώσσα υψηλού επιπέδου στη μνήμη, αποθήκευε τον αντικειμενικό κώδικα (object program) και στη συνέχεια το φόρτωνε και το συνέδεε με τις λοιπές συναρτήσεις.



Σχήμα 1.4. Εξέλιξη λειτουργικών συστημάτων

1.3.2 Δεύτερο στάδιο εξέλιξης

Το δεύτερο στάδιο εξέλιξης (αρχές–μέσα της δεκαετίας 1960) περιλαμβάνει τα απλά μαζικά συστήματα επεξεργασίας (batch process). Ο λόγος ανάπτυξης τους ήταν η μείωση του νεκρού χρόνου στις πολύ ακριβές υπολογιστικές μηχανές της εποχής και βασίζονται στην κεντρική ιδέα της χρήση ενός λογισμικού που λέγεται παρακολουθητής (monitor). Ο χρήστης υποβάλλει την εργασία του σε ένα χειριστή ο οποίος τις βάζει όλες μαζί σε μια συσκευή εισόδου για να χρησιμοποιηθεί από τον παρακολουθητή, ενώ κάθε πρόγραμμα είναι φτιαγμένο έτσι ώστε όταν ολοκληρώνεται να επιστρέφει στο monitor για να φορτωθεί άλλο πρόγραμμα.



Σχήμα 1.5. Διαχωρισμός μνήμης για λειτουργικό σύστημα και δεδομένα χρήστη

Σύμφωνα με τα παραπάνω, ο παρακολουθητής (**monitor**) είναι ένα ειδικό λογισμικό που ελέγχει τα προγράμματα που τρέχουν και ομαδοποιεί τις εργασίες..

Επίσης, ο παρακολουθητής παραμένει στην κεντρική μνήμη και είναι πάντοτε διαθέσιμος ενώ μόλις ένα πρόγραμμα ολοκληρωθεί ο έλεγχος μεταφέρεται σε αυτόν. Η υποβολή εργασιών στα συστήματα επεξεργασίας του δεύτερου σταδίου εξέλιξης πραγματοποιείται με μια ειδική γλώσσα ελέγχου εργασιών, την Job Control Language (JCL), η οποία παρέχει εντολές ελέγχου στον παρακολουθητή, επιτρέπει τον καθορισμό των μεταγλωττιστών που θα χρησιμοποιηθούν αλλά και των δεδομένων που απαιτούνται από τις εργασίες.

Τα χαρακτηριστικά του υλικού μέρους που είναι επιθυμητά στα απλά μαζικά συστήματα επεξεργασίας είναι τα ακόλουθα:

- Προστασία μνήμης: το πρόγραμμα χρήστη που εκτελεί ο παρακολουθητής δεν πρέπει να τροποποιεί την περιοχή μνήμης όπου υπάρχει ο παρακολουθητής.
- Χρονομετρητής: πρέπει να αποφεύγεται η μονοπώληση του συστήματος από μια μοναδική εργασία (π.χ. κάρτες σε προγράμματα FORTRAN)
- Προνομιούχες εντολές: εκτελούνται μόνον από τον παρακολουθητή, κυρίως για διαδικασίες I/O
- Διακοπές: μηχανισμοί του ΛΣ για παραχώρηση και ανάκτηση του ελέγχου του συστήματος

1.3.3 Τρίτο στάδιο εξέλιξης

Το τρίτο στάδιο εξέλιξης (1965-1980) περιλαμβάνει τα πολυπρογραμματιζόμενα μαζικά συστήματα με βασικό λόγο ανάπτυξης το γεγονός ότι ο επεξεργαστής παραμένει ανενεργός λόγω διαφοράς ταχύτητας με τις συσκευές I/O.

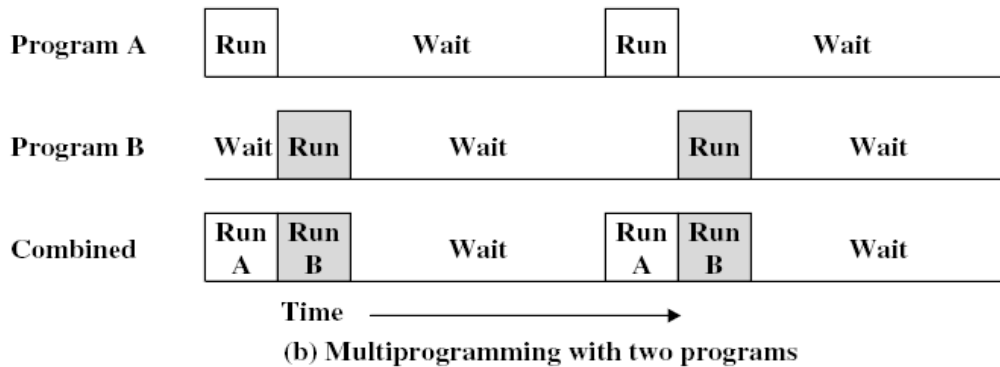
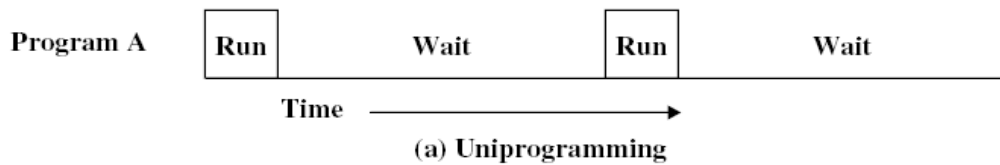
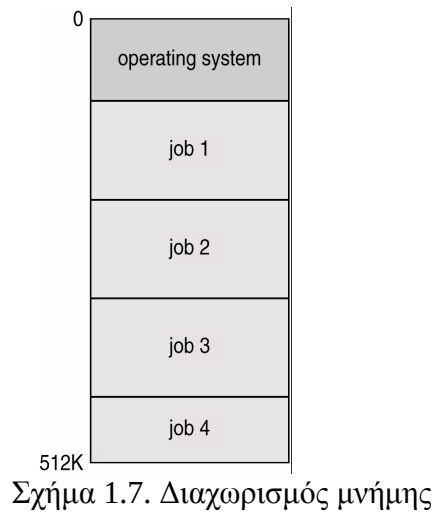
Read one record from file	0.0015 seconds
Execute 100 instructions	0.0001 seconds
Write one record to file	<u>0.0015 seconds</u>
TOTAL	0.0031 seconds

$$\text{Percent CPU Utilization} = \frac{0.0001}{0.0031} = 0.032 = 3.2\%$$

Σχήμα 1.6. Ποσοστό αξιοποίησης επεξεργαστή

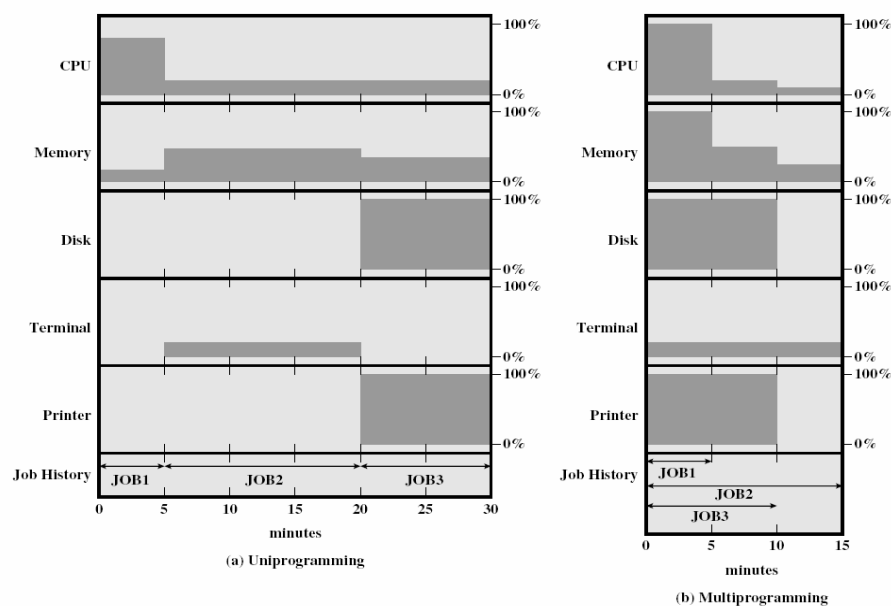
Τα απλά μαζικά συστήματα επεξεργασίας χαρακτηρίζονταν από τον μονοπρογραμματισμό, σύμφωνα με τον οποίον όταν η εκτελούμενη εργασία διέκοπτε, αναμένοντας να ολοκληρωθεί μια διεργασία I/O, η CPU έμενε ανενεργή. Ενώ σε επιστημονικές εφαρμογές η σπατάλη χρόνου είναι ελάχιστη αφού διαδικασίες I/O συμβαίνουν σπάνια, ενώ σε εμπορικές εφαρμογές η σπατάλη είναι 80-90%.

Αντίθετα, στην περίπτωση του πολυπρογραμματισμού έχουμε διαμερισμό της μνήμης σε τμήματα ώστε κάθε διαφορετική διεργασία να καταλαμβάνει άλλο τμήμα μνήμης. Όταν μια διεργασία περιμένει μια I/O, κάποια άλλη χρησιμοποιεί την CPU. Αν μπορούσαν να παραμείνουν στη μνήμη πολλές διεργασίες ταυτόχρονα τότε το ποσοστό χρήσης της CPU έφθανε και το 100%. Η βασική απαίτηση για την υποστήριξη πολυπρογραμματισμού είναι η προστασία της μνήμης από επικαλύψεις, ένας μηχανισμός που ήταν διαθέσιμος μέσω του υλικού.



Σχήμα 1.8. Παράδειγμα μονοπρογραμματισμού και πολυπρογραμματισμού

Στην περίπτωση του πολυπρογραμματισμού, τα μεγέθη που χαρακτηρίζουν την απόδοση ενός λειτουργικού συστήματος είναι η μέση χρησιμοποίηση των πόρων, η ρυθμοαπόδοση (throughput) και ο χρόνος απόκρισης των εργασιών. Τα πολυπρογραμματιζόμενα μαζικά συστήματα βασίζονται σε συγκεκριμένα χαρακτηριστικά του υλικού μέρους του Η/Υ (διακοπές, I/O, DMA), στη διαχείριση μνήμης και στους αλγόριθμους δρομολόγησης.

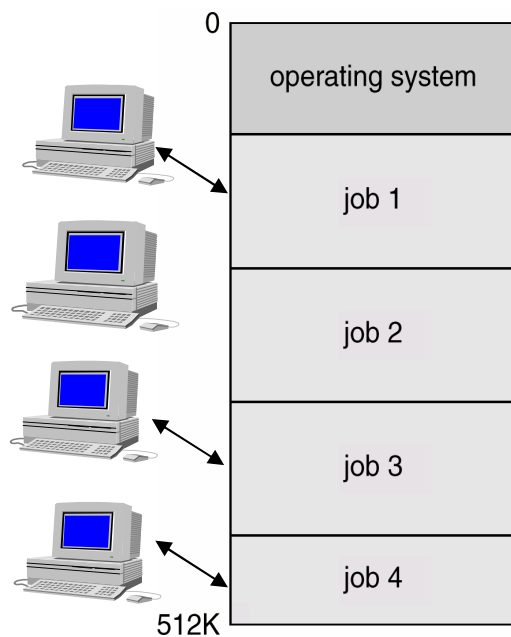


Σχήμα 1.9. Παράδειγμα μονοπρογραμματισμού και πολυπρογραμματισμού

	Uniprogramming	Multiprogramming
Processor use	22%	43%
Memory use	30%	67%
Disk use	33%	67%
Printer use	33%	67%
Elapsed time	30 min	15 min
Throughput rate	6 jobs/hr	12 jobs/hr
Mean response time	18 min	10 min

Σχήμα 1.10. Πολυπρογραμματισμός και αξιοποίηση πόρων

Τα συστήματα διαμοιραζόμενου χρόνου προέκυψαν από τις διαμαρτυρίες όσων περίμεναν ώρες ή ημέρες για να πάρουν αποτελέσματα από τα πολυπρογραμματιζόμενα μαζικά συστήματα. Στην περίπτωση τους, κάθε χρήστης συνδέεται μέσω τερματικού και η CPU εξυπηρετεί εκ περιτροπής κάθε πρόγραμμα χρήστη με ένα σύντομο καταιγισμό (burst) ή ένα κβάντο (quantum) υπολογισμού. Με τον τρόπο αυτό, ο ΗΥ προσφέρει μια διαλογική εξυπηρέτηση με την τεχνική του καταμερισμού χρόνου και εκμεταλλεύεται τον σχετικά βραδύ χρόνο της ανθρώπινης αντίδρασης.



Σχήμα 1.11. Σύστημα διαμερισμού χρόνου

Συγκρίνοντας τον πολυπρογραμματισμό με τη διαμοίραση χρόνου, διακρίνουμε στην περίπτωση του πρώτου την μεγιστοποίηση της χρησιμοποίησης επεξεργαστή και την εισαγωγή οδηγιών προς το λειτουργικό σύστημα μέσω γλώσσας ελέγχου εργασιών, ενώ στην διαμοίραση χρόνου την ελαχιστοποίηση του χρόνου απόκρισης και τις εντολές προς το λειτουργικό σύστημα που εισάγονται άμεσα από το τερματικό.

1.3.4 Τέταρτο στάδιο εξέλιξης

Η τέταρτη γενιά των λειτουργικών συστημάτων (1980 – 1990) επηρεάζεται από την εμφάνιση των ολοκληρωμένων κυκλωμάτων τύπου LSI. Την περίοδο αυτή τα λειτουργικά συστήματα γίνονται φιλικά προς το χρήστη ενώ κάνουν την εμφάνισή τους τα λειτουργικά συστήματα δικτύων, όπου κάθε ΗΥ τρέχει το δικό του ΛΣ, και τα κατανεμημένα (distributed) λειτουργικά συστήματα. Τα τελευταία εμφανίζονται ως παραδοσιακά συστήματα ενός επεξεργαστή, στα οποία οι χρήστες δεν ενδιαφέρονται που εκτελούνται τα προγράμματά τους ή που βρίσκονται τα αρχεία τους. Τα κατανεμημένα λειτουργικά συστήματα επιτρέπουν στα προγράμματα να εκτελούνται σε διαφορετικούς επεξεργαστές την ίδια χρονική στιγμή και απαιτούν πολύπλοκους αλγόριθμους χρονοδρομολόγησης (scheduling). Παράλληλα κάνουν την εμφάνισή τους και τα συστήματα συναλλαγής πραγματικού χρόνου, π.χ. σύστημα κρατήσεων αεροπορικών εταιρειών)

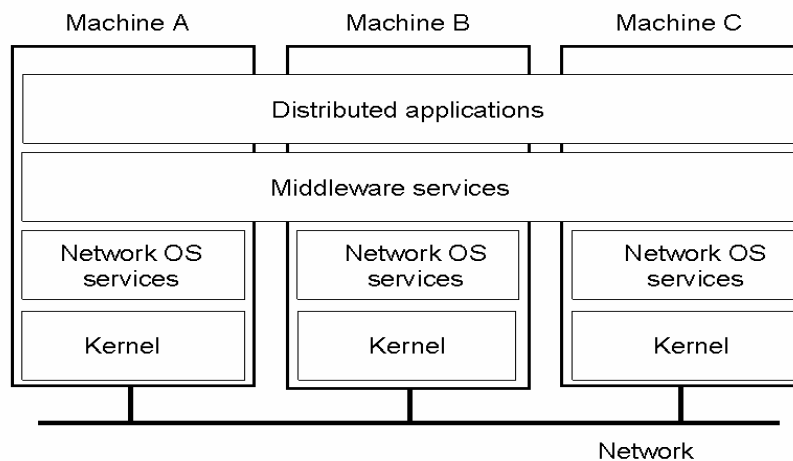
1.3.5 Πέμπτο στάδιο εξέλιξης

Το τελευταίο στάδιο εξέλιξης των λειτουργικών συστημάτων περιλαμβάνει τις σύγχρονες εξελίξεις τους, οι οποίες μπορούν να διαχωριστούν σε αυτές της δεκαετίας του 1990 και αυτής που διανύουμε.

Στην δεκαετία του 1990 έχουμε την αλματώδη αύξηση της απόδοσης του υλικού (MIPS), ενώ η δημιουργία του παγκοσμίου ιστού αύξησε την κατανεμημένη επεξεργασία και οδήγησε στην ανάγκη ενσωμάτωσης διαδικτυακών διεργασιών. Παράλληλα έχουμε την καθιέρωση της αντικειμενοστραφούς τεχνολογίας, την διάδοση

και ανάπτυξη της τεχνολογίας ανοικτού κώδικα με σημαντικότερο γεγονός την εμφάνιση του LINUX.

Κατά την τρέχουσα δεκαετία έχουμε την εμφάνιση του middleware, το οποίο αποτελεί κατάλληλο λογισμικό που συνδέει δύο ξεχωριστές εφαρμογές, που συχνά βρίσκονται σε δίκτυο. Επίσης, κυριαρχούν εφαρμογές (web services) που δημοσιεύονται στο Internet και χρησιμοποιούνται από χρήστες μέσω συνδέσεων υψηλών ταχυτήτων ενώ κάνουν την εμφάνιση τους βελτιωμένες αρχιτεκτονικές δικτύων. Σημειώνεται σημαντική πρόοδος και αύξηση της παράλληλης επεξεργασίας ενώ χρησιμοποιούνται λειτουργικά συστήματα τύπου POSIX (Portable Operating System Interface). Τέλος, εμφανίζεται η υπολογιστική δυνατότητα ακόμα και σε φορητές συσκευές (PDA's, cell phones κλπ).



Σχήμα 1.12 Συστήματα ενδιάμεσου λογισμικού

Ερωτήσεις Επανάληψης

1. Τι είναι ένα λειτουργικό σύστημα
2. Ποια είναι τα βασικά στοιχεία ενός υπολογιστικού συστήματος;
3. Ποια είναι τα βασικά χαρακτηριστικά των λειτουργικών συστημάτων που σχετίζονται με τον πολυπρογραμματισμό;
4. Τι ήταν ο παρακολουθητής (monitor) του δευτέρου σταδίου εξέλιξης;
5. Τι είναι ο πολυπρογραμματισμός και ποια τα χαρακτηριστικά του;

Ασκήσεις

1. Λειτουργικό σύστημα (operating system) είναι:
 - α) ένα υπολογιστικό σύστημα
 - β) ένα πρόγραμμα επικοινωνίας
 - γ) ένα ειδικό πρόγραμμα μεταξύ χρηστών και υλικού
 - δ) μία ομάδα υπολογιστικών κόμβων
2. Βασικά στοιχεία ενός υπολογιστικού συστήματος είναι τα ακόλουθα:
 - α) το υλικό και οι χρήστες
 - β) το υλικό και το λειτουργικό σύστημα
 - γ) το λειτουργικό σύστημα και τα προγράμματα εφαρμογών
 - δ) όλα τα παραπάνω

3. Σύμφωνα με την έννοια του πολυπρογραμματισμού
- α) το λειτουργικό σύστημα εκτελεί μια εργασία κάθε φορά
 - β) το λειτουργικό σύστημα μπορεί να φορτώσει πολλές εργασίες στη μνήμη
 - γ) οι χρηστές χρησιμοποιούν διάφορες γλώσσες προγραμματισμού
 - δ) οι εργασίες του συστήματος καταλαμβάνουν τα ίδια τμήματα μνήμης
4. Δεν αποτελεί βασικό χαρακτηριστικό των λειτουργικών συστημάτων που σχετίζονται με τον πολυπρογραμματισμό
- α) διαχείριση διεργασιών
 - β) διαχείριση μνήμης
 - γ) χρονοπρογραμματισμός του επεξεργαστή
 - δ) οργάνωση των αρχείων του χρήστη
5. Ο αριθμός των σταδίων εξέλιξης των λειτουργικών συστημάτων είναι:
- α) τρία
 - β) τέσσερα
 - γ) πέντε
 - δ) έξι
6. Ποιο από τα παρακάτω δεν αποτελεί πρόβλημα του πρώτου σταδίου εξέλιξης των λειτουργικών συστημάτων
- α) σειριακή επεξεργασία
 - β) χαμηλές ταχύτητες μετάδοσης δεδομένων
 - γ) σπατάλη υπολογιστικού χρόνου
 - δ) υψηλός χρόνος εγκατάστασης προγραμμάτων
7. Ο παρακολουθητής (monitor) του δευτέρου σταδίου εξέλιξης
- α) ελέγχει και ομαδοποιεί τις εργασίες που εκτελούνται
 - β) είναι πρόγραμμα που φορτώνεται στη μνήμη από τον χρήστη
 - γ) είναι ο διαχειριστής (administrator) του υπολογιστικού συστήματος
 - δ) ενεργοποιείται μετά την ολοκλήρωση όλων των προγραμμάτων
8. Ποιο από τα παρακάτω δεν χαρακτηρίζει την απόδοση ενός λειτουργικού συστήματος που υποστηρίζει πολυπρογραμματισμού
- α) μέση χρησιμοποίηση των πόρων
 - β) ταχύτητα μεταφοράς δεδομένων από τον σκληρό δίσκο στη μνήμη
 - γ) ρυθμοαπόδοση
 - δ) χρόνος απόκρισης των εργασιών
9. Χαρακτηριστικό του τετάρτου σταδίου εξέλιξης
- α) καταμερισμός χρόνου
 - β) πολυπρογραμματισμός
 - γ) χρήση αντικειμενοστραφούς τεχνολογίας
 - δ) κατανεμημένα λειτουργικά συστήματα
10. Η τεχνολογία ανοικτού κώδικα σχετίζεται με
- α) την ανάπτυξη αποδοτικών προγραμμάτων
 - β) το λειτουργικό σύστημα Windows
 - γ) το λειτουργικό σύστημα Linux
 - δ) τα γραφικά περιβάλλοντα προγραμματισμού

Κεφάλαιο 2 – Σκοποί Λειτουργικών Συστημάτων

1. Σκοποί και λειτουργίες των ΛΣ
2. Σημαντικά σημεία εξέλιξης των ΛΣ
3. Χαρακτηριστικά των σύγχρονων ΛΣ

2.1. Σκοποί και Λειτουργίες των ΛΣ

Οι κύριοι σκοποί και λειτουργίες του λειτουργικού συστήματος είναι η κατάλληλη προστασία του υλικού, η επικοινωνία με τον χρήστη, η διαχείριση, αξιοποίηση και έλεγχος των πόρων του συστήματος και η ικανότητα και ευκολία εξέλιξης του.

2.1.1 Προστασία Υλικού

Η προστασία υλικού που παρέχεται από το λειτουργικό σύστημα περιλαμβάνει τις εξής περιπτώσεις:

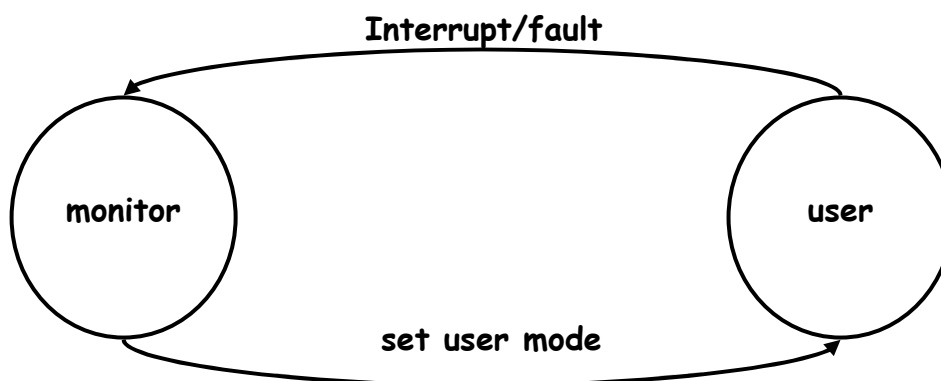
- α) Λειτουργία dual-mode
- β) Προστασία I/O
- γ) Προστασία μνήμης
- δ) Προστασία CPU

α) Λειτουργία dual-mode

Οι διαμοιραζόμενοι πόροι του συστήματος απαιτούν από το ΛΣ να εξασφαλίσει ότι ένα λανθασμένο πρόγραμμα δεν θα μπορεί να γίνει αφορμή άλλα προγράμματα να εκτελούνται λανθασμένα. Για το λόγο αυτό, το ΛΣ παρέχει υποστήριξη υλικού για 2 καταστάσεις λειτουργίας:

- User mode: η εκτέλεση γίνεται εκ μέρους του χρήστη.
- Monitor mode (επίσης supervisor mode ή system mode): η εκτέλεση γίνεται εκ μέρους του ΛΣ

Η λειτουργία dual-mode υλοποιείται με τη χρήση ενός mode bit, που προστίθεται στο υλικό για να δείχνει την τρέχουσα κατάσταση: monitor (0) ή user (1). Όταν συμβεί μια διακοπή ή ένα λάθος το υλικό εναλλάσσεται σε monitor mode. Επιπλέον, είναι διαθέσιμες κάποιες προνομιούχες εντολές (Privileged instructions), οι οποίες μπορούν να προκύψουν μόνον σε monitor mode.



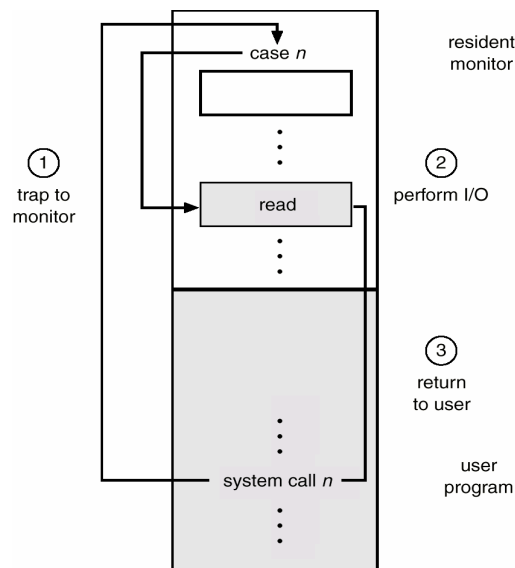
Σχήμα 2.1 Λειτουργία dual mode

Η προστασία του πυρήνα του λειτουργικού συστήματος πραγματοποιείται με το mode register bit, το οποίο δείχνει αν η CPU εκτελεί ένα πρόγραμμα χρήστη ή βρίσκεται σε κατάσταση προστασίας του πυρήνα του λειτουργικού συστήματος. Επιπλέον, ορισμένες εντολές ή προσπελάσεις σε δεδομένα είναι δυνατές μόνον όταν η CPU λειτουργεί σε kernel mode.

β) Προστασία I/O

Όλες οι εντολές I/O είναι προνομιούχες εντολές. Εξασφαλίζεται έτσι ότι ένα πρόγραμμα χρήστη δεν θα μπορεί ποτέ να αποκτήσει τον έλεγχο του υπολογιστή σε monitor mode. Για παράδειγμα, είναι αδύνατο για ένα πρόγραμμα χρήστη, κατά την εκτέλεσή του να αποθηκεύσει μια νέα διεύθυνση στον πίνακα διανυσμάτων διακοπών.

Ένα πρόγραμμα χρήστη εκτελεί I/O μέσω κλήσης συστήματος, που είναι ουσιαστικά η μέθοδος που χρησιμοποιείται από μια διεργασία για να απαιτήσει ενέργεια από το ΛΣ. Η κλήση συστήματος έχει συνήθως τη μορφή παγίδευσης (trap) σε μια καθορισμένη θέση του διανύσματος διακοπών. Ο έλεγχος περνά διαμέσου του διανύσματος διακοπών σε μια ρουτίνα εξυπηρέτησης στο ΛΣ και το mode bit τίθεται σε monitor (supervisor) mode. Ο παρακολουθητής (monitor) διαπιστώνει ότι οι παράμετροι είναι σωστοί και αποδεκτοί, εκτελεί την απαίτηση και επιστρέφει τον έλεγχο στην εντολή που ακολουθεί την κλήση συστήματος (system call).

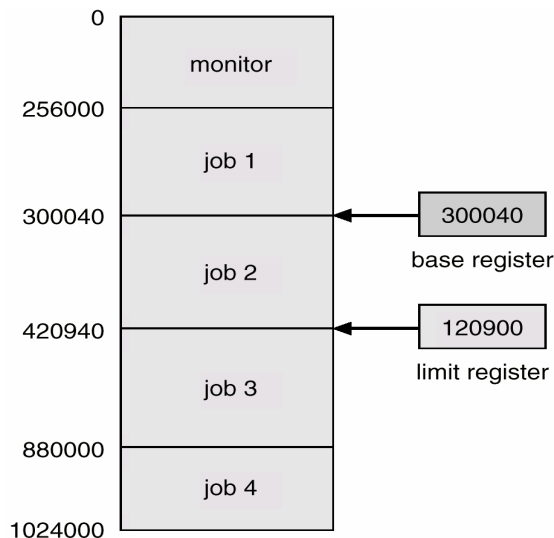


Σχήμα 2.3. Κλήση συστήματος για εκτέλεση I/O

γ) Προστασία Μνήμης

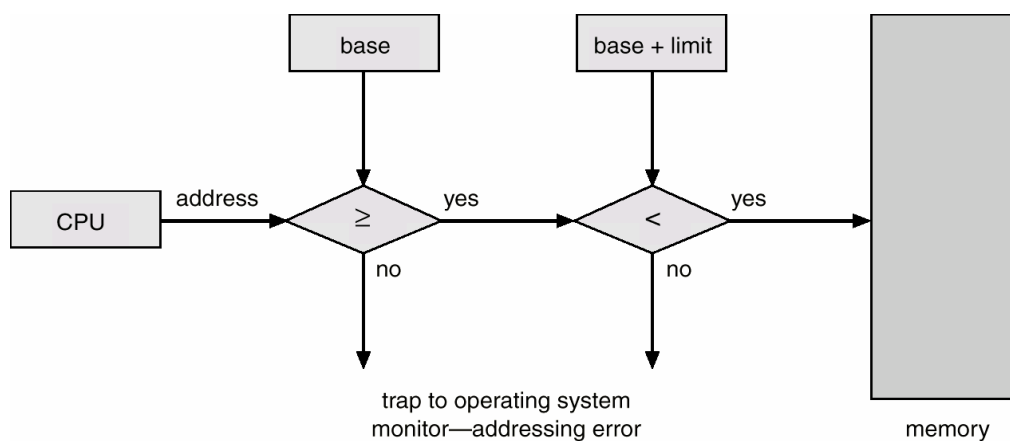
Το ΛΣ πρέπει να παρέχει προστασία μνήμης τουλάχιστον για τον πίνακα διανυσμάτων διακοπών και για τις ρουτίνες εξυπηρέτησης διακοπών. Για την προστασία της μνήμης, χρησιμοποιούνται δύο καταχωρητές που καθορίζουν το εύρος των αποδεκτών διευθύνσεων μνήμης που μπορεί να χρησιμοποιήσει ένα πρόγραμμα. Η μνήμη εκτός της καθορισμένης περιοχής είναι προστατευμένη. Πιο συγκεκριμένα, οι δύο καταχωρητές που χρησιμοποιούνται για την προστασία μνήμης είναι οι ακόλουθοι:

- base register (καταχωρητής βάσης): κρατά τη μικρότερη αποδεκτή φυσική διεύθυνση μνήμης.
- limit register (καταχωρητής ορίου): περιέχει το μέγεθος της περιοχής



Σχήμα 2.4. Καθορισμός λογικού χώρου διευθύνσεων

Κατά τη λειτουργία σε κατάσταση επόπτη (monitor mode), το ΛΣ έχει απεριόριστη πρόσβαση σε ολόκληρη τη μνήμη. Οι εντολές φόρτωσης (load) για τους καταχωρητές base & limit είναι προνομιούχες εντολές.



Σχήμα 2.5. Μετατροπή λογικής διεύθυνσης σε φυσική

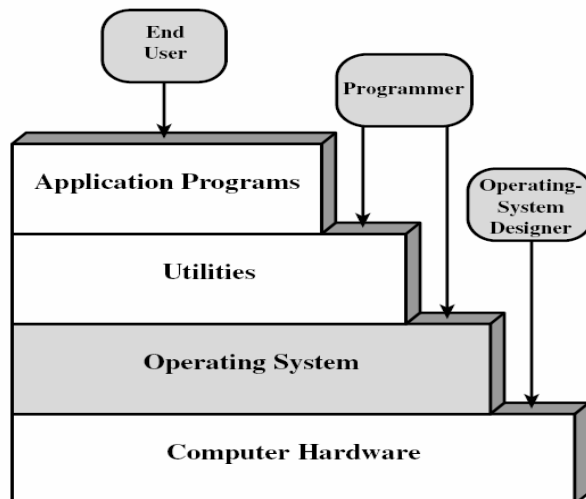
δ) Προστασία CPU

Ο επεξεργαστής διαθέτει κάποιον χρονιστή (Timer), ο οποίος διακόπτει τον υπολογιστή μετά από μια καθορισμένη περίοδο για να εξασφαλίσει ότι το ΛΣ διατηρεί τον έλεγχο. Ο χρονιστής: μειώνεται με κάθε clock tick και όταν φθάσει την τιμή 0, προκαλείται μια διακοπή. Ο χρονιστής χρησιμοποιείται συχνά σε συστήματα καταμερισμού χρόνου (time sharing) καθώς επίσης και για τον υπολογισμό της τρέχουσας ώρας. Η φόρτωση του timer είναι μια προνομιούχος εντολή.

2.1.2. Επικοινωνία με τον χρήστη

Το ΛΣ παρέχει αποδέσμευση από το υλικό (hardware) για την ανάπτυξη προγραμμάτων. Ο προγραμματισμός συστήματος πραγματοποιείται με κατάλληλα εργαλεία και προγράμματα όπως οι μεταγλωττιστές (compilers), οι διερμηνευτές (interpreters) και οι κειμενογράφοι (editors). Αντίθετα, το υλικό μέρος του υπολογιστή περιλαμβάνει τη γλώσσα μηχανής, τον μικροπρογραμματισμό και τις φυσικές συσκευές.

Ένα μικροπρόγραμμα είναι ένας διερμηνευτής (interpeter) που υποδέχεται εντολές σε γλώσσα μηχανής (ADD, MOVE, JUMP) και τις μεταφράζει σε μια σειρά από μικρά βήματα. Το μικροπρόγραμμα τοποθετείται σε μνήμη ROM, ενώ το σύνολο των εντολών που διερμηνεύει το μικροπρόγραμμα είναι η **γλώσσα μηχανής**, η οποία δεν αποτελεί μέρος του υλικού. Ένα τυπικό μέγεθος της γλώσσας μηχανής περιλαμβάνει 50-300 εντολές.



Σχήμα 2.6. Θέση του λειτουργικού συστήματος σε ένα υπολογιστικό σύστημα

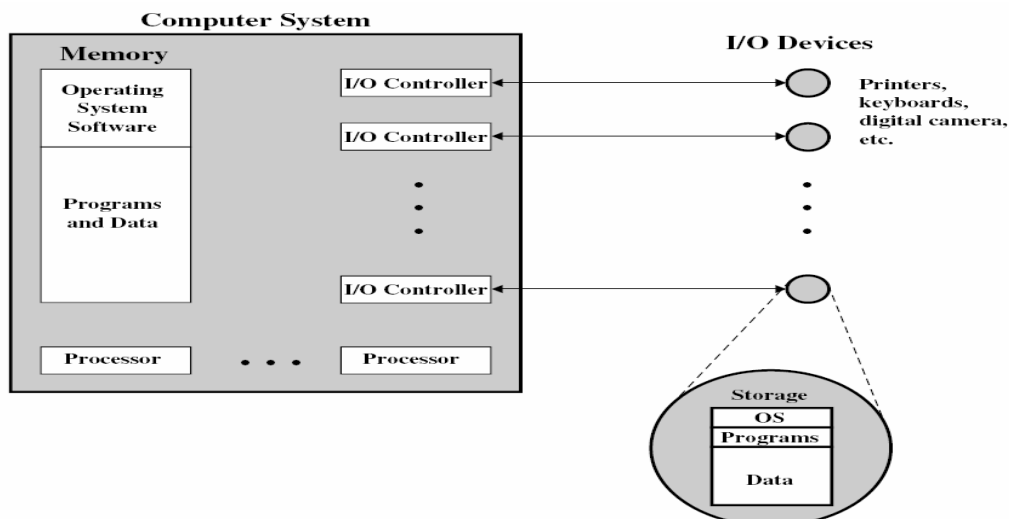
Μια κύρια αποστολή του ΛΣ είναι να αποκρύψει την πολυπλοκότητα που δημιουργείται από τη γλώσσα μηχανής και να δώσει στο χρήστη ένα περισσότερο εύχρηστο σύνολο εντολών για να εργαστεί αποτελεσματικά.

Στην κορυφή του ΛΣ βρίσκεται το υπόλοιπο λογισμικό συστήματος, που δεν αποτελεί μέρος του ΛΣ και περιλαμβάνει τον command interpreter ή φλοιός (shell), τους μεταγλωττιστές (compilers) και τους κειμενογράφους (editors). Το ΛΣ είναι το μέρος του υλικού που εκτελείται (τρέχει) σε κατάσταση πυρήνα (kernel mode) ή κατάσταση επόπτη (supervisor mode). Αντίθετα, οι compilers και οι interpreters τρέχουν σε κατάσταση χρήστη (user mode). Ο χρήστης μπορεί να γράψει ένα δικό του command interpreter όχι όμως π.χ. τον δικό του χειριστή διακοπών δίσκου που προστατεύεται από το ΛΣ. Το ΛΣ καλύπτει τις λεπτομέρειες του υλικού μέρους από τον προγραμματιστή και του παρέχει έναν κατάλληλο ενδιαμέσο για τη χρήση του συστήματος.

Ο πυρήνας είναι ένα τμήμα του λειτουργικού συστήματος που περιέχει τις πιο συχνά χρησιμοποιούμενες συναρτήσεις καθώς και άλλα τμήματα του ΛΣ που είναι σε τρέχουσα χρήση, όλα αυτά στην κύρια μνήμη του υπολογιστικού συστήματος. Οι υπηρεσίες που παρέχει στο λειτουργικό σύστημα στους χρήστες είναι η δυνατότητα ανάπτυξης και εκτέλεσης προγραμμάτων, η προσπέλαση σε συσκευές I/O και η ελεγχόμενη προσπέλαση σε αρχεία, η ανίχνευση σφαλμάτων και η κατάλληλη απόκρυψή τους και, τέλος, η απαραίτητη λογιστική που διατηρεί κατάλληλα στατιστικά στοιχεία χρήσης.

2.1.3 Διαχείριση Πόρων

Το ΛΣ παρέχει μια ελεγχόμενη κατανομή των επεξεργαστών, μνημών και των άλλων συσκευών (I/O) ανάμεσα στα διάφορα προγράμματα που ανταγωνίζονται για τη χρήση αυτών των πόρων.



Σχήμα 2.7. Διαχείριση Πόρων

Ένα ΛΣ λειτουργεί όπως ένα συνηθισμένο λογισμικό ΗΥ, το οποίο εκχωρεί συχνά εκχωρεί τον έλεγχο του συστήματος και πρέπει να βασίζεται στον επεξεργαστή ώστε να επανακτήσει τον έλεγχο. Επίσης, το λειτουργικό σύστημα παρέχει εντολές για τον επεξεργαστή, όπως και τα άλλα προγράμματα). Η κύρια διαφορά του όμως βρίσκεται στους στόχους του, αφού καθοδηγεί τον επεξεργαστή στη χρησιμοποίηση άλλων πόρων του συστήματος καθώς και στο χρονοισμό εκτέλεσης άλλων προγραμμάτων. Για την επίτευξη του τελευταίου, το λειτουργικό σύστημα εκχωρεί και επανακτήσει τον έλεγχο του επεξεργαστή.

2.1.4 Ικανότητα και ευκολία εξέλιξης λειτουργικού συστήματος

Το λειτουργικό σύστημα είναι σημαντικό να είναι σχεδιασμένο ώστε να μην επηρεάζεται από την αναβάθμιση του υλικού και να επιτρέπει την χρήση νέων τύπων του υλικού μέρους του υπολογιστικού συστήματος. Χαρακτηριστικά προς την κατεύθυνση αυτή είναι η σελιδοποίηση τμημάτων μνήμης και η υποστήριξη τερματικών γραφικών. Το λειτουργικό σύστημα πρέπει επίσης να είναι ανοιχτό στην εισαγωγή νέων υπηρεσιών, όπως έχει συμβεί με τη χρήση παραθύρων και την ενσωμάτωση ποικίλων στατιστικά εργαλείων. Τέλος, όπως κάθε πρόγραμμα, το λειτουργικό σύστημα πρέπει να επιδέχεται με εύκολο τρόπο διορθωμένες εκδόσεις του, που περιλαμβάνουν διορθώσεις σε υπάρχοντα σφάλματα και ανανεώσεις που βελτιώνουν την αποδοτικότητά του.

2.2 Σημαντικά σημεία εξέλιξης των ΛΣ

Τα σημαντικά σημεία στην εξέλιξη των λειτουργικών συστημάτων είναι η χρήση διεργασιών, η υποστήριξη για διαχείριση μνήμης, η προστασία και ασφάλεια πληροφοριών, η δρομολόγηση και διαχείριση πόρων και η δομή συστήματος.

2.2.1 Διεργασίες

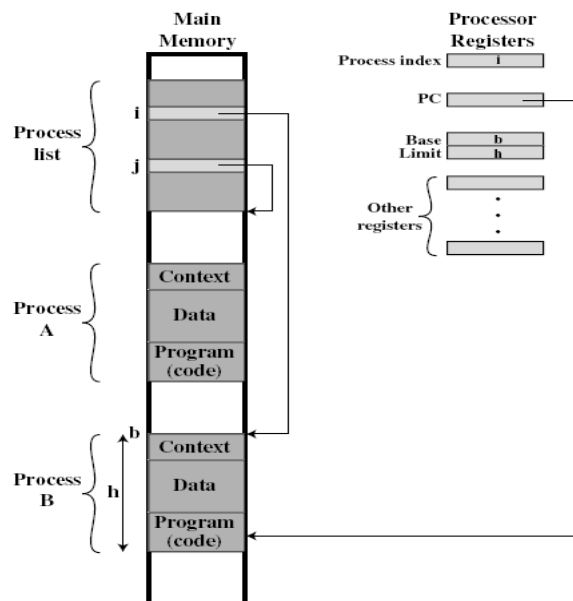
Μια διεργασία είναι ένα πρόγραμμα ή το στιγμιότυπο ενός προγράμματος που εκτελείται σε κάποιον υπολογιστή. Αποτελεί μία οντότητα που μπορεί να ανατεθεί και να εκτελεστεί σε έναν επεξεργαστή. Σε πολλά ΛΣ οι πληροφορίες που σχετίζονται με κάθε διεργασία αποθηκεύονται σε έναν πίνακα που δημιουργεί το ΛΣ και λέγεται

πίνακας διεργασιών, είναι στην ουσία μια δομή συνδεδεμένης λίστας (linked list), με κόμβους τις διεργασίες. Μια αναστελλόμενη διεργασία αποτελείται από το χώρο διευθύνσεων της, δηλαδή την εικόνα μνήμης που τις αντιστοιχεί, και τις πληροφορίες της, οι οποίες αποθηκεύονται στον πίνακα διεργασιών.

Το ΛΣ περιέχει θεμελιώδεις κλήσεις για τη διαχείριση διεργασιών που σχετίζονται με τη δημιουργία και τον τερματισμό των διεργασιών. Παραδείγματα αποτελούν η διακοπή ενός προγράμματος μέσω του πληκτρολογίου ή όταν μια διεργασία δημιουργεί θυγατρικές. Η σχεδίαση ενός ΛΣ σχετικά με τον συντονισμό όλων των διεργασιών είναι πολύ δύσκολη, αφού τα σφάλματα και τα λάθη προκύπτουν μόνον έπειτα από συγκεκριμένες και σπάνιες αλληλουχίες ενεργειών. Επιπλέον, η ανίχνευση του σφάλματος δε καθορίζει αυτόματα και την αιτία από την οποία προήλθε.

Οι βασικοί λόγοι για τη δημιουργία σφαλμάτων σε ένα λειτουργικό σύστημα είναι οι ακόλουθοι:

- Ανακριβής συγχρονισμός: τα σήματα που παράγονται από τις διακοπές να μην διαχειρίζονται σωστά από τον μηχανισμό σηματοδότησης.
- Αποτυχημένος αμοιβαίος αποκλεισμός: η προσπάθεια από χρήστες ή προγράμματα να κάνουν χρήση ενός διαμοιραζόμενου πόρου την ίδια χρονική στιγμή
- Ακαθόριστη λειτουργία προγράμματος: πολλά προγράμματα που διαμοιράζονται μνήμη να παρεμβαίνουν μεταξύ τους επανεγγράφοντας κοινές περιοχές της μνήμης με μη προβλέψιμους τρόπους. Η σειρά με την οποία δρομολογούνται διάφορα προγράμματα μπορεί να επηρεάσει το αποτέλεσμα
- Αδιέξοδα (deadlocks): δύο ή περισσότερα προγράμματα είναι σε αναμονή περιμένοντας το ένα το άλλο



Σχήμα 2.8 Τυπική υλοποίηση διεργασίας

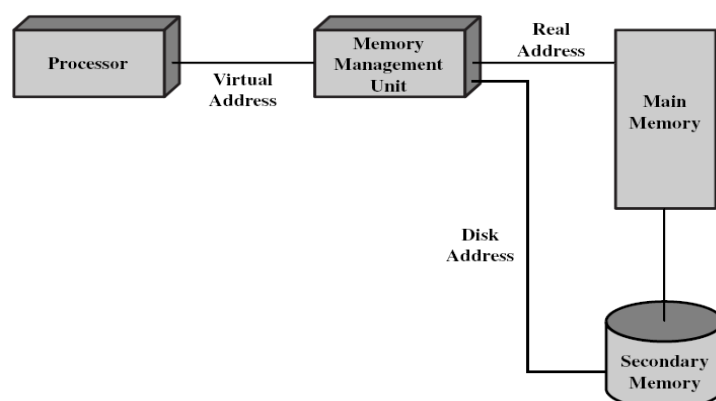
Το σχήμα 2.8 παρουσιάζει την τυπική υλοποίηση διεργασίες σε ένα λειτουργικό σύστημα. Η μνήμη του συστήματος οργανώνεται σε τμήματα και σε καθεμιά από τις δύο διεργασίες A και B έχει ανατεθεί ένα ξεχωριστό μπλοκ (τμήμα) μνήμης. Κάθε τμήμα μνήμης αποτελείται από το εκτελέσιμο πρόγραμμα, τα δεδομένα που απαιτούνται για την εκτέλεση, το περιεχόμενο εκτέλεσης δηλαδή την κατάσταση της διεργασίας, οποιαδήποτε πληροφορία απαιτείται από το ΛΣ για να διαχειριστεί τη

διεργασία (π.χ. προτεραιότητα) αλλά και οποιαδήποτε πληροφορία απαιτείται από τον επεξεργαστή για την κατάλληλη εκτέλεση της διεργασίας (π.χ. περιεχόμενο καταχωρητών όπως ο μετρητής προγράμματος, οι καταχωρητές δεδομένων κλπ.).

Για τη σωστή διαχείρισή τους, κάθε διεργασία εγγράφεται σε μια λίστα διεργασιών που συντηρείται από το ΛΣ. Η λίστα διεργασιών περιέχει μια είσοδο για κάθε διεργασία, η οποία αποτελεί ένα δείκτη στη θέση του μπλοκ της μνήμης που περιέχει τη διεργασία. Ο καταχωρητής δείκτη διεργασίας περιέχει τον δείκτη στη λίστα διεργασιών, για τη διεργασία που ελέγχει τον επεξεργαστή εκείνη τη στιγμή. Οι διεργασίες που έχουν διακοπεί εγγράφουν το περιεχόμενο των καταχωρητών τους κατά τη στιγμή της διακοπής, στο δικό τους περιεχόμενο εκτέλεσης. Η εναλλαγή διεργασίας περιλαμβάνει την αποθήκευση των πληροφοριών της τρέχουσας διεργασίας και την φόρτωση (επαναφορά) της διεργασίας που θα συνεχίσει τώρα να εκτελείται. Μια διεργασία μπορεί είτε να εκτελείται είτε να είναι σε αναμονή για εκτέλεση.

2.2.2 Διαχείριση μνήμης

Για να ικανοποιήσει τις απαιτήσεις των χρηστών, το ΛΣ έχει πέντε βασικές ευθύνες σχετικά με τη διαχείριση της αποθήκευσης: α) απομόνωση διεργασίας, β) αυτόματη τοποθέτηση και διαχείριση (όρια μνήμης, χάρτης μνήμης), γ) υποστήριξη σπονδυλωτού προγραμματισμού (δυναμική παραχώρηση μνήμης, διαχείριση δυναμικής μνήμης), δ) προστασία και έλεγχος προσπέλασης, και ε) μακροπρόθεσμη αποθήκευση. Οι παραπάνω απαιτήσεις ικανοποιούνται από το λειτουργικό σύστημα μέσω της Ιδεατής Μνήμης (virtual memory) και του Συστήματος Αρχείων.

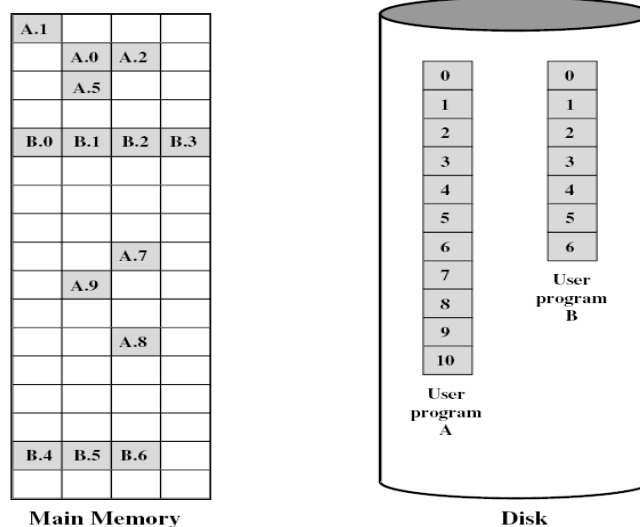


Σχήμα 2.9. Διευθυνσιοδότηση ιδεατής μνήμης

Η ιδεατή μνήμη είναι μια υπηρεσία που επιτρέπει σε προγράμματα να διευθυνσιοδοτούν τη φυσική μνήμη του συστήματος με λογικό τρόπο, χωρίς να λαμβάνουν υπόψη το διαθέσιμο μέγεθος της φυσικής μνήμης. Με την ιδεατή μνήμη αντιμετωπίζεται η απαίτηση πολλαπλές εργασίες να βρίσκονται ταυτόχρονα στην κεντρική μνήμη. Όταν ο επεξεργαστής εναλλάσσεται μεταξύ ενός πλήθους διεργασιών είναι δύσκολο να τις ομαδοποιήσει συμπαγώς στην κύρια μνήμη, επειδή οι διεργασίες ποικίλουν σε μέγεθος. Για το λόγο αυτό χρησιμοποιούνται συστήματα σελιδοποίησης.

Η σελιδοποίηση μνήμης επιτρέπει στις διεργασίες να αποτελούνται από μπλοκ μνήμης σταθερού μεγέθους που λέγονται σελίδες (π.χ. με μέγεθος 4K), με την κάθε σελίδα να μπορεί να τοποθετηθεί οπουδήποτε στην κύρια μνήμη. Κάθε πρόγραμμα αναφέρει μια εικονική διεύθυνση και το σύστημα σελιδοποίησης την μετατρέπει σε φυσική δηλ. πραγματική διεύθυνση (real address). Σημαντική απαίτηση είναι να

ελαχιστοποιηθεί η απαίτηση όλες οι σελίδες μιας διεργασίας να βρίσκονται ταυτόχρονα στην κύρια μνήμη. Για το λόγο αυτό, όσες δεν βρίσκονται μεταφέρονται από την δευτερεύουσα μέσω του συστήματος διαχείρισης μνήμης (όλες οι σελίδες μιας διεργασίας συντηρούνται στο δίσκο). Σαν αποτέλεσμα, το υλικό μέρος του επεξεργαστή μαζί με το ΛΣ παρέχει στο χρήστη έναν «ιδεατό επεξεργαστή», με προσπέλαση σε μια ιδεατή μνήμη. Οι εντολές μιας γλώσσας προγραμματισμού μπορούν αναφέρονται σε θέσεις προγράμματος και δεδομένων στην ιδεατή μνήμη. Η απομόνωση μιας διεργασίας μπορεί να επιτευχθεί δίνοντας σε κάθε διεργασία μια μοναδική, μη επικαλυπτόμενη ιδεατή μνήμη.



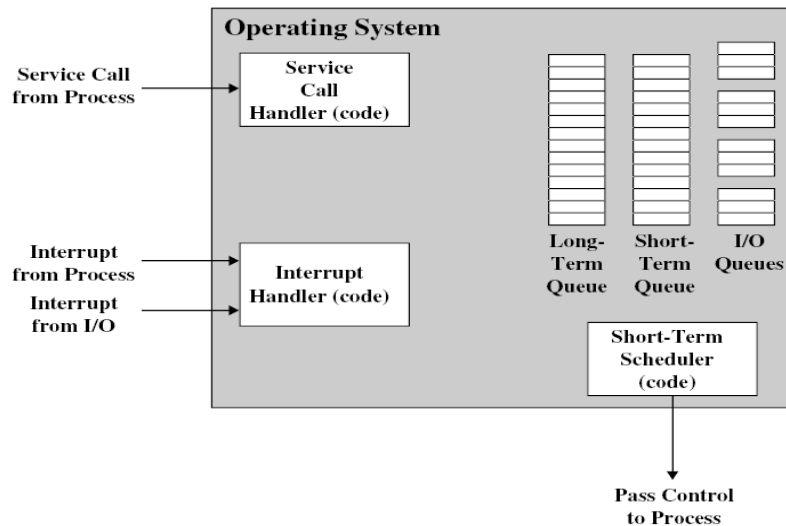
Σχήμα 2.10. Οργάνωση ιδεατής μνήμης

2.2.3 Προστασία και ασφάλεια πληροφοριών

Ένα από τα βασικά στοιχεία ενός λειτουργικού συστήματος είναι ο έλεγχος προσπέλασης που παρέχει και ο οποίος υλοποιείται με τη κατάλληλη ρύθμιση της πρόσβασης των χρηστών στο σύστημα. Το λειτουργικό σύστημα ελέγχει επίσης τη ροή πληροφοριών με τη σωστή ρύθμιση της ροής δεδομένων στο σύστημα, ενώ παρέχει κάθε στιγμή πιστοποίηση με τους μηχανισμούς προσπέλασης και ελέγχου να εκτελούνται σύμφωνα με τις προδιαγραφές και τις πολιτικές ασφαλείας που έχουν καθοριστεί.

2.2.4 Δρομολόγηση και διαχείριση πόρων

Η σωστή δρομολόγηση των εργασιών και η δίκαιη διαχείριση των υπολογιστικών πόρων από το λειτουργικό σύστημα προϋποθέτει αμεροληψία, που εξασφαλίζει με τον ορισμό κλάσεων προτεραιότητας, διαφορική απόκριση, που σημαίνει ότι το ΛΣ πρέπει να παίρνει αποφάσεις για τη δρομολόγηση των απαιτήσεων που προκύπτουν δυναμικά, και αποτελεσματικότητα. Η τελευταία συνίσταται στη μεγιστοποίηση της απόδοσης του συστήματος, στην ελαχιστοποίηση του χρόνου απόκρισης και στην καλύτερη δυνατή εξυπηρέτηση μεγάλου αριθμού χρηστών. Η δρομολόγηση και διαχείριση πόρων είναι πρόβλημα επιχειρησιακής έρευνας.



Σχήμα 2.11. Κύρια στοιχεία για πολυπρογραμματισμό

Ο πολυπρογραμματισμός περιλαμβάνει τη δρομολόγηση διεργασιών και τη διαχείριση πόρων σε περιβάλλοντα πολυπρογραμματισμού. Το λειτουργικό σύστημα διατηρεί έναν αριθμό ουρών κάθε μια από τις οποίες είναι μια λίστα διεργασιών που αναμένουν για κάποιον πόρο. Μια βραχυπρόθεσμη ουρά περιλαμβάνει τις διεργασίες που βρίσκονται στην κύρια μνήμη και είναι έτοιμες να εκτελεσθούν, ενώ ο βραχυπρόθεσμος δρομολογητής ή διεκπεραιωτής (dispatcher), επιλέγει την επόμενη προς εκτέλεση διεργασία σύμφωνα με κάποιες στρατηγικές επιλογής που κατά κανόνα περιλαμβάνουν εκ περιτροπής εξυπηρέτηση (round-robin) και επίπεδα προτεραιότητας. Είναι ακόμα διαθέσιμη μια μακροπρόθεσμη ουρά που παρέχει μια λίστα από νέες διεργασίες που περιμένουν να χρησιμοποιήσουν τον επεξεργαστή, όπως και ουρές για τις συσκευές I/O, στις οποίες περιέχονται όλες οι διεργασίες που περιμένουν να χρησιμοποιήσουν μια συσκευή.

2.2.5 Δομή συστήματος

Η δομή του συστήματος, δηλαδή ο τρόπος σχεδίασης του λειτουργικού συστήματος, είναι ιδιαίτερα σημαντική αφού μπορεί να αντιμετωπίσει αποτελεσματικά προβλήματα που σχετίζονται με την αύξηση του μεγέθους και την πολυπλοκότητα των λειτουργικών συστημάτων, όπως είναι η καθυστερημένη χρονική παράδοση, η ύπαρξη μη εμφανών προγραμματιστικών λαθών και η μειωμένη απόδοση. Σχεδιαστικές τάσεις της δομής του ΛΣ που έχουν καθιερωθεί είναι ο σπονδυλωτός προγραμματισμός, η ελαχιστοποίηση του interface μεταξύ των τμημάτων του λογισμικού και η χρήση ιεραρχικών επιπέδων.

Η ιεραρχική δομή των σύγχρονων ΛΣ χωρίζει τις λειτουργίες σύμφωνα με την πολυπλοκότητά τους δημιουργώντας μια σειρά επιπέδων. Κάθε επίπεδο εκτελεί ένα υποσύνολο λειτουργιών, βασίζεται στο αμέσως προηγούμενο για την εκτέλεση πιο πρωταρχικών λειτουργιών και παρέχει υπηρεσίες στο υψηλότερο επίπεδο. Τα χαμηλότερα επίπεδα απασχολούνται σε μικρότερη χρονική κλίμακα, αλληλεπιδρώντας άμεσα με το υλικό.

Level	Name	Objects	Example Operations
13	Shell	User programming environment	Statements in shell language
12	User processes	User processes	Quit, kill, suspend, resume
11	Directories	Directories	Create, destroy, attach, detach, search, list
10	Devices	External devices, such as printers, displays, and keyboards	Open, close, read, write
9	File system	Files	Create, destroy, open, close, read, write
8	Communications	Pipes	Create, destroy, open, close, read, write
7	Virtual memory	Segments, pages	Read, write, fetch
6	Local secondary store	Blocks of data, device channels	Read, write, allocate, free
5	Primitive processes	Primitive processes, semaphores, ready list	Suspend, resume, wait, signal
4	Interrupts	Interrupt-handling programs	Invoke, mask, unmask, retry
3	Procedures	Procedures, call stack, display	Mark stack, call, return
2	Instruction set	Evaluation stack, microprogram interpreter, scalar and array data	Load, store, add, subtract, branch
1	Electronic circuits	Registers, gates, buses, etc.	Clear, transfer, activate, complement

Shaded area represents hardware.

Σχήμα 2.12 Ιεραρχική δομή λειτουργικών συστημάτων

2.3 Χαρακτηριστικά σύγχρονων λειτουργικών συστημάτων

Τα σύγχρονα λειτουργικά συστήματα έχουν επηρεαστεί σε μεγάλο βαθμό από τις εξελίξεις που έχουν πραγματοποιηθεί στο επίπεδο του υλικού και του λογισμικού αλλά και στη γενικότερη σχεδίαση και αρχιτεκτονική τους.

Η εξέλιξη του υλικού περιλαμβάνει την διαθεσιμότητα και χρησιμοποίηση μεγάλου αριθμού επεξεργαστών, την υψηλή ταχύτητα συνδέσεων δικτύου και τις πολλές και μεγάλες σε χωρητικότητα συσκευές αποθήκευσης. Όσον αφορά το λογισμικό, έχουμε την επικράτηση των πολυμεσικών εφαρμογών, την εύκολη και γρήγορη πρόσβαση στο διαδίκτυο και την καθιέρωση του υπολογιστικού μοντέλου πελάτη/εξυπηρετητή (client/server). Τέλος, η εξέλιξη στη σχεδίαση και αρχιτεκτονική του ΛΣ περιλαμβάνει την αρχιτεκτονική μικροπυρήνα, την πολυνημάτωση (multithreading), τα συστήματα πολυεπεξεργασίας και τα παράλληλα συστήματα, καθώς επίσης και τα συστήματα πραγματικού χρόνου (real time systems) και τα κατακευμασμένα ΛΣ (distributed systems).

Ερωτήσεις Επανάληψης

1. Τι περιλαμβάνει η προστασία υλικού που παρέχεται από το λειτουργικό σύστημα;
2. Τι είναι η λειτουργία dual-mode και τι μικροπρόγραμμα;
3. Ποιοι είναι οι βασικοί λόγοι για τη δημιουργία σφαλμάτων σε ένα λειτουργικό σύστημα;
4. Ποια είναι τα σημαντικά σημεία εξέλιξης των λειτουργικών συστημάτων;
5. Αναφέρετε τα χαρακτηριστικά του υλικού και του λογισμικού των σύγχρονων λειτουργικών συστημάτων.

Ασκήσεις

1. Η προστασία υλικού που παρέχεται από το λειτουργικό σύστημα εν περιλαμβάνει:
 - α) προστασία εισόδου-εξόδου
 - β) προστασία αρχείων χρηστών
 - γ) προστασία μνήμης
 - δ) προστασία επεξεργαστή

2. Στη λειτουργία user-mode:
 - α) η εκτέλεση γίνεται εκ μέρους του χρήστη
 - β) η εκτέλεση γίνεται εκ μέρους του λειτουργικού συστήματος
 - γ) εκτελούνται προνομιούχες εντολές του επεξεργαστή
 - δ) πραγματοποιείται η αντιμετώπιση των σφαλμάτων εκτέλεσης
3. Κλήση συστήματος:
 - α) είναι μια προνομιούχα εντολή του επεξεργαστή
 - β) παρέχει στο χρήστη πλήρη έλεγχο του συστήματος
 - γ) είναι μηχανισμός αιτήσεων εξυπηρέτησης προς το λειτουργικό
 - δ) είναι ένας τρόπος τερματισμού της λειτουργίας του συστήματος
4. Ένα μικροπρόγραμμα είναι
 - α) ένα ειδικό πρόγραμμα χρήστη
 - β) μεθοδολογία προγραμματισμού του επεξεργαστή
 - γ) ένα σύντομο πρόγραμμα σε γλώσσα υψηλού επιπέδου
 - δ) ένας διερμηνευτής που επεξεργάζεται εντολές σε γλώσσα μηχανής
5. Δεν αποτελεί σημαντικό σημείο εξέλιξης των λειτουργικών συστημάτων
 - α) υποστήριξη μονοπρογραμματισμού
 - β) προστασία και ασφάλεια πόρων
 - γ) διεργασίες
 - δ) διαχείριση πόρων
6. Οι διεργασίες
 - α) χρησιμοποιούνται για τη μεταφορά δεδομένων
 - β) σχετίζονται με τη οργάνωση των αρχείων στο σκληρό δίσκο
 - γ) διατηρούν πληροφορίες του λειτουργικού συστήματος
 - δ) εκτελούνται στον επεξεργαστή
7. Δεν αποτελεί κύριο λόγο για τη δημιουργία σφαλμάτων σε ένα λειτουργικό σύστημα
 - α) ανακριβής συγχρονισμός
 - β) μικρό μέγεθος κύριας μνήμης
 - γ) αποτυχημένος αμοιβαίος αποκλεισμός
 - δ) αδιέξοδο
8. Η ιδεατή μνήμη
 - α) επεκτείνει τη φυσική μνήμη του συστήματος
 - β) οργάνωνει τη φυσική μνήμη του συστήματος με λογικό τρόπο
 - γ) αποτελεί ειδικό υλικό του υπολογιστικού συστήματος
 - δ) επιτρέπει την επικοινωνία προγραμμάτων – λειτουργικού συστήματος
9. Η ιεραρχική δομή των λειτουργικών συστημάτων
 - α) αυξάνει σημαντικά το μέγεθος τους
 - β) μειώνει την απόδοσή τους
 - γ) μειώνει την πολυπλοκότητά τους
 - δ) αλληλεπιδρά άμεσα με το υλικό του συστήματος

10. Χαρακτηριστικό του λογισμικού των σύγχρονων λειτουργικών συστημάτων αποτελεί:

- α) η χρησιμοποίηση μεγάλου αριθμού επεξεργαστών
- β) η καθιέρωση του υπολογιστικού μοντέλου πελάτη/εξυπηρετητή
- γ) η πολυνημάτωση
- δ) η ανάπτυξη συστημάτων πραγματικού χρόνου

Κεφάλαιο 3 – Διεργασίες

1. Εισαγωγή - ορισμοί
2. Καταστάσεις διεργασίας
3. Διαγράμματα καταστάσεων
4. Μπλοκ ελέγχου διεργασίας – PCB
5. Υπηρεσίες Λ.Σ. για διαχείριση διεργασιών

3. 1. Εισαγωγή

Η έννοια της διεργασίας (process) είναι θεμελιώδης για την κατανόηση του τρόπου με τον οποίο οι σύγχρονοι Η/Υ επιτελούν και ελέγχουν πλήθος ταυτόχρονων δραστηριοτήτων. Μια διεργασία μπορεί να θεωρηθεί ως ένα πρόγραμμα σε εκτέλεση, μια ασύγχρονη δραστηριότητα αλλά και μια εκτελούμενη διαδικασία που συσχετίζεται με την ύπαρξη μιας δομής δεδομένων στο ΛΣ, τον περιγραφέα διεργασίας (process descriptor) ή μπλοκ ελέγχου διεργασίας (Process Control Block). Το πρόγραμμα για μια διεργασία είναι ότι και μια παρτιτούρα σε μια συμφωνική ορχήστρα, που εκτελεί το αντίστοιχο μουσικό κομμάτι που περιέχει η παρτιτούρα

Μια διεργασία είναι μια οντότητα με τον δικό της χώρο διευθύνσεων, που αποτελείται από τα εξής:

- την περιοχή κώδικα (text region), που περιέχει τον κώδικα που εκτελεί ο επεξεργαστής
- την περιοχή δεδομένων (data region), όπου αποθηκεύονται οι μεταβλητές και η δυναμικά παραχωρούμενη μνήμη που χρησιμοποιεί ο επεξεργαστής κατά τη διάρκεια της εκτέλεσης
- την περιοχή στοίβας (stack region), όπου αποθηκεύονται οι εντολές και οι τοπικές μεταβλητές για τις ενεργές κλήσεις διαδικασιών

Η διαχείριση των διεργασιών αποτελεί το θεμελιώδες αντικείμενο οποιουδήποτε ΛΣ. Το ΛΣ διατηρεί μια δομή δεδομένων για κάθε διεργασία που περιγράφει την κατάσταση της και την κατοχή πόρων του συστήματος, και δίνει τη δυνατότητα στο Λ.Σ. να ασκεί επιρροή στον έλεγχο της διεργασίας.

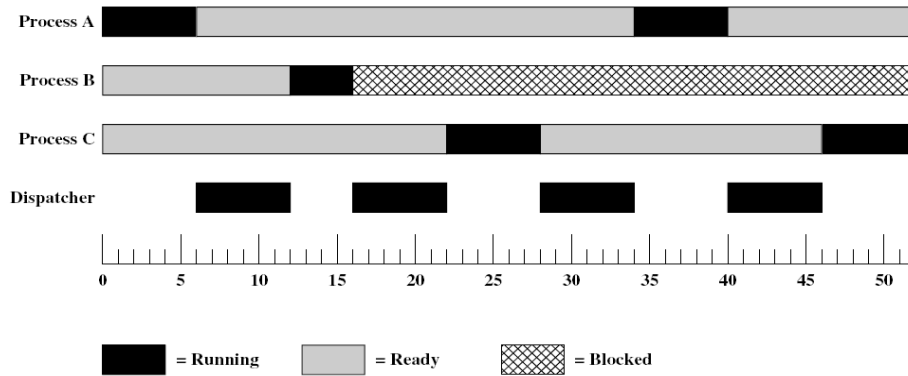
Οι περισσότερες απαιτήσεις που πρέπει να αντιμετωπίσει το ΛΣ εκφράζονται με αναφορά στις διεργασίες. Έτσι, το ΛΣ πρέπει να παρεμβάλει την εκτέλεση πολλαπλών διεργασιών με σκοπό τη μεγιστοποίηση της χρήσης του επεξεργαστή και την ελαχιστοποίηση του χρόνου απόκρισης των διεργασιών. Επίσης το ΛΣ πρέπει να αναθέτει πόρους στις διεργασίες με μια συγκεκριμένη πολιτική και να αποφεύγει ταυτόχρονα το αδιέξοδο. Τέλος, το ΛΣ χρειάζεται να υποστηρίζει την επικοινωνία των διεργασιών και τη δημιουργία διεργασιών από τους χρήστες

3. 2. Καταστάσεις διεργασίας

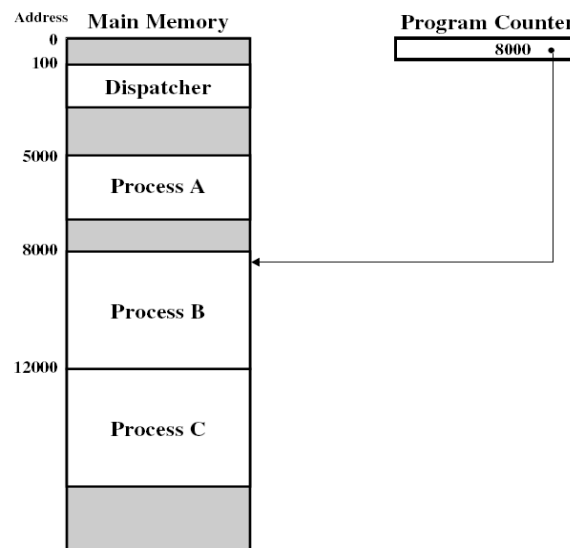
Το ΛΣ πρέπει να εξασφαλίσει ότι κάθε διεργασία αναλαμβάνει για αρκετό χρόνο τη χρήση του επεξεργαστή. Σε κάθε σύστημα μπορούν να υπάρχουν τόσες ταυτόχρονες διεργασίες όσοι είναι και οι επεξεργαστές. Στην πραγματικότητα σε κάθε επεξεργαστή υπάρχουν περισσότερες διεργασίες. Κατά τη διάρκεια ζωής της κάθε διεργασία μπορεί να βρεθεί σε μια σειρά από διακριτές καταστάσεις. Διάφορα γεγονότα αναγκάζουν μια διεργασία να μεταβάλει την κατάστασή της.

Οι βασικές γενικές καταστάσεις στις οποίες μπορεί να βρεθεί μια διεργασία είναι οι παρακάτω:

- Εκτελούμενη (running): η διεργασία χρησιμοποιεί τη CPU εκείνη τη στιγμή.
- Έτοιμη (ready): η διεργασία έχει διακοπεί προσωρινά για να εκτελεστεί κάποια άλλη
- Υπό αναστολή (blocked): η διεργασία δεν μπορεί να εκτελεστεί μέχρι να συμβεί κατάλληλο εξωτερικό συμβάν



Σχήμα 3.1 Δρομολόγηση διεργασιών και καταστάσεις



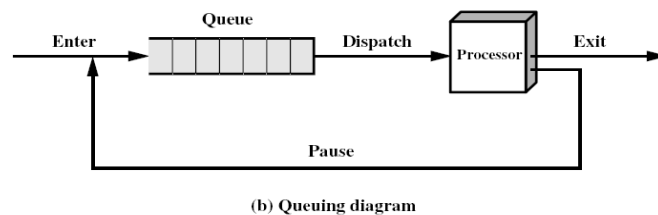
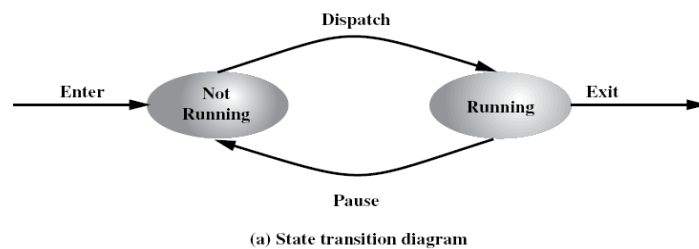
Σχήμα 3.2 Στιγμιότυπο εκτέλεσης διεργασιών

3.3 Διαγράμματα καταστάσεων

Κατά την εκτέλεση των προγραμμάτων των χρηστών, οι διεργασίες δημιουργούνται και εισέρχονται στη λίστα των έτοιμων διεργασιών. Με την πάροδο του χρόνου μια διεργασία προωθείται προς την κεφαλή της λίστας και μόλις ο επεξεργαστής γίνει διαθέσιμος η διεργασία ανατίθεται στον επεξεργαστή και η κατάστασή της αλλάζει σε εκτελούμενη. Η διαδικασία της ανάθεσης της πρώτης διεργασίας από τη λίστα των έτοιμων διεργασιών στον επεξεργαστή ονομάζεται διεκπεραίωση και υλοποιείται από μια οντότητα του συστήματος, τον διεκπεραιωτή (dispatcher).

Για να αποφευχθεί η μονοπώληση της χρήσης του επεξεργαστή, το ΛΣ χρησιμοποιεί μια συσκευή που παράγει μια διακοπή (interrupt) μετά από την πάροδο ενός προκαθορισμένου χρονικού ορίου, που ονομάζεται κβάντο χρόνου. Αν η διεργασία

δεν εκχωρήσει από μόνη της τον επεξεργαστή, πριν παρέλθει το κβάντο χρόνου, το ΛΣ μέσω της διακοπής που παράγεται, αναλαμβάνει τον έλεγχο του επεξεργαστή. Στην περίπτωση αυτή, η κατάσταση της τρέχουσας διεργασίας αλλάζει σε έτοιμη και προωθείται προς εκτέλεση η αμέσως επόμενη διεργασία. Αν μια εκτελούμενη διεργασία απαιτήσει μια λειτουργία I/O πριν λήξει το κβάντο χρόνου της, παραιτείται από τη χρήση του επεξεργαστή αναμένοντας την ολοκλήρωση της I/O λειτουργίας. Η διεργασία τότε τελεί υπό αναστολή. Οι υπό αναστολή διεργασίες θεωρούνται κοιμώμενες (sleep), επειδή δεν μπορούν να εκτελεστούν, ακόμη και αν ο επεξεργαστής καταστεί διαθέσιμος. Όταν ολοκληρωθεί μια διαδικασία I/O η αντίστοιχη διεργασία αλλάζει την κατάστασή της (wake up) από υπό αναστολή σε έτοιμη και εισέρχεται στη λίστα των έτοιμων διεργασιών. Οι καταστάσεις διεργασιών αποτυπώνονται στα διαγράμματα καταστάσεων, ενώ η μετάβαση από μια κατάσταση σε μία άλλη γίνεται λόγω της ύπαρξης συγκεκριμένων γεγονότων.



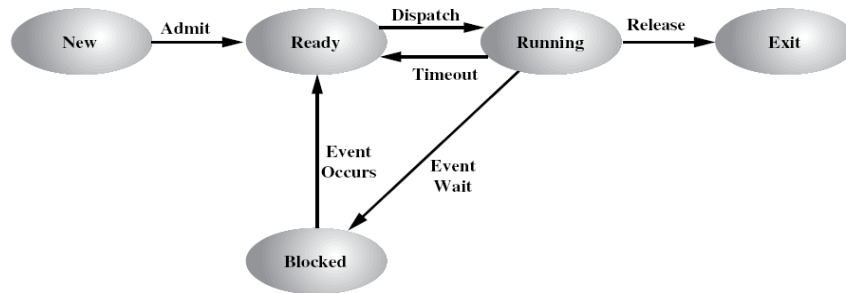
Σχήμα 3.3 Μοντέλο διεργασίας δύο καταστάσεων

Σύμφωνα με τα παραπάνω, οι βασικές καταστάσεις μιας διεργασίας είναι οι ακόλουθες:

- Running:: η διεργασία που είναι σε εκτέλεση
- Ready: είναι έτοιμη για εκτέλεση
- Blocked: δεν μπορεί να εκτελεστεί μέχρι να προκύψει κάποιο γεγονός
- New: έχει μόλις δημιουργηθεί από το ΛΣ
- Exit: έχει απελευθερωθεί από τη δεξαμενή εκτελέσιμων εργασιών από το ΛΣ

Αντίστοιχα, οι βασικοί τύποι γεγονότων, σύμφωνα με τους οποίους αλλάζει κατάσταση μια διεργασία, είναι οι εξής:

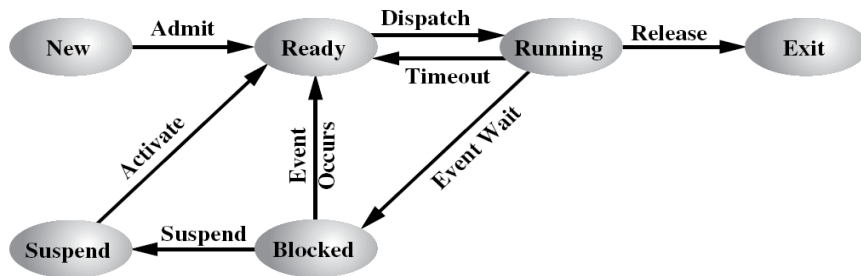
- New→Ready: μια επιπρόσθετη διεργασία εισάγεται όταν δεν παραβιάζεται ένα μέγιστο όριο που εξαρτάται από τις υπάρχουσες διεργασίες και το μέγεθος της ιδεατής μνήμης.
- Running→Ready: η εκτελούμενη διεργασία έχει φθάσει στο μέγιστο επιτρεπτό όριο μη διακοπτόμενης εκτέλεσής της
- Running→Blocked: η διεργασία απαιτεί μια υπηρεσία από το ΛΣ για την οποία το ΛΣ δεν είναι προετοιμασμένο να την εκτελέσει άμεσα



Σχήμα 3.4 Μοντέλο διεργασίας πέντε καταστάσεων

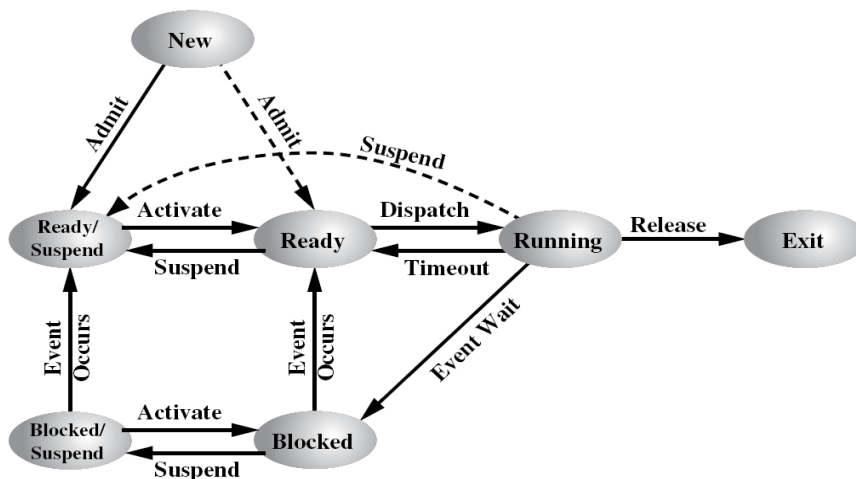
Οι διεργασίες μπορούν να βρεθούν σε κατάσταση αναστολής επειδή ο επεξεργαστής είναι πολύ πιο γρήγορος από τις συσκευές E/E με συνέπεια να είναι δυνατό όλες οι διεργασίες να περιμένουν κάποια εντολή E/E. Στην περίπτωση αυτή στέλνονται κάποιες διεργασίες στο σκληρό δίσκο για να ελευθερωθεί κύρια μνήμη (RAM), με αποτέλεσμα να προκύπτουν δύο νέες καταστάσεις διεργασίας:

- Μπλοκαρισμένη, σε αναστολή (Blocked / Suspend)
- Έτοιμη, σε αναστολή (Ready / Suspend)



(a) With One Suspend State

Σχήμα 3.5 Μοντέλο διεργασίας με μία κατάσταση αναστολής

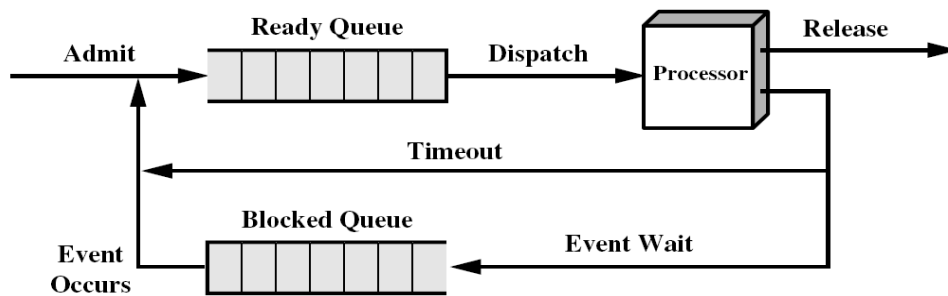


(b) With Two Suspend States

Σχήμα 3.6 Μοντέλο διεργασίας με δύο καταστάσεις αναστολής

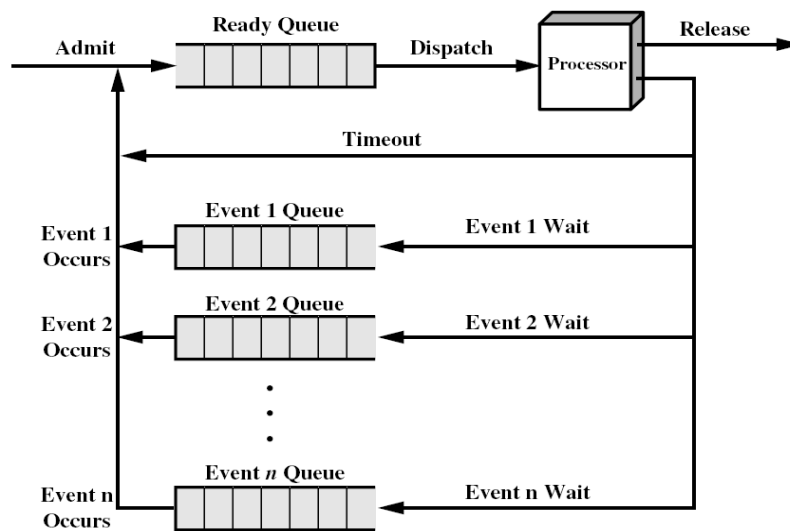
Όταν μια διεργασία βρίσκεται σε μια ουρά, το ΛΣ με κάποιον τρόπο πρέπει να επιλέξει μια διεργασία από την ουρά. Η λειτουργία αυτή επιτελείται από έναν

χρονοδρομολογητή. Ο μακροπρόθεσμος δρομολογητής (Long-term scheduler ή job scheduler) αποφασίζει και επιλέγει τις διεργασίες που θα περιληφθούν στην ουρά των έτοιμων προς εκτέλεση διεργασιών (ready queue), ελέγχοντας έτσι τον βαθμό πολυπρογραμματισμού. Ο βραχυπρόθεσμος δρομολογητής (Short-term scheduler ή CPU scheduler) επιλέγει ποια διεργασία θα είναι η επόμενη προς εκτέλεση και εκχωρεί τη χρήση της CPU. Τέλος, ο μεσοπρόθεσμος δρομολογητής (medium-term scheduler) χρησιμοποιείται ειδικά σε συστήματα καταμερισμού χρόνου ως ένα ενδιάμεσο επίπεδο χρονοδρομολόγησης, μετακινώντας περιοδικά διεργασίες από τη μνήμη



(a) Single blocked queue

Σχήμα 3.7 Απλή ουρά διεργασιών σε αναμονή



(b) Multiple blocked queues

Σχήμα 3.8 Πολλαπλές ουρές διεργασιών σε αναμονή

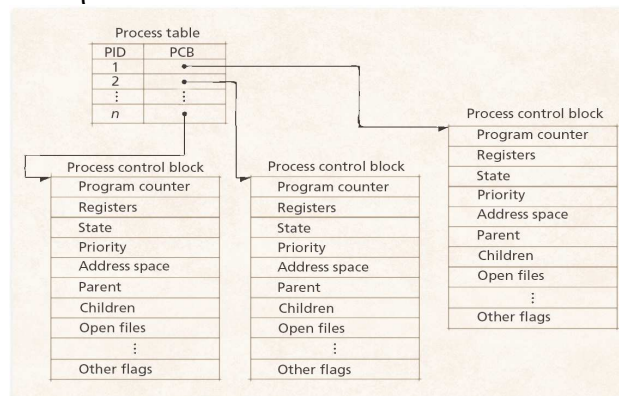
3.4. Μπλοκ ελέγχου διεργασίας

Το ΛΣ εκτελεί αρκετές λειτουργίες όταν δημιουργεί μια νέα διεργασία. Καταρχήν πρέπει να μπορεί να ταυτοποιήσει κάθε νέα διεργασία, έτσι αναθέτει σε κάθε μία ένα μοναδικό αριθμό (Process Identification number – PID). Στη συνέχεια δημιουργεί το μπλοκ ελέγχου διεργασίας (Process Control Block – PCB), το οποίο ονομάζεται και περιγραφέας διεργασίας, που διατηρεί τις πληροφορίες που απαιτούνται για τη διαχείριση της διεργασίας. Το PCB είναι το σύνολο των χαρακτηριστικών που χρησιμοποιούνται από το ΛΣ για τον έλεγχο της διεργασίας.

Το μπλοκ ελέγχου διεργασίας είναι συστατικό της εικόνας της διεργασίας που περιλαμβάνει επίσης το πρόγραμμα, τα δεδομένα και τη στοίβα. Μια εικόνα διεργασίας αποτελείται από ένα σύνολο θέσεων μνήμης (μεταβλητού ή σταθερού μήκους) που

αποθηκεύονται σε μη γειτονικές θέσεις. Το ΛΣ μπορεί να ασχολείται κάθε στιγμή μόνο με ένα τμήμα μιας συγκεκριμένης διεργασίας. Σε κάθε χρονική στιγμή ένα τμήμα της εικόνας διεργασίας μπορεί να βρίσκεται στην κύρια μνήμη και το υπόλοιπο στη δευτερεύουσα μνήμη.

Για να υλοποιηθεί οποιοδήποτε μοντέλο κατάστασης διεργασιών, το ΛΣ οικοδομεί έναν πίνακα διεργασιών (process table), ο οποίος περιλαμβάνει μια καταχώρηση για κάθε διεργασία. Στην καταχώρηση αυτή εγγράφονται πληροφορίες σχετικές με την κατάσταση της διεργασίας (εικόνα της διεργασίας) ώστε να συνεχίσει να εκτελείται μετά από κάποια διακοπή



Σχήμα 3.9 Process Control Blocks (PCBs)

Τα πεδία καταχώρησης του πίνακα διεργασιών χωρίζονται σε τρεις κατηγορίες, που σχετίζονται με τη διαχείριση των διεργασιών, της μνήμης και των αρχείων. Αναλυτικότερα, είναι τα εξής:

1. Διαχείριση διεργασιών
 - Καταχωρητές
 - Δείκτης εντολών προγράμματος
 - Λέξη κατάστασης προγράμματος
 - Δείκτης στοίβας
 - Κατάσταση διεργασίας
 - Χρόνος εκκίνησης διεργασίας
 - Χρόνος χρήσης CPU
 - Χρόνος CPU θυγατρικών διεργασιών
 - Χρόνος επόμενης εγρήγορσης
 - Δείκτης ουράς μηνυμάτων
2. Διαχείριση μνήμης
 - Δείκτης σε τμήμα κειμένου
 - Δείκτης σε τμήμα δεδομένων
 - Κατάσταση εξόδου
 - Κατάσταση σήματος
 - Ταυτότητα διεργασίας
 - Γονική διεργασία
 - Ομάδα διεργασιών
 - Πραγματική και λειτουργική ταυτότητα χρήστη και ομάδας χρήστη
 - Χάρτης δυαδικών ψηφίων για σήματα
3. Διαχείριση αρχείων
 - Μάσκα δικαιωμάτων
 - Πρωταρχική διαδρομή

- Διαδρομή εργασίας
- Περιγραφείς αρχείων
- Λειτουργική ταυτότητα χρήστη και ομάδας
- Παράμετροι κλήσεων συστήματος
- Διάφοροι ενδείκτες

Οι λόγοι για τους οποίους μπορεί να συμβεί εναλλαγή μιας εκτελούμενης διεργασίας είναι οι εξής:

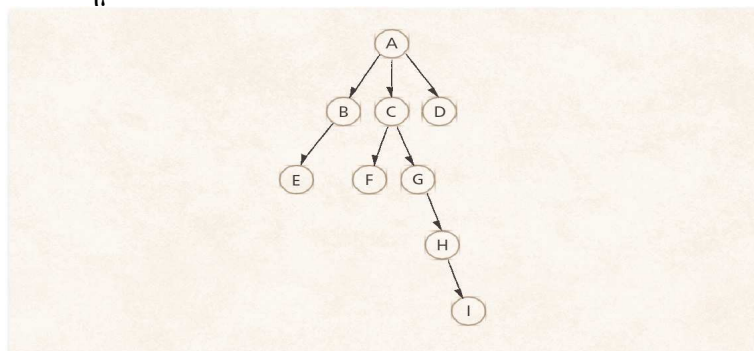
- Διακοπή ρολογιού (Clock interrupt): η διεργασία έχει εξαντλήσει το μέγιστο επιτρεπόμενο χρονικό όριο (κβάντο χρόνου)
- Διακοπή E/E (I/O interrupt)
- Σφάλμα μνήμης: η διεύθυνση μνήμης βρίσκεται στην εικονική μνήμη και πρέπει να μεταφερθεί στην κύρια μνήμη
- Παγίδευση (Trap): έχει συμβεί σφάλμα
- Κλήση επόπτη (κλήση συστήματος), π.χ. άνοιγμα αρχείου

Κατά την αλλαγή κατάστασης μιας διεργασίας από εκτελούμενη (running) σε κάποια άλλη κατάσταση, αρχικά γίνεται αποθήκευση του πλαισίου του επεξεργαστή που περιλαμβάνει τον PC και άλλους καταχωρητές (στο PCB) και μετακίνηση του PCB στην αντίστοιχη ουρά (ready, blocked, ...). Στη συνέχεια επιλέγεται μια άλλη διεργασία προς εκτέλεση, ενημερώνεται το PCB της διεργασίας που έχει επιλεγεί καθώς και οι δομές δεδομένων που σχετίζονται με τη διαχείριση μνήμης. Τέλος πραγματοποιείται επαναφορά πλαισίου (στον επεξεργαστή) της διεργασίας που έχει επιλεγεί και κατά συνέπεια αλλαγή της κατάστασης της συγκεκριμένης διεργασίας.

Όταν η CPU εναλλάσσει την τρέχουσα διεργασία με μια άλλη διεργασία, το σύστημα πρέπει να αποθηκεύσει την κατάσταση (state) της παλιάς διεργασίας και να φορτώσει την αποθηκευμένη κατάσταση για την νέα διεργασία. Ο χρόνος εναλλαγής πλαισίου αποτελεί καθυστέρηση (overhead), αφού το σύστημα δεν κάνει κάποια χρήσιμη λειτουργία κατά τη διάρκεια της εναλλαγής. Ο χρόνος εναλλαγής πλαισίου εξαρτάται από την υποστήριξη εκ μέρους του υλικού.

3.5. Υπηρεσίες ΛΣ για διαχείριση διεργασιών

Τα ΛΣ πρέπει να μπορούν να υλοποιούν συγκεκριμένες λειτουργίες που αφορούν τις διεργασίες. Οι πυρήνες των ΛΣ πολυπρογραμματισμού παρέχουν υπηρεσίες (system calls) για τη διαχείριση των διεργασιών. Ως υπηρεσίες εννοούνται οι προκαθορισμένες κλήσεις συστήματος που μπορούν να τεθούν σε λειτουργία από το χρήστη, είτε άμεσα μέσω κλήσεων του supervisor που ενσωματώνονται στον κώδικα του χρήστη, είτε έμμεσα, μέσω εντολών που δίνονται στο τερματικό και μεταφράζονται σε κλήσεις από το λειτουργικό σύστημα.



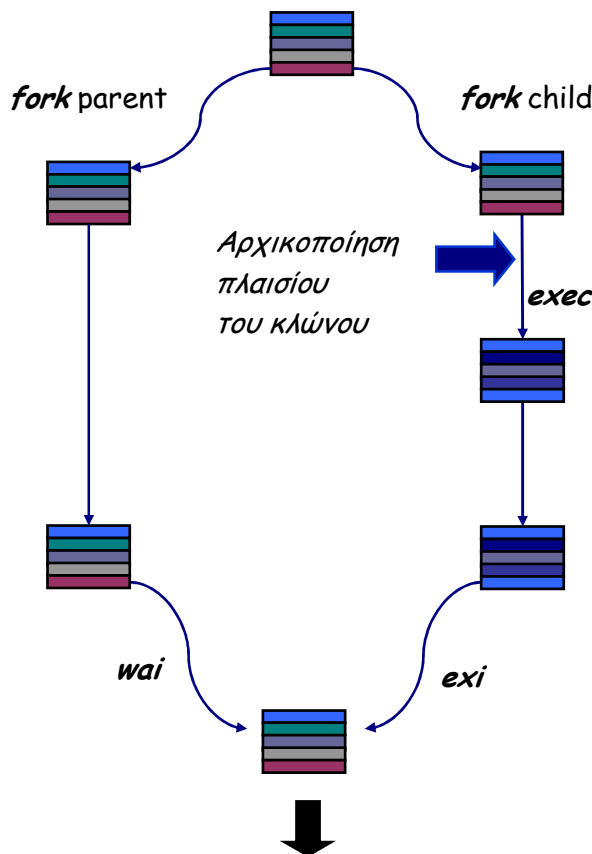
Σχήμα 3.10 Ιεραρχία δημιουργίας διεργασιών

Τα διάφορα λειτουργικά συστήματα, αν και συχνά διαφέρουν στον σχεδιασμό και τη φιλοσοφία παρουσιάζουν μεγάλες ομοιότητες στα εσωτερικά επίπεδα του πυρήνα, όσον αφορά τον τύπο και το εύρος της στοιχειώδους διαχείρισης διεργασιών που παρέχουν. Οι λεπτομέρειες και οι παράμετροι διαφέρουν αναπόφευκτα από το ένα σύστημα στο άλλο, αλλά οι συναρτήσεις που παρέχονται από το σύνολο των ΛΣ είναι παρόμοιες. Υπάρχει ένα ελάχιστο σύνολο συναρτήσεων και υπηρεσιών που θεωρούνται απαραίτητες σε ένα πολυδιεργασιακό περιβάλλον. Μερικές υπηρεσίες που θεωρούνται βασικές σε ένα ΛΣ μπορούν να υλοποιούνται ως συνδυασμός άλλων σε διαφορετικό ΛΣ.

Όταν πρόκειται να προστεθεί μια καινούρια διεργασία, σε αυτές που ήδη υπάρχουν, το ΛΣ δημιουργεί τις δομές δεδομένων που χρησιμοποιούνται για τη διαχείρισή της και εκχωρεί στην νέα διεργασία χώρο στην κύρια μνήμη. Οι βασικές αιτίες για τη δημιουργία διεργασίας είναι η απόκριση στην υποβολή μιας εργασίας, η προσπάθεια σύνδεσης ενός νέου χρήστη, η απαίτηση για παροχή κάποιας υπηρεσίας από μια εφαρμογή και η άμεση παραγωγή από υπάρχουσα διεργασία.

Στο ΛΣ Unix η κλήση συστήματος για τη δημιουργία διεργασίας ονομάζεται `fork()`. Η κλήση συστήματος `fork` δημιουργεί μια θυγατρική διεργασία που είναι κλώνος της διεργασίας – γονέα. Η θυγατρική διεργασία διατηρεί ένα εικονικό αντίγραφο της εικονικής μνήμης του γονέα, εκτελεί το ίδιο πρόγραμμα όπως και ο γονέας και ξεκινά με τις τιμές καταχωρητών ίδιες με αυτές του γονέα. Η θυγατρική διεργασία είναι πιθανόν να εκτελεί στο πλαίσió της ένα διαφορετικό πρόγραμμα, με μια ξεχωριστή κλήση συστήματος `exec()`.

Παράδειγμα Unix Fork/Exec/Exit/Wait



int pid = fork();

Δημιουργεί μια νέα διεργασία που είναι κλώνος της γονικής διεργασίας.

exec("program");

Επικάλυψη της εικονικής μνήμης της καλούμενης διεργασίας με ένα νέο πρόγραμμα και μεταφορά του ελέγχου σε αυτό.

exit(status);

Έξοδος με καταστροφή της διεργασίας - κλώνου.

int pid = wait(&status);

Αναμένει για έξοδο (ή άλλη αλλαγή κατάστασης) του απογόνου.

Ερωτήσεις Επανάληψης

1. Τι είναι μια διεργασία και από τι αποτελείται; .
2. Σχεδιάστε και περιγράψτε το μοντέλο διεργασίας των 5 καταστάσεων;
3. Εξηγήστε τι συμβαίνει όταν έχουμε διεργασίες σε αναστολή.
4. Σχεδιάστε τα μοντέλα διεργασίας με μία και δύο καταστάσεις αναστολής;
5. Πότε γίνεται η αλλαγή της εκτελούμενης διεργασίας;

Ασκήσεις

1. Διεργασία (process) είναι:
 - α) ένα πρόγραμμα.
 - β) ένας επεξεργαστής.
 - γ) ένα ειδικό αρχείο.
 - δ) ένα πρόγραμμα σε εκτέλεση.
2. Στην περιοχή στοίβας μιας διεργασίας:
 - α) περιέχεται ο κώδικας και οι μεταβλητές
 - β) αποθηκεύονται οι τοπικές μεταβλητές των συναρτήσεων
 - γ) αποθηκεύονται οι μεταβλητές και η δυναμικά παραχωρούμενη μνήμη
 - δ) περιέχεται ο κώδικας που εκτελεί ο επεξεργαστής
3. Μια διεργασία είναι σε κατάσταση «έτοιμη»
 - α) όταν έχει διακοπεί προσωρινά για να εκτελεστεί κάποια άλλη
 - β) όταν χρησιμοποιεί τον επεξεργαστή εκείνη τη στιγμή.
 - γ) όταν δεν εκτελείται μέχρι να συμβεί κατάλληλο εξωτερικό συμβάν
 - δ) όταν έχει μόλις δημιουργηθεί από το λειτουργικό σύστημα
4. Κατά την μετάβαση της κατάστασης εκτέλεσης από «εκτελούμενη» σε «έτοιμη»
 - α) μια επιπρόσθετη διεργασία εισάγεται στο σύστημα
 - β) η διεργασία έχει φθάσει στο μέγιστο επιτρεπτό όριο εκτέλεσής της
 - γ) η διεργασία απαιτεί μια υπηρεσία από το ΛΣ που δεν θα εκτελεστεί άμεσα
 - δ) η διεργασία αναμένει την απελευθέρωση μνήμης
5. Οι διεργασίες μπορούν να βρεθούν σε κατάσταση αναστολής επειδή
 - α) ο επεξεργαστής είναι πολύ πιο αργός από τις συσκευές εισόδου/εξόδου
 - β) ο σκληρός δίσκος δεν επιτρέπει τη δημιουργία νέων αρχείων
 - γ) ο επεξεργαστής είναι πολύ πιο γρήγορος από τις συσκευές εισόδου/εξόδου
 - δ) μία νέα διεργασία εισάγεται στο σύστημα
6. Η λίστα έτοιμων διεργασιών περιλαμβάνει:
 - α) τις διεργασίες που περιμένουν για είσοδο/έξοδο.
 - β) τις διεργασίες που περιμένουν σε κάποιο μηχανισμό συγχρονισμού
 - γ) τις διεργασίες που περιμένουν επιπλέον μνήμη.
 - δ) τις διεργασίες που περιμένουν για εξυπηρέτηση από τον επεξεργαστή
7. Το μπλοκ ελέγχου διεργασίας
 - α) είναι ένας ακέραιος αριθμός που χαρακτηρίζει μια διεργασία
 - β) μπορεί να τροποποιηθεί από έναν χρήστη
 - γ) αποτελεί συστατικό της εικόνας μιας διεργασίας

δ) δημιουργείται μόνο για διεργασίας με ειδικά δικαιώματα

8. Ο μακροπρόθεσμος δρομολογητής

- α) επιλέγει ποια διεργασία θα είναι η επόμενη προς εκτέλεση
- β) επιλέγει τις διεργασίες που θα περιληφθούν στην ουρά των έτοιμων προς εκτέλεση διεργασιών
- γ) διαχειρίζεται τη μνήμη του υπολογιστικού συστήματος
- δ) μετακινεί περιοδικά διεργασίες από τη μνήμη

9. Ο δείκτης στοίβας αποτελεί πεδίο του μπλοκ ελέγχου διεργασίας που σχετίζεται με

- α) τη διαχείριση μνήμης
- β) τη διαχείριση αρχείων
- γ) τη διαχείριση διεργασιών
- δ) κανένα από τα παραπάνω

10. Η κλήση συστήματος fork

- α) επιστρέφει το αναγνωριστικό μιας διεργασίας
- β) δημιουργεί ένα νέο αρχείο
- γ) δημιουργεί ένα αντίγραφο της τρέχουσας διεργασίας
- δ) αναμένει τον τερματισμό μιας διεργασίας

Κεφάλαιο 4 – Αρχιτεκτονικές ΛΣ

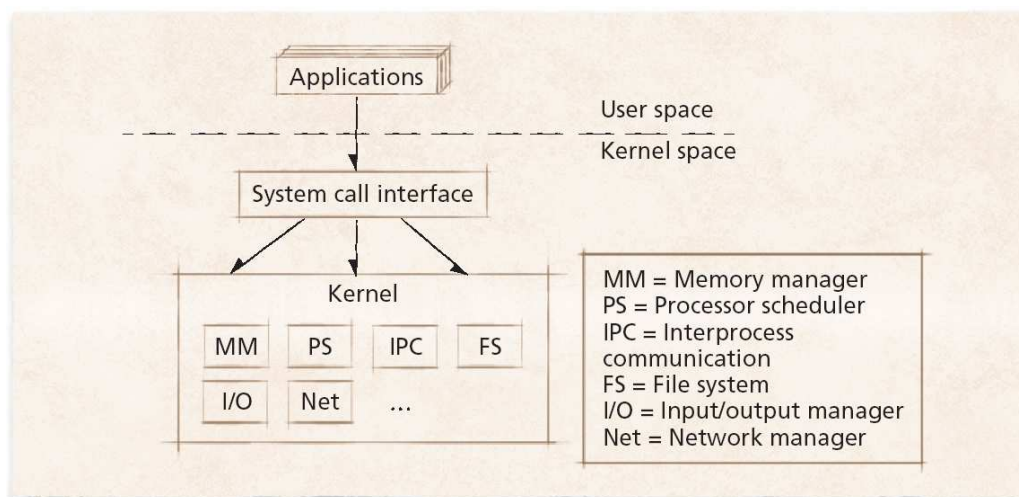
1. Μονολιθικά συστήματα
2. Στρωματοποιημένη αρχιτεκτονική
3. Αρχιτεκτονική μικροπυρήνα
4. Νήματα (threads) - Πολυνημάτωση (multithreading)
5. Συστήματα πολυεπεξεργασίας
6. Παράλληλα συστήματα
7. Συστήματα πραγματικού χρόνου
8. Κατανεμημένα συστήματα

4.1. Μονολιθικά συστήματα

Η απλοϊκότερη αρχιτεκτονική λειτουργικών συστημάτων περιλαμβάνει τα μονολιθικά συστήματα, τα οποία αποτελούν μια προσέγγιση χωρίς καμιά δομή. Το ΛΣ είναι μια συλλογή από διαδικασίες, κάθε μια από τις οποίες μπορεί να καλέσει οποιαδήποτε άλλη, όταν τη χρειαστεί. Επιπλέον, η επικοινωνία μεταξύ των διαδικασιών γίνεται με παραμέτρους ενώ κάθε διαδικασία είναι ορατή σε οποιαδήποτε άλλη.

Κατά την εκτέλεση μιας κλήσης συστήματος σε μονολιθικά συστήματα, το πρόγραμμα του χρήστη δημιουργεί μια παγίδα στον πυρήνα εκτελώντας μια ειδική εντολή ή κλήση πυρήνα – kernel call. Το ΛΣ προσδιορίζει τον αριθμό εξυπηρέτησης που ζητήθηκε και στη συνέχεια εντοπίζει και καλεί τη διαδικασία εξυπηρέτησης. Μετά την εκτέλεση, ο έλεγχος επιστρέφεται στο πρόγραμμα του χρήστη.

Σύμφωνα με τη μονολιθική οργάνωση, η βασική δομή για το λειτουργικό σύστημα περιλαμβάνει ένα κύριο πρόγραμμα που ζητά την ενεργοποίηση μιας διαδικασίας εξυπηρέτησης, ένα σύνολο διαδικασιών εξυπηρέτησης, οι οποίες υλοποιούν τις κλήσεις συστήματος και ένα σύνολο βοηθητικών προγραμμάτων, τα οποία υποβοηθούν τις διαδικασίες εξυπηρέτησης.



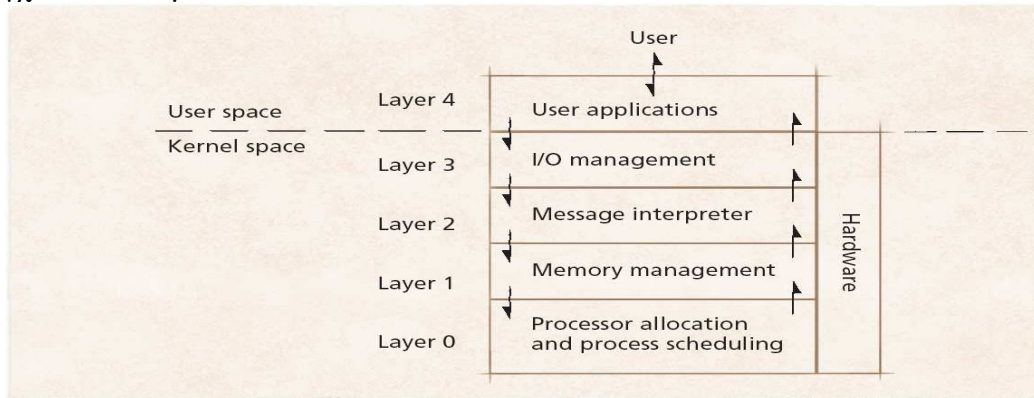
Σχήμα 4.1 Αρχιτεκτονική πυρήνα μονολιθικού ΛΣ

Τα κύρια συστατικά ενός τυπικού πυρήνα (kernel) στα μονολιθικά συστήματα είναι ο δρομολογητής διεργασιών (process scheduler), ο διαχειριστής μνήμης (memory manager), ο διαχειριστής συσκευών E/E (I/O manager), ο διαχειριστής διαδιεργασιακής

επικοινωνίας (interprocessor communication (IPC) manager) και ο διαχειριστής συστήματος αρχείων (file system manager).

4. 2. Στρωματοποιημένη αρχιτεκτονική

Η στρωματοποιημένη αρχιτεκτονική προσπαθεί να βελτιώσει το σχεδιασμό των μονολιθικών πυρήνων ομαδοποιώντας συστατικά που υλοποιούν παρόμοιες λειτουργίες σε επίπεδα. Κάθε επίπεδο επικοινωνεί μόνον με τα γειτονικά του (επάνω και κάτω), ενώ οι απαιτήσεις των διεργασιών διαπερνούν αρκετά επίπεδα πριν ολοκληρωθούν. Η ρυθμοαπόδοση (throughput) μπορεί να είναι μικρότερη από τα ΛΣ με τους μονολιθικούς πυρήνες και απαιτούνται επιπλέον μέθοδοι για τη μεταβίβαση και τον έλεγχο των δεδομένων.

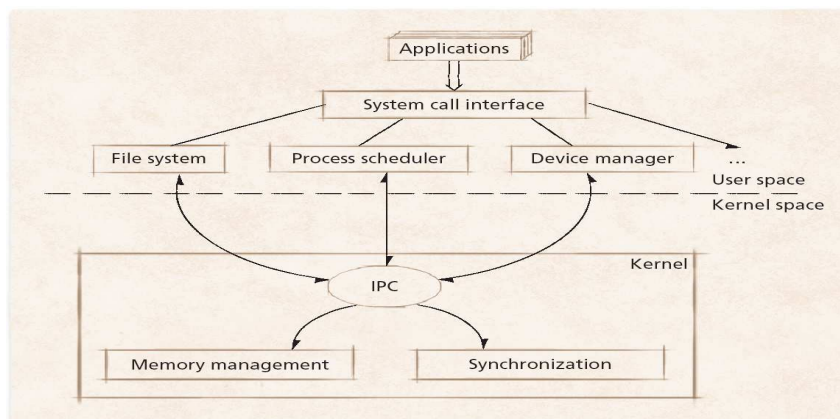


Σχήμα 4.2 Πολλαπλά επίπεδα

4. 3. Αρχιτεκτονική μικροπυρήνα

Ένας μονολιθικός πυρήνας περιλαμβάνει τη δρομολόγηση, το σύστημα αρχείων, τη δικτύωση, τους οδηγούς συσκευής, τη διαχείριση μνήμης κ.α. Υλοποιείται ως μια μοναδική διεργασία και όλα τα στοιχεία διαμοιράζονται τον ίδιο χώρο διευθύνσεων.

Αντίθετα, η αρχιτεκτονική μικροπυρήνα αναθέτει λίγες λειτουργίες στον πυρήνα και τις υπόλοιπες τις αναθέτει σε εξυπηρετητές που εκτελούνται σε κατάσταση χρήστη. Η διεργασία του χρήστη (client process) στέλνει την απαίτηση στη διεργασία εξυπηρετητή (server process) η οποία επιτελεί τη διεργασία και επιστρέφει την απάντηση. Ο μικροπυρήνας διαχειρίζεται την επικοινωνία μεταξύ clients και servers.



Σχήμα 4.3 Αρχιτεκτονική μικροπυρήνα

Το σχήμα της τοποθέτησης λογισμικού πάνω από τον πυρήνα ώστε να χειρίζεται τη διαδικασία client-server δεν είναι απόλυτα ρεαλιστικό. Κάποιες λειτουργίες του χρήστη είναι δύσκολο, ακόμη και ακατόρθωτο να πραγματοποιηθούν από το χώρο προγραμμάτων του χρήστη. Μια λύση του παραπάνω προβλήματος είναι οι κρίσιμες διεργασίες του εξυπηρετητή (drivers I/O) να τρέχουν σε κατάσταση πυρήνα με πλήρη πρόσβαση στο υλικό αλλά να συνεχίσουν να επικοινωνούν με τις άλλες διεργασίες. Εναλλακτικά, μπορεί να εμφυτευθεί ένας στοιχειώδης μηχανισμός στον πυρήνα αλλά η πολιτική αποφάσεων να παραμείνει στους εξυπηρετητές.

Τα πλεονεκτήματα της αρχιτεκτονικής μικροπυρήνα συνοψίζονται στα ακόλουθα:

- **Επεκτασιμότητα:** η αρχιτεκτονική επιτρέπει την προσθήκη και αφαίρεση υπηρεσιών και χαρακτηριστικών.
- **Μεταφερσιμότητα:** οι αλλαγές που απαιτούνται για τη μεταφορά του συστήματος σε νέο επεξεργαστή γίνονται στον μικροπυρήνα και όχι στις άλλες υπηρεσίες.
- **Αντικειμενοστραφής σχεδιασμός:** τα συστατικά του ΛΣ είναι αντικείμενα με σαφώς καθορισμένες διεπαφές που μπορούν να διασυνδεθούν.
- **Αξιοπιστία:** οφείλεται στο αρθρωτό σχεδιασμό του λειτουργικού συστήματος και στο ότι ο μικρού μεγέθους μικροπυρήνας μπορεί να ελεγχθεί με ακρίβεια.

4.4. Νήματα - Πολυνημάτωση (multithreading)

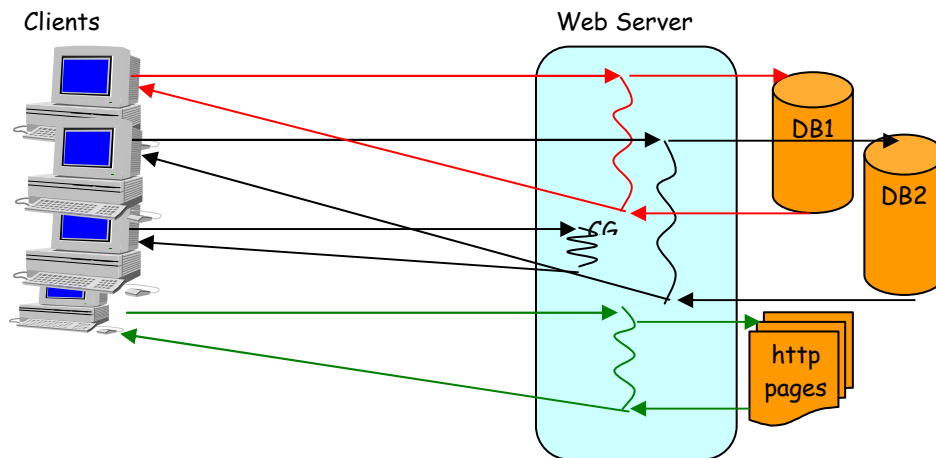
Ένα νήμα (thread) είναι μια απλή ροή δεδομένων διαμέσου του επεξεργαστή του συστήματος. Κάθε εφαρμογή παράγει το δικό της ή τα δικά της νήματα εξαρτώμενα από τον τρόπο που αυτή εκτελείται. Στα πολυδιεργασιακά συστήματα ένας επεξεργαστής μπορεί να διαχειριστεί ένα νήμα κάθε στιγμή, έτσι το σύστημα εναλλάσσεται ταχύτατα μεταξύ των νημάτων για να επεξεργαστεί τα δεδομένα δίνοντας την αίσθηση της συνεξέλιξης των νημάτων. Από την πλευρά του λειτουργικού συστήματος, ένα νήμα είναι μια μονάδα εργασίας που διεκπεραιώνεται και μπορεί να διακόπτεται, και περιλαμβάνει μετρητή προγράμματος, δείκτη στοίβας και τη δική της περιοχή δεδομένων.

Ένα νήμα εκτελείται σειριακά και είναι διακοπτόμενο, ώστε ο επεξεργαστής να μπορεί να επιστρέφει σε άλλο νήμα. Ένα πρόγραμμα μπορεί να περιέχει αρκετά στοιχεία που διαμοιράζονται δεδομένα και μπορούν να εκτελούνται συγχρόνως. Για παράδειγμα, ένας Web browser μπορεί να περιέχει διαφορετικά συστατικά για την ανάγνωση ιστοσελίδων σε μορφή HTML, την ανάκτηση των συστατικών τους (εικόνες, video κλπ) και την εμφάνιση των σελίδων στο παράθυρο του browser. Αυτά τα συστατικά του προγράμματος που εκτελούνται ανεξάρτητα αλλά υλοποιούνται ως λειτουργίες σε μια κοινή περιοχή μνήμης ονομάζονται νήματα (threads).

Μια διεργασία είναι μια συλλογή νημάτων μαζί με συσχετιζόμενους πόρους του συστήματος. Η διάσπαση μιας εφαρμογής σε πολλά νήματα έχει ως αποτέλεσμα το χρονισμό των γεγονότων της εφαρμογής. Τα νήματα προσφέρουν έναν εναλλακτικό τρόπο λειτουργίας σε σχέση με το μοντέλο της διεργασίας. Υπάρχει δυσκολία και πολυπλοκότητα όταν χρησιμοποιείται το μοντέλο διεργασίας, που επιπλέον δεν είναι αρκετά αποτελεσματικό. Η πολυνημάτωση (multi-threading) είναι χρήσιμη σε εφαρμογές που εκτελούν πολλές ανεξάρτητες μεταξύ τους διεργασίες που δεν χρειάζεται να είναι σε σειρά, π.χ. ένας web server (Σχήμα 4.4).

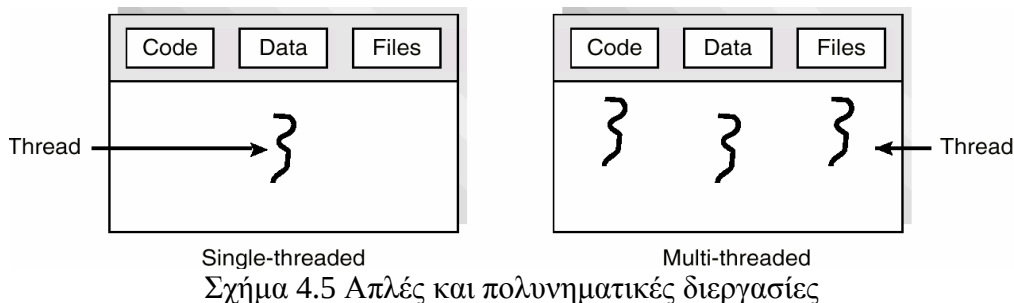
Το μοντέλο νήματος διαιρεί τη διεργασία σε δύο τμήματα, τη διεργασία και το νήμα. Το τμήμα διεργασίας περιέχει τους πόρους που χρησιμοποιούνται σε όλη τη διάρκεια του προγράμματος, όπως τις καθολικές (global) εντολές και καθολικά

δεδομένα. Το τμήμα νήματος περιέχει πληροφορίες που σχετίζονται με την κατάσταση εκτέλεσης, όπως μετρητή προγράμματος και στοίβα.



Σχήμα 4.4 Πολυνηματικός web server

Είναι φανερό ότι οι διεργασίες μπορούν να διακριτού σε απλές και πολυνηματικές. Οι διεργασίες είναι οντότητες που δεσμεύουν οποιονδήποτε πόρο τις αφορά όπως τον χώρο διευθύνσεων, τα δεδομένα, τον κώδικα και τα αρχεία Έχουν μεγάλο κόστος σε σχέση με τη δημιουργία, τον τερματισμό και την εναλλαγή πλαισίου. Τα νήματα διαμοιράζονται τον χώρο διευθύνσεων, τα δεδομένα, τον κώδικα και τα αρχεία εφόσον ανήκουν στην ίδια διεργασία Επίσης, έχουν μικρό κόστος σε σχέση με τη δημιουργία, τον τερματισμό και την εναλλαγή πλαισίου.



Single-threaded

Multi-threaded

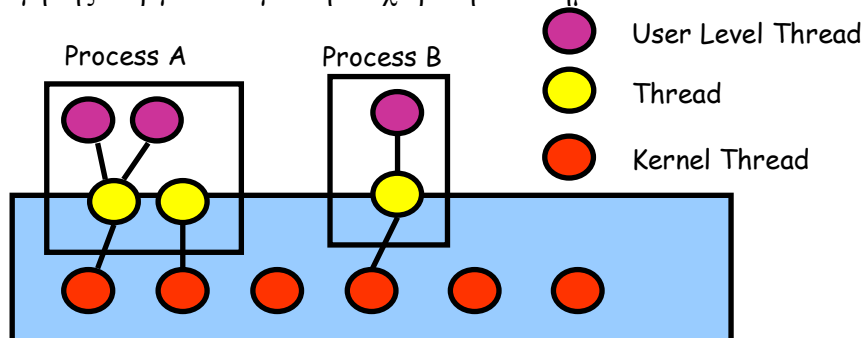
Σχήμα 4.5 Απλές και πολυνηματικές διεργασίες

Το νήμα αποτελούν τη βασική μονάδα χρήσης του επεξεργαστή, με κάθε νήμα να κατέχει μετρητή προγράμματος, ένα σύνολο καταχωρητών και χώρο στοίβας, και να διαμοιράζεται με τα υπόλοιπα νήματα της διεργασίας τα τμήματα κώδικα, δεδομένων και τους πόρους του λειτουργικού συστήματος.

Ανάλογα με τον τρόπο υλοποίησής τους, τα νήματα διαχωρίζονται σε δύο μοντέλα: νήματα χρήστη και νήματα πυρήνα. Στα νήματα επιπέδου χρήστη, η διαχείριση των νημάτων γίνεται από τη βιβλιοθήκη των νημάτων χρήστη ενώ ο πυρήνας δεν ενδιαφέρεται για το πλήθος των νημάτων χρήστη. Αντίθετα τα νήματα πυρήνα παρέχονται από το λειτουργικό σύστημα και υποστηρίζονται από τον πυρήνα.

Αναλυτικότερα, τα νήματα χρήστη είναι άγνωστα στο ΛΣ, το οποίο είναι ενημερωμένο μόνο για τη διεργασία όπου περιέχονται. Το λειτουργικό σύστημα δρομολογεί τις διεργασίες και όχι τα νήματα ενώ ο προγραμματιστής χρησιμοποιεί τη βιβλιοθήκη νημάτων για να τα διαχειριστεί (δημιουργία, διαγραφή, συγχρονισμός και δρομολόγηση). Αντίθετα, τα νήματα επιπέδου πυρήνα είναι γνωστά στο ΛΣ, η

εναλλαγή μεταξύ νημάτων πυρήνα στην ίδια διεργασία είναι αρκετά δαπανηρή σε σχέση με τα νήματα επιπέδου χρήστη ενώ οι τιμές των καταχωρητών, του μετρητή προγράμματος και των δεικτών στοίβας μεταβάλλονται. Οι πληροφορίες που αφορούν τη διαχείριση μνήμης δεν αλλάζουν ενώ ο πυρήνας χρησιμοποιεί αλγορίθμους δρομολόγησης διεργασιών για τη διαχείριση των νημάτων.



Σχήμα 4.7. Νήματα επιπέδου χρήστη και νήματα επιπέδου πυρήνα

Σε κάθε περίπτωση, βασικό πλεονέκτημα των νημάτων είναι η ταχύτητα που παρέχουν σε σύγκριση με τις διεργασίες. Η δημιουργία μιας νέας διεργασίας είναι περισσότερο δαπανηρή, λόγω του χρόνου που απαιτείται για την κλήση στον πυρήνα του ΛΣ ενώ μπορεί να προκαλέσει επαναδρομολόγηση οπότε θα προκύψει εναλλαγή πλαισίου με αντικατάσταση ολόκληρης της διεργασίας. Αντίστοιχα, η επικοινωνία και ο συγχρονισμός κοστίζουν διότι απαιτείται κλήση στον πυρήνα. Τα νήματα μπορούν να δημιουργηθούν χωρίς να αντικατασταθεί ολόκληρη η διεργασία. Το περισσότερο έργο για τη δημιουργία του νήματος γίνεται στο χώρο διευθύνσεων του χρήστη παρά στον πυρήνα του ΛΣ. Τα νήματα συγχρονίζονται με την παρακολούθηση μιας μεταβλητής, σε αντίθεση με τις διεργασίες που απαιτούν κλήση πυρήνα.

Ένα νήμα δεν διαθέτει δικό του τμήμα δεδομένων και σωρό, δεν μπορεί να υπάρξει μόνο του αλλά πρέπει να υπάρχει στα πλαίσια μιας διεργασίας. Υπάρχουν περισσότερα από ένα νήματα σε μια διεργασία, το πρώτο νήμα είναι το κύριο και κατέχει τη στοίβα της διεργασίας. Η δημιουργία ενός νήματος καθώς και η εναλλαγή πλαισίου δεν είναι δαπανηρή, ενώ αν ένα νήμα εκλείψει η στοίβα του επιστρέφεται στους πόρους του συστήματος. Αντίθετα, μια διεργασία έχει τμήμα κώδικα, δεδομένων και σωρό, όπως και άλλα τμήματα, κάθε διεργασία έχει ένα τουλάχιστον νήμα, και τα νήματα της διαμοιράζονται τα τμήματα κώδικα και δεδομένων, το σωρό και την είσοδο-έξοδο αλλά το καθένα έχει τη δική του στοίβα και τους δικούς του καταχωρητές. Η δημιουργία των διεργασιών καθώς και οι εναλλαγές πλαισίου είναι δαπανηρές, ενώ αν μια διεργασία εκλείψει οι πόροι της επιστρέφονται στο σύστημα και όλα της τα νήματα εκλείπουν.

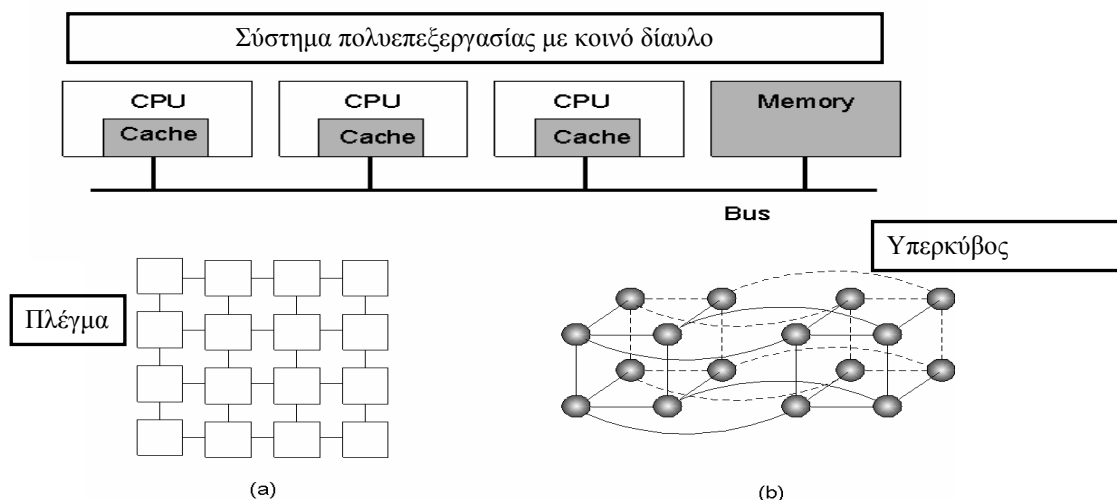
Παράδειγμα νημάτων αποτελούν τα Posix Threads του πρότυπο IEEE, τα οποία έχουν καθορισμένη διεπαφή, η υλοποίησή τους εξαρτάται από όσους τα αναπτύσσουν και αποτελούν τα πιο συνηθισμένα σε όλα τα σύγχρονα λειτουργικά συστήματα UNIX, ενώ είναι διαθέσιμα και στα Windows. Άλλο παράδειγμα νημάτων είναι τα Java threads, τα οποία υποστηρίζονται από την JVM και των οποίων η υλοποίηση επίσης εξαρτάται από όσους τα αναπτύσσουν.

Μια πρόσφατη εξέλιξη στην περιοχή των νημάτων είναι η τεχνολογία Hyper-threading που υλοποιείται από την Intel. Η τεχνολογία hyper-threading που μοιάζει με την πολυνημάτωση, υλοποιείται σε συστήματα με έναν πυρήνα και τα κάνει να εμφανίζονται σαν να διαθέτουν πολλαπλούς επεξεργαστές. Εκείνο το οποίο επιτυγχάνεται είναι η αύξηση του ρυθμού με τον οποίο το σύστημα μπορεί να εναλλάσσεται μεταξύ πολλών νημάτων. Έτσι ενισχύεται ο πολυδιεργασιακός χαρακτήρας των προσωπικών υπολογιστών.

4.5. Συστήματα πολυεπεξεργασίας

Πολυεπεξεργασία (multiprocessing) είναι η χρήση πολλαπλών ταυτόχρονων διεργασιών σε ένα σύστημα. Τα συστήματα πολυεπεξεργασίας διακρίνονται στις εξής δύο κατηγορίες:

- Συμπαγώς συνδεδεμένα συστήματα: περιέχουν πολλαπλές CPUs που συνδέονται σε επίπεδο διαύλου. Έχουν πρόσβαση σε μια κεντρική διαμοιραζόμενη μνήμη ή μπορούν να συμμετέχουν σε μια ιεραρχία μνήμης διαθέτοντας και τοπική και διαμοιραζόμενη μνήμη
- Χαλαρά συνδεδεμένα συστήματα: κάθε επεξεργαστής έχει τη δική του τοπική μνήμη. Οι επεξεργαστές επικοινωνούν μεταξύ τους μέσω γραμμών επικοινωνίας όπως δίαυλοι υψηλής ταχύτητας (gigabit Ethernet) ή τηλεφωνικές γραμμές



Σχήμα 4.8 Αρχιτεκτονικές συστημάτων πολυεπεξεργασίας

4.6. Παράλληλα συστήματα

Τα παράλληλα συστήματα είναι συστήματα πολυεπεξεργασίας (multiprocessor systems) με περισσότερες από μία CPU σε επικοινωνία μεταξύ τους. Ο όρος των παράλληλων συστημάτων καλύπτει μια πληθώρα αρχιτεκτονικών που περιλαμβάνουν τη συμμετρική πολυεπεξεργασία (SMP), τις συστοιχίες συστημάτων SMP και τα μαζικά παράλληλα συστήματα (MPP). Κριτήριο διάκρισης αποτελεί το είδος της διασύνδεσης των επεξεργαστών, που είναι γνωστοί ως επεξεργαστικά στοιχεία (processing elements), καθώς και το είδος διασύνδεσης μεταξύ επεξεργαστών και μνημών.

Η ταξινόμηση Flynn κατηγοριοποιεί τα συστήματα ανάλογα με το αν όλοι οι επεξεργαστές εκτελούν τις ίδιες εντολές την ίδια χρονική στιγμή (single instruction / multiple data – SIMD) ή αν κάθε επεξεργαστής εκτελεί διαφορετικές εντολές (multiple instruction / multiple data – MIMD). Οι μηχανές παράλληλης επεξεργασίας διακρίνονται επίσης σε συμμετρικά και μη συμμετρικά συστήματα πολυεπεξεργασίας. Αυτό καθορίζεται από το αν όλοι οι επεξεργαστές είναι σε θέση να εκτελούν ολόκληρο τον κώδικα του ΛΣ και να έχουν προσπέλαση στις συσκευές I/O ή αν κάποιοι επεξεργαστές έχουν περισσότερα ή λιγότερα προνόμια.

Τα πλεονεκτήματα παράλληλων συστημάτων είναι ο αυξημένος βαθμός χρήσης της και κατά συνέπεια οι υψηλές επιδόσεις τους, η οικονομία, η αυξημένη αξιοπιστία τους, η διαθεσιμότητα, η επεκτασιμότητα και η κλιμάκωσή τους.

Η συμμετρική πολυεπεξεργασία (SMP) αποτελεί την πλέον συνήθη προσέγγιση για τη δημιουργία ενός πολυεπεξεργαστικού συστήματος, στο οποίο δύο ή περισσότεροι επεξεργαστές εργάζονται μαζί στο ίδιο motherboard. Οι επεξεργαστές συντονίζονται και διαμοιράζονται πληροφορίες διαμέσου του διαύλου συστήματος καθώς επίσης ρυθμίζουν μεταξύ τους το υπολογιστικό φορτίο. Κάθε επεξεργαστής τρέχει ένα πανομοιότυπο αντίγραφο του ΛΣ, ενώ πολλές διεργασίες μπορούν να εκτελούνται ταυτόχρονα χωρίς να υπάρχει χειροτέρευση της απόδοσης. Τα περισσότερα σύγχρονα ΛΣ υποστηρίζουν την συμμετρική πολυεπεξεργασία ενώ οι βασικοί περιορισμοί των SMP σχετίζονται με το λογισμικό και την υποστήριξη εκ μέρους του ΛΣ.

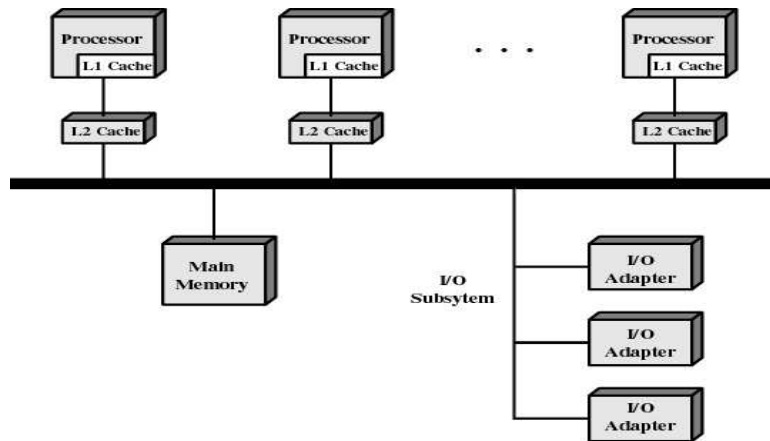


Figure 4.9 Symmetric Multiprocessor Organization

Σχήμα 4.9 Αρχιτεκτονική συμμετρικής πολυεπεξεργασίας

Σε αντίθεση με την προηγούμενη περίπτωση, στην ασύμμετρη πολυεπεξεργασία κάθε επεξεργαστής ανατίθεται σε μια ορισμένη διεργασία ενώ ο πρωτεύων επεξεργαστής δρομολογεί και αναθέτει τις διεργασίες στους άλλους (slave) επεξεργαστές. Η συγκεκριμένη προσέγγιση είναι πιο συνηθισμένη σε πολύ μεγάλα συστήματα.

4.7. Συστήματα πραγματικού χρόνου

Τα συστήματα πραγματικού χρόνου συχνά χρησιμοποιούνται ως μια συσκευή ελέγχου σε μια συγκεκριμένη εφαρμογή όπως ο έλεγχος επιστημονικών πειραμάτων, ο έλεγχος βιομηχανικών συστημάτων, σε συστήματα επεξεργασίας εικόνας ιατρικών εφαρμογών κλπ. Διαθέτουν καλά σχεδιασμένους περιορισμούς χρόνου και διακρίνονται σε δύο κατηγορίες:

- Hard real-time systems: χαρακτηρίζονται από την περιορισμένη χρήση δευτερεύουσας μνήμης και τα δεδομένα αποθηκεύονται σε μνήμες βραχείας διάρκειας ή σε ROM.
- Soft real-time systems: παρέχουν περιορισμένη χρησιμότητα σε βιομηχανικό έλεγχο και σε ρομποτική αλλά είναι ιδιαίτερα χρήσιμα σε εφαρμογές (multimedia, virtual reality) που απαιτούν εξειδικευμένα χαρακτηριστικά ΛΣ.

4.8. Κατανεμημένα συστήματα

Τα κατανεμημένα συστήματα διαμοιράζουν τη διαδικασία υπολογισμών σε πολλούς φυσικούς επεξεργαστές και παρέχουν την ψευδαίσθηση ενός μοναδικού χώρου για την κύρια μνήμη και ενός ξεχωριστού μοναδικού χώρου για τη δευτερεύουσα μνήμη, μαζί με άλλες ευκολίες όπως ένα κατανεμημένο σύστημα αρχείων. Ένα κατανεμημένο

σύστημα αφορά στην ουσία ένα πολύ-υπολογιστικό σύστημα δηλ. μια συλλογή από οντότητες (H/Y), που η κάθε μια έχει τη δική της κύρια, δευτερεύουσα μνήμη και άλλα στοιχεία I/O.

Τα κατανεμημένα συστήματα αποκρύπτουν από τους χρήστες και τα πρόγραμμα τον τρόπο πρόσβασης σε έναν πόρο και το χώρο όπου βρίσκεται κάποιος πόρος, αναλαμβάνουν τη διαμοίραση πόρων από πολλούς χρήστες που ανταγωνίζονται για τη χρήση τους, επιτρέπουν τη μετακίνηση ενός πόρου σε άλλο μέρος ενώ είναι σε χρήση και αποκρύβουν τις διαφορές στην αναπαράσταση δεδομένων.

Τα πλεονεκτήματα των κατανεμημένων συστημάτων είναι η διαμοίραση πόρων, η αύξηση της ταχύτητας υπολογισμού, η υψηλή αξιοπιστία τους και οι δυνατότητες επικοινωνίας που παρέχουν. Τα κυρία μειονεκτήματά τους είναι τα πρόβλημα ασφάλειας και προστασίας που αντιμετωπίζουν.

Τα μοντέλα σχεδίασης των κατανεμημένων συστημάτων είναι τα εξής:

- Μοντέλο μικροϋπολογιστή κάθε χρήστης έχει τη δική του μηχανή (τοπικά) και η επεξεργασία είναι τοπική αλλά μπορούν να προσκομιστούν δεδομένα (αρχεία, βάσεις δεδομένων).
- Μοντέλο σταθμών εργασία: η επεξεργασία μπορεί να γίνεται και απομακρυσμένα.
- Μοντέλο δικτυακού πελάτη-εξυπηρετητή: ο χρήστης έχει το δικό του τοπικό σταθμό εργασίας ενώ ισχυροί υπολογιστές λειτουργούν ως εξυπηρετητές (π.χ. αρχείων, εκτύπωσης, βάσεων δεδομένων).
- Μοντέλο δεξαμενής επεξεργαστών: υπάρχουν τερματικά γραφικών ή τερματικά χωρίς δίσκο ενώ μια δεξαμενή επεξεργαστών διεκπεραιώνει την επεξεργασία

Ερωτήσεις Επανάληψης

1. Τι είναι η αρχιτεκτονική μικροπυρήνα και ποια τα πλεονεκτήματά της;
2. Τι είναι τα νήματα, ποια τα βασικά χαρακτηριστικά και ποια τα πλεονεκτήματα των νημάτων;
3. Σε τι διαφέρουν τα νήματα επιπέδου χρήστη από τα νήματα επιπέδου πυρήνα;
4. Περιγράψτε τα πλεονεκτήματα των νημάτων
5. Τι είναι η συμμετρική πολυεπεξεργασία;

Ασκήσεις

1. Σύμφωνα με τη μονολιθική οργάνωση, η βασική δομή για το λειτουργικό σύστημα δεν περιλαμβάνει
 - α) ένα κύριο πρόγραμμα που ζητά την ενεργοποίηση μιας διαδικασίας εξυπηρέτησης
 - β) ένα σύνολο επιπέδων που υλοποιούν παρόμοιες λειτουργίες
 - γ) ένα σύνολο διαδικασιών εξυπηρέτησης
 - δ) ένα σύνολο βοηθητικών προγραμμάτων
2. Η στρωματοποιημένη αρχιτεκτονική:
 - α) παρέχει πάντα χαμηλότερη ρυθμοαπόδοση από τους μονολιθικούς πυρήνες
 - β) απαιτεί μεθόδους για τη μεταβίβαση και τον έλεγχο των δεδομένων
 - γ) εξυπηρετεί άμεσα τις απαιτήσεις των διεργασιών
 - δ) επιτρέπει την επικοινωνία μεταξύ οποιωνδήποτε επιπέδων

3. Σύμφωνα με την αρχιτεκτονική μικροπυρήνα
- α) χρησιμοποιείται μια μοναδική διεργασία
 - β) χρησιμοποιεί εξυπηρετητές που εκτελούνται σε κατάσταση πυρήνα
 - γ) αναθέτει όλες τις λειτουργίες στον πυρήνα
 - δ) χρησιμοποιεί εξυπηρετητές που εκτελούνται σε κατάσταση χρήστη
4. Ο μικροπυρήνας
- α) υλοποιεί το σύστημα αρχείων
 - β) είναι υπεύθυνος για τη δρομολόγηση των διεργασιών
 - γ) διαχειρίζεται την επικοινωνία μεταξύ προγραμμάτων και εξυπηρετητών
 - δ) διαχειρίζεται τη μνήμη του συστήματος
5. Δεν αποτελεί βασικό πλεονέκτημα της αρχιτεκτονικής μικροπυρήνα
- α) ανταλλαγή μηνυμάτων
 - β) επεκτασιμότητα
 - γ) μεταφερσιμότητα
 - δ) αξιοπιστία
6. Η μεταφερσιμότητα της αρχιτεκτονικής μικροπυρήνα
- α) ορίζει σαφώς καθορισμένες διεπαφές
 - β) διευκολύνει την υλοποίηση του συστήματος σε νέο επεξεργαστή
 - γ) επιτρέπει την προσθήκη και αφαίρεση υπηρεσιών και χαρακτηριστικών
 - δ) παρέχει ευκολία ελέγχου
7. Ένα νήμα
- α) είναι μια διεργασία
 - β) αποτελεί οντότητα άγνωστη στο λειτουργικό σύστημα
 - γ) διαθέτει μετρητή προγράμματος και δείκτη στοίβας
 - δ) δεν διακόπτεται ποτέ όταν εκτελείται στον επεξεργαστή
8. Τα νήματα πυρήνα:
- α) δρομολογούνται από τον πυρήνα
 - β) είναι άγνωστα στο λειτουργικό σύστημα
 - γ) είναι ταχύτερα από τα νήματα επιπέδου χρήστη
 - δ) επικοινωνούν με μηνύματα
9. Τα νήματα:
- α) έχουν μεγαλύτερες καθυστερήσεις από τις διεργασίες
 - β) απαιτούν κλήσεις στον πυρήνα του λειτουργικού συστήματος
 - γ) έχουν δικό τους χώρο δεδομένων και σωρό
 - δ) συγχρονίζονται με την παρακολούθηση μιας μεταβλητής
10. Τα παράλληλα συστήματα
- α) αυξάνουν τη διαθέσιμη μνήμη του συστήματος
 - β) εκτελούν ένα πρόγραμμα κάθε φορά
 - γ) παρέχουν αυξημένη αξιοπιστία
 - δ) απαιτούν ειδικά λειτουργικά συστήματα

Κεφάλαιο 5 – Αμοιβαίος Αποκλεισμός

1. Εισαγωγή
2. Κρίσιμα τμήματα (Critical Sections)
3. Υλοποίηση του αμοιβαίου αποκλεισμού
 1. Προσεγγίσεις λογισμικού
 2. Υποστήριξη εκ μέρους του υλικού
 3. Σημαφόροι

5.1. Εισαγωγή

Ο αμοιβαίος αποκλεισμός είναι ο αποκλεισμός μιας διεργασίας από μια ενέργεια που επιτελεί ταυτοχρόνως κάποια άλλη διεργασία. Απαιτεί μόνο μία διεργασία να πραγματοποιεί λειτουργίες σε κοινούς πόρους. Αν δύο διεργασίες προσπαθήσουν να αποκτήσουν ένα κοινό πόρο, τότε η μία θα πρέπει να αναμένει. Για την αποφυγή των παρενεργειών λόγω ανταγωνισμού είναι ζωτικής σημασίας η πρόβλεψη και η αποτροπή χρήσης διαμοιραζόμενων πόρων από περισσότερες από μια διεργασίες την ίδια χρονική στιγμή. Είναι απαραίτητος για την προστασία των κρίσιμων τμημάτων των διεργασιών.

Κατάθεση 100 €	
Ανάγνωση τρέχοντος υπολοίπου = 500 €	Απαίτηση ανάληψης = 100 €
	Ανάγνωση τρέχοντος υπολοίπου = 500 €
	Νέο υπόλοιπο = 500 € – 100 € = 400 €
Νέο υπόλοιπο = 500 € + 100 € = 600 €	
Εγγραφή νέου υπολοίπου στο δίσκο = 600 €	
	Εγγραφή νέου υπολοίπου στο δίσκο = 400 €

Πίνακας 5.1 Παράδειγμα αναγκαιότητας αμοιβαίου αποκλεισμού

Κατάθεση 100 €	
Απαίτηση για κλείδωμα του λογαριασμού	Απαίτηση ανάληψης = 100 €
Ανάγνωση τρέχοντος υπολοίπου = 500 €	Απαίτηση για κλείδωμα του λογαριασμού
	Αναμονή
Νέο υπόλοιπο = 500 € + 100 € = 600 €	
Εγγραφή νέου υπολοίπου στο δίσκο = 600 €	
Απελευθέρωση λογαριασμού	
	Δέσμευση λογαριασμού
	Ανάγνωση τρέχοντος υπολοίπου = 600 €
	Νέο υπόλοιπο = 600 € – 100 € = 500 €
	Εγγραφή νέου υπολοίπου στο δίσκο = 400 €
	Απελευθέρωση λογαριασμού

Πίνακας 5.2 Αντιμετώπιση προβλήματος με χρήση αμοιβαίου αποκλεισμού

5.2. Κρίσιμα τμήματα (Critical Sections)

Μια ακολουθία εντολών μιας διεργασίας που πρέπει να εκτελείται αδιαίρετα, ονομάζεται κρίσιμο τμήμα. Η εκτέλεση του κρίσιμου τμήματος θα πρέπει να είναι “όλα ή τίποτα”, που σημαίνει ότι δεν θα πρέπει να διακόπτεται στο μέσον. Η αποτελεσματικότητα της πολυεπεξεργασίας εξαρτάται από το μήκος του κρίσιμου τμήματος κάθε ανταγωνιστικής διεργασίας και πρέπει να είναι όσο το δυνατόν

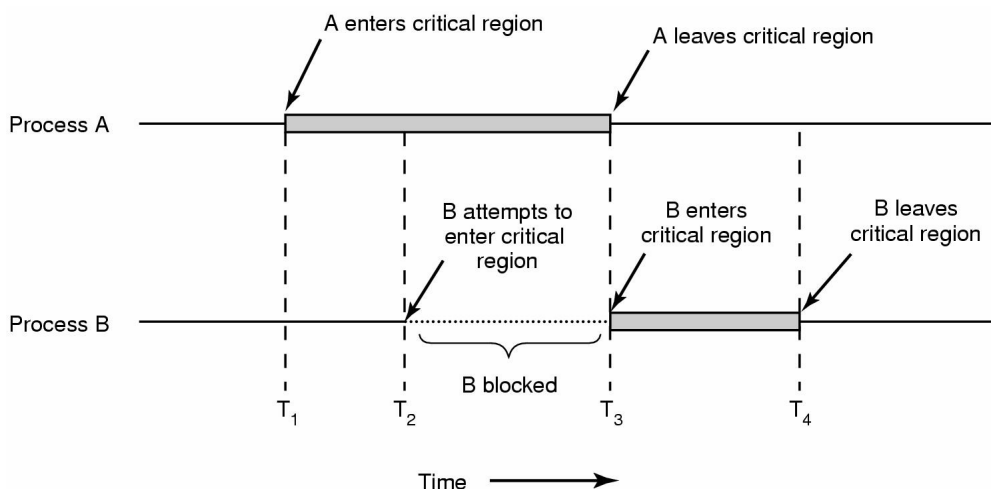
μικρότερο. Σύμφωνα με το πλέον αναποτελεσματικό σενάριο, όλο το πρόγραμμα είναι ένα κρίσιμο τμήμα με συνέπεια να μην υπάρχει πολυπρογραμματισμός.

Όταν μια διεργασία εκτελεί το κρίσιμο τμήμα της (KT), σε καμιά άλλη δεν μπορεί να επιτραπεί να εκτελεί το δικό της κρίσιμο τμήμα. Αν μπορούσε να διασφαλισθεί ότι ποτέ δύο ή περισσότερες διεργασίες δεν θα βρίσκονται ταυτόχρονα σε κρίσιμα τμήματα, τότε θα είχε επιτευχθεί η αποφυγή των συνθηκών ανταγωνισμού. Ωστόσο αυτό δεν είναι αρκετό για τη σωστή και αποδοτική συνεργασία δύο ή περισσότερων παράλληλων διεργασιών.

Μια καλή λύση για κρίσιμα τμήματα προϋποθέτει τις παρακάτω συνθήκες:

- Κάθε διεργασία μπορεί να διαιρεθεί σε δύο ακολουθίες (τμήματα ή περιοχές εντολών): το κρίσιμο τμήμα, δηλ. αυτό που προσπελάζει τον κοινό πόρο και δεν πρέπει να καταμεριστεί και το μη κρίσιμο τμήμα.
- Δυο διεργασίες δεν μπορούν να βρίσκονται ταυτόχρονα στα κρίσιμα τμήματά τους.
- Μια διεργασία μπορεί να σταματήσει μόνον μέσα στο μη κρίσιμο τμήμα της, χωρίς να επηρεάσει (να αναστείλει) άλλες διεργασίες.
- Δεν επιτρέπονται υποθέσεις σε ότι αφορά την ταχύτητα ή το πλήθος των επεξεργαστών.
- Όταν μια διεργασία δεν εκτελεί το κρίσιμο τμήμα της δεν μπορεί να αναστείλει την εκτέλεση άλλης διεργασίας.
- Το πρόγραμμα δεν επιτρέπεται να καταλήξει σε αδιέξοδο. Αν περισσότερες της μιας διεργασίες προσπαθήσουν να εισέλθουν ταυτόχρονα στις κρίσιμες περιοχές τους, τότε μόνο μια πρέπει να επιτύχει.
- Δεν επιτρέπεται η επ' αόριστον αναμονή μιας διεργασίας, μέχρι να εισέλθει στο κρίσιμο τμήμα της.
- Στην περίπτωση έλλειψης συναγωνισμού για ταυτόχρονη είσοδο σε κρίσιμες περιοχές, μια διεργασία που επιθυμεί να εισέλθει στην κρίσιμη περιοχή της θα πρέπει να το επιτύχει με την ελάχιστη δυνατή επιβάρυνση.
- Μια διεργασία παραμένει μέσα στο κρίσιμο τμήμα της για ορισμένο χρονικό διάστημα.

Οι κίνδυνοι που προκύπτουν στην ομαλή λειτουργία του συστήματος λόγω του αμοιβαίου αποκλεισμού οφείλονται σε λανθασμένες ή ελλιπείς λύσεις που μπορούν να οδηγήσουν σε παρατεταμένη στέρηση (starvation), αδιέξοδο (deadlock) και ανεπάρκεια του συστήματος.



Σχήμα 5.1 Αμοιβαίος αποκλεισμός με κρίσιμα τμήματα

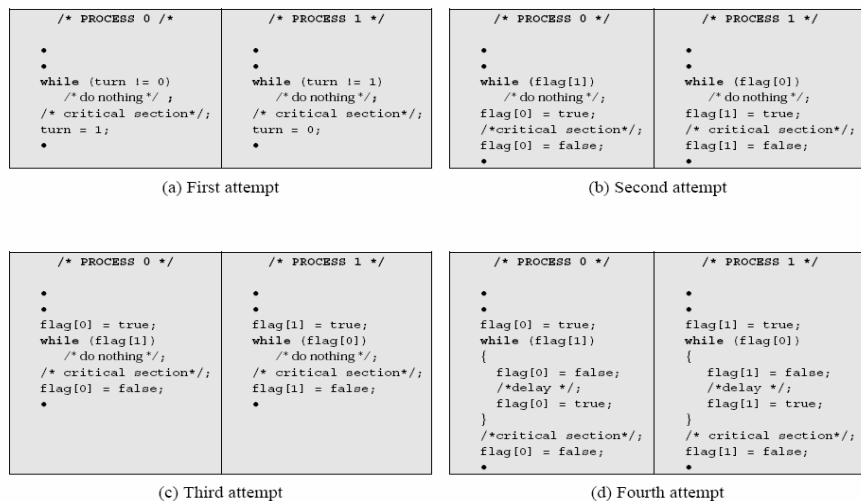
5.3. Υλοποίηση του αμοιβαίου αποκλεισμού

Για την υλοποίηση αμοιβαίου αποκλεισμού, υπάρχουν διάφορες προσεγγίσεις λογισμικού κατά τις οποίες γίνεται παραχώρηση της ευθύνης στις διεργασίες χωρίς υποστήριξη από τη γλώσσα προγραμματισμού ή το ΛΣ (αλγόριθμοι Dekker και Peterson). Άλλες προσεγγίσεις είναι η υποστήριξη εκ μέρους του υλικού με χρήση εντολών μηχανής ειδικού σκοπού (ατομικές λειτουργίες test και set) και η παροχή κάποιου επιπέδου υποστήριξης από το ΛΣ ή τη γλώσσα προγραμματισμού, όπως είναι οι σημαφόροι ή σηματοφόρες (semaphores), παρακολουθητές (monitors) και η μεταβίβαση μηνυμάτων. Όλες οι παραπάνω λύσεις περιλαμβάνουν συνήθως κρίσιμα τμήματα.

5.3.1. Προσεγγίσεις λογισμικού

Οι λύσεις σε επίπεδο λογισμικού υλοποιούνται για ταυτόχρονες διεργασίες που εκτελούνται σε έναν ή πολλαπλούς επεξεργαστές με διαμοιραζόμενη κύρια μνήμη. Οι ταυτόχρονες προσβάσεις (ανάγνωσης ή εγγραφής) στην ίδια περιοχή της μνήμης διατάσσονται σειριακά, με κάποιο τρόπο διαχείρισης. Η παραχώρηση της πρόσβασης δεν είναι εκ των προτέρων καθορισμένη. Δεν υπάρχει υποστήριξη σε επίπεδο υλικού, λειτουργικού συστήματος ή γλώσσας προγραμματισμού.

Χαρακτηριστικό παράδειγμα προσέγγισης λογισμικού είναι ο αλγόριθμος του Dekker. Σύμφωνα με τον αλγόριθμο, κάθε προσπάθεια για αμοιβαίο αποκλεισμό πρέπει να στηρίζεται σε βασικούς μηχανισμούς απαγορεύσεων σε επίπεδο υλικού. Κατά τον πλέον συνήθη μηχανισμό, σε μια περιοχή (θέση) μνήμης επιτρέπεται μια πρόσβαση κάθε φορά. Για την υλοποίηση του αλγορίθμου Dekker υπάρχουν διάφορες προσπάθειες αυξανόμενης δυσκολίας, χωρίς να είναι όλες ορθές όσον αφορά την ανίχνευση των λαθών.



Σχήμα 5.2. Οι 4 προσπάθειες του αλγορίθμου του Dekker

Η πρώτη προσπάθεια υλοποίησης του αλγορίθμου Dekker Υλοποιείται ο αμοιβαίος αποκλεισμός αλλά καταναλώνει χρόνο του επεξεργαστή (ενεργός αναμονή) και οδηγεί σε αυστηρή εναλλαγή των διεργασιών κατά την εκτέλεση των κρίσιμων τμημάτων τους.

var turn: 0..1; /* shared */	
<u>Διεργασία 0:</u> while (turn != 0) do nothing; <critical section> turn = 1;	<u>Διεργασία 1:</u> while (turn != 1) do nothing; <critical section> turn = 0;

Σχήμα 5.3 Πρώτη προσπάθεια υλοποίησης του αλγορίθμου Dekker

Σύμφωνα με τη δεύτερη προσπάθεια υλοποίησης του αλγορίθμου, μια διεργασία μπορεί να μεταβάλλει την κατάστασή της αφού έχει ελεγχθεί από την άλλη, αλλά πριν η άλλη διεργασία εισέλθει στο κρίσιμο τμήμα της. Στην περίπτωση αυτή είναι δυνατόν και οι δύο διεργασίες να βρεθούν στο κρίσιμο τμήμα τους.

var flag: array[0..1] of Boolean; /* initialize both to false */ /* shared */	
<u>Διεργασία 0:</u> while (flag[1] != 0) do nothing; flag[0] = true <critical section> flag[0] = false;	<u>Διεργασία 1:</u> while (flag[0] != 1) do nothing; flag[1] = false;; <critical section> flag[1] = false;

Σχήμα 5.4 Δεύτερη προσπάθεια υλοποίησης του αλγορίθμου Dekker

5.3.2. Υποστήριξη εκ μέρους του υλικού

Τα βασικά προβλήματα με τις προσεγγίσεις λογισμικού είναι η ενεργός αναμονή (busy waiting), που είναι ο επαναλαμβανόμενος έλεγχος μιας μεταβλητής με αποτέλεσμα να δαπανάται χρόνος της CPU και πρέπει να αποφεύγεται, καθώς και οι πολυπλοκότητα των αλγορίθμων. Επίσης, οι αλγόριθμοι δουλεύουν μόνον με 2 διεργασίες και μπορούν να επεκταθούν και σε περισσότερες διεργασίες αλλά το πλήθος τους πρέπει να είναι εξ αρχής γνωστό και επιπλέον οι αλγόριθμοι γίνονται ακόμη πιο σύνθετοι. Γενικά, η εφαρμογή ενός μηχανισμού αμοιβαίου αποκλεισμού είναι δύσκολη.

Το υλικό (hardware) μπορεί να βοηθήσει στην περίπτωση του αμοιβαίου αποκλεισμού με σκοπό να κάνει τη λύση περισσότερο αποτελεσματική και να μπορεί να εφαρμοστεί σε περισσότερες από δύο διεργασίες. Το υλικό που απαιτείται για να υποστηρίξει κρίσιμα τμήματα πρέπει κατ' ελάχιστο να έχει αδιαίρετες εντολές και ατομικές (atomic) εντολές, π.χ. load, store, test. Αν δύο ατομικές εντολές εκτελεστούν ταυτόχρονα, πρέπει να συμπεριφερθούν σαν να εκτελούνται σειριακά. Οι σύγχρονοι μικροεπεξεργαστές διαθέτουν εντολές υλικού που υποστηρίζουν τον αμοιβαίο αποκλεισμό.

5.3.2.1 Απενεργοποίηση Διακοπών

Οι διακοπές απενεργοποιούνται κατά τη χρονική περίοδο που απαιτείται ο αμοιβαίος αποκλεισμός., ενώ χωρίς διακοπές δεν μπορεί να συμβεί εναλλαγή διεργασιών. Η λύση

αυτή είναι επικίνδυνη αφού μια διεργασία μπορεί να απασχολήσει αποκλειστικά όλο το σύστημα και για αυτό χρησιμοποιείται σε συστήματα ειδικής χρήσης με περιορισμένο hardware.

```
while (true)
{
    /* disable interrupts */;
    /* critical section */;
    /* enable interrupts */;
    /* remainder */;
}
```

Στα μειονεκτήματα της απενεργοποίησης διακοπών περιλαμβάνεται το υψηλό κόστος της και η μείωση της αποτελεσματικότητας της εκτέλεσης διότι περιορίζεται η δυνατότητα του επεξεργαστή να εναλλάσσει προγράμματα. Είναι μια χρήσιμη τεχνική για τον πυρήνα, αλλά δεν ενδείκνυται ως γενικός μηχανισμός αμοιβαίου αποκλεισμού για τις διεργασίες χρήστη. Επιπλέον, σε συστήματα πολλών επεξεργαστών δεν εγγυάται ο αμοιβαίος αποκλεισμός, αφού η απενεργοποίηση διακοπών επηρεάζει μόνον την υπεύθυνη CPU ενώ οι υπόλοιπες επιτρέπουν τη μεταβολή των περιεχομένων της κοινής περιοχής μνήμης.

5.3.2.2 Ειδικές Εντολές Μηχανής

Πολλές CPU παρέχουν εντολές υλικού για την ανάγνωση, τροποποίηση και την εγγραφή μιας θέσης μνήμης ατομικά. Οι πιο κοινές εντολές με αυτή τη δυνατότητα είναι οι TAS (Test-And-Set) και XCHG (exchange). Η βασική ιδέα είναι η δυνατότητα ανάγνωσης του περιεχομένου μιας θέσης μνήμης και η μεταβολή του, όλα σε έναν μη διακοπτόμενο κύκλο εντολής. Οι λειτουργίες αυτές πραγματοποιούνται σε ένα κύκλο μηχανής και επομένως δεν υπόκεινται σε παρεμβολές από άλλες εντολές.

Πλεονέκτημα της τεχνικής ειδικών εντολών μηχανής για αμοιβαίο αποκλεισμό είναι η δυνατότητα εφαρμογής της σε οποιοδήποτε αριθμό διεργασιών. Επίσης μπορεί να χρησιμοποιηθεί με έναν ή πολλούς επεξεργαστές που διαμοιράζονται μια κοινή μνήμη, είναι απλή και εύκολη στην επαλήθευση και μπορεί να χρησιμοποιηθεί για να υποστηρίξει πολλαπλά κρίσιμα τμήματα, π.χ. ορίζοντας μια ξεχωριστή μεταβλητή για κάθε κρίσιμο τμήμα.

Μειονέκτημα της τεχνικής είναι η απασχόληση ενεργούς αναμονής (busy waiting), καθώς μια διεργασία που αναμένει για πρόσβαση στο κρίσιμο τμήμα συνεχίζει να καταναλώνει χρόνο του επεξεργαστή. Επίσης είναι πιθανή η παρατεταμένη στέρηση (starvation), επειδή η επιλογή για είσοδο σε μια διεργασία είναι αυθαίρετη όταν πολλαπλές διεργασίες συναγωνίζονται για να εισέλθουν στο κρίσιμο τμήμα. Τέλος υπάρχει η πιθανότητα αδιεξόδου: αν μια διεργασία χαμηλής προτεραιότητας P1 εκτελεί την ειδική εντολή μηχανής, εισέρχεται στο κρίσιμο τμήμα της και μια μεγαλύτερης προτεραιότητας διεργασία P2 πάρει τον έλεγχο του επεξεργαστή, η P2 θα περιμένει να αποκτήσει τον έλεγχο του πόρου, εισερχόμενη σε βρόγχο ενεργού αναμονής.

Οι διατυπώσεις αμοιβαίου αποκλεισμού που παρουσιάστηκαν βασίζονται στην ενεργό αναμονή, η οποία όμως δεν είναι επιθυμητή. Αν υποθέσουμε ότι τρέχουν δύο διεργασίες, έστω **Y**, υψηλής προτεραιότητας, και **X**, χαμηλής προτεραιότητας, ο δρομολογητής εκτελεί πάντοτε την **Y**, κάθε φορά που αυτή είναι σε κατάσταση ετοιμότητας. Αν σε μια συγκεκριμένη στιγμή η **X** είναι στο κρίσιμο τμήμα της και η **Y** γίνει εκτελέσιμη, τότε η διεργασία **Y** ξεκινά την ενεργό αναμονή περιμένοντας να

εισέλθει στο κρίσιμο τμήμα της. Κατά συνέπεια, η **X** δεν δρομολογείται ποτέ να εγκαταλείψει το κρίσιμο τμήμα της με αποτέλεσμα να δημιουργείται αδιέξοδος.

Εναλλακτικά στην ενεργό αναμονή, που είναι αναποτελεσματική και επιρρεπής σε αδιέξοδα, μπορεί να χρησιμοποιηθεί η προσέγγιση sleep/wakeup. Η προσέγγιση αυτή εφαρμόζεται από τους σηματοφόρους ή σηματοφορείς (semaphores), γίνεται δηλαδή προσέγγιση χρησιμοποιώντας μηχανισμούς του ΛΣ και των γλωσσών προγραμματισμού.

5.3.2.3 Σηματοφόροι (semaphores)

Σύμφωνα με το Dijkstra, η σχεδίαση του ΛΣ αποτελεί μια συλλογή συνεργαζόμενων σειριακών διεργασιών μαζί με έναν αποτελεσματικό μηχανισμό για την υποστήριξη της συνεργασίας. Οι μηχανισμοί αυτοί χρησιμοποιούνται και από διεργασίες χρηστών, αρκεί να τους παρέχει ο επεξεργαστής και το ΛΣ. Η βασική αρχή: δύο ή περισσότερες διεργασίες μπορούν να συνεργαστούν με τη χρήση απλών σημάτων. Κάθε διεργασία επιβάλλεται να σταματήσει σε μια συγκεκριμένη θέση, μέχρι να παραλάβει ένα συγκεκριμένο σήμα. Για τη δημιουργία σημάτων χρησιμοποιούνται ειδικές μεταβλητές, οι σηματοφόροι (ή σηματοφορείς).

Ένας σηματοφόρος είναι μια μεταβλητή συγχρονισμού που λαμβάνει ακέραιες θετικές τιμές ή μηδέν και μπορεί να αρχικοποιηθεί με μια μη-αρνητική τιμή. Οι σηματοφόροι ανακαλύφθηκαν από τον Dijkstra στα μέσα της δεκαετίας του 1960 για να επιλύουν προβλήματα συγχρονισμού μεταξύ διεργασιών. Οι σηματοφόροι είναι απλοί και κομψοί και επιλύουν πολλά ενδιαφέροντα προβλήματα και όχι μόνον τον αμοιβαίο αποκλεισμό.

Βασικές (πρωταρχικές) λειτουργίες σηματοφόρων είναι οι **P** ή **wait**, και η **V** ή **signal**. Η **P** (semaphore) ατομική λειτουργία που αναμένει για τον σηματοφόρο να γίνει θετικός, έπειτα τον μειώνει κατά 1, ενώ η **V** (semaphore) είναι μια ατομική λειτουργία που αυξάνει τον σηματοφόρο κατά 1. Οι πρωταρχικές αυτές λειτουργίες θεωρούνται ατομικές.

Υπάρχουν δύο είδη σηματοφόρων: οι δυαδικοί σηματοφόροι (binary semaphores), οι οποίοι παίρνουν τιμές 0 και 1 και χρησιμοποιούνται για να επιτευχθεί ο αμοιβαίος αποκλεισμός, και οι μετρητές σηματοφόροι (counting semaphores) που μπορούν να λάβουν οποιαδήποτε μη αρνητική τιμή και χρησιμοποιούνται για τη διαχείριση των περιορισμένων πόρων των συστημάτων.

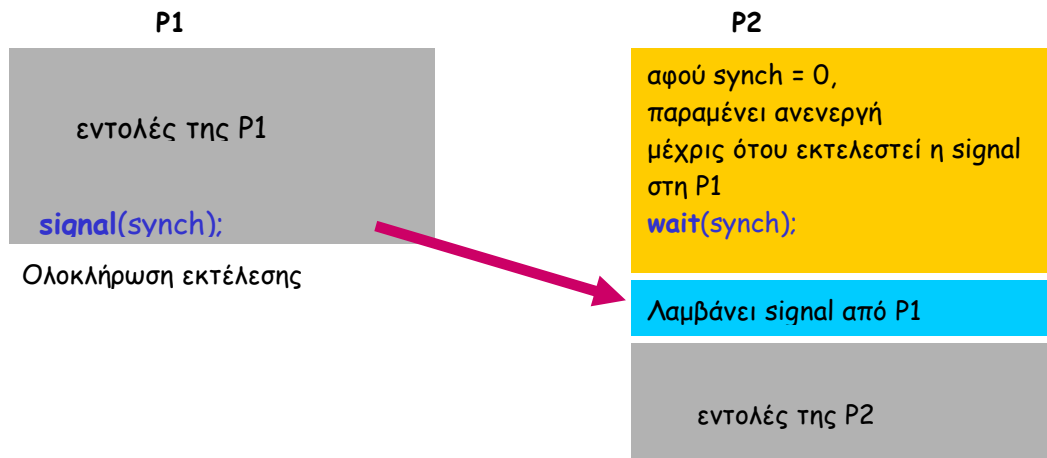
Οι σηματοφόροι είναι ανεξάρτητοι του υλικού, είναι απλοί, λειτουργούν με πολλές διεργασίες και επιτρέπουν την ύπαρξη πολλών κρίσιμων τμημάτων στον κώδικα, το καθένα με τον δικό του σηματοφόρο. Οι σηματοφόροι μπορούν να αποκτούν πολλούς πόρους ταυτόχρονα και ο καθένας έχει μια ακέραια τιμή και μια λίστα από συσχετιζόμενες διεργασίες τις οποίες εξυπηρετεί.

Σύμφωνα με την εσωτερική λειτουργία σηματοφόρων, όταν μια διεργασία μπλοκάρεται από μόνη της σε ένα σηματοφόρο, τότε προστίθεται σε μια ουρά που περιλαμβάνει τις διεργασίες που αναμένουν. Η λειτουργία signal σε ένα σηματοφόρο αφαιρεί μια διεργασία από την ουρά και την αφυπνίζει. Η πιο δίκαιη πολιτική διαχείρισης της ουράς είναι η FIFO, το οποίο ισχύει για την περίπτωση των ισχυρών σηματοφόρων. Αντίθετα, στην περίπτωση ασθενών σηματοφόρων, δεν καθορίζεται η σειρά με την οποία εξέρχονται οι διεργασίες από την ουρά. Οι ισχυροί σηματοφόροι εγγυώνται την απαλλαγή από την παρατεταμένη στέρηση. Είναι το είδος των σηματοφόρων που παρέχουν τα ΛΣ

Παράδειγμα 1

Έστω οι διεργασίες P1 και P2. Στόχος μας είναι η διεργασία P2 να εκτελεστεί υποχρεωτικά (δηλ. να εξαναγκαστεί) μετά από την P1. Χρησιμοποιείται ένας κοινός

σημαφόρος με όνομα `synch` για να συγχρονίσει τις λειτουργίες των δύο σύγχρονων διεργασιών. Οι εντολές `wait`, `signal` χρησιμοποιούνται για να καθυστερήσουν την P2 μέχρι να ολοκληρωθεί η P1. Ο σημαφόρος `synch` αρχικοποιείται με τιμή 0



Παράδειγμα 2

Οι σημαφόροι μπορούν να χρησιμοποιηθούν ως εργαλείο συγχρονισμού. Στόχος είναι η εκτέλεση του τμήματος B στη διεργασία P2 μετά από την εκτέλεση του τμήματος A στη διεργασία P1. Χρησιμοποιείται ένας σημαφόρος `flag` με αρχική τιμή 0.

Διεργασία P1	Διεργασία P2
A	<code>wait(flag);</code>
<code>signal(flag);</code>	B

Παράδειγμα 3

Οι σημαφόροι πρέπει να χρησιμοποιούνται με προσοχή αφού μπορούν να οδηγήσουν σε αδιέξοδο. Σύμφωνα με τον παρακάτω κώδικα, η διεργασία P1 περιμένει την P2 να εκτελέσει `signal(Q)`. Ταυτόχρονα, η διεργασία P2 περιμένει την P1 να εκτελέσει `signal(S)`. Επομένως και οι δύο διεργασίες βρίσκονται σε αδιέξοδο.

Διεργασία P1	Διεργασία P2
<code>wait(S);</code>	<code>wait(Q);</code>
<code>wait(Q);</code>	<code>wait(S);</code>

Παράδειγμα 4

Τρεις διεργασίες διαμοιράζονται έναν πόρο στον οποίο, μία σχεδιάζει ένα A, μία δεύτερη ένα B και τέλος μία τρίτη σχεδιάζει ένα C. Ζητούμενο είναι να υλοποιηθεί μια μορφή συγχρονισμού έτσι ώστε να εμφανίζονται κατά σειρά τα γράμματα A B C.

Χωρίς κάποια μορφή συγχρονισμού τα τρία γράμματα μπορούν να εμφανιστούν με οποιαδήποτε αυθαίρετη σειρά. Ο κώδικας στην περίπτωση αυτή θα έχει την ακόλουθη μορφή:

Διεργασία P1	Διεργασία P2	Διεργασία P3
<code>Compute();</code>	<code>Compute();</code>	<code>Compute();</code>
<code>Write(A);</code>	<code>Write(B);</code>	<code>Write(C);</code>

Στην περίπτωση αξιοποίησης κάποιου μηχανισμού συγχρονισμού, χρησιμοποιούμε δύο δυαδικούς σημαφόρους (`b`, `c`) που έχουν αρχικοποιηθεί στην τιμή 0, και παρέχουν την επιθυμητή λύση στο συγκεκριμένο πρόβλημα.

Semaphore b = 0, c = 0;		
<u>Διεργασία P1</u>	<u>Διεργασία P2</u>	<u>Διεργασία P3</u>
Compute();	Wait(b);	Wait(c);
Write(A);	Compute();	Compute();
Signal(b);	Write(B);	Write(C);
	Signal(c);	

Ερωτήσεις Επανάληψης

1. Τι είναι κρίσιμα τμήματα διεργασιών;
2. Ποιες είναι οι συνθήκες για μια καλή λύση για κρίσιμα τμήματα;
3. Αναφέρετε και περιγράψτε συνοπτικά τις προσεγγίσεις αμοιβαίου αποκλεισμού.
4. Τι είναι οι σημαφόροι και ποιες οι βασικές λειτουργίες τους;
5. Τι είναι η ενεργός αναμονή και ποιες οι ιδιότητές της;

Ασκήσεις

1. Κρίσιμα τμήματα (critical sections) διεργασιών (processes):
 - α) απαγορεύεται να εκτελούνται ταυτόχρονα από πολλές διεργασίες, χωρίς να αποκλείεται κάποιες να εκτελούν ταυτόχρονα μη κρίσιμα τμήματά τους
 - β) απαγορεύεται να εκτελούνται ταυτόχρονα από πολλές διεργασίες και αποκλείεται κάποιες να εκτελούν ταυτόχρονα μη κρίσιμα τμήματά τους
 - γ) επιτρέπεται να εκτελούνται ταυτόχρονα από πολλές διεργασίες
 - δ) όταν εκτελούνται τερματίζονται οι αντίστοιχες διεργασίες
2. Ποιο από τα παρακάτω αποτελεί συνθήκη για λύση κρίσιμων τμημάτων:
 - α) μια διεργασία μπορεί να σταματήσει μέσα στο κρίσιμο τμήμα της
 - β) δυο διεργασίες δεν μπορούν να είναι ταυτόχρονα στο ίδιο κρίσιμο τμήμα
 - γ) επιτρέπονται υποθέσεις σε ότι αφορά την ταχύτητα του επεξεργαστή
 - δ) επιτρέπεται η επ'αόριστον αναμονή μιας διεργασίας ώστε να εισέλθει στο κρίσιμο τμήμα της.
3. Ποιο από τα παρακάτω δεν αποτελεί συνθήκη για λύση κρίσιμων τμημάτων:
 - α) μια διεργασία παραμένει μέσα στο κρίσιμο τμήμα της για ορισμένο χρονικό διάστημα
 - β) η επιβάρυνση εισαγωγής στο κρίσιμο τμήμα είναι πάντοτε υψηλή
 - γ) δυο διεργασίες δεν μπορούν να είναι ταυτόχρονα στο ίδιο κρίσιμο τμήμα
 - δ) κάθε διεργασία αποτελεί ένα κρίσιμο τμήμα
4. Ποια από τις παρακάτω προσεγγίσεις αμοιβαίου αποκλεισμού δεν απαιτεί υποστήριξη εκ μέρους του υλικού
 - α) απενεργοποίηση διακοπών
 - β) ειδικές εντολές μηχανής
 - γ) σημαφόροι
 - δ) έλεγχος μεταβλητής σε κοινή μνήμη
5. Η ενεργός αναμονή
 - α) αποτελεί μειονέκτημα των προσεγγίσεων λογισμικού
 - β) αξιοποιεί καλύτερα τον επεξεργαστή του συστήματος

- γ) παρέχει την ταχύτερη λύση αμοιβαίου αποκλεισμού
- δ) αναστέλλει την εκτέλεση των διεργασιών

6. Στην περίπτωση του αμοιβαίου αποκλεισμού, το υλικό
- α) βοηθάει στην περίπτωση εκτέλεσης δύο μόνο διεργασιών
 - β) δεν παρέχει πιο αποτελεσματικές λύσεις σε σχέση με το λογισμικό
 - γ) αναστέλλει ταχύτερα την εκτέλεση των διεργασιών
 - δ) πρέπει να έχει ατομικές εντολές

7. Η τεχνική ειδικών εντολών μηχανής για αμοιβαίο αποκλεισμό
- α) είναι δύσκολη στην επαλήθευση της
 - γ) δεν οδηγεί ποτέ σε παρατεταμένη στέρηση
 - γ) εφαρμόζεται σε οποιοδήποτε αριθμό διεργασιών
 - δ) είναι περίπλοκη στην εφαρμογή της

8. Οι σημαφόροι (semaphores) χρησιμεύουν για να:
- α) γίνεται πιο εύκολα η μετάφραση προγραμμάτων
 - β) συγχρονίζονται οι διεργασίες
 - γ) επιταχύνεται η εκτέλεση προγραμμάτων
 - δ) αντικαταστήσουν πολύπλοκες δομές δεδομένων

9. Έστω το παρακάτω σενάριο χρήσης σημαφόρων από δύο διεργασίες, με τα A,B,C,D να αποτελούν ανεξάρτητα τμήματα κώδικα.

<u>Διεργασία P1</u>	<u>Διεργασία P2</u>
wait(S);	wait(Q);
wait(Q);	wait(S);
A;	C;
B;	D;
signal(Q);	signal(S);
signal(S);	signal(Q);

Ποια από τα παρακάτω δε μπορεί να συμβεί;

- α) Σειρά εκτέλεσης: A, B, C, D
- β) Σειρά εκτέλεσης: A, C, D, B
- η) Σειρά εκτέλεσης: C, D, A, B
- δ) Τίποτε από τα παραπάνω, αφού θα συμβεί αδιέξοδος.

10. Ένας υπολογιστής έχει 3 επαναχρησιμοποιούμενους πόρους A, B, C. Τρεις διεργασίες X, Y, Z εκτελούνται στον υπολογιστή. Η διεργασία X αποκτά τον A και ζητά τον B. Η διεργασία Y αποκτά τον B και ζητά τον C. Η διεργασία Z αποκτά τον C και ζητά τον A. Μόλις μια διεργασία πάρει και τους δύο πόρους, τους χρησιμοποιεί και μετά τους απελευθερώνει.

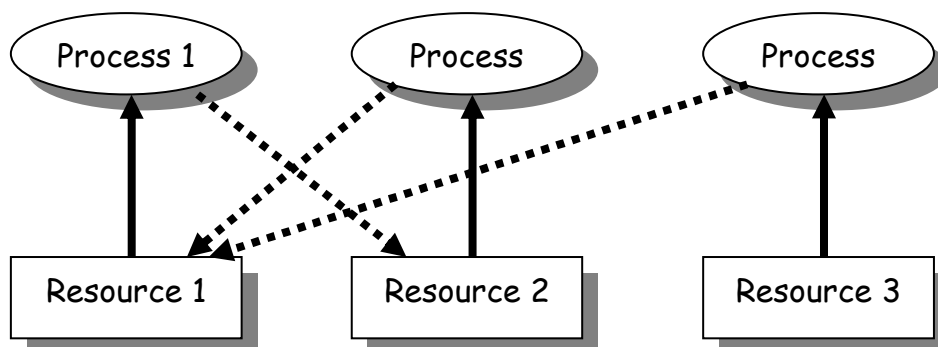
- α) Οι διεργασίες θα εκτελεστούν με τη σειρά: A, B, C
- β) Οι τρεις διεργασίες θα οδηγηθούν σε αδιέξοδο
- γ) Οι διεργασίες θα εκτελεστούν με τη σειρά: A, C, B
- δ) Οι διεργασίες θα εκτελεστούν με τη σειρά: C, B, A

Κεφάλαιο 6 – Αδιέξοδο

1. Ορισμοί – είδη πόρων
2. Γράφοι εκχώρησης πόρων
3. Συνθήκες αδιεξόδου
4. Προσεγγίσεις αδιεξόδου
 1. Πρόληψη
 2. Αποφυγή
 3. Ανίχνευση
5. Το πρόβλημα των συνδαιτημόνων φιλοσόφων

6.1. Εισαγωγή

Αδιέξοδο (deadlock) είναι η μόνιμη ή επ' αόριστον αναμονή ενός συνόλου διεργασιών που είτε συναγωνίζονται για πόρους του συστήματος είτε επικοινωνούν μεταξύ τους. Σε ένα σύστημα πολυπρογραμματισμού η συνολική απαίτηση πόρων από όλες τις ενεργές διεργασίες υπερβαίνει κατά πολύ το συνολικό ποσό των διαθέσιμων πόρων. Όλα τα αδιέξοδα εμπεριέχουν τις συγκρουόμενες ανάγκες για πόρους από δύο ή περισσότερες διεργασίες. Βασικός σκοπός είναι η σχεδίαση συστημάτων όπου το αδιέξοδο δεν θα μπορεί να συμβεί, αν και γενικά δεν υπάρχει αποτελεσματική λύση.



Σχήμα 6.1 Παράδειγμα αδιεξόδου

Οι πόροι (Resources) ενός υπολογιστικού συστήματος διακρίνονται σε προεκχωρούμενοι πόροι (preemptable resources), οι οποίοι μπορούν να απομακρυνθούν από μια διεργασία χωρίς παρενέργειες, και σε μη προεκχωρούμενους πόρους (nonpreemptable resources), που προξενούν αποτυχία στη διεργασία όταν απομακρυνθούν. Παραδείγματα πόρων αποτελούν οι εκτυπωτές, tape drives, μνήμη, CD - Recorders κλπ. Η σειρά των γεγονότων που απαιτούνται για τη χρήση ενός πόρου περιλαμβάνει την απαίτηση (request) για απόκτηση του πόρου, τη χρήση του και την απελευθέρωση του πόρου. Αν η απαίτηση για απόκτηση δεν ικανοποιηθεί, τότε έχουμε είτε αναμονή, με την αιτούμενη διεργασία να αναστέλλεται είτε η αιτούμενη διεργασία αποτυγχάνει εμφανίζοντας μήνυμα λάθους

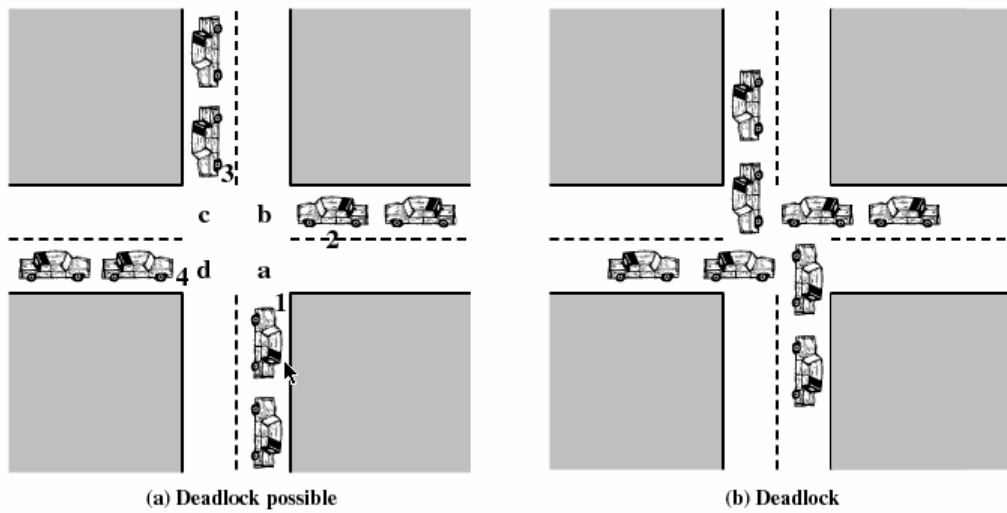
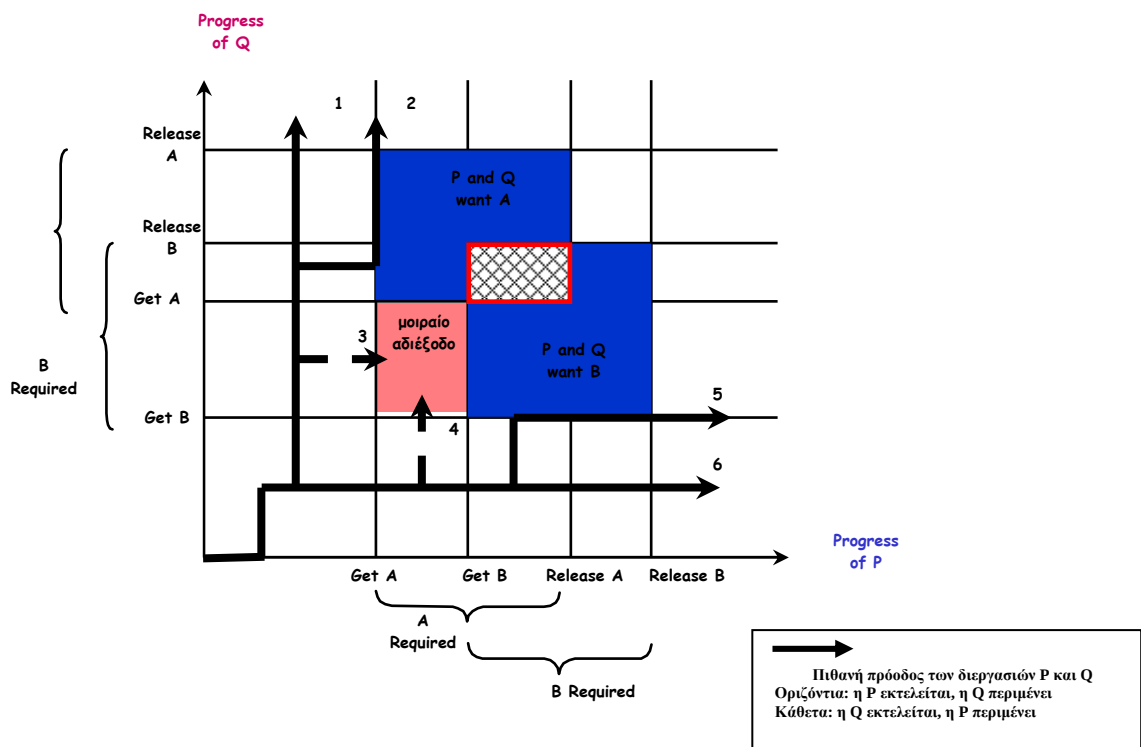
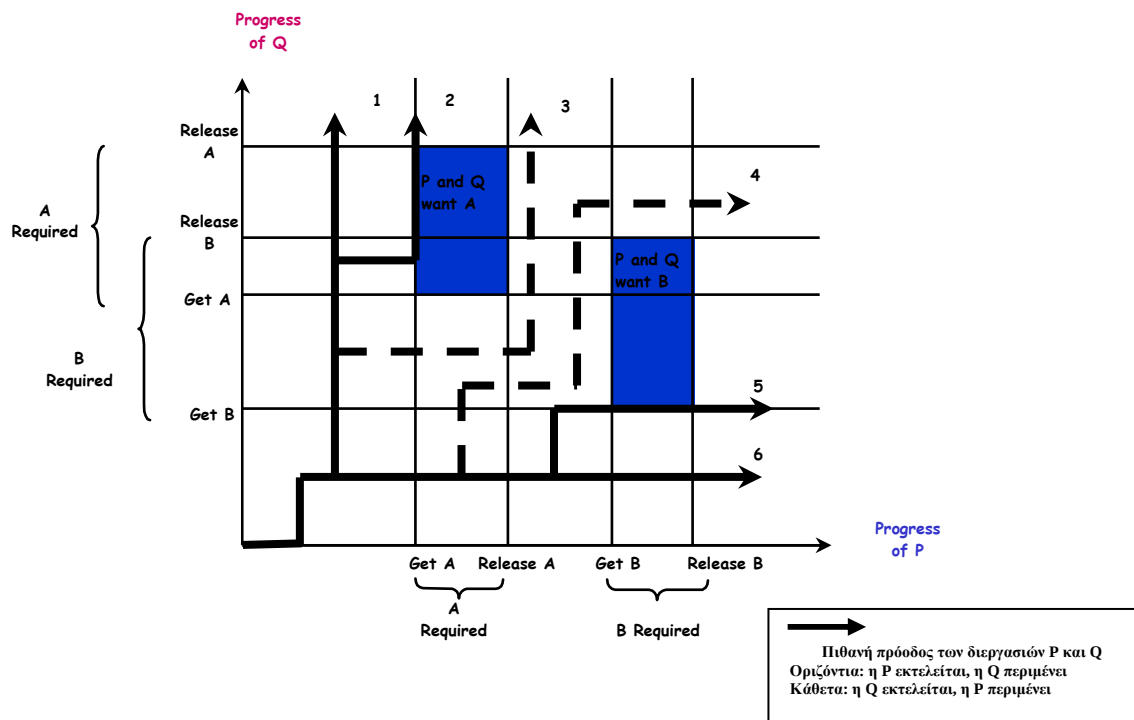


Figure 6.1 Illustration of Deadlock
Σχήμα 6.2 Αναπαράσταση αδιεξόδου



Σχήμα 6.3 Παράδειγμα αδιεξόδου



Σχήμα 6.4 Παράδειγμα χωρίς αδιέξοδο

Επαναχρησιμοποιήσιμοι πόροι είναι εκείνοι που χρησιμοποιούνται με ασφάλεια από μια διεργασία σε κάθε χρονική στιγμή και δεν εξαντλούνται από αυτή τη χρήση. Οι διεργασίες αποκτούν πόρους που θα απελευθερώσουν στη συνέχεια ώστε να χρησιμοποιηθούν από άλλες διεργασίες. Στους πόρους αυτούς περιλαμβάνονται επεξεργαστές, I/O κανάλια, κύρια και δευτερεύουσα μνήμη, αρχεία, βάσεις δεδομένων, και σημαφόροι. Το αδιέξοδο προκύπτει όταν μια διεργασία δεσμεύει ένα πόρο και απαιτεί έναν άλλο.

Καταναλώσιμοι πόροι είναι εκείνοι που δημιουργούνται (παράγονται) και καταστρέφονται (καταναλώνονται) από μια διεργασία. Όταν ένας πόρος δεσμεύεται από μια διεργασία παύει να υπάρχει. Παραδείγματα καταναλώσιμων πόρων είναι οι διακοπές (Interrupts), τα σήματα (signals), καθώς και τα μηνύματα και οι πληροφορίες σε ενταμιευτές (buffers) I/O. Το αδιέξοδο μπορεί να συμβεί αν ένα μήνυμα που αποστέλλεται από μια διεργασία δεν παραλαμβάνεται από την άλλη, ενώ ακόμα και κάποιος σπάνιος συνδυασμός γεγονότων μπορεί να οδηγήσει σε αδιέξοδο.

Παράδειγμα αδιεξόδου 1

Αν θεωρηθεί η σειρά εκτέλεσης: $p_0 p_1 q_0 q_1 p_2 q_2$, στο παράδειγμα του παρακάτω σχήματος, τότε είναι προφανές ότι προκύπτει κατάσταση αδιεξόδου.

Process P		Process Q	
Step	Action	Step	Action
p_0	Request (D)	q_0	Request (T)
p_1	Lock (D)	q_1	Lock (T)
p_2	Request (T)	q_2	Request (D)
p_3	Lock (T)	q_3	Lock (D)
p_4	Perform function	q_4	Perform function
p_5	Unlock (D)	q_5	Unlock (T)
p_6	Unlock (T)	q_6	Unlock (D)

Figure 6.4 Example of Two Processes Competing for Reusable Resources

Σχήμα 6.5 Παράδειγμα αδιεξόδου

Η αιτία αδιεξόδων αυτού του είδους είναι συχνά η πολύπλοκη λογική των προγραμμάτων. Μια στρατηγική αντιμετώπισης είναι η επιβολή περιορισμών που να αφορούν τη σειρά με την οποία ζητούνται οι πόροι.

Παράδειγμα αδιεξόδου 2

Αν θεωρηθεί ότι ο διαθέσιμος χώρος για κατανομή στην κύρια μνήμη είναι 200Kbytes, και πραγματοποιείται η παρακάτω σειρά αιτημάτων, θα προκύψει αδιέξοδος αν και οι δύο διεργασίες προχωρήσουν στο δεύτερο αίτημά τους.

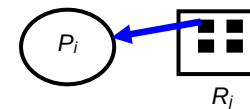
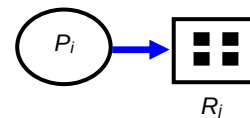
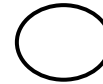
Διεργασία P1	Διεργασία P2
...	...
Request 80KB;	Request 70KB;
...	...
Request 60KB;	Request 80KB;

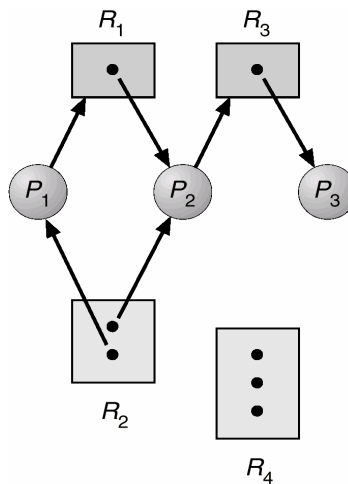
6.2. Γράφοι εκχώρησης πόρων

Οι γράφοι εκχώρησης φανερώνουν την ύπαρξη ή μη μιας κατάστασης αδιεξόδου. Αν ο γράφος δεν περιέχει κύκλους τότε δεν υπάρχει αδιέξοδος. Αντίθετα, αν ο γράφος περιέχει ένα κύκλο και αν υπάρχει μόνον ένα στιγμιότυπο ανά τύπο πόρου, τότε υπάρχει αδιέξοδος. Στην περίπτωση που υπάρχουν αρκετά στιγμιότυπα ανά τύπο πόρου, τότε υπάρχει πιθανότητα να συμβεί αδιέξοδος.

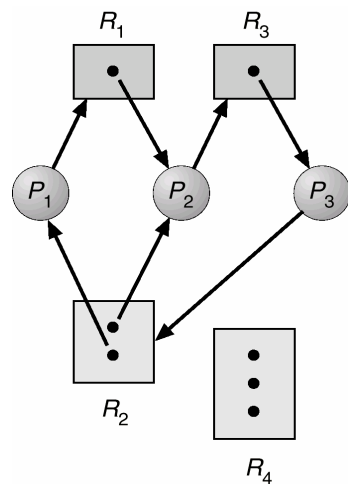
Τα βασικά στοιχεία ενός γράφου εκχώρησης πόρων είναι τα ακόλουθα:

- Διεργασία
- Τύπος πόρου με 4 στιγμιότυπα
- Η διεργασία P_i απαιτεί ένα στιγμιότυπο του πόρου R_j
- Η διεργασία P_i δεσμεύει ένα στιγμιότυπο του πόρου R_j
- Η διεργασία P_i απελευθερώνει ένα στιγμιότυπο του R_j





Σχήμα 6.6 Παράδειγμα γράφου εκχώρησης πόρων (μπορεί να συμβεί αδιέξοδο;)



Σχήμα 6.7 Παράδειγμα γράφου εκχώρησης πόρων με αδιέξοδο

Οι γράφοι εκχώρησης φανερώνουν την ύπαρξη ή μη μιας κατάστασης αδιεξόδου. Αν ο γράφος δεν περιέχει κύκλους τότε δεν υπάρχει αδιέξοδο. Αντίθετα, αν ο γράφος περιέχει ένα κύκλο και αν υπάρχει μόνον ένα στιγμιότυπο ανά τύπο πόρου, τότε υπάρχει αδιέξοδο. Στην περίπτωση που υπάρχουν αρκετά στιγμιότυπα ανά τύπο πόρου, τότε υπάρχει πιθανότητα να συμβεί αδιέξοδο.

6.3. Συνθήκες αδιεξόδου

Το αδιέξοδο συμβαίνει όταν ισχύουν όλα τα παρακάτω (συνθήκες αδιεξόδου):

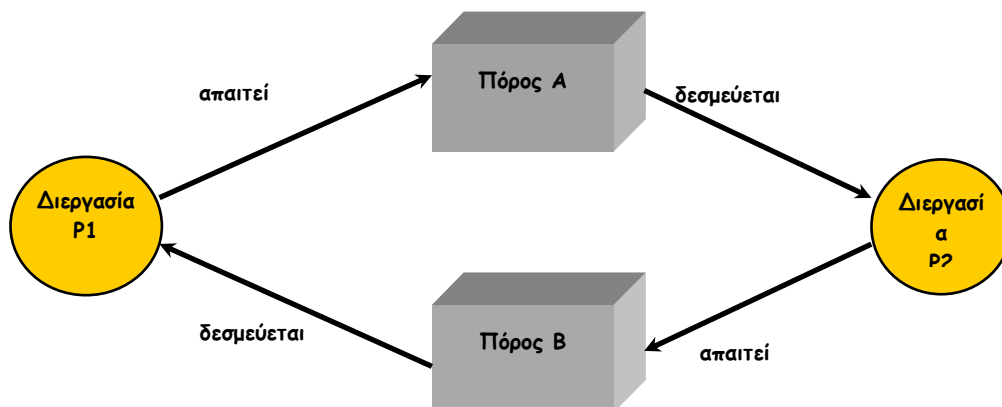
- Αμοιβαίος αποκλεισμός: οι εκχωρούμενοι πόροι είναι στην αποκλειστική κυριότητα της διεργασίας. Κάθε πόρος εκχωρείται σε μία διεργασία ή είναι διαθέσιμος.
- Κατοχή και αναμονή: η διεργασία μπορεί να δεσμεύει έναν πόρο ενώ περιμένει έναν άλλο.
- Μη προεκχώρηση: κανένας πόρος δεν μπορεί να αποσπασθεί δια της βίας από τη διεργασία που την κατέχει .
- Κυκλική αναμονή: ύπαρξη μιας κλειστής αλυσίδας διεργασιών 2 ή περισσότερων διεργασιών. Κάθε μία αναμένει ένα πόρο που κατέχεται από το επόμενο μέλος της αλυσίδας.

Σύμφωνα με τον αμοιβαίο αποκλεισμό, μόνο μια διεργασία τη φορά μπορεί να χρησιμοποιεί ένα πόρο. Παράδειγμα αποτελεί ο εκτυπωτής, όπου μόνον ο printer daemon μπορεί να χρησιμοποιεί τον πόρο. Φυσικά, δεν μπορούν όλες οι συσκευές να λειτουργήσουν παρόμοια

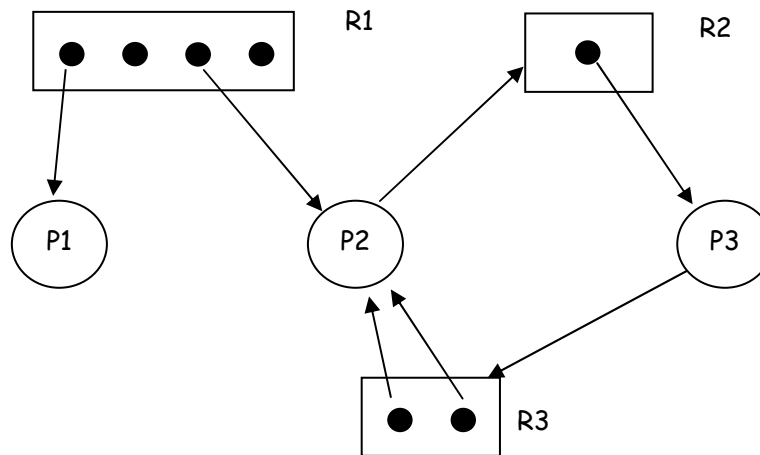
Στη συνθήκη κατοχής και αναμονής (Hold-and-wait), μια διεργασία μπορεί να κατέχει πόρους καθώς αναμένει την εκχώρηση κάποιων άλλων πόρων. Μια διεργασία απαιτεί όλους τους πόρους πριν ξεκινήσει, και επομένως δεν θα περιμένει ποτέ στη συνέχεια. Επίσης μια διεργασία απαιτεί όλους τους πόρους όταν δεν διαθέτει κανέναν, ενώ μπορεί να διακόψει τη χρήση των πόρων. Προβλήματα είναι το ότι δεν είναι γνωστές οι απαιτήσεις πόρων κατά την έναρξη της διεργασίας, η πιθανή παρατεταμένη στέρηση που μπορεί να συμβεί και το ότι δεσμεύονται πόροι που θα μπορούσαν να χρησιμοποιηθούν από άλλες διεργασίες.

Σύμφωνα με τη μη προεκχώρηση (No preemption), αν μια διεργασία που δεσμεύει συγκεκριμένους πόρους αρνείται μια επιπλέον απαίτηση τότε η διεργασία πρέπει να αποδεσμεύσει τους αρχικούς πόρους. Επίσης, αν μια διεργασία απαιτήσει έναν πόρο που δεσμεύεται από κάποια άλλη διεργασία, το Λ.Σ. θα μπορεί να προεκχωρήσει τη δεύτερη διεργασία και να απαιτήσει από την πρώτη να απελευθερώσει τους πόρους που κατέχει. Για παράδειγμα, η διακοπή μιας εγγραφής CD δημιουργεί ένα σημαντικό κόστος και αποτελεί μια προφανής λύση, ενώ ο οδηγός συσκευής του CD-R απαγορεύει μια δεύτερη λειτουργία open(). Κανένας πόρος δεν μπορεί να αποσπασθεί δια της βίας από τη διεργασία που τον κατέχει. Εφαρμόζεται σε πόρους, η κατάσταση των οποίων μπορεί να αποθηκευθεί και αργότερα να γίνει επαναφορά, όπως είναι οι καταχωρητές της CPU και χώρος μνήμης. Σε γενικές γραμμές δεν είναι μια βιώσιμη επιλογή για την πλειονότητα των πόρων.

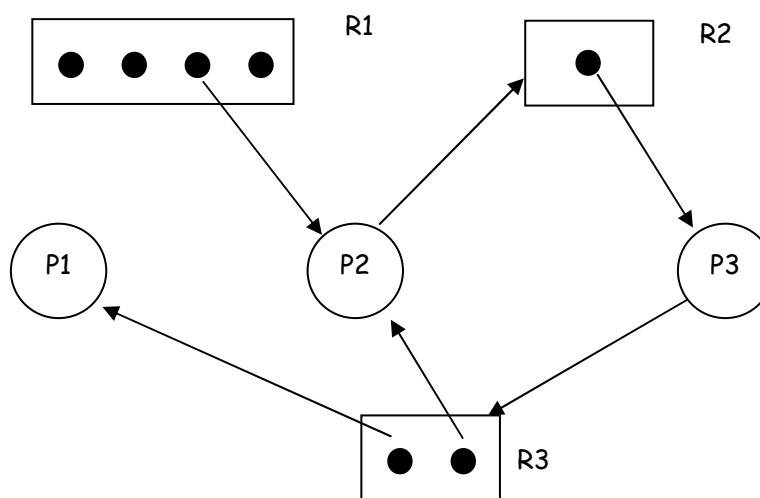
Τέλος, στη συνθήκη της κυκλικής αναμονής πρέπει να υπάρχει μια κυκλική αλυσίδα με 2 ή περισσότερες διεργασίες. Κάθε μια διεργασία περιμένει έναν πόρο που κατέχεται από άλλη διεργασία της αλυσίδας. Η συγκεκριμένη συνθήκη μπορεί να αποφευχθεί με τον ορισμό μια γραμμικής διάταξης των πόρων.



Σχήμα 6.8. Παράδειγμα κυκλικής αναμονής



Σχήμα 6.9. Παράδειγμα με κυκλική αναμονή



Σχήμα 6.10. Παράδειγμα χωρίς κυκλική αναμονή

6.4. Προσεγγίσεις αντιμετώπισης αδιεξόδου

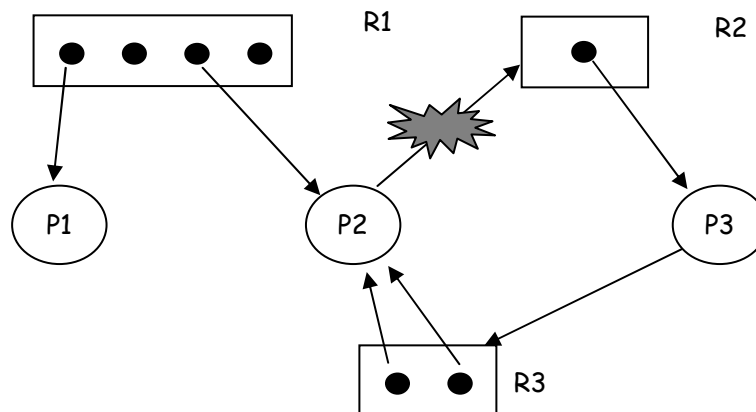
Το αδιέξοδο είναι μια γενική κατάσταση, για την οποία δεν υπάρχει μια ενιαία στρατηγική αντιμετώπισης κάθε είδους αδιεξόδου. Απαιτείται η ανάλυση όλων των διεργασιών που χρειάζονται πόρους και δεν μπορεί να ληφθεί μια τοπική απόφαση που θα στηρίζεται στις ανάγκες μιας διεργασίας.

Υπάρχουν 4 προσεγγίσεις αντιμετώπισης αδιεξόδου:

- Πρόληψη (Prevention): δεν επιτρέπεται ποτέ να συμβεί αδιέξοδο
- Αποφυγή (Avoidance): το σύστημα λαμβάνει απόφαση για να αποτρέψει μελλοντική κατάσταση αδιεξόδου
- Ανίχνευση (Detection) & Επαναφορά (Recovery): έλεγχος για αδιέξοδο (περιοδικά ή σποραδικά), στη συνέχεια επαναφορά
- Χειροκίνητη μεσολάβηση: ο χειριστής κάνει επανεκκίνηση του συστήματος, αν φαίνεται υπερβολικά αργό.

6.4.1. Πρόληψη αδιεξόδου

Βασική ιδέα στην προσέγγιση πρόληψης αδιεξόδου είναι ο σχεδιασμός του συστήματος έτσι ώστε να παραβιάζει μία από τις 4 αναγκαίες συνθήκες. Για την πρόληψη κατοχής και αναμονής γίνεται εξ αρχής απαίτηση όλων των πόρων και αναμονή μέχρι να καταστούν διαθέσιμοι όλοι οι πόροι. Η πρόληψη της κυκλικής αναμονής πραγματοποιείται με τον καθορισμό τον ορισμό μια γραμμικής διάταξης των τύπων των πόρων, οπότε μια διεργασία που δεσμεύει κάποιους πόρους μπορεί να απαιτήσει μόνον τύπους πόρων υψηλότερης τάξης. Το μειονέκτημα της πρόληψης κυκλικής αναμονής είναι η μείωση της απόδοσης λόγω καθυστέρησης στην εκτέλεση και μικρής παραλληλίας, ενώ η πρόληψη της κατοχής και αναμονής είναι πολυέξοδη αφού δεσμεύει περισσότερους πόρους από όσους απαιτούνται .



Σχήμα 6.11. Παράδειγμα πρόληψης αδιεξόδου

6.4.2. Αποφυγή αδιεξόδου

Για την αποφυγή αδιεξόδου, οι πόροι εκχωρούνται με τρόπο που εγγυάται ότι δεν θα βρεθεί ποτέ σημείο στο οποίο θα συμβεί αδιέξοδος. Η αποφυγή του αδιεξόδου επιτρέπει τις τρεις απαραίτητες συνθήκες αλλά κάνει διακριτικές επιλογές ώστε να εξασφαλιστεί ότι δεν θα προκύψει ποτέ αδιέξοδος. Έτσι επιτρέπεται μεγαλύτερος συγχρονισμός σε σχέση με την πρόληψη.

Η συγκεκριμένη προσέγγιση απαιτεί γνώση των μελλοντικών απαιτήσεων της διεργασίας και προϋποθέτει ότι το σύστημα έχει κάποια πρόσθετη και εκ των προτέρων διαθέσιμη πληροφορία. Λαμβάνεται δυναμικά μια απόφαση σχετικά με το αν η ικανοποίηση μιας απαίτησης, σε σχέση με την τρέχουσα εκχώρηση πόρων, θα οδηγήσει σε αδιέξοδο, που βασίζεται στο συνολικό ποσό των διαθέσιμων πόρων, στους τρέχοντες διαθέσιμους πόρους, στις απαιτήσεις πόρων εκ μέρους των διεργασιών και στην πρόσφατη εκχώρηση πόρων στις διεργασίες.

Το απλούστερο και πλέον χρήσιμο μοντέλο απαιτεί ότι κάθε διεργασία δηλώνει το μέγιστο πλήθος των πόρων κάθε τύπου που είναι πιθανόν να χρειαστεί. Ο αλγόριθμος αποφυγής αδιεξόδου εξετάζει δυναμικά την κατάσταση ανάθεσης πόρων για να εξασφαλίσει ότι δεν μπορεί να προκύψει κατάσταση κυκλικής αναμονής. Η κατάσταση ανάθεσης πόρων ορίζεται από το πλήθος των διαθέσιμων και εκχωρούμενων πόρων και από το μέγιστο πλήθος των απαιτήσεων εκ μέρους των διεργασιών.

Στις προσεγγίσεις αποφυγής αδιεξόδου, μια διεργασία δεν ξεκινά αν οι απαιτήσεις της μπορούν να οδηγήσουν σε αδιέξοδο. Επίσης, δεν ικανοποιείται μια αυξημένη απαίτηση για τη χρήση ενός πόρου από μια διεργασία αν αυτή η εκχώρηση του πόρου μπορεί να οδηγήσει σε αδιέξοδο. Ως αποτέλεσμα, η αποφυγή αδιεξόδου μπορεί να οδηγήσει σε μικρή χρήση πόρων και μειωμένη ρυθμοαπόδοση (throughput) του συστήματος.

Αλγόριθμος του τραπεζίτη

Ο αλγόριθμος του τραπεζίτη είναι γνωστός και ως άρνηση ανάθεσης πόρων αφού δεν επιτρέπει την ικανοποίηση αυξημένων απαιτήσεων για πόρους σε μια διεργασία αν αυτή η εκχώρηση πόρων μπορεί να οδηγήσει σε αδιέξοδο. Κατάσταση του συστήματος είναι η τρέχουσα ανάθεση πόρων στις διεργασίες, και χαρακτηρίζεται ασφαλής εφόσον υπάρχει μια τουλάχιστον ακολουθία εκτέλεσης των διεργασιών που μπορεί να εκτελεστεί μέχρι το τέλος (δηλαδή δεν οδηγεί σε αδιέξοδο) και μη ασφαλής κατάσταση σε κάθε άλλη περίπτωση.

Όταν μια διεργασία απαιτεί ένα διαθέσιμο πόρο, το σύστημα πρέπει να αποφασίσει αν η άμεση ανάθεση του πόρου θα το διατηρήσει σε ασφαλή κατάσταση. Το σύστημα είναι σε ασφαλή κατάσταση αν υπάρχει μια ασφαλής ακολουθία για όλες τις διεργασίες. Η ακολουθία $\langle P_1, P_2, \dots, P_n \rangle$ είναι ασφαλής αν για κάθε διεργασία P_i , οι πόροι που η P_i μπορεί ακόμη να απαιτήσει μπορούν να ικανοποιηθούν από τους τρέχοντες διαθέσιμους πόρους συν τους πόρους που δεσμεύονται από όλες τις διεργασίες P_j , με $j < i$. Αν οι ανάγκες σε πόρους της P_i δεν είναι άμεσα διαθέσιμοι, τότε η P_i μπορεί να περιμένει μέχρις ότου να τελειώσουν όλες οι P_j . Όταν τελειώνει η P_j η P_i μπορεί να αποκτήσει τους πόρους που χρειάζεται, να εκτελεστεί, να επιστρέψει τους πόρους που της ανατέθηκαν και να τερματιστεί. Όταν τερματίσει η P_i , η P_{i+1} μπορεί να αποκτήσει τους πόρους που χρειάζεται κ.ο.κ.

Βασικά γεγονότα στον αλγόριθμο του τραπεζίτη είναι ότι αν το σύστημα είναι σε ασφαλή κατάσταση τότε δεν υπάρχει αδιέξοδο, ενώ αν το σύστημα δεν είναι σε ασφαλή κατάσταση τότε υπάρχει πιθανότητα αδιεξόδου. Η αποφυγή αδιεξόδου συνίσταται στην εξασφάλιση ότι το σύστημα δεν θα εισέλθει ποτέ σε μη ασφαλή κατάσταση.

Παραδοχές του αλγορίθμου είναι ότι υπάρχουν πολλαπλά στιγμιότυπα των πόρων, κάθε διεργασία πρέπει εκ των προτέρων να διεκδικεί τη μέγιστη χρήση των πόρων, ενώ όταν μια διεργασία απαιτεί έναν πόρο ίσως χρειαστεί να περιμένει και όταν λάβει όλους τους πόρους πρέπει να τους επιστρέψει σε πεπερασμένο χρονικό διάστημα. Η μέγιστη απαίτηση για πόρους πρέπει να δηλώνεται εκ των προτέρων, ενώ οι σημαντικές διεργασίες πρέπει να είναι ανεξάρτητες και δεν υπόκεινται σε απαιτήσεις συγχρονισμού. Τέλος, πρέπει να υπάρχει ένας σταθερός αριθμός πόρων προς ανάθεση ενώ καμιά διεργασία δεν μπορεί να περιέλθει σε κατάσταση εξόδου (exit) ενώ δεσμεύει πόρους.

Δομές δεδομένων του αλγορίθμου

- n = το πλήθος των διεργασιών.
- m = το πλήθος τύπων πόρων.
- Available: Διάνυσμα μήκους m . Αν $available[j] = k$, υπάρχουν k στιγμιότυπα του τύπου πόρου R_j διαθέσιμα.
- Max: $n \times m$ πίνακας. Αν $Max[i,j] = k$, τότε η διεργασία P_i μπορεί να απαιτήσει κατά μέγιστο k στιγμιότυπα του τύπου πόρου R_j .
- Allocation: $n \times m$ πίνακας. Αν $Allocation[i, j] = k$ τότε στη διεργασία P_i εκχωρούνται k στιγμιότυπα του πόρου R_j .
- Need: $n \times m$ πίνακας. Αν $Need[i, j] = k$, τότε η διεργασία P_i μπορεί να χρειαστεί k επιπλέον στιγμιότυπα του πόρου R_j για να ολοκληρωθεί.
- $Need[i,j] = Max[i,j] - Allocation[i,j]$

Παράδειγμα

Εφαρμογή του αλγορίθμου για καθορισμό μιας ασφαλούς κατάστασης. Υπάρχουν 3 τύποι πόρων με πλήθος $R(1) = 9$, $R(2) = 3$, $R(3) = 6$. Πρέπει να καθοριστεί αν η παρακάτω κατάσταση ασφαλής;

Βήμα 1

	Απαιτήσεις			Δεσμεύσεις			Συνολικοί πόροι		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	9	3	6
<u>P2</u>	6	1	3	6	1	2	Διαθέσιμοι 0 1 1		
P3	3	1	4	2	1	1			
P4	4	2	2	0	0	2			

Βήμα 2

	Απαιτήσεις			Δεσμεύσεις			Συνολικοί πόροι		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	9	3	6
P2	0	0	0	0	0	0	Διαθέσιμοι 6 2 3		
P3	3	1	4	2	1	1			
P4	4	2	2	0	0	2			

Βήμα 3

	Απαιτήσεις			Δεσμεύσεις			Συνολικοί πόροι		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
<u>P1</u>	0	0	0	0	0	0	9	3	6
P2	0	0	0	0	0	0	Διαθέσιμοι 7 2 3		
P3	3	1	4	2	1	1			
P4	4	2	2	0	0	2			

Βήμα 4

	Απαιτήσεις			Δεσμεύσεις			Συνολικοί πόροι		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	0	0	0	0	0	0	9	3	6
P2	0	0	0	0	0	0	Διαθέσιμοι 9 3 4		
<u>P3</u>	0	0	0	0	0	0			
P4	4	2	2	0	0	2			

Βήμα 5

	Απαιτήσεις			Δεσμεύσεις			Συνολικοί πόροι		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	0	0	0	0	0	0	9	3	6
P2	0	0	0	0	0	0			
P3	0	0	0	0	0	0			
P4	0	0	0	0	0	0	Διαθέσιμοι	9	3 6

Η κατάσταση είναι ασφαλής: $P2 \rightarrow P1 \rightarrow P3 \rightarrow P4$

6.4.3. Ανίχνευση αδιεξόδου

Ο αλγόριθμος του τραπέζιτη είναι απαισιόδοξος αφού πάντοτε υποθέτει ότι μια διεργασία δεν θα απελευθερώσει τους πόρους που κατέχει μέχρι να αποκτήσει όλους τους πόρους που χρειάζεται. Ως συνέπεια προκύπτουν μικρό ποσό παραλληλίας και πολύπλοκοι έλεγχοι για κάθε απαίτηση εκχώρησης πόρου (πολυπλοκότητα $O(n^2)$). Οι στρατηγικές ανίχνευσης αδιεξόδου δεν οριοθετούν την πρόσβαση σε πόρους και δεν περιορίζουν τις ενέργειες των διεργασιών. Οι απαιτούμενοι πόροι εκχωρούνται στις διεργασίες, όποτε αυτό είναι δυνατό.

Περιοδικά το ΛΣ εφαρμόζει έναν αλγόριθμο ανίχνευσης αδιεξόδου που δίνει τη δυνατότητα να εντοπισθεί η συνθήκη της κυκλικής αναμονής. Το πότε και πόσο συχνά καλείται ο αλγόριθμος εξαρτάται από πόσο συχνά συνηθίζεται να συμβαίνει αδιέξοδο και από το πλήθος των διεργασιών που πρέπει να επιστρέψουν σε προηγούμενη κατάσταση. Αν ο αλγόριθμος καλείται αυθαίρετα δεν εξασφαλίζεται η ανίχνευση της διεργασίας που προκαλεί το αδιέξοδο Στην πράξη, τα περισσότερα Λ.Σ. εθελουφλούν και αντιμετωπίζουν το πρόβλημα με συνδυασμό τεχνικών, όπως οι ακόλουθες:

- η διακοπή κατοχής και αναμονής: όταν μια διεργασία δεν μπορεί να αποκτήσει έναν πόρο, τότε δεν μπορεί να ολοκληρωθεί και αποτυγχάνει.
- χρήση ορίων (Quotas)
- υψηλής απόδοσης τεχνικές προγραμματισμού: απαίτηση χρήσης σηματοφόρων με καθορισμένη προτεραιότητα.

Οι δυνατές στρατηγικές που ακολουθούνται όταν ανιχνευθεί αδιέξοδο είναι οι παρακάτω:

- Τερματισμός διεργασίας & προεκχώρηση πόρων
- Διακοπή όλων των διεργασιών που περιήλθαν σε αδιέξοδο
- Δημιουργία αντιγράφου ασφαλείας κάθε διεργασίας που βρίσκεται σε αδιέξοδο, σε κάποιο προηγούμενο σημείο ελέγχου και επανεκκίνηση όλων των διεργασιών, αν και το αρχικό αδιέξοδο μπορεί να συμβεί ξανά
- Διαδοχική διακοπή όλων των διεργασιών που βρίσκονται σε αδιέξοδο ώστε να μην υπάρχει πλέον αδιέξοδο
- Διαδοχική προεκχώρηση πόρων έως ότου δεν θα υπάρχει αδιέξοδο

Υπάρχουν διάφορα κριτήρια επιλογής των διεργασιών που βρίσκονται σε αδιέξοδο και θα τερματιστούν. Έτσι, μπορεί να επιλεγεί η διεργασία:

- που έχει καταναλώσει το μικρότερο ποσό χρόνου επεξεργασίας μέχρι την τρέχουσα στιγμή
- που παράγει το μικρότερο πλήθος γραμμών εξόδου μέχρι την τρέχουσα στιγμή
- με τον μεγαλύτερο εκτιμώμενο χρόνο
- με το μικρότερο πλήθος πόρων που της έχουν εκχωρηθεί
- με τη μικρότερη προτεραιότητα

Η συνδυασμένη ανίχνευση αδιεξόδου περιλαμβάνει το συνδυασμό των τριών προσεγγίσεων που μελετήσαμε, δηλαδή της πρόληψης, της αποφυγής και της ανίχνευσης, και επιτρέπει τη χρήση της βέλτιστης προσέγγισης για κάθε πόρο του συστήματος. Μια κατά το δυνατόν βέλτιστη προσέγγιση περιλαμβάνει τη διαμοίραση των πόρων σε ιεραρχικά διατεταγμένες κλάσεις και χρήση των πλέον κατάλληλων τεχνικών για τη διαχείριση αδιεξόδων μέσα σε κάθε κλάση.

6.5. Πρόβλημα συνδαιτημόνων φιλοσόφων

Πέντε φιλόσοφοι κάθονται γύρω από ένα κυκλικό τραπέζι. Κάθε φιλόσοφος καταναλώνει το χρόνο του διαδοχικά σκεπτόμενος και τρώγοντας. Στο κέντρο του τραπεζιού υπάρχει ένα μεγάλο πιάτο με spaghetti. Κάθε φιλόσοφος χρειάζεται δύο πιρούνια για να φάει λίγο spaghetti. Υπάρχει ένα πιρούνι ανάμεσα σε κάθε ζεύγος φιλοσόφων και όλοι συμφωνούν ότι θα χρησιμοποιούν μόνον τα πιρούνια που βρίσκονται δεξιά και αριστερά από τον καθένα.

Κάθε φιλόσοφος είναι μια διεργασία και κάθε πιρούνι είναι ένας διαμοιραζόμενος πόρος με ενέργειες δέσμευσης και απελευθέρωσης. Αν ένας φιλόσοφος πεινάσει, πρέπει πρώτα να πάρει τα πιρούνια δεξιά και αριστερά του δεξιό για να μπορέσει να ξεκινήσει να τρώει.

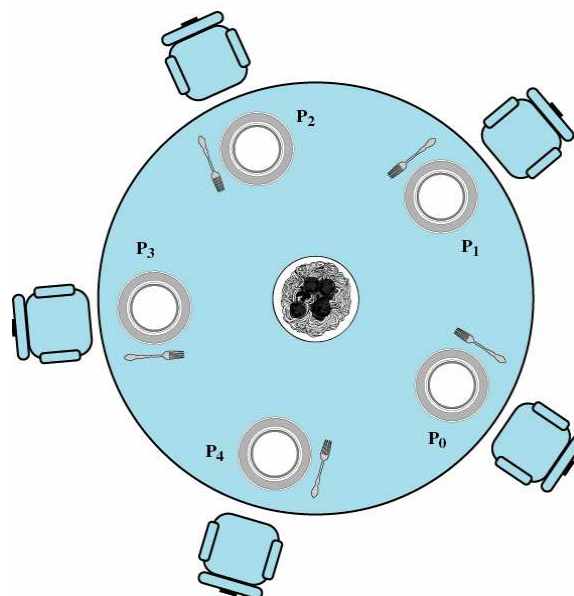


Figure 6.11 Dining Arrangement for Philosophers

Σχήμα 6.12. Καθορισμός του δείπνου των φιλοσόφων

Το πρόβλημα αυτό είναι ένα πρότυπο που χρησιμοποιείται για την αποτίμηση μεθόδων σχετικών με το συγχρονισμό ταυτόχρονων διεργασιών. Καταδεικνύει τη

δυσκολία της εκχώρησης πόρων μεταξύ διεργασιών χωρίς αδιέξοδα και παρατεταμένες στερήσεις. Στόχος είναι η ανάπτυξη ενός πρωτοκόλλου απόκτησης των πηρουινών που θα εξασφαλίζει την απαλλαγή από αδιέξοδα, τη δικαιοσύνη, ώστε κανένας φιλόσοφος να μην πρέπει να υποφέρει από παρατεταμένη στέρηση, και ο μέγιστος δυνατός συγχρονισμός.

Για το συγκεκριμένο πρόβλημα δεν υπάρχει συμμετρική λύση. Πιθανές λύσεις είναι η προσθήκη ενός ακόμη πιρουνιού ή ο περιορισμός του μέγιστου αριθμού φιλοσόφων στο τραπέζι σε τέσσερις (κυκλική αναμονή). Επίσης, μπορεί να ακολουθείται διαφορετική αλληλουχία ενεργειών για τους φιλοσόφους με άρτιο και περιττό αύξοντα αριθμό δηλαδή δημιουργία δύο ομάδων φιλοσόφων. Η μία ομάδα (περιττός α/α) θα αποκτά πρώτα το δεξιό και μετά το αριστερό πιρούνι και η άλλη ομάδα (άρτιος α/α) πρώτα το αριστερό και μετά το δεξιό πιρούνι. Άλλες λύσεις είναι ένας φιλόσοφος να επιτρέπεται να αποκτήσει τα πιρούνια μόνον όταν και τα δύο είναι διαθέσιμα (κρίσιμο τμήμα) - (κατοχή και αναμονή) αλλά και ο σχεδιασμός του συστήματος έτσι ώστε ένας φιλόσοφος να «κλέψει» ένα πιρούνι που δεν είναι γειτονικό του.

Ερωτήσεις Επανάληψης

1. Τι είναι κατάσταση αδιεξόδου; Δώστε ένα παράδειγμα αδιεξόδου.
2. Εξηγήστε τις κατηγορίες πόρων: προεκχωρούμενοι, μη προεκχωρούμενοι πόροι, επαναχρησιμοποιήσιμοι, καταναλώσιμοι.
3. Περιγράψτε τις αναγκαίες συνθήκες για τη δημιουργία αδιεξόδου.
4. Αναφέρετε και περιγράψτε συνοπτικά τις προσεγγίσεις αντιμετώπισης αδιεξόδου;
5. Ποια είναι τα κριτήρια επιλογής των διεργασιών που θα τερματιστούν κατά την ανίχνευση αδιεξόδου;

Ασκήσεις

1. Επαναχρησιμοποιήσιμοι είναι οι υπολογιστικοί πόροι που:
 - α) μπορούν να απομακρυνθούν από μια διεργασία χωρίς παρενέργειες
 - β) προξενούν αποτυχία στη διεργασία όταν απομακρυνθούν
 - γ) χρησιμοποιούνται με ασφάλεια από μια διεργασία σε κάθε χρονική στιγμή
 - δ) εξαντλούνται από τη χρήση τους από τις διεργασίες
2. Ποιο από τα παρακάτω δεν αποτελεί προσέγγιση αντιμετώπισης αδιεξόδου;
 - α) αναμονή τερματισμού διεργασιών
 - β) πρόληψη
 - γ) ανίχνευση και επαναφορά
 - δ) επανεκκίνηση του συστήματος
3. Στην προσέγγιση αποφυγής αδιεξόδου
 - α) διακόπτονται όλες οι διεργασίες που βρίσκονται σε αδιέξοδο
 - β) οι πόροι εκχωρούνται με κατάλληλη σειρά
 - γ) εξετάζεται στατικά η κατάσταση ανάθεσης πόρων
 - δ) η εκτέλεση των διεργασιών ξεκινάει άμεσα
4. Να σχεδιάσετε ένα γράφο εκχώρησης πόρων για τα παρακάτω:
 - Η διεργασία P1 απαιτεί τον πόρο R1
 - Η διεργασία P2 απαιτεί τον πόρο R3

- Η διεργασία P3 απαιτεί τον πόρο R3
- Ο πόρος R1 εκχωρείται στη διεργασία P2
- Ο πόρος R2 εκχωρείται στη διεργασία P1
- Ο πόρος R3 εκχωρείται στη διεργασία P2

Ποια διεργασία θα εκτελεστεί πρώτη;

- α) Η διεργασία P1
- β) Η διεργασία P2
- γ) Η διεργασία P3
- δ) Καμία αφού θα συμβεί αδιέξοδο

5. Ένα σύστημα διαθέτει 6 όμοια tape drives και n σε πλήθος διεργασίες που ανταγωνίζονται για τη χρήση τους. Κάθε διεργασία μπορεί να απαιτήσει 2 tape drives. Ποια από τις παρακάτω είναι η μέγιστη δυνατή τιμή του n ώστε το σύστημα να είναι απαλλαγμένο από αδιέξοδο;

- α) 7
- β) 6
- γ) 5
- δ) 4

6. Στο γράφο του σχήματος 6.6, τα συνολικά στιγμιότυπα πόρων είναι:

- α) 3
- β) 8
- γ) 4
- δ) 7

7. Σύμφωνα με το γράφο του σχήματος 6.6, ποια διεργασία θα εκτελεστεί δεύτερη;

- α) Η διεργασία P1
- β) Η διεργασία P2
- γ) Η διεργασία P3
- δ) Καμία αφού θα συμβεί αδιέξοδο

8. Σύμφωνα με το γράφο του σχήματος 6.7, ποια διεργασία θα εκτελεστεί τελευταία;

- α) Καμία αφού θα συμβεί αδιέξοδο
- β) Η διεργασία P1
- γ) Η διεργασία P2
- δ) Η διεργασία P3

9. Σύμφωνα με το γράφο του σχήματος 6.9, ποια διεργασία θα εκτελεστεί δεύτερη;

- α) Η διεργασία P1
- β) Η διεργασία P2
- γ) Η διεργασία P3
- δ) Καμία αφού θα συμβεί αδιέξοδο

10. Σύμφωνα με το γράφο του σχήματος 6.10, ποια διεργασία θα εκτελεστεί τελευταία;

- α) Η διεργασία P1
- β) Η διεργασία P2
- γ) Η διεργασία P3
- δ) Καμία αφού θα συμβεί αδιέξοδο

Κεφάλαιο 7 – Διαχείριση Μνήμης

1. Εισαγωγή
2. Βασική διαχείριση μνήμης
3. Μνήμη και πολυπρογραμματισμός
4. Τμηματοποίηση σταθερού μεγέθους
 1. Ίσα τμήματα
 2. Άνισα τμήματα
5. Δυναμική τμηματοποίηση
6. Εναλλαγή

7.1. Εισαγωγή

Η διαχείριση μνήμης είναι η λειτουργία της υποδιαίρεσης της μνήμης από το λειτουργικό σύστημα με δυναμικό τρόπο ώστε να εξυπηρετούνται όσο το δυνατόν περισσότερες διεργασίες. Αποτελεί απαραίτητο στοιχείο των λειτουργικών συστημάτων διότι η μνήμη είναι ένας ανεπαρκής πόρος και είναι απαραίτητη η αποτελεσματική χρήση της. Η διαχείριση μνήμης διευκολύνει τον προγραμματισμό, ενισχύει τον πολυπρογραμματισμό και παρέχει ασφάλεια και προστασία στις εκτελούμενες διεργασίες. Οι προγραμματιστές επιζητούν την ελαχιστοποίηση του χρόνου προσπέλασης και τη μεγιστοποίηση του μεγέθους της μνήμης για την εκτέλεση των προγραμμάτων.

<i>Operating System</i>	<i>Release Date</i>	<i>Minimum Memory Requirement</i>	<i>Recommended Memory</i>
Windows 1.0	November 1985	256KB	
Windows 2.03	November 1987	320KB	
Windows 3.0	March 1990	896KB	1MB
Windows 3.1	April 1992	2.6MB	4MB
Windows 95	August 1995	8MB	16MB
Windows NT 4.0	August 1996	32MB	96MB
Windows 98	June 1998	24MB	64MB
Windows ME	September 2000	32MB	128MB
Windows 2000 Professional	February 2000	64MB	128MB
Windows XP Home	October 2001	64MB	128MB
Windows XP Professional	October 2001	128MB	256MB

Σχήμα 7.1 Απαιτήσεις μνήμης του λειτουργικού συστήματος Windows

Ο διαχειριστής μνήμης είναι ένα συστατικό του ΛΣ που ασχολείται με την οργάνωση και τις στρατηγικές διαχείρισης της μνήμης. Τα βασικά χαρακτηριστικά των διαχειριστών μνήμης είναι ότι εκχωρούν την πρωτεύουσα μνήμη σε διεργασίες, αντιστοιχούν το χώρο διευθύνσεων της διεργασίας στην κύρια μνήμη ελαχιστοποιούν το χρόνο προσπέλασης χρησιμοποιώντας cost-effective τεχνικές, στατικές ή δυναμικές καθώς επίσης και ότι αλληλεπιδρούν με ειδικό hardware για τη διαχείριση της μνήμης (MMU) για να βελτιώσουν την απόδοση.

Οι στρατηγικές διαχείρισης μνήμης σχεδιάζονται έτσι ώστε να είναι εφικτή η βέλτιστη δυνατή χρήση της κύριας μνήμης και διακρίνονται στις ακόλουθες κατηγορίες:

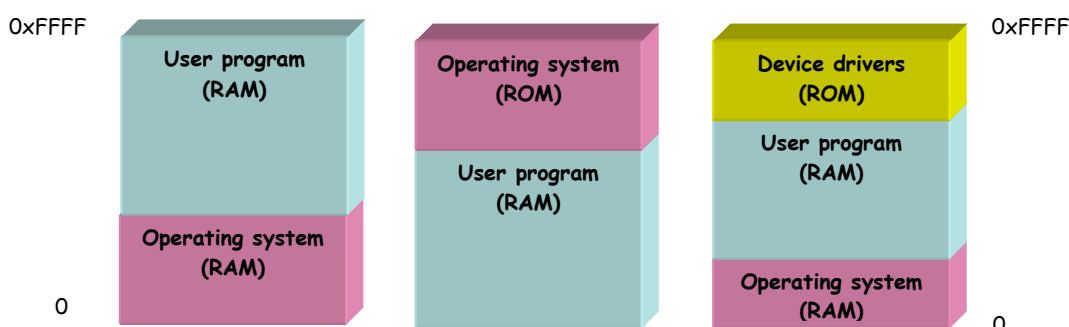
- Στρατηγικές προσκόμισης: καθορίζουν το σημείο όπου θα τοποθετηθεί το επόμενο τμήμα προγράμματος ή δεδομένων, καθώς μετακινείται από τη δευτερεύουσα μνήμη.
- Στρατηγικές τοποθέτησης: καθορίζουν το σημείο της κύριας μνήμης όπου το σύστημα θα μπορούσε να τοποθετήσει τμήματα δεδομένων.
- Στρατηγικές επανατοποθέτησης: καθορίζουν ποιο τμήμα θα αφαιρεθεί από την κύρια μνήμη στις περιπτώσεις όπου η κύρια μνήμη είναι αρκετά πλήρης ώστε να παρέχει χώρο σε ένα νέο πρόγραμμα.

Η εκχώρηση μνήμης στα προγράμματα μπορεί να είναι συνεχόμενη ή μη συνεχόμενη. Η συνεχόμενη εκχώρηση μνήμης αφορά τα πρώτα υπολογιστικά συστήματα όπου αν το πρόγραμμα ήταν μεγαλύτερο από τη διαθέσιμη μνήμη το σύστημα δεν μπορούσε να το εκτελέσει. Στην μη συνεχόμενη εκχώρηση μνήμης, το πρόγραμμα διαιρείται σε τεμάχια ή τμήματα που τοποθετούνται από το σύστημα σε μη γειτονικές σχισμές στην κύρια μνήμη. Η τεχνική αυτή κάνει εφικτή τη χρήση περιοχών που είναι πολύ μικρές για να χωρέσουν ολόκληρο πρόγραμμα. Αν και με τον τρόπο αυτό εισάγεται στο σύστημα πολυπλοκότητα αυτή δικαιολογείται από την αύξηση που επιτυγχάνεται στο βαθμό πολυπρογραμματισμού.

7.2. Βασική διαχείριση μνήμης

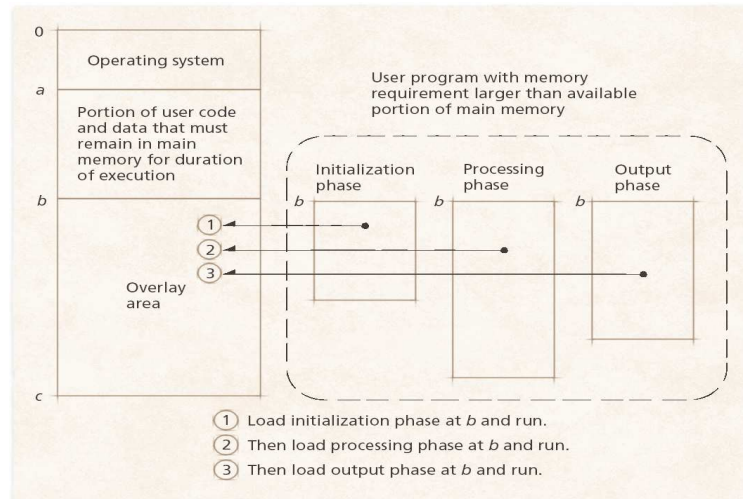
Η βασική διαχείριση μνήμης στηρίζεται σε δύο εναλλακτικές προσεγγίσεις: τον μονοπρογραμματισμό και τις επικαλύψεις.

Κατά τον μονοπρογραμματισμό, ένας χρήστης μονοπωλεί τη χρήση του συστήματος και όλοι οι πόροι είναι αφιερωμένοι σ' αυτόν. Επομένως, η προστασία μνήμης δεν τίθεται ως πρόβλημα στην προκειμένη περίπτωση. Το σχήμα 7.2 παρουσιάζει τρεις απλούς τρόπους οργάνωσης μνήμης σε λειτουργικό σύστημα με μια μόνο διεργασία χρήστη.



Σχήμα 7.2 Οργάνωση μνήμης σε περίπτωση μονοπρογραμματισμού

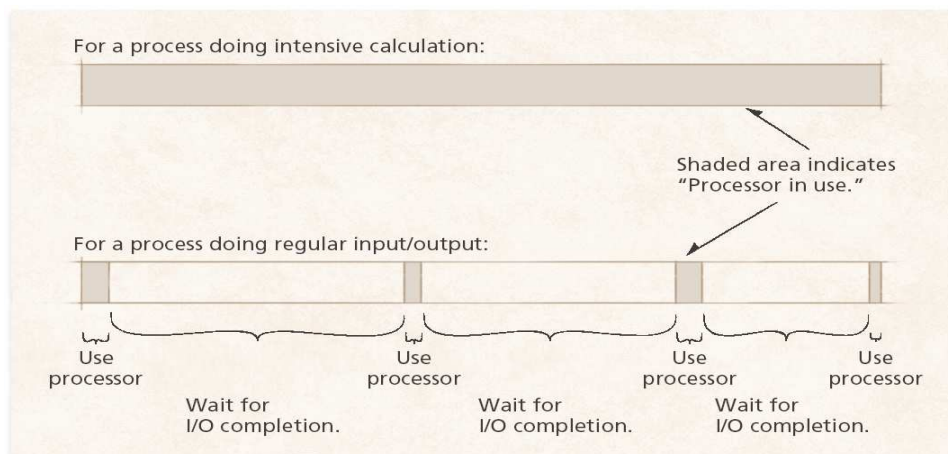
Οι επικαλύψεις (overlays) είναι μια τεχνική που επιτρέπει σε ένα σύστημα να εκτελεί προγράμματα που είναι μεγαλύτερα από την κύρια μνήμη. Ο προγραμματιστής διαιρεί το πρόγραμμα σε λογικές ενότητες και όταν αυτό δεν χρειάζεται μνήμη για ένα τμήμα, το σύστημα μπορεί να αντικαταστήσει όλη ή μέρη της κύριας μνήμης για να καλύψει μια ανάγκη (δηλ. να φορτώσει μια άλλη ενότητα).



Σχήμα 7.3 Οργάνωση μνήμης με επικαλύψεις

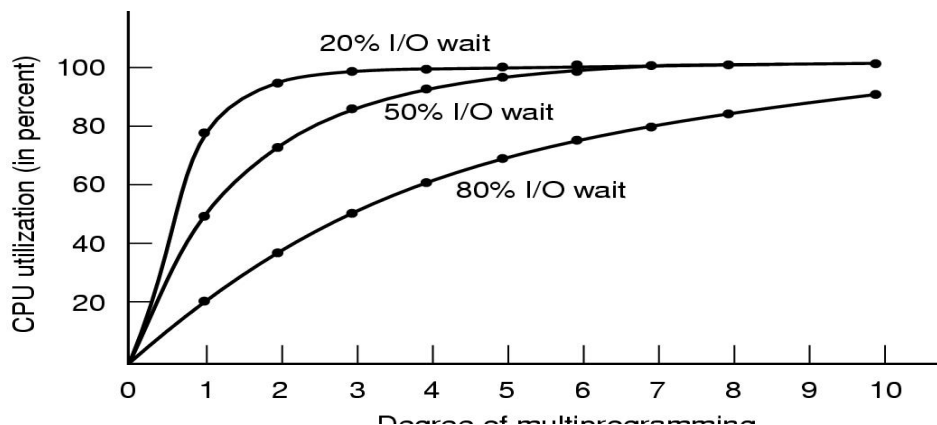
7.3. Μνήμη και πολυπρογραμματισμός

Η χρήση του επεξεργαστή από μία διεργασία διακόπτεται συχνά, λόγω της ανάγκης για λειτουργίες I/O που είναι υπερβολικά αργές συγκρινόμενες με την ταχύτητα της CPU. Η αύξηση της χρήσης της CPU επιτυγχάνεται με τα συστήματα πολυπρογραμματισμού, όπου αρκετοί χρήστες ανταγωνίζονται συγχρόνως για τους πόρους του συστήματος. Με τον τρόπο αυτό, αρκετές διεργασίες πρέπει να βρίσκονται στην κύρια μνήμη την ίδια στιγμή, ώστε αν κάποια υλοποιεί λειτουργίες I/O κάποια άλλη να χρησιμοποιεί την CPU ώστε να αυξάνεται το ποσοστό χρήσης της CPU και η απόδοση (throughput) του συστήματος.



Σχήμα 7.4 Χρήση της CPU σε σύστημα ενός χρήστη

Για τον καθορισμό του πλήθους των διεργασιών που μπορούν να υπάρχουν συγχρόνως στην κύρια μνήμη πρέπει να ληφθεί υπόψη το ότι περισσότερες διεργασίες χρησιμοποιούν καλύτερα την CPU αλλά απαιτείται καλύτερη διαχείριση και προστασία της μνήμης, να εξισορροπηθεί με το γεγονός ότι λιγότερες διεργασίες χρησιμοποιούν και λιγότερη μνήμη, καθώς και ότι περισσότερη αναμονή για I/O σημαίνει μικρότερη χρήση επεξεργαστή.



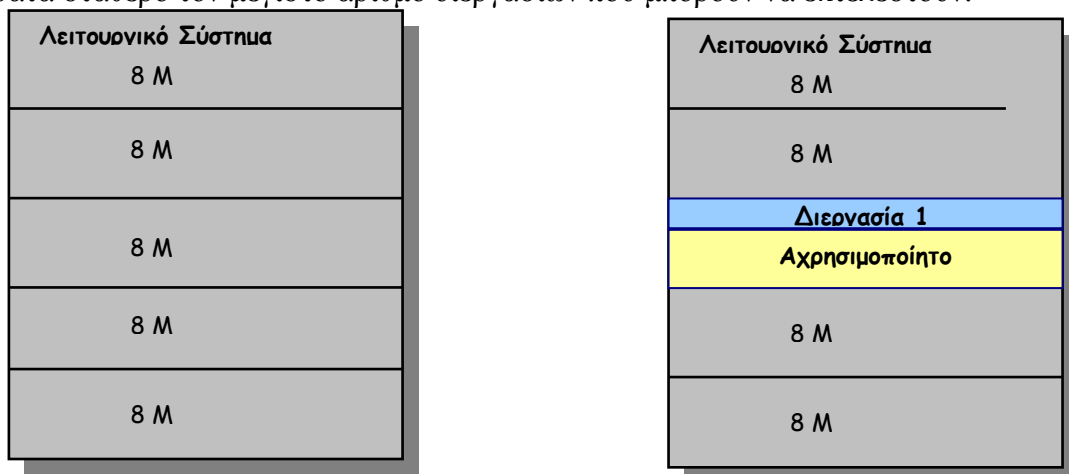
Σχήμα 7.5 Χρήση CPU – Πλήθος διεργασιών

7. 4. Τμηματοποίηση σταθερού μεγέθους

Κατά την τμηματοποίηση σταθερού μεγέθους, η μνήμη του συστήματος χωρίζεται σε σταθερά τμήματα των οποίων το μέγεθος μπορεί να ίσο ή άνισα.

Στην τμηματοποίηση σταθερού μεγέθους (fixed Partitioning) με ίσα τμήματα, κάθε διεργασία με μέγεθος μικρότερο ή ίσο με το μέγεθος του τμήματος μπορεί να φορτωθεί στο διαθέσιμο τμήμα. Αν όλα τα τμήματα είναι γεμάτα, το Λ.Σ. μπορεί να κάνει εναλλαγή μιας διεργασίας. Ένα πρόγραμμα είναι πιθανό να μη χωρά σε ένα τμήμα, οπότε ο προγραμματιστής πρέπει να σχεδιάσει το πρόγραμμα με επικαλύψεις.

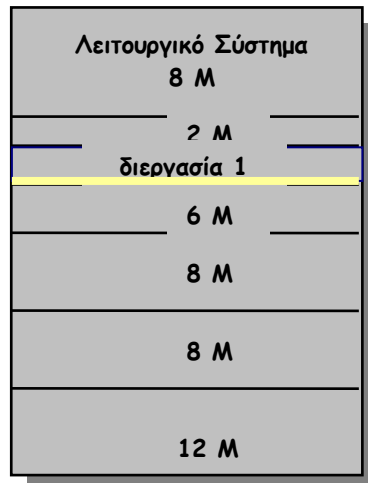
Πλεονέκτημα της χρήσης ίσων τμημάτων είναι η μικρή επιβάρυνση στο λειτουργικό σύστημα, αφού δεν υπάρχουν ιδιαίτερες απαιτήσεις για την υλοποίηση της. Με τη μέθοδο αυτή, ωστόσο, η χρησιμοποίηση της κύριας μνήμης είναι εξαιρετικά αναποτελεσματική. Αυτό οφείλεται στο ότι κάθε πρόγραμμα, όσο μικρό και να είναι, καταλαμβάνει ένα ολόκληρο τμήμα. Ο ανεκμετάλλευτος χώρος εσωτερικά σε ένα τμήμα αναφέρεται ως εσωτερικός κατακερματισμός (internal fragmentation). Επομένως, οι μικρές διεργασίες δεν χρησιμοποιούν αποτελεσματικά τον χώρο των τμημάτων και η ανεπαρκής χρήση της μνήμης λόγω του εσωτερικού κατακερματισμού κρατά σταθερό τον μέγιστο αριθμό διεργασιών που μπορούν να εκτελεστούν.



Σχήμα 7.5 Τμηματοποίηση σταθερού μεγέθους με ίσα τμήματα και εσωτερικός κατακερματισμός

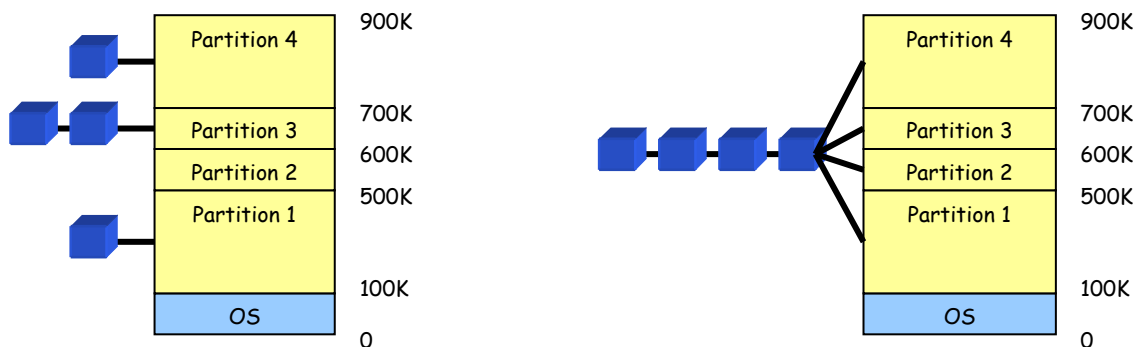
Η τμηματοποίηση σταθερού μεγέθους με άνισα τμήματα μειώνει τα προβλήματα της τμηματοποίησης ίσων τμημάτων, αφού αποφεύγεται σε σημαντικό βαθμό ο εσωτερικός κατακερματισμός και γίνεται αποτελεσματικότερη χρήση της κύριας

μνήμης, σε σχέση με την προηγούμενη μέθοδο. Μειονέκτημά της όμως αποτελεί η ανεπαρκής χρήση του επεξεργαστή λόγω της ανάγκης για συμπίεση για την αντιμετώπιση του εξωτερικού κατακερματισμού. Σύμφωνα με τα όσα έχουν ήδη αναφέρει, εσωτερικός κατακερματισμός είναι η μνήμη που δεν χρησιμοποιείται (δαπανάται) και είναι ορατή μόνον από τη διεργασία που ζητά μνήμη. Συμβαίνει επειδή η ποσότητα μνήμης που θα εκχωρηθεί στη διεργασία πρέπει να είναι μεγαλύτερη ή ίση από την αιτούμενη ποσότητα. Αντίθετα, εξωτερικός κατακερματισμός είναι η μνήμη που δεν χρησιμοποιείται (δαπανάται) και είναι ορατή από το σύστημα εκτός των διεργασιών που απαιτούν μνήμη. Συμβαίνει επειδή όλες οι απαιτήσεις μνήμης δεν είναι του ίδιου μεγέθους.



Σχήμα 7.5 Τμηματοποίηση σταθερού μεγέθους με άνισα τμήματα

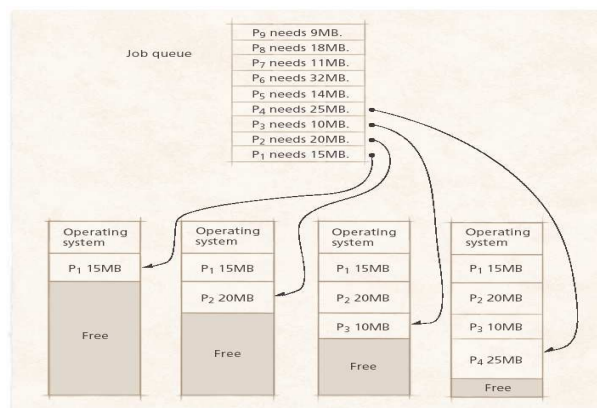
Στην τμηματοποίηση σταθερού μεγέθους υπάρχουν κάποιοι αλγόριθμοι τοποθέτησης που καθορίζουν ποιο τμήμα μνήμης θα χρησιμοποιηθεί. Όταν έχουμε ίσου μεγέθους τμήματα, δεν έχει σημασία ποιο χρησιμοποιείται αφού όλα τα τμήματα είναι πανομοιότυπα. Σε περίπτωση που όλα τα τμήματα είναι κατειλημμένα τότε πραγματοποιείται γίνεται εναλλαγή κάποιες διεργασίες. Στην περίπτωση τμημάτων διαφορετικού μεγέθους, είναι δυνατόν να χρησιμοποιηθεί μια ουρά αναμονής για κάθε τμήμα, ενώ κάθε διεργασία μπορεί να αντιστοιχηθεί στο μικρότερο τμήμα στο οποίο χωρά. Οι διεργασίες αντιστοιχούνται με τρόπο ώστε να ελαχιστοποιείται η σπατάλη μνήμης μέσα σε ένα τμήμα, δηλαδή ο εσωτερικός κατακερματισμός. Εναλλακτικά, μπορεί να υπάρχει μια μοναδική ουρά για όλες τις διεργασίες και όταν η διεργασία πρέπει να φορτωθεί στη μνήμη επιλέγεται το μικρότερο διαθέσιμο τμήμα. Η τελευταία λύση διαθέτει καλύτερη ικανότητα για τη βελτιστοποίηση χρήσης της CPU.



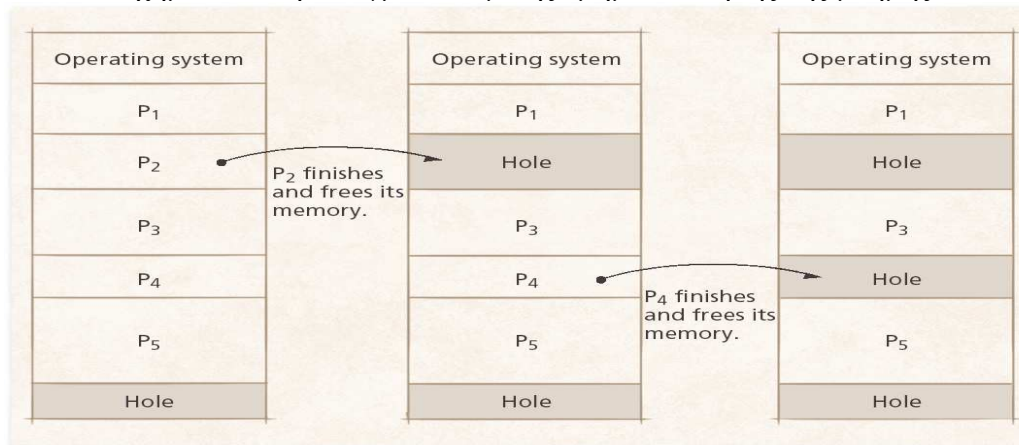
Σχήμα 7.6 Αλγόριθμοι τοποθέτησης σε σταθερά τμήματα άνισου μεγέθους

7.5 Δυναμική τμηματοποίηση

Σε αντίθεση με την τμηματοποίηση σταθερού μεγέθους, η δυναμική τμηματοποίηση χρησιμοποιεί τμήματα μεταβλητού μεγέθους και πλήθους. Με τον τρόπο αυτό, μια διεργασία αντιστοιχείται ακριβώς στην ποσότητα μνήμης που απαιτείται. Ως αποτέλεσμα μπορεί να προκύψουν τελικά κενά στη μνήμη, δηλαδή εξωτερικός κατακερματισμός (external fragmentation). Επομένως πρέπει να χρησιμοποιηθεί συμπίεση (compaction) που θα μετατοπίσει τις διεργασίες έτσι ώστε να είναι συνεχόμενες και όλη η ελεύθερη μνήμη να αποτελεί μια ενότητα (block). Η συμπίεση σπαταλά το χρόνο της CPU και προϋποθέτει τη δυνατότητα δυναμικής μετατόπισης, π.χ. μεταφορά ενός προγράμματος σε άλλη περιοχή μνήμης χωρίς να ακυρώνονται οι αναφορές της μνήμης.



Σχήμα 7.7 Παράδειγμα δυναμικής τμηματοποίησης της μνήμης



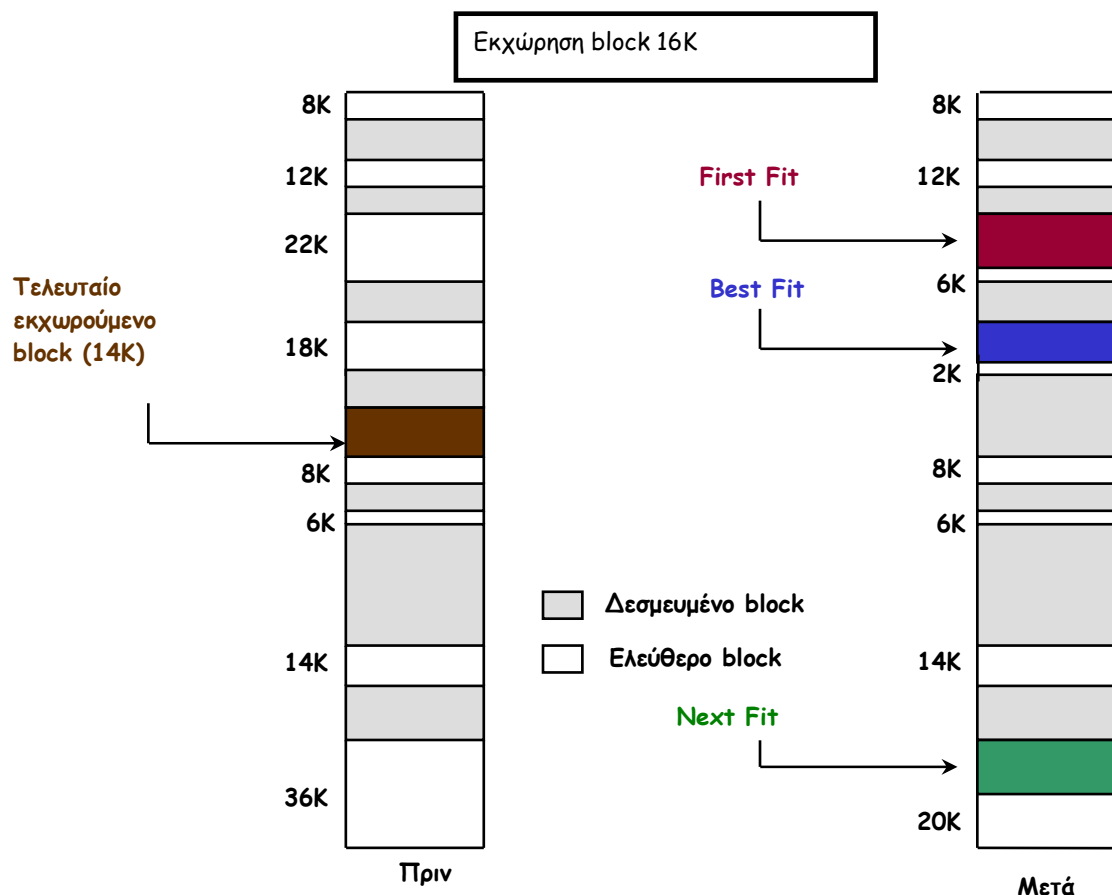
Σχήμα 7.8 Δημιουργία κενών τμημάτων κατά τη δύναμης τμηματοποίησης μνήμης

Στη δυναμική τμηματοποίηση το ΛΣ πρέπει να αποφασίσει ποιο ελεύθερο τμήμα της μνήμης θα εκχωρήσει σε μια διεργασία σύμφωνα με κάποιον από τους τρεις δυνατούς αλγόριθμους τοποθέτησης, που είναι οι εξής:

- Αλγόριθμος καλύτερης τοποθέτησης (best-fit algorithm): ο αλγόριθμος αυτός επιλέγει το τμήμα μνήμης (block) που είναι πλησιέστερα στο μέγεθος που απαιτείται. Ο αλγόριθμος έχει τη χειρότερη απόδοση επειδή βρίσκει το μικρότερο block για τη διεργασία με αποτέλεσμα η κύρια μνήμη γεμίζει γρήγορα από blocks που είναι πολύ μικρά και η συμπίεση τους πραγματοποιείται πιο συχνά.

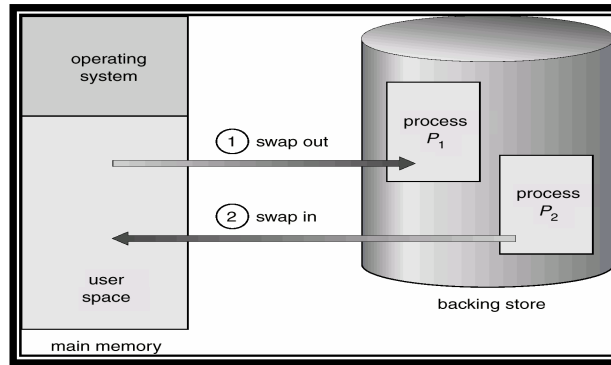
- Αλγόριθμος πρώτης τοποθέτησης (first-fit algorithm): ξεκινά και σαρώνει τη μνήμη από την αρχή και επιλέγει το πρώτο διαθέσιμο μπλοκ που είναι αρκετά μεγάλο. Είναι ο ταχύτερος αλγόριθμός αλλά έχει ως αποτέλεσμα πολλές διεργασίες να φορτώνονται στο εμπρός τμήμα της μνήμης, το οποίο θα πρέπει να εξετάζεται κάθε φορά που γίνεται προσπάθεια για την εύρεση ενός ελεύθερου block.
- Αλγόριθμος επόμενης τοποθέτησης (next-fit algorithm): ο αλγόριθμος ξεκινά να σαρώνει τη μνήμη από την τελευταία τοποθέτηση και επιλέγει το επόμενο αρκετά μεγάλο διαθέσιμο μπλοκ. Όταν φτάσει στο τέλος της μνήμης συνεχίζει, αν απαιτείται, τη σάρωση από την αρχή της μνήμης ώστε να εξετάσει όλα τα ελεύθερα τμήματα μνήμης. Με τον τρόπο αυτό, εκχωρείται συχνά ένα block μνήμης που βρίσκεται στο τέλος της μνήμης, όπου βρίσκεται το μεγαλύτερο block. Σταδιακά το μεγαλύτερο block μνήμης διασπάται σε μικρότερα blocks και απαιτείται συμπίεση για να αποκτηθεί ένα μεγάλο block στο τέλος της μνήμης.

Παράδειγμα χρήσης των αλγορίθμων τοποθέτησης

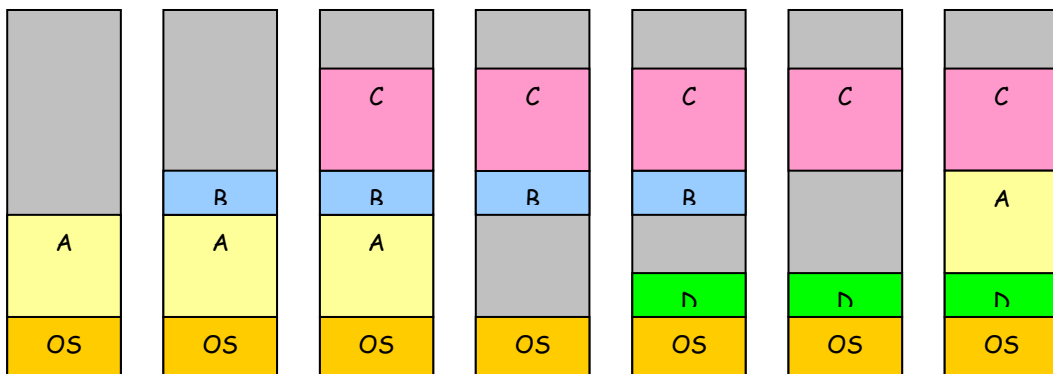


7.6 Εναλλαγή (swapping)

Κατά το χρόνο λειτουργίας ενός υπολογιστικού συστήματος, η εκχώρηση μνήμης που πραγματοποιείται μέσω του λειτουργικού συστήματος μεταβάλλεται καθώς εισάγονται νέες διεργασίες στη μνήμη ενώ κάποιες άλλες εγκαταλείπουν τη μνήμη αφού εναλλάσσονται στον επεξεργαστή και κατά συνέπεια μεταφέρονται στο δίσκο ή ολοκληρώνουν την εκτέλεσή τους.



Σχήμα 7.8 Μεταφορά διεργασιών από το δίσκο στη μνήμη και αντίστροφα



Σχήμα 7.9 Παράδειγμα εναλλαγής – οι γκρι περιοχές είναι κενά τμήματα μνήμης

Παρά τη χρησιμότητά της, η εναλλαγή έχει και περιορισμούς που προκύπτουν από τα προβλήματα που σχετίζονται με το γεγονός ότι η διεργασία πρέπει να χωρά στη φυσική μνήμη και επομένως είναι αδύνατη η εκτέλεση μεγαλύτερων διεργασιών. Η μνήμη κατακερματίζεται και απαιτείται συμπίεση για την επανασυναρμολόγηση μεγαλύτερων ελεύθερων περιοχών. Ακόμα, οι διεργασίες μπορούν να βρίσκονται και στη μνήμη και στο δίσκο. Η μέθοδος των επικαλύψεων επιλύει το πρόβλημα εκτέλεσης μεγάλων διεργασιών κατατάσσοντας τη διεργασία στη διάρκεια του χρόνου (κυρίως τα δεδομένα), χωρίς ωστόσο να επιλύει το πρόβλημα του κατακερματισμού.

Ερωτήσεις Επανάληψης

1. Αναφέρετε και τους τρεις αλγόριθμους τοποθέτησης (δέσμευσης) μνήμης.
2. Περιγράψτε τον αλγόριθμους πρώτης τοποθέτησης (first-fit), βέλτιστης τοποθέτησης (best-fit) και επόμενης τοποθέτησης (next-fit) και αναφέρετε τα χαρακτηριστικά τους (πλεονεκτήματα-μειονεκτήματα).
3. Περιγράψτε την τμηματοποίηση σταθερού μεγέθους.
4. Περιγράψτε τη μέθοδο σταθερής διαίρεσης μνήμης και τα μειονεκτήματά της.
5. Περιγράψτε τη μέθοδο δυναμικής τμηματοποίησης.

Ασκήσεις

1. Δεν αποτελεί βασικό χαρακτηριστικό των διαχειριστών μνήμης
 - α) εκχωρούν πρωτεύουσα μνήμη στις διεργασίες
 - β) αντιστοιχούν το χώρο διευθύνσεων της διεργασίας στην κύρια μνήμη
 - γ) δρομολογούν τις διεργασίες στον επεξεργαστή
 - δ) ελαχιστοποιούν το χρόνο προσπέλασης στη μνήμη

2. Οι στρατηγικές τοποθέτησης καθορίζουν
- α) το σημείο όπου θα τοποθετηθεί το επόμενο τμήμα προγράμματος καθώς μετακινείται από τη δευτερεύουσα μνήμη
 - β) το σημείο της κύριας μνήμης όπου το σύστημα θα μπορούσε να τοποθετήσει τμήματα δεδομένων
 - γ) το σημείο όπου θα τοποθετηθεί το επόμενο τμήμα δεδομένων καθώς μετακινείται από τη δευτερεύουσα μνήμη
 - δ) ποιο τμήμα θα αφαιρεθεί από την κύρια μνήμη στις περιπτώσεις όπου η κύρια μνήμη είναι αρκετά πλήρης ώστε να παρέχει χώρο σε ένα νέο πρόγραμμα

3. Αποτελεί χαρακτηριστικό της χρήσης ίσων τμημάτων:
- α) μεγάλη επιβάρυνση στο λειτουργικό σύστημα
 - β) αυξημένες απαιτήσεις υλοποίησης
 - γ) αποτελεσματική χρησιμοποίηση της κύριας μνήμης
 - δ) δημιουργία εσωτερικού κατακερματισμού

4. Ο αλγόριθμος επόμενης τοποθέτησης:
- α) επιλέγει το τμήμα μνήμης που είναι πλησιέστερα στο μέγεθος που απαιτείται
 - β) ξεκινά και σαρώνει τη μνήμη από την αρχή
 - γ) ξεκινά και σαρώνει τη μνήμη από το τέλος προς την αρχή
 - δ) επιλέγει ένα τμήμα που να είναι αρκετά μεγάλο

5. Ένα σύστημα τοποθετεί διεργασίες στη μνήμη χρησιμοποιώντας δυναμική πολιτική τοποθέτησης. Κατά την πλέον πρόσφατη χρονική στιγμή έγινε φόρτωση μιας διεργασίας που χρειαζόταν 12KB μνήμης και η εικόνα μνήμης του συστήματος διαμορφώθηκε ως εξής :

20		30		12		32		24		48
----	--	----	--	----	--	----	--	----	--	----

Οι σκιασμένες περιοχές δηλώνουν μη διαθέσιμα τμήματα μνήμης, οι λευκές τα διαθέσιμα τμήματα ενώ η περιοχή με μαύρο χρώμα τη θέση όπου έγινε η τελευταία τοποθέτηση. Οι αριθμοί δηλώνουν το μέγεθος σε KB. Για την τοποθέτηση μιας νέας διεργασίας που χρειάζεται 22KB μνήμης, το τμήμα μνήμης που θα χρησιμοποιηθεί σύμφωνα με τον αλγόριθμο first-fit είναι εκείνο με μέγεθος (σε KB):

- α) 20KB
 - β) 30KB
 - γ) 32KB
 - δ) 24KB
6. Στην άσκηση (5), αν χρησιμοποιηθεί ο αλγόριθμος best-fit τότε το τμήμα μνήμης που θα χρησιμοποιηθεί είναι εκείνο με μέγεθος (σε KB):
- α) 12KB
 - β) 30KB
 - γ) 32KB
 - δ) 24KB
7. Στην άσκηση (5), αν χρησιμοποιηθεί ο αλγόριθμος next-fit τότε το τμήμα μνήμης που θα χρησιμοποιηθεί είναι εκείνο με μέγεθος (σε KB):
- α) 30KB
 - β) 48KB

- γ) 24KB
- δ) 32KB

8. Υποθέστε ότι έχετε ελεύθερη μνήμη σε τμήματα μεγέθους 1^ο:150KB, 2^ο:500KB, 3^ο:200KB, 4^ο:300KB, και 5^ο:600KB (με αυτή τη σειρά) και υπάρχουν κατά σειρά απαιτήσεις μνήμης για Α:212KB, Β:417KB, Γ:112KB, και Δ:426KB. Ο αλγόριθμος πρώτης τοποθέτησης (first-fit) θα διευθετήσει την απαίτηση Δ:

- α) η απαίτηση Δ δεν θα ικανοποιηθεί
- β) στο 2^ο τμήμα
- γ) στο 4^ο τμήμα
- δ) στο 5^ο τμήμα

9. Θεωρήσετε τα δεδομένα της άσκησης (4) και ότι χρησιμοποιείται ο αλγόριθμος βέλτιστης τοποθέτησης (best-fit). Η διευθέτηση των απαιτήσεων θα είναι η εξής (αίτηση → τμήμα):

- α) Α→4, Β→5, Γ→1, Δ→2
- β) Α→4, Β→2, Γ→3, Δ→5
- γ) Α→4, Β→2, Γ→1, Δ→5
- δ) Α→5, Β→2, Γ→4, Δ→3

10. Θεωρήσετε τα δεδομένα της άσκησης (4) και ότι χρησιμοποιείται ο αλγόριθμος επόμενης τοποθέτησης (next-fit) με την τελευταία δεσμευμένη να έχει γίνει πριν το μπλοκ 3 (200KB). Η διευθέτηση των απαιτήσεων θα είναι η εξής:

- α) Α→4, Β→5, Γ→1, Δ→2
- β) Α→4, Β→2, Γ→3, Δ→5
- γ) Α→4, Β→2, Γ→1, Δ→5
- δ) Α→5, Β→2, Γ→4, Δ→3

Κεφάλαιο 8 – Ιδεατή Μνήμη

1. Εισαγωγή
2. Ιδεατές και πραγματικές διευθύνσεις
3. Λογική οργάνωση
4. Τμηματοποίηση ιδεατής μνήμης
 1. Σελιδοποίηση
 2. Κατάτμηση

8.1. Εισαγωγή

Η κύρια μνήμη είναι, μετά από το χρόνο χρήσης της CPU, ο δεύτερος πιο σημαντικός πόρος σε ένα υπολογιστικό σύστημα. Ακόμη και με σχετικά μεγάλο μέγεθος η ποσότητα της διαθέσιμης κύριας μνήμης συχνά δεν είναι ικανοποιητική, αφού ένας μεγάλος αριθμός διεργασιών πρέπει να συνυπάρχουν στη μνήμη. Η λήψη πληροφοριών από τον σκληρό δίσκο αντί της κύριας μνήμης καθυστερεί υπέρμετρα το σύστημα επειδή ο χρόνος προσπέλασης της κύριας μνήμης είναι της τάξης των νανοδευτερολέπτων (π.χ. 60 ns) ενώ ο μέσος χρόνος προσπέλασης των σκληρών δίσκων είναι πολλές τάξεις μεγέθους μεγαλύτερος (π.χ. 10 ms, δηλαδή 10×10^6 ns).

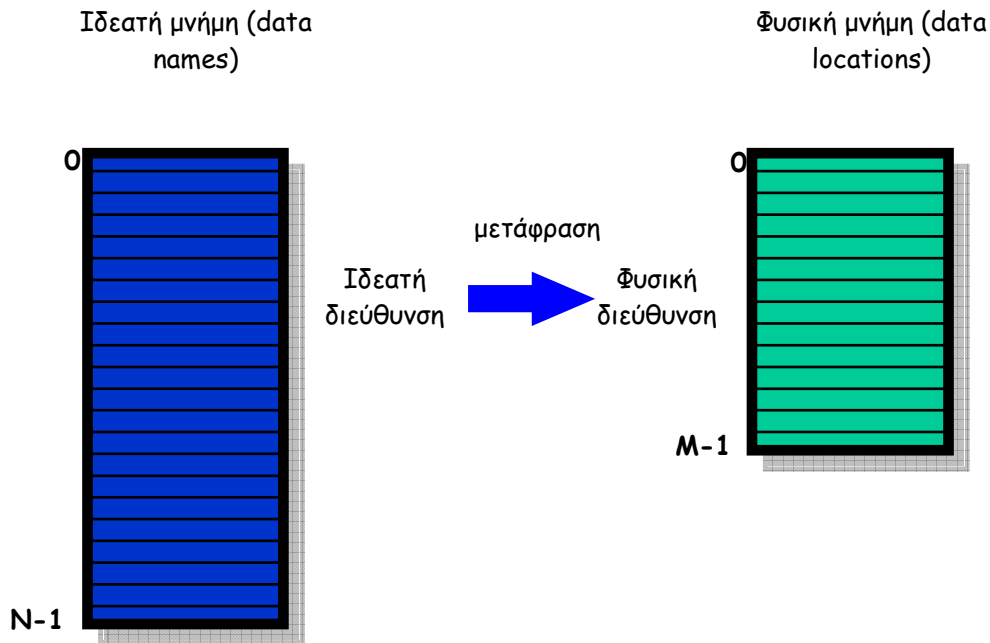
Η διαχείριση μνήμης επιτυγχάνεται μέσω μιας πολύπλοκης σχέσης μεταξύ του υλικού μέρους του επεξεργαστή και του λογισμικού του λειτουργικού συστήματος. Οι βασικές τεχνικές διαχείρισης μνήμης ανταγωνίζονται για τη δέσμευση περιορισμένου χώρου στην κύρια μνήμη. Η λύση της μεγαλύτερης κύριας μνήμης είναι συνήθως απαγορευτικά δαπανηρή. Η δεύτερη λύση είναι η δημιουργία της ψευδαίσθησης ότι υπάρχει περισσότερη μνήμη από όση είναι εγκατεστημένη και αποτελεί τη βασική ιδέα της ιδεατής μνήμης (virtual memory).

8.2. Ιδεατές και πραγματικές διευθύνσεις

Τα συστήματα ιδεατής μνήμης καλύπτουν τις ανάγκες των διεργασιών μέσω της ψευδαίσθησης ότι έχουν στη διάθεσή τους περισσότερη κύρια μνήμη από όση διαθέτει το υπολογιστικό σύστημα. Έτσι υπάρχουν δύο τύποι διευθύνσεων στα συστήματα ιδεατής μνήμης : αυτές στις οποίες αναφέρονται οι διεργασίες και καλούνται ιδεατές ή εικονικές διευθύνσεις (virtual addresses) και αυτές που είναι διαθέσιμες στην κύρια μνήμη και καλούνται φυσικές ή πραγματικές διευθύνσεις (physical ή real addresses)

8.3. Λογική οργάνωση

Η κύρια μνήμη σε ένα υπολογιστικό σύστημα οργανώνεται ως ένας γραμμικός, μονοδιάστατος χώρος διευθύνσεων. Η δευτερεύουσα μνήμη, σε φυσικό επίπεδο, οργανώνεται με παρόμοιο τρόπο. Ακόμα, τα προγράμματα οργανώνονται και γράφονται σε ενότητες (modules). Οι ενότητες αυτές γράφονται και μεταφράζονται ανεξάρτητα και σε αυτές δίνονται διαφορετικοί βαθμοί προστασίας, π.χ. μόνο ανάγνωσης (read-only), μόνο εκτέλεσης (execute-only) κλπ. Επιπρόσθετα, οι ενότητες αυτές μπορούν να διαμοιράζονται μεταξύ των διεργασιών.



Σχήμα 8.1 Εικονική και φυσική μνήμη

Στην περίπτωση του πολυπρογραμματισμού, το λειτουργικό σύστημα πρέπει να διατηρεί ξεχωριστά τη μνήμη για κάθε διεργασία, που προστατεύεται από άλλες που θέλουν να διαβάσουν ή να γράψουν στη δική της περιοχή μνήμης αλλά κι από κάθε τροποποίηση της δικής της μνήμης με ανεπιθύμητο τρόπο (π.χ γράφοντας στο τμήμα κώδικα).

Από την άλλη πλευρά, το λειτουργικό σύστημα πρέπει να επιτρέπει σε πολλές διεργασίες να έχουν πρόσβαση στην ίδια περιοχή της μνήμης. Για παράδειγμα, είναι προτιμότερο να επιτρέπεται η πρόσβαση σε μια διεργασία (σε ένα άτομο) στο ίδιο αντίγραφο του προγράμματος από το να υπάρχει ένα αντίγραφο για κάθε μια διεργασία.

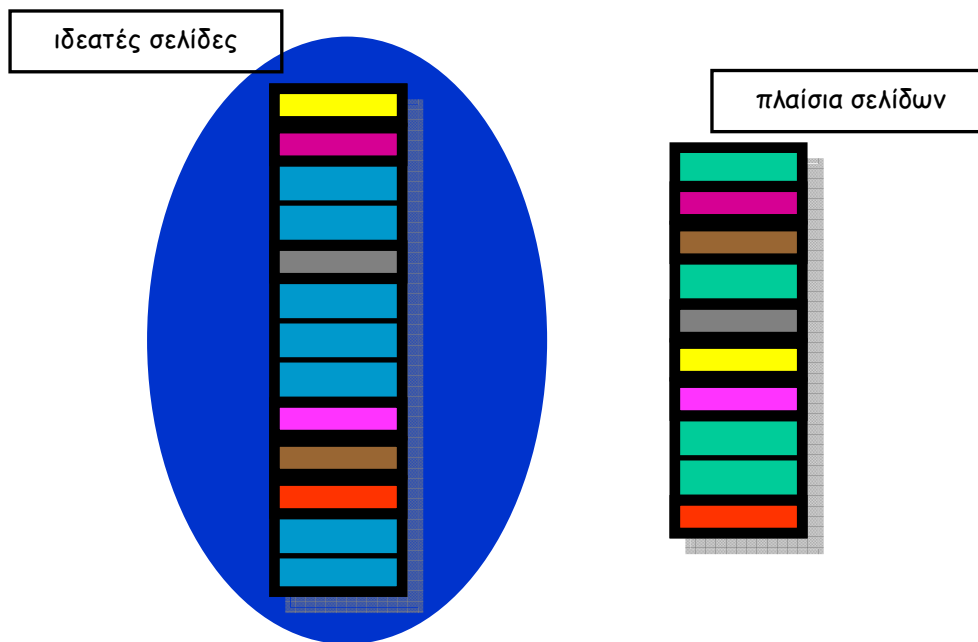
Για να ικανοποιηθούν αυτές τις ανάγκες χρησιμοποιείται ως εργαλείο ή τμηματοποίηση της ιδεατής μνήμης.

8.4. Τμηματοποίηση ιδεατής μνήμης

Δύο βασικές τεχνικές που χρησιμοποιεί η ιδεατή μνήμη είναι η σελιδοποίηση (paging) και η κατάτμηση (segmentation).

8.4.1 Σελιδοποίηση

Σελιδοποίηση είναι ο διαχωρισμός της μνήμης σε μικρά ίσου μεγέθους τμήματα (blocks, chunks) και η διαίρεση κάθε διεργασίας σε τμήματα του ίδιου μεγέθους. Τα τμήματα μιας διεργασίας λέγονται σελίδες (pages) και τα τμήματα της μνήμης πλαίσια (frames). Ο εικονικός χώρος διευθύνσεων διαμοιράζεται σε σελίδες σταθερού μεγέθους, ενώ η φυσική μνήμη διαμοιράζεται σε πλαίσια σελίδας (page frames), μεγέθους ίδιου με τη σελίδα. Σύμφωνα με τη σελιδοποίηση, μια σελίδα μπορεί να τοποθετηθεί σε οποιοδήποτε πλαίσιο σελίδας.



Σχήμα 8.2 Σελίδες και πλαίσια

Η σελιδοποίηση είναι ανάλογη με την τμηματοποίηση σταθερού μεγέθους, με τη διαφορά ότι τα τμήματα μνήμης είναι αρκετά μικρά και δεν χρειάζεται να είναι συνεχόμενα, ενώ ένα πρόγραμμα μπορεί να απασχολεί περισσότερα από ένα τμήματα. Η σπατάλη μνήμης οφείλεται στον εσωτερικό κατακερματισμό που είναι κλάσμα της τελευταίας σελίδας της διεργασίας. Αντίθετα, εξωτερικός κατακερματισμός δεν υπάρχει.

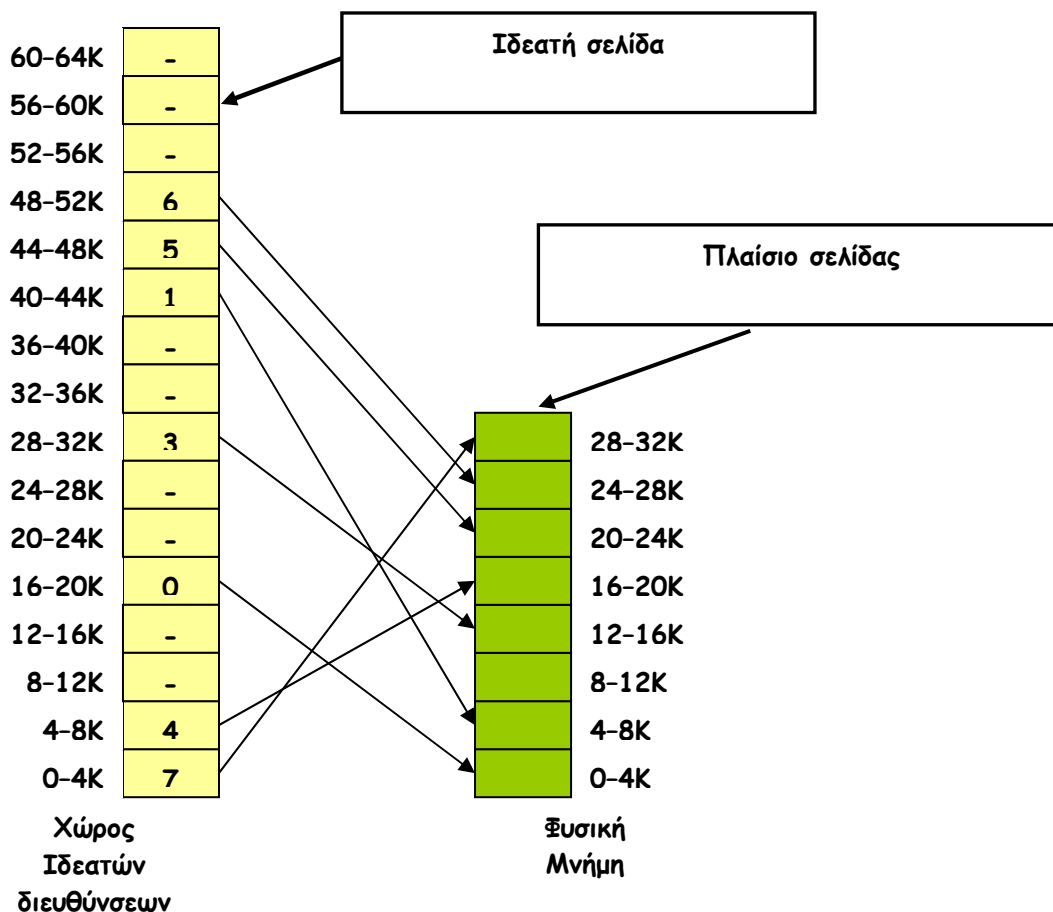
Το μέγεθος σελίδας και πλαισίου είναι δύναμη του 2, και κυμαίνεται συνήθως μεταξύ 512 bytes και 8192 bytes. Τα πλεονεκτήματα από τη χρήση μικρής σελίδας είναι η μείωση του εσωτερικού κατακερματισμού, η επίτευξη καλύτερου ταιριάσματος για διάφορες δομές δεδομένων και τμήματα κώδικα, και η φόρτωση λιγότερου μη χρησιμοποιούμενου προγράμματος στη μνήμη. Το κύριο μειονέκτημα της μικρής σελίδας είναι ότι α προγράμματα χρειάζονται πολλές σελίδες και μεγαλύτερους πίνακες σελίδων.

4.2 Κατάτμηση (segmentation)

Κατάτμηση είναι ο τρόπος οργάνωσης της ιδεατής μνήμης σε τμήματα. Ένα τμήμα (segment) είναι ένα μεταβλητού μεγέθους σύνολο συνεχόμενων διευθύνσεων μνήμης στον ιδεατό χώρο διευθύνσεων μιας διεργασίας που οργανώνεται και διαχειρίζεται από το λειτουργικό σύστημα ως μια ενιαία μονάδα.

Τα τμήματα αυτά δεν είναι ίσα και η κατάτμηση είναι παρόμοια με τη δυναμική τμηματοποίηση. Όπως είναι αναμενόμενο, με την κατάτμηση μειώνεται ο εσωτερικός κατακερματισμός. Επιπλέον, τα τμήματα μπορούν να έχουν δυναμικό μέγεθος ώστε να απλοποιείται η διαχείριση δυναμικών δομών δεδομένων. Η κατάτμηση επιτρέπει στα προγράμματα να τροποποιούνται και να μεταφράζονται εκ νέου ανεξάρτητα και είναι κατάλληλη για διαμοίραση και προστασία δεδομένων.

Κάθε διεύθυνση αποτελείται από δύο μέρη – έναν αριθμό τμήματος και μια μετατόπιση (offset).

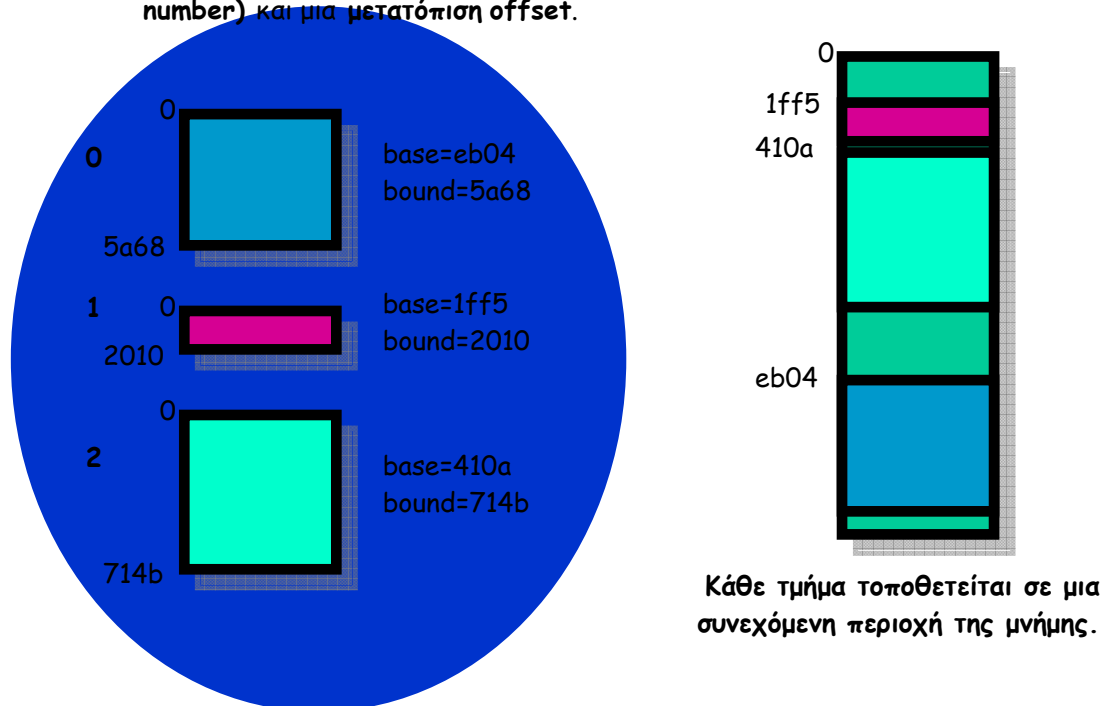


Σχήμα 8.3 Απεικόνιση σελίδων σε πλαίσια

Σε αντίθεση με τη σελιδοποίηση, η κατάτμηση είναι φανερό στον προγραμματιστή και παρέχεται ως διευκόλυνση για την οργάνωση προγραμμάτων και δεδομένων. Ο προγραμματιστής βλέπει το πρόγραμμα σαν συλλογή από τμήματα, χωρίς να υπάρχει μια απλή συσχέτιση μεταξύ των λογικών και των φυσικών διευθύνσεων.

Πλεονέκτημα της κατάτμησης είναι ότι η εικόνα της μνήμης είναι η εικόνα που έχει ο προγραμματιστής. Επίσης, τα τμήματα προστατεύονται μεταξύ τους, με κάθε τμήμα να περιέχει έναν τύπο πληροφορίας. Με την κατάτμηση, η διαμοίραση τμημάτων είναι λογική και εύκολη. Αν όλες οι εντολές είναι σε ένα τμήμα και όλα τα δεδομένα σε κάποιο άλλο, το τμήμα εντολών μπορεί να διαμοιραστεί ελεύθερα σε διαφορετικές διεργασίες, με κάθε διεργασία ωστόσο να διαθέτει τα δικά της δεδομένα.

Μια εικονική διεύθυνση είναι ένας αριθμός τμήματος (**segment number**) και μια μετατόπιση **offset**.



Σχήμα 8.4 Κατάτμηση μνήμης

Ερωτήσεις Επανάληψης

1. Πως οργανώνεται λογικά η μνήμη σε ένα υπολογιστικό σύστημα;
2. Ποιες είναι οι βασικές τεχνικές οργάνωσης της ιδεατής μνήμης;
3. Περιγράψτε τη μέθοδο της σελιδοποίησης.
4. Ποιες είναι τα τυπικά μεγέθη σελίδας και πλαισίου;
5. Περιγράψτε τη μέθοδο της τμηματοποίησης

Ασκήσεις

1. Ποιο από τα παρακάτω είναι αληθές
 - α) η κύρια μνήμη δεν είναι σημαντικός πόρος ενός υπολογιστικού συστήματος
 - β) η ποσότητα της διαθέσιμης κύριας μνήμης συχνά δεν είναι ικανοποιητική
 - γ) ο μέσος χρόνος προσπέλασης των σκληρών δίσκων είναι πολλές τάξεις μεγέθους μεγαλύτερος από τον αντίστοιχο της κύριας μνήμης
 - δ) ο χρόνος προσπέλασης της κύριας μνήμης είναι της τάξης των δευτερολέπτων
2. Η ιδεατή μνήμη
 - α) επεκτείνει την ποσότητα διαθέσιμης κύριας μνήμης
 - β) καθιστά ταχύτερη τη πρόσβαση στην κύρια μνήμη του συστήματος
 - γ) αυξάνει τη χωρητικότητα του σκληρού δίσκου
 - δ) καθιστά ταχύτερη την πρόσβαση στην δευτερεύουσα μνήμη του συστήματος
3. Η τμηματοποίηση της ιδεατής μνήμης
 - α) δεν απαιτείται στην περίπτωση πολυπρογραμματισμού

- β) επιτρέπει την τροποποίηση της μνήμης μεταξύ διεργασιών
- γ) δημιουργεί αντίγραφα προγραμμάτων για κάθε διεργασία χρήστη
- δ) επιτρέπει σε πολλές διεργασίες την πρόσβαση στην ίδια περιοχή μνήμης

4. Στη σελιδοποίηση

- α) κάθε διεργασία διαιρείται σε τμήματα άνισου μεγέθους
- β) τα τμήματα μιας διεργασίας λέγονται πλαίσια
- γ) η μνήμη διαχωρίζεται σε τμήματα ίσου μεγέθους
- δ) η μνήμη χωρίζεται σε σελίδες

5. Η σελιδοποίηση

- α) τοποθετεί μια σελίδα σε συγκεκριμένο πλαίσιο
- β) δεν εμφανίζει εξωτερικό κατακερματισμό
- γ) απαιτεί τα τμήματα μνήμης να είναι συνεχόμενα
- δ) απαιτεί ένα πρόγραμμα να απασχολεί ένα τμήμα

6. Το μέγεθος μια σελίδας

- α) κυμαίνεται μεταξύ 64 bytes και 4096 bytes
- β) κυμαίνεται μεταξύ 512 bytes και 8192 bytes
- γ) κυμαίνεται μεταξύ 256 bytes και 8192 bytes
- δ) κυμαίνεται μεταξύ 512 bytes και 4096 bytes

7. Ποιο από τα παρακάτω δεν είναι αποτελεσιμα της χρήσης μικρής σελίδας

- α) χρήση μικρών πινάκων σελίδων
- β) φόρτωση λιγότερου μη χρησιμοποιούμενου προγράμματος στη μνήμη
- γ) επίτευξη καλύτερου ταιριάσματος
- δ) μείωση του εσωτερικού κατακερματισμού

8. Στη μέθοδο της κατάτμησης, ένα τμήμα είναι ένα

- α) σταθερού μεγέθους σύνολο συνεχόμενων διευθύνσεων ιδεατής μνήμης
- β) μεταβλητού μεγέθους σύνολο συνεχόμενων διευθύνσεων ιδεατής μνήμης
- γ) μεταβλητού μεγέθους σύνολο συνεχόμενων διευθύνσεων κύριας μνήμης
- δ) σταθερού μεγέθους σύνολο μη συνεχόμενων διευθύνσεων ιδεατής μνήμης

9. Στη μέθοδο της κατάτμησης

- α) τα τμήματα έχουν ίσο μέγεθος
- β) διευκολύνεται η διαμοίραση και προστασία δεδομένων
- γ) δημιουργείται εσωτερικός κατακερματισμός.
- δ) γίνεται πολύπλοκη η διαχείριση δυναμικών δομών δεδομένων

10. Δεν αποτελεί πλεονέκτημα της κατάτμησης

- α) η κατάτμηση είναι φανερό στον προγραμματιστή
- β) τα τμήματα προστατεύονται μεταξύ τους
- γ) όλα τα τμήματα περιέχουν τον ίδιο τύπο πληροφορίας
- δ) η διαμοίραση των τμημάτων είναι λογική και εύκολη

Κεφάλαιο 9 – Δρομολόγηση Διεργασιών

1. Εισαγωγή
2. Κριτήρια αποτίμησης της απόδοσης
3. Κριτήρια βελτιστοποίησης
4. Τύποι δρομολόγησης του επεξεργαστή
5. Πολιτικές δρομολόγησης
6. Αλγόριθμοι Δρομολόγησης
 1. First Come First Served (FCFS)
 2. Shortest Job First (SJF)
 3. Shortest Remaining Time First (SRTF)
 4. Round Robin (RR)
7. Σύγκριση αλγορίθμων δρομολόγησης

9.1. Εισαγωγή

Το κεφάλαιο αυτό μελετάει τη δρομολόγηση διεργασιών στην περίπτωση υπολογιστικών συστημάτων ενός επεξεργαστή. Μερικοί στόχοι της δρομολόγησης είναι το υψηλό ποσοστό χρήσης του επεξεργαστή (CPU utilization), η επίτευξη υψηλής ρυθμοαπόδοσης (High throughput), που εκφράζεται με το πλήθος των διεργασιών που ολοκληρώνονται στη μονάδα του χρόνου και ο μικρός χρόνος απόκρισης (response time) των διεργασιών, που είναι ο χρόνος που μεσολαβεί από την υποβολή μιας απαίτησης μέχρι την έναρξη της απόκρισης του συστήματος. Μολονότι οι στόχοι της δρομολόγησης σημαίνουν την επίτευξη υψηλών επιδόσεων για ένα υπολογιστικό σύστημα, μπορεί να είναι μεταξύ τους και αλληλοσυγκρουόμενοι.

9.2. Κριτήρια αποτίμησης της απόδοσης

Τα κριτήρια με τα οποία αποτιμάται η απόδοση ενός συστήματος και επομένως η αποτελεσματικότητα της δρομολόγησης διεργασιών από το λειτουργικό σύστημα είναι τα ακόλουθα:

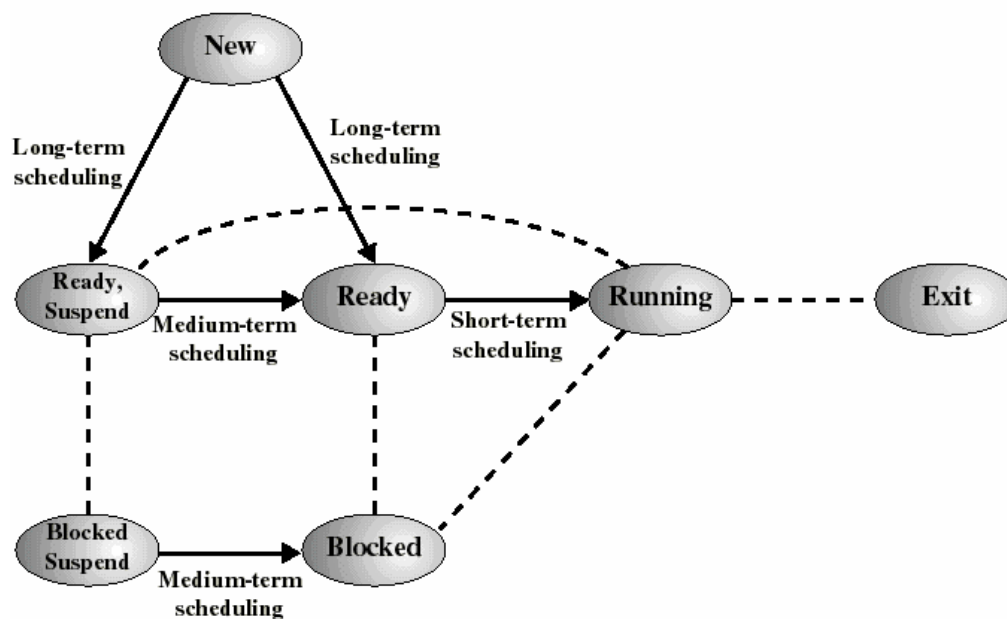
- Δικαιοσύνη (fairness): αναφέρεται συχνή χρήση της CPU από κάθε διεργασία.
- Χρησιμοποίηση (utilization): το ποσοστό του χρόνου κατά το οποίο μια συσκευή χρησιμοποιείται, όπως εκφράζεται από το πηλίκο του χρόνου χρήσης προς το συνολικό χρόνο.
- Ρυθμοαπόδοση (throughput): ο αριθμός των εργασιών που ολοκληρώνονται σε μια χρονική περίοδο (εργασίες / second).
- Χρόνος επιστροφής (turnaround time): ο χρόνος εκτέλεσης μιας διεργασίας, από την αρχή μέχρι την ολοκλήρωσή της.
- Χρόνος αναμονής (waiting time): ο χρόνος κατά τον οποίο μια διεργασία βρίσκεται σε μια ουρά αναμονής έτοιμων διεργασιών (seconds).
- Χρόνος απόκρισης (response time): ο χρόνος από τότε που μια απαίτηση υποβάλλεται μέχρι τη στιγμή που παράγεται η πρώτη απόκριση και όχι τη στιγμή εξόδου.
- Εναλλαγές πλαισίων (context switches): αντιπροσωπεύει χρόνο που σπαταλάται διατηρώντας τον επεξεργαστή άεργο.
- Πολυπλοκότητα του αλγορίθμου δρομολόγησης: χρόνος που απαιτείται για την επιλογή της επόμενης διεργασίας

9.3. Κριτήρια βελτιστοποίησης

Τα κριτήρια βελτιστοποίησης που καλείται να ικανοποιήσει η δρομολόγηση διεργασιών είναι καταρχάς η μεγιστοποίηση της χρήσης του επεξεργαστή και της ρυθμοαπόδοσης του συστήματος. Ταυτόχρονα όμως, βασικά κριτήρια βελτιστοποίησης αποτελεί η ελαχιστοποίηση των χρόνων επιστροφής, αναμονής και απόκρισης των διεργασιών. Όπως θα δούμε και στη συνέχεια, τα κριτήρια αυτά είναι συχνά αλληλοσυγκρουόμενα

9.4. Τύποι δρομολόγησης του επεξεργαστή

Υπάρχουν τρεις κατηγορίες δρομολόγησης του επεξεργαστή και είναι η μακροπρόθεσμη, η μεσοπρόθεσμη και η βραχυπρόθεσμη. Η μακροπρόθεσμη (long-term) δρομολόγηση καθορίζει ποια διεργασία θα γίνει αποδεκτή για επεξεργασία από το σύστημα. Η μεσοπρόθεσμη (medium-term) δρομολόγηση αποφασίζει ποια διεργασία θα προστεθεί στις διεργασίες που βρίσκονται ολόκληρες ή εν μέρει στην κύρια μνήμη. Τέλος, η βραχυπρόθεσμη (short-term) δρομολόγηση επιλέγει ποια διαθέσιμη διεργασία θα εκτελεστεί στον επεξεργαστή.



Σχήμα 9.1 Τύπου δρομολόγησης του επεξεργαστή

Αναλυτικότερα, η μακροπρόθεσμη δρομολόγηση καθορίζει ποια προγράμματα θα γίνουν αποδεκτά για επεξεργασία από το σύστημα, ελέγχοντας με τον τρόπο αυτό το βαθμό πολυπρογραμματισμού του συστήματος. Είναι φανερό πως αν γίνουν αποδεκτές περισσότερες διεργασίες, τότε λιγότερες διεργασίες θα ανασταλούν και θα υπάρξει καλύτερη χρήση της CPU, αν και κάθε διεργασία θα λάβει λιγότερο ποσοστό χρόνου της CPU. Ο μακροπρόθεσμος δρομολογητής προσπαθεί να διατηρήσει μια ισορροπία ανάμεσα στις προοριζόμενες για τον επεξεργαστή διεργασίες (processor-bound) και στις προοριζόμενες για λειτουργίες I/O διεργασίες (I/O-bound). Στην μεσοπρόθεσμη δρομολόγηση, οι αποφάσεις για εναλλαγή των διεργασιών βασίζονται στην ανάγκη της διαχείρισης του πολυπρογραμματισμού.

Η μεσοπρόθεσμη δρομολόγηση πραγματοποιείται από το λογισμικό του λειτουργικού συστήματος που είναι υπεύθυνο για τη διαχείρισης μνήμης.

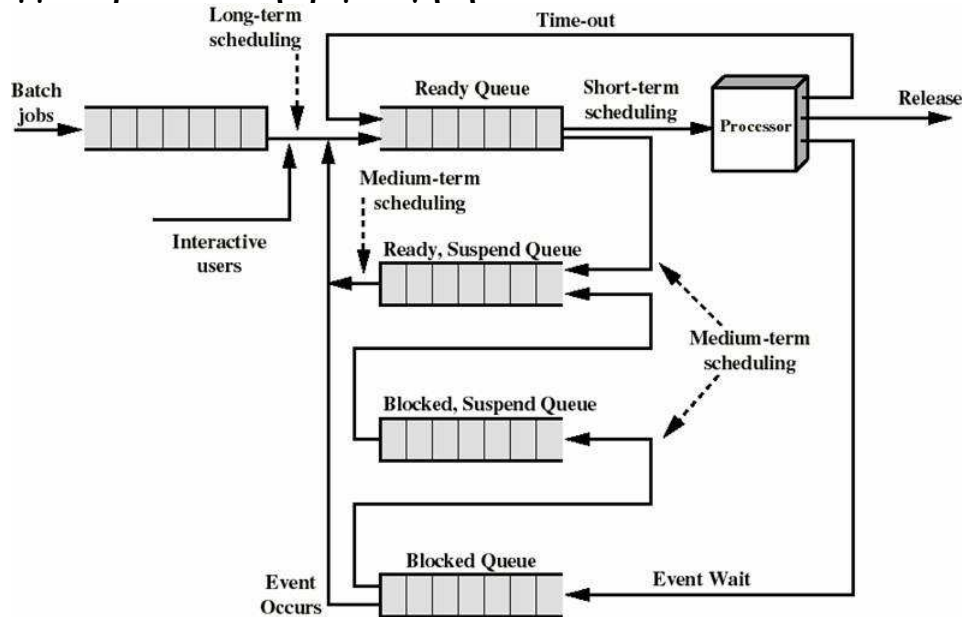
Η βραχυπρόθεσμη δρομολόγηση αποφασίζει ποια διεργασία πρόκειται να εκτελεστεί ως επόμενη. Αναφέρεται συνήθως ως CPU scheduling, και είναι γνωστή και ως διεκπεραίωση, ενώ ο βραχυπρόθεσμος δρομολογητής λέγεται διεκπεραιωτής (dispatcher). Η βραχυπρόθεσμη δρομολόγηση ενεργοποιείται από ένα γεγονός που μπορεί να οδηγήσει στην επιλογή μιας άλλης διεργασίας για εκτέλεση. Τέτοια γεγονότα είναι οι διακοπές ρολογιού, οι διακοπές I/O, οι κλήσεις του λειτουργικού συστήματος και τα σήματα.

Στην περίπτωση της βραχυπρόθεσμης δρομολόγησης, τα κριτήρια βελτιστοποίησης διακρίνονται σε εκείνα που είναι προσανατολισμένα στο χρήστη και περιλαμβάνουν το χρόνο απόκρισης και το χρόνος επιστροφής, και σε εκείνα που είναι προσανατολισμένα στο σύστημα και είναι η χρησιμοποίηση επεξεργαστή, η δικαιοσύνη και η ρυθμοαπόδοση.

Ο έλεγχος του επεξεργαστή δίνεται στη διεργασία που έχει επιλεγεί από τον βραχυπρόθεσμο δρομολογητή (διεκπεραιωτή). Η εκχώρηση αυτή περιλαμβάνει την εναλλαγή πλαισίου, την εναλλαγή σε κατάσταση χρήστη και τη μεταβίβαση στην κατάλληλη διεύθυνση του προγράμματος του χρήστη για την εκκίνηση του προγράμματος. Λανθάνουσα κατάσταση διεκπεραιωτή ή καθυστέρηση διεκπεραίωσης (dispatch latency) είναι ο χρόνος που χρειάζεται ο διεκπεραιωτής για να σταματήσει μια διεργασία και να αρχίσει την εκτέλεση μιας άλλης

Η δρομολόγηση διεργασιών περιλαμβάνει τη χρήση προτεραιοτήτων, που εφαρμόζονται με την ύπαρξη πολλαπλών ουρών έτοιμων διεργασιών, με την κάθε ουρά αντιπροσωπεύει ένα επίπεδο προτεραιότητας. Ο δρομολογητής επιλέγει πάντοτε μια διεργασία υψηλότερης προτεραιότητας έναντι μιας με μικρότερη προτεραιότητα. Μια διεργασία χαμηλής προτεραιότητας μπορεί να υποφέρει από παρατεταμένη στέρηση, ενώ επιτρέπεται σε μια διεργασία να αλλάζει την προτεραιότητά της, ανάλογα με το διάστημα που βρίσκεται στο σύστημα ή με το ιστορικό εκτέλεσής της.

Διάγραμμα ουρών κατά τη δρομολόγηση



Σχήμα 9.2. Διάγραμμα ουρών κατά τη δρομολόγηση

Ο κύκλος καταιγισμού (burst cycle) επεξεργαστή-I/O χαρακτηρίζει την εκτέλεση της διεργασίας, που εναλλάσσεται μεταξύ των δραστηριοτήτων της CPU και των I/O. Οι χρόνοι του επεξεργαστή είναι γενικά πολύ μικρότεροι των χρόνων για I/O. Οι διεργασίες απαιτούν εναλλασσόμενη χρήση του επεξεργαστή και των μονάδων, με επαναλαμβανόμενο τρόπο. Κάθε κύκλος αποτελείται από ένα καταιγισμό επεξεργαστή (CPU burst), διάρκειας συνήθως μερικών εκατοστών του δευτερολέπτου (msecs), ακολουθούμενο από ένα (συνήθως μεγαλύτερο) καταιγισμό I/O. Είναι προφανές ότι μια διεργασία τερματίζεται κατά τη διάρκεια ενός καταιγισμού επεξεργαστή, ενώ δηλαδή εκτελείται. Οι προοριζόμενες για τον επεξεργαστή διεργασίες έχουν μεγαλύτερους καταιγισμούς επεξεργαστή CPU από εκείνων όσων προορίζονται για I/O.

9.5. Πολιτικές δρομολόγησης

Μια πολιτική δρομολόγησης είναι ουσιαστικά μια συνάρτηση επιλογής που αποφασίζει ποια διεργασία στην ουρά των έτοιμων διεργασιών επιλέγεται ως επόμενη για εκτέλεση. Η κατάσταση απόφασης: καθορίζει τα στιγμιότυπα του χρόνου στα οποία εξετάζεται η συνάρτηση επιλογής. Στην περίπτωση δρομολόγησης χωρίς προεκχώρηση (Non-preemption), κάθε φορά που μια διεργασία βρίσκεται σε εκτελούμενη κατάσταση, συνεχίζει να εκτελείται μέχρι να τερματιστεί ή να ανασταλεί από μόνη της περιμένοντας για την ολοκλήρωση I/O. Αντίθετα, όταν έχουμε προεκχώρηση (Preemption), η τρέχουσα εκτελούμενη διεργασία μπορεί να έχει διακοπεί και να έχει μετακινηθεί σε κατάσταση Ready από το λειτουργικό σύστημα. Η προεκχώρηση συντελεί στην καλύτερη εξυπηρέτηση μια και κάθε διεργασία δεν θα μονοπωλεί τη χρήση του επεξεργαστή για μεγάλο χρονικό διάστημα.

Ο δρομολογητής του επεξεργαστή επιλέγει από τις έτοιμες διεργασίες που βρίσκονται στην κύρια μνήμη και αποτελεί βασικό συστατικό του λειτουργικού συστήματος. Είναι από μόνος του μια διεργασία που χρησιμοποιεί τους πόρους του συστήματος, ειδικότερα το χρόνο του επεξεργαστή και τη μνήμη, χωρίς ωστόσο να καταναλώνει σημαντικό χρόνο αφού διαφορετικά η επιβράδυνση στην εκτέλεση των διεργασιών θα είναι μεγάλη.

Παράγοντες που επιδρούν στη δρομολόγηση είναι αν η διεργασία είναι CPU-bound ή I/O-bound, αν είναι αλληλεπιδραστική ή batch, η προτεραιότητα που έχει κάθε διεργασία, ο χρόνος που έχει εκτελεστεί και ο χρόνος που απαιτείται για να ολοκληρωθεί. Επίσης, σημαντικό ρόλο παίζει η συχνότητα προεκχώρησης και η συχνότητα εμφάνισης σφαλμάτων σελίδας. Ανεξάρτητα από τον αλγόριθμο δρομολόγησης που χρησιμοποιείται, δεν είναι δυνατόν να ωφεληθεί μια κατηγορία διεργασιών χωρίς να υπάρξει επίπτωση στις υπόλοιπες. Για παράδειγμα, μια μικρή βελτίωση για τις μικρές διεργασίες (π.χ. στον χρόνο αναμονής) προκαλεί δυσανάλογη επιβράδυνση στις μεγάλες διεργασίες.

9.6. Αλγόριθμοι Δρομολόγησης

Οι αλγόριθμοι δρομολόγησης καθορίζουν τον τρόπο επιλογής της επόμενης προς εκτέλεση διεργασίας καθώς και τη σειρά με την οποία εκτελούνται οι διεργασίες. Ουσιαστικά ορίζουν τις ενέργειες εκ μέρους του δρομολογητή και χωρίζονται σε προεκχωρούμενους και μη προεκχωρούμενους αλγόριθμους. Παραδείγματα αλγορίθμων δρομολόγησης είναι οι FCFS, SJF, SRTF, priority-based, round robin, και οι multilevel multilevel feedback.

Στην περίπτωση ενός μη προεκχωρούμενου αλγόριθμου δρομολόγησης, μια διεργασία παραμένει στον επεξεργαστή μέχρι να απελευθερώσει από μόνη της τον

επεξεργαστή. Κατά συνέπεια, προκύπτουν μεγάλοι χρόνοι αναμονής και απόκρισης και υπάρχει το ενδεχόμενο παρατεταμένης στέρησης. Η υλοποίηση μη προεκχωρούμενων αλγορίθμων δρομολόγησης είναι απλή και εύκολη αλλά δεν είναι κατάλληλη για πολυχρηστικά συστήματα.

Στους προεκχωρούμενους αλγόριθμους δρομολόγησης, η εκτέλεση μιας διεργασίας μπορεί να διακόπτεται από το λειτουργικό σύστημα οποιαδήποτε στιγμή. Πιθανές αιτίες διακοπής της εκτέλεσης είναι η άφιξη μιας νέας διεργασίας με υψηλότερη προτεραιότητα, η πρόκληση μιας διακοπής, η αλλαγή της κατάστασης μιας διεργασίας και η υπέρβαση ενός χρονικού ορίου. Με τον τρόπο αυτό εμποδίζεται η μονοπώληση της χρήσης του επεξεργαστή από μία διεργασία αλλά μπορεί να οδηγήσει σε συνθήκες ανταγωνισμού, οι οποίοι επιλύονται χρησιμοποιώντας συγχρονισμό μεταξύ των διεργασιών.

Παραδείγματα αλγορίθμων δρομολόγησης, τα οποία θα μελετήσουμε αναλυτικότερα στη συνέχεια, είναι τα εξής:

- FCFS (First Come First Served): πρώτη ήλθε πρώτη εξυπηρετήθηκε
- SJF ή SPN (Shortest Job First ή Shortest Process Next) : η συντομότερη διεργασία εκτελείται πρώτη.
- Shortest-Remaining-Time-First (SRTF): η διεργασία με το μικρότερο εναπομείναντα χρόνο εκτελείται πρώτη.
- RR (Round Robin): εξυπηρέτηση εκ περιτροπής

9.6.1. First-Come, First-Served (FCFS)

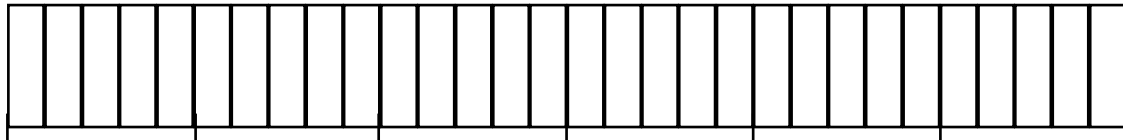
Οι βασικές αρχές του αλγόριθμου FCFS είναι ότι οι διεργασίες εξυπηρετούνται με τη σειρά που φθάνουν και ακόμη και αν φθάνουν την ίδια χρονική στιγμή, η σειρά άφιξης είναι διακριτή, αλλιώς η διεργασία που θα εξυπηρετηθεί κατά προτεραιότητα επιλέγεται τυχαία. Ο αλγόριθμος δεν περιλαμβάνει προεκχώρηση, που σημαίνει ότι κάθε διεργασία εκτελείται μέχρι να ανασταλεί από μόνη της. Πρόκειται για έναν πολύ απλό αλγόριθμο που υλοποιείται εύκολα αλλά είναι ακατάλληλος για συστήματα πολλών χρηστών.

Ο αλγόριθμος FCFS χρησιμοποιεί μια ουρά FIFO και η επιλογή της επόμενης διεργασίας είναι ταχύτατη και ανεξάρτητη από το πλήθος των διεργασιών στην ουρά των έτοιμων διεργασιών. Συχνά προκύπτουν μεγάλοι χρόνοι αναμονής και απόκρισης. Μια διεργασία που δεν χρησιμοποιεί I/O θα μονοπωλεί τη χρήση του επεξεργαστή, οπότε ευνοούνται οι προοριζόμενες για τη CPU διεργασίες ενώ οι προοριζόμενες για I/O διεργασίες θα περιμένουν μέχρι να ολοκληρωθούν όλες οι προηγούμενες. Η αναμονή αυτή θα ισχύει ακόμη και αν οι I/O λειτουργίες τους έχουν ολοκληρωθεί. Κατά συνέπεια, στον αλγόριθμο FCFS υπάρχουν μεγάλες διακυμάνσεις στον μέσο χρόνο επιστροφής των διεργασιών και αυτό ακριβώς δικαιολογεί την ακαταλληλότητά του για αλληλεπιδραστικά συστήματα.

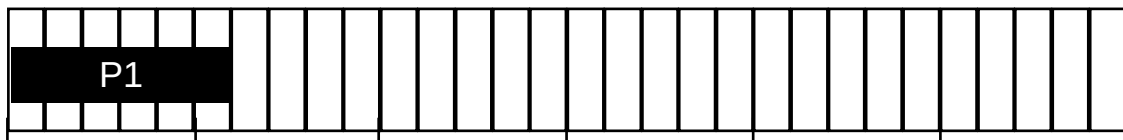
Παράδειγμα αλγορίθμου FCFS

Στοιχεία εκτέλεσης διεργασιών και αρχική κατάσταση

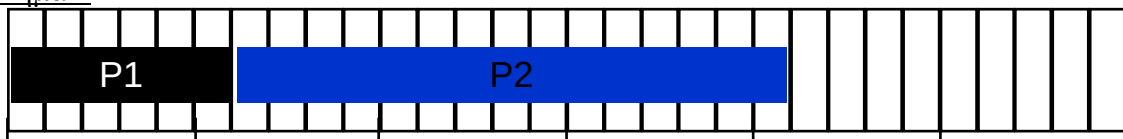
Process #	Arrival Time	Burst Length	Priority
P1	0	6	1
P2	0	15	1
P3	0	3	1
P4	0	4	1
P5	0	2	1



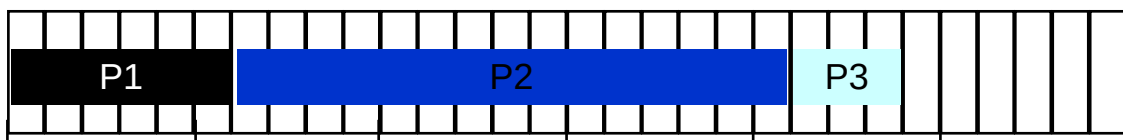
Βήμα 1



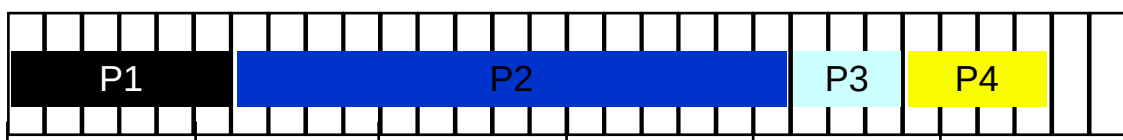
Βήμα 2



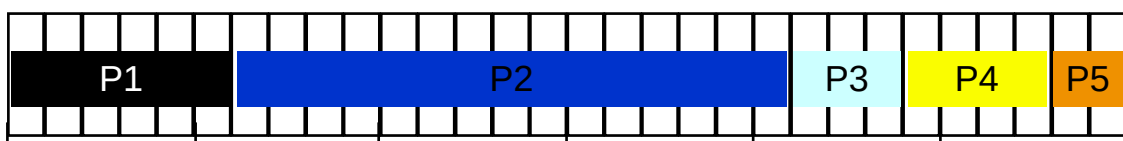
Βήμα 3



Βήμα 4



Βήμα 5



Τελικά αποτελέσματα

<i>Process #</i>	<i>Waiting Time</i>	<i>Response Time</i>	<i>Turnaround Time</i>	<i>#of Context Switches</i>
P1	0	0	6	1
P2	6	6	21	1
P3	21	21	24	1
P4	24	24	28	1
P5	28	28	30	1
Average	79/5 = 15.8	79/5 = 15.8	21.8	1

9.6.2. Shortest-Job-First (SJF)

Σύμφωνα με τον αλγόριθμο δρομολόγησης SJF, κάθε διεργασία δηλώνει το χρόνο καταιγισμού της στην CPU. Υπάρχουν δύο σχήματα του αλγορίθμου ανάλογα με τη χρήση προεκχώρησης ή μη. Στην περίπτωση μη προεκχώρησης, αν εκχωρηθεί ο επεξεργαστής τότε η διεργασία δεν προεκχωρείται μέχρι να ολοκληρωθεί ο καταιγισμός της. Στην περίπτωση της προεκχώρησης, αν υπάρξει μια νέα διεργασία με χρόνο καταιγισμού της CPU μικρότερο από τον χρόνο που απομένει στη τρέχουσα διεργασία τότε την αντικαθιστά. Η περίπτωση με προεκχώρηση είναι γνωστή και ως ο αλγόριθμος: Shortest-Remaining-Time-First (SRTF)., δηλαδή «η διεργασία με το μικρότερο εναπομείναντα χρόνο πρώτη».

Ο αλγόριθμος SJF αποτελεί η βέλτιστη λύση που δίνει τον ελάχιστο μέσο χρόνο αναμονής για ένα δεδομένο σύνολο διεργασιών. Επιλέγεται η διεργασία με το μικρότερο χρόνο καταιγισμού στη CPU, ενσωματώνονται αναμφίβολα προτεραιότητες αφού οι συντομότερες διεργασίες έχουν δεδομένη προτεραιότητα. Ο αλγόριθμος SJF αποτελεί μια προφανή βελτίωση του αλγορίθμου FCFS, αλλά απαιτείται ο υπολογισμός του χρόνου καταιγισμού (CPU burst time) για κάθε διεργασία.

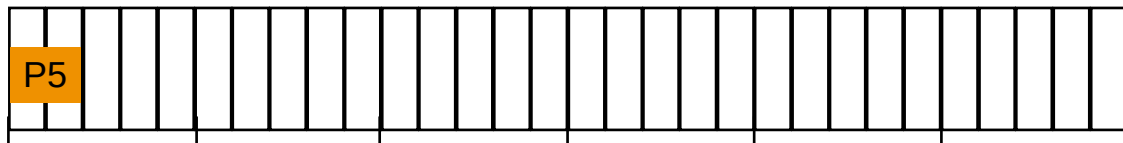
Ο αλγόριθμος SJF δίνει πολύ καλύτερο μέσο χρόνο αναμονής σε σχέση με τον αλγόριθμο FCFS. Χωρίς προεκχώρηση ωστόσο η έλλειψη προεκχώρησης δεν είναι κατάλληλη σε ένα περιβάλλον καταμερισμού χρόνου. Απαιτείται η γνώση των χρόνων καταιγισμού στη CPU που γενικά είναι δύσκολο και συνήθως ακατόρθωτο ενώ και η επιλογή είναι περισσότερο σύνθετη από αυτήν του FCFS. Τέλος ο αλγόριθμος SJF είναι πιθανό να οδηγήσει σε παρατεταμένη στέρηση, αφού στην περίπτωση που νέες μικρής διάρκειας διεργασίας φθάνουν στο σύστημα τότε οι προγενέστερες, μεγάλης διάρκειας διεργασίες, δεν θα εξυπηρετηθούν ποτέ.

Παράδειγμα αλγορίθμου SJF (1)

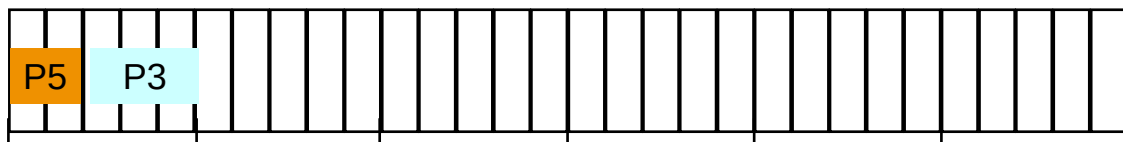
Στοιχεία εκτέλεσης διεργασιών

<i>Process #</i>	<i>Arrival Time</i>	<i>Burst Length</i>	<i>Priority</i>
P1	0	6	1
P2	0	15	1
P3	0	3	1
P4	0	4	1
P5	0	2	1

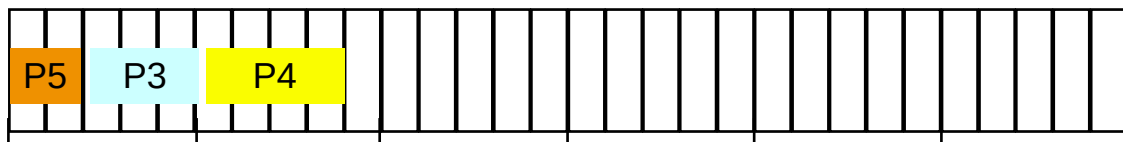
Βήμα 1



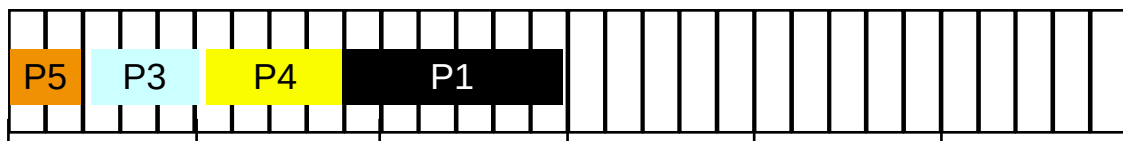
Βήμα 2



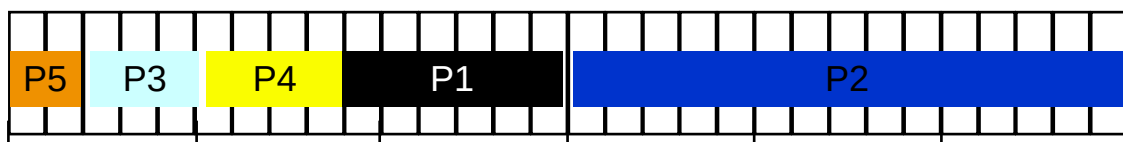
Βήμα 3



Βήμα 4



Βήμα 5



Τελικά αποτελέσματα

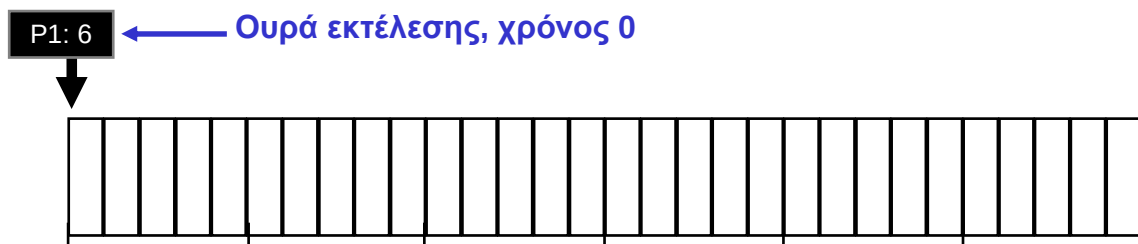
<i>Process #</i>	<i>Waiting Time</i>	<i>Response Time</i>	<i>Turnaround Time</i>	<i>#of Context Switches</i>
P1	9	9	15	1
P2	15	15	30	1
P3	2	2	6	1
P4	5	5	9	1
P5	0	0	2	1
Average	$31/5 = 6.2$	$31/5 = 6.2$	$62/5 = 12.4$	1

Παράδειγμα αλγορίθμου SJF (2)

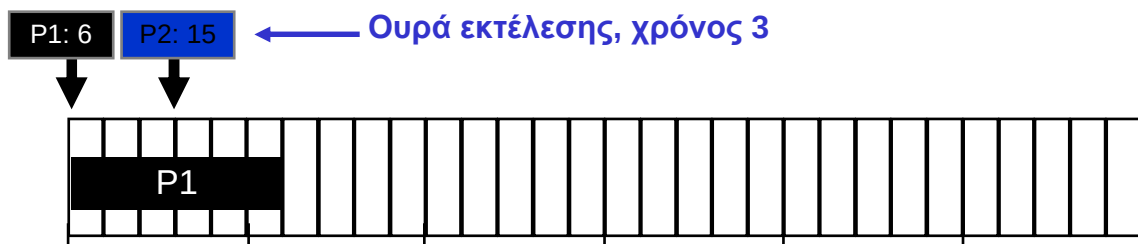
Στοιχεία εκτέλεσης διεργασιών και αρχική κατάσταση

Process #	Arrival Time	Burst Length	Priority
P1	0	6	1
P2	3	15	1
P3	5	3	1
P4	8	4	1
P5	14	2	1

Βήμα 1



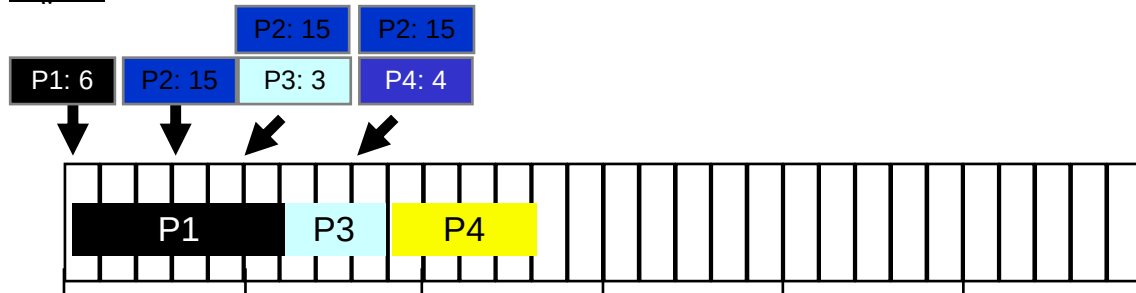
Βήμα 2



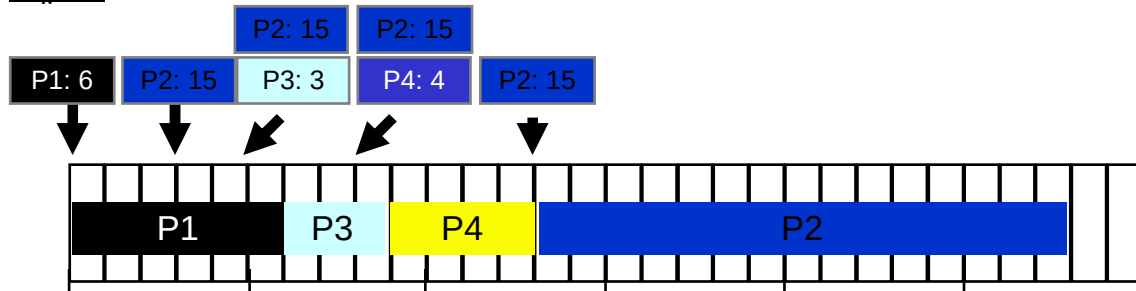
Βήμα 3



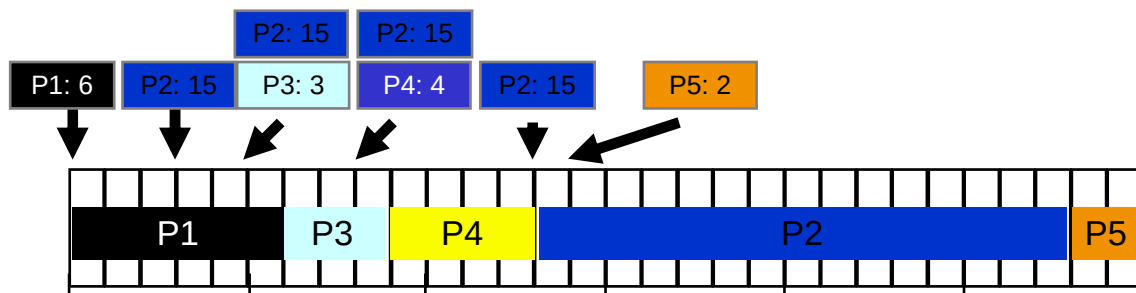
Βήμα 4



Βήμα 5



Βήμα 6



Τελικά αποτελέσματα

Process #	Waiting Time	Response Time	Turnaround Time	#of Context Switches
P1	0	0	6	1
P2	$13-3 = 10$	$13-3 = 10$	$28-3 = 25$	1
P3	$6-5 = 1$	$6-5 = 1$	$9-5 = 4$	1
P4	$9-8 = 1$	$9-8 = 1$	$13-8 = 5$	1
P5	$28-14 = 14$	$28-14 = 14$	$30-14 = 16$	1
Average	$26/5 = 5.2$	$26/5 = 5.2$	$50/5 = 10$	1

9.6.3. Shortest Remaining Time First (SRTF)

Οι βασικές αρχές του αλγορίθμου SRTF είναι ότι επιλέγεται η διεργασία με το μικρότερο εναπομείναντα χρόνο. Ο χρόνος που απομένει είναι ο συνολικός χρόνος καταιγισμού μείον το χρόνο που παρέμεινε προς εξυπηρέτηση η διεργασία στη CPU. Αν φθάσει μια διεργασία με μικρότερο χρόνο καταιγισμού από τον υπολειπόμενο χρόνο

καταιγισμού της τρέχουσας διεργασίας, η τρέχουσα διεργασία προεκχωρείται. Δεν είναι πρακτική εξ αιτίας της αναγκαστικής πρόβλεψης που απαιτείται για τους χρόνους καταιγισμού.

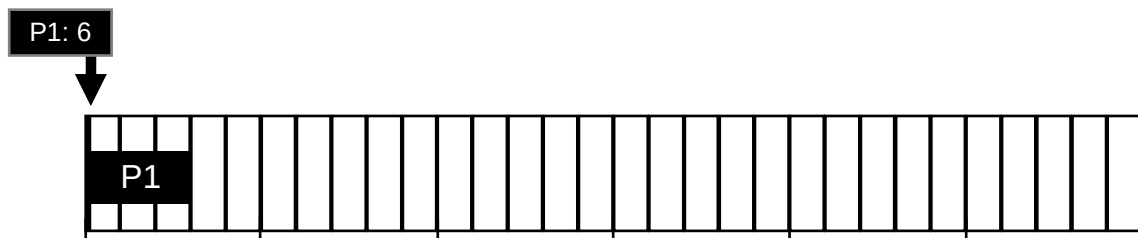
Ο χρόνος άφιξης μιας νέας διεργασίας είναι σημαντικός και είναι χρήσιμη η καταγραφή των διεργασιών που βρίσκονται στην ουρά των έτοιμων διεργασιών, στην οποία οδηγούνται και οι προεκχωρούμενες διεργασίες σύμφωνα με την πολιτική του αλγορίθμου. Η απόφαση για δρομολόγηση λαμβάνεται όταν μια διεργασία έχει ολοκληρώσει το χρόνο καταιγισμού της στη CPU ή μια νέα διεργασία φθάνει στην ουρά των έτοιμων διεργασιών. Ο αλγόριθμος SRTF δίνει καλούς χρόνους απόκρισης στις διεργασίες μικρής διάρκειας γι' αυτό προτιμάται στα συστήματα πολλών χρηστών. Αν ληφθούν υπόψη οι εναλλαγές πλαισίων τότε η χρονική επιβάρυνση του αλγορίθμου είναι σημαντική, ενώ η παρατεταμένη στέρηση είναι πιθανή.

Παράδειγμα αλγορίθμου SRTF

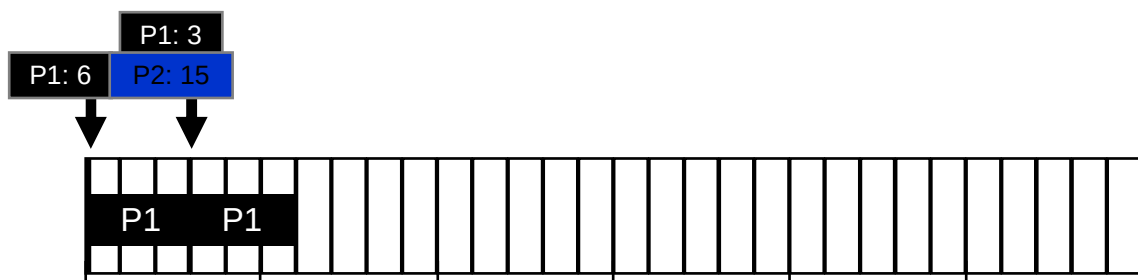
Στοιχεία εκτέλεσης διεργασιών

<i>Process #</i>	<i>Arrival Time</i>	<i>Burst Length</i>	<i>Priority</i>
P1	0	6	1
P2	3	15	1
P3	9	3	1
P4	14	4	1
P5	17	2	1

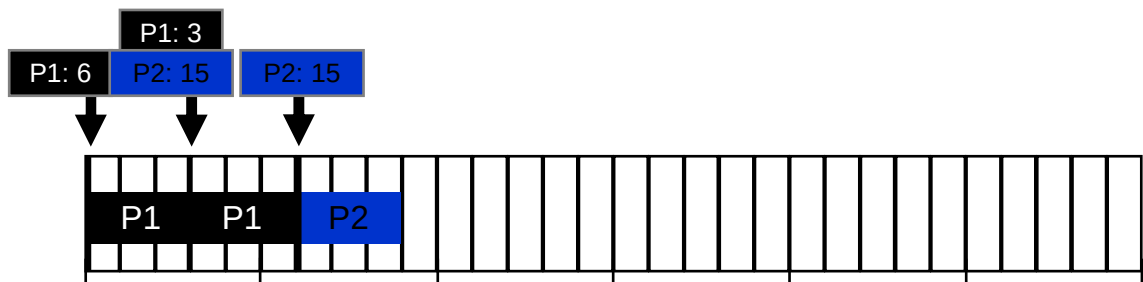
Βήμα 1



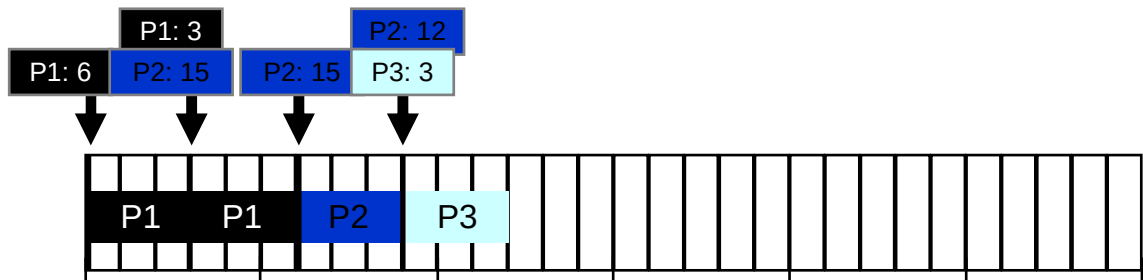
Βήμα 2



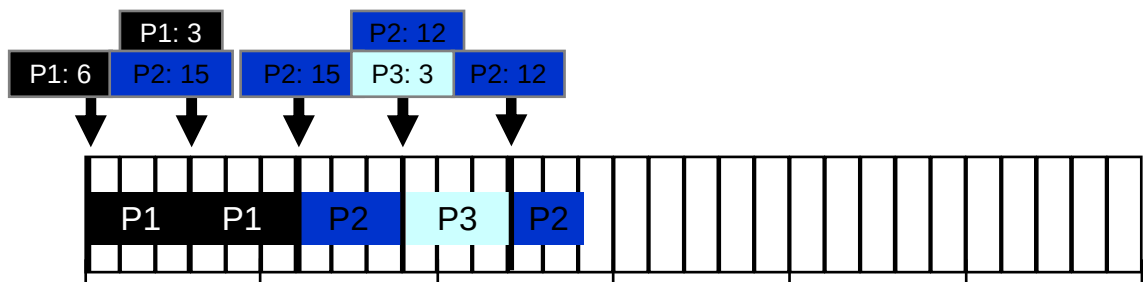
Βήμα 3



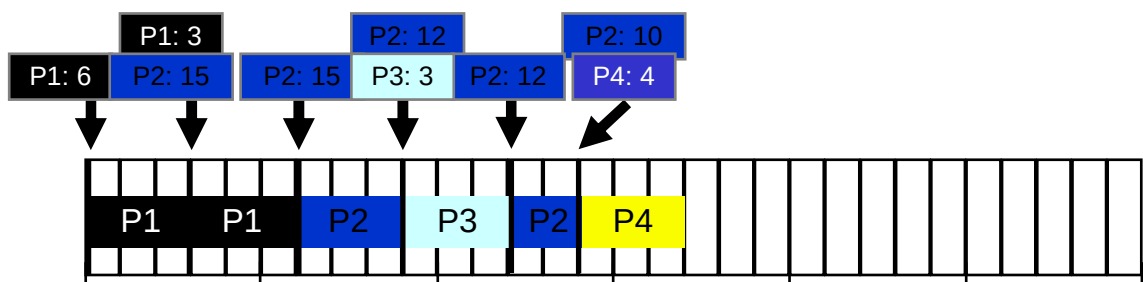
Βήμα 4



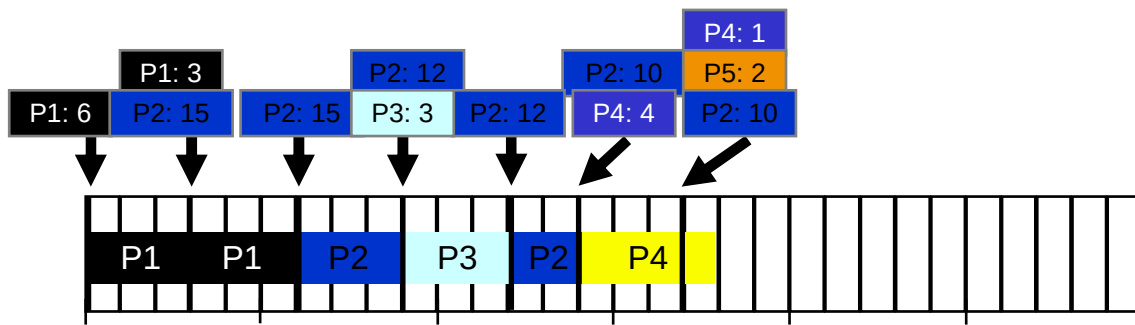
Βήμα 5



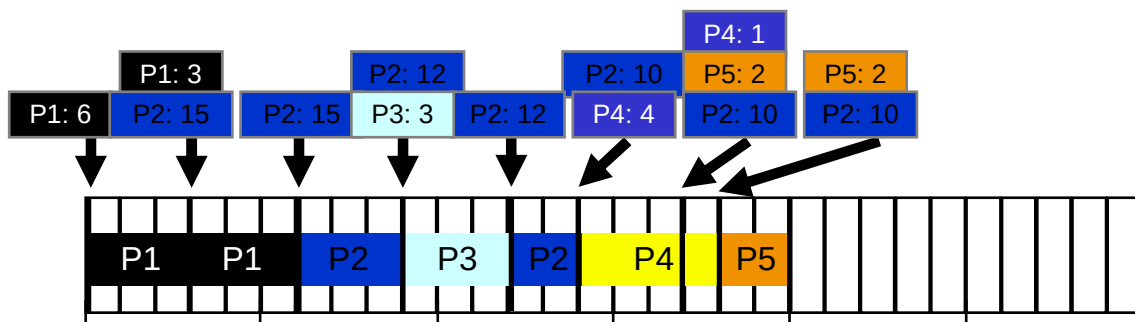
Βήμα 6



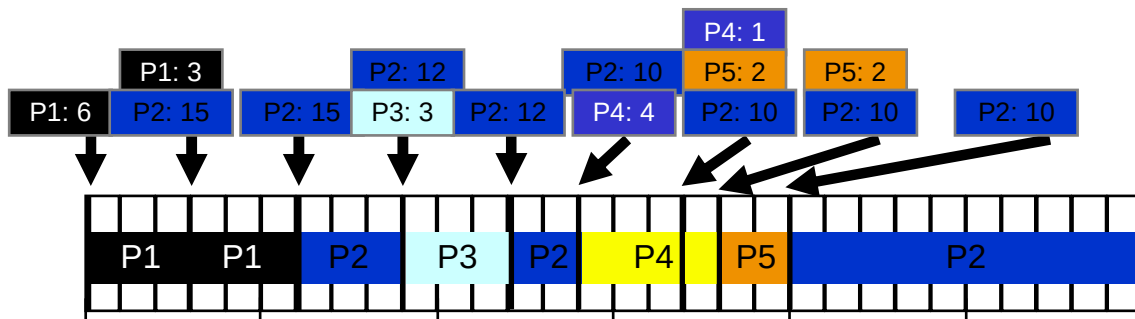
Βήμα 7



Βήμα 8



Βήμα 9



Τελικά αποτελέσματα

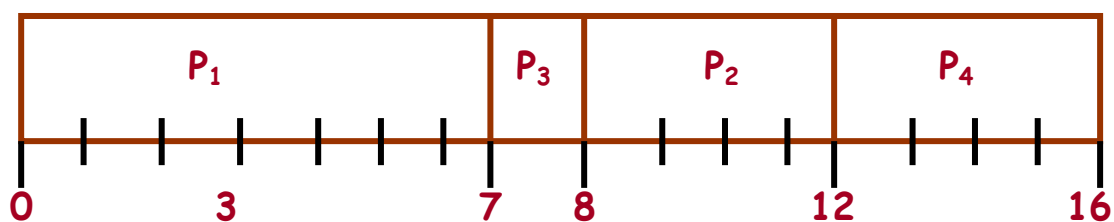
Process #	Waiting Time	Response Time	Turnaround Time	#of Context Switches
P1	0	0	6	1 (2)
P2	$(6-3) + (12-9) + (20-14) = 12$	$(6-3) = 12$	$(30 - 3) = 27$	3
P3	0	0	$(12-9) = 3$	1
P4	0	0	$(18-14) = 3$	1 (2)
P5	$(18-17) = 1$	$(18-17) = 1$	$(20-17) = 3$	1
Average	$13/5 = 2.6$	$13/5 = 2.6$	$36/5 = 7.2$	7

Παράδειγμα σύγκρισης αλγορίθμων SJF και SRTF

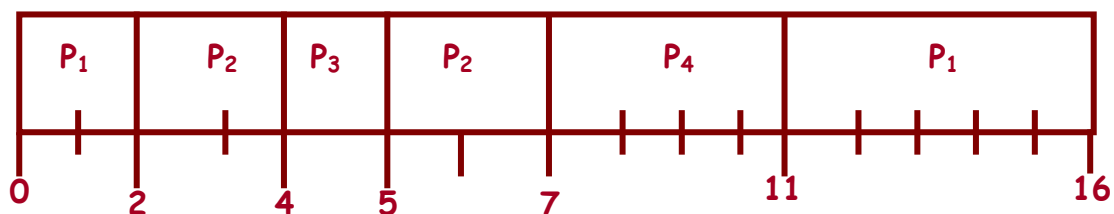
Στοιχεία εκτέλεσης διεργασιών

Διεργασία	Χρόνος άφιξης	Χρόνος καταιγισμού
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Αλγόριθμος SJF: Μέσος χρόνος αναμονής = $(0 + 6 + 3 + 7)/4 = 4$



Αλγόριθμος SRTF: Μέσος χρόνος αναμονής = $(9 + 1 + 0 + 2)/4 = 3$



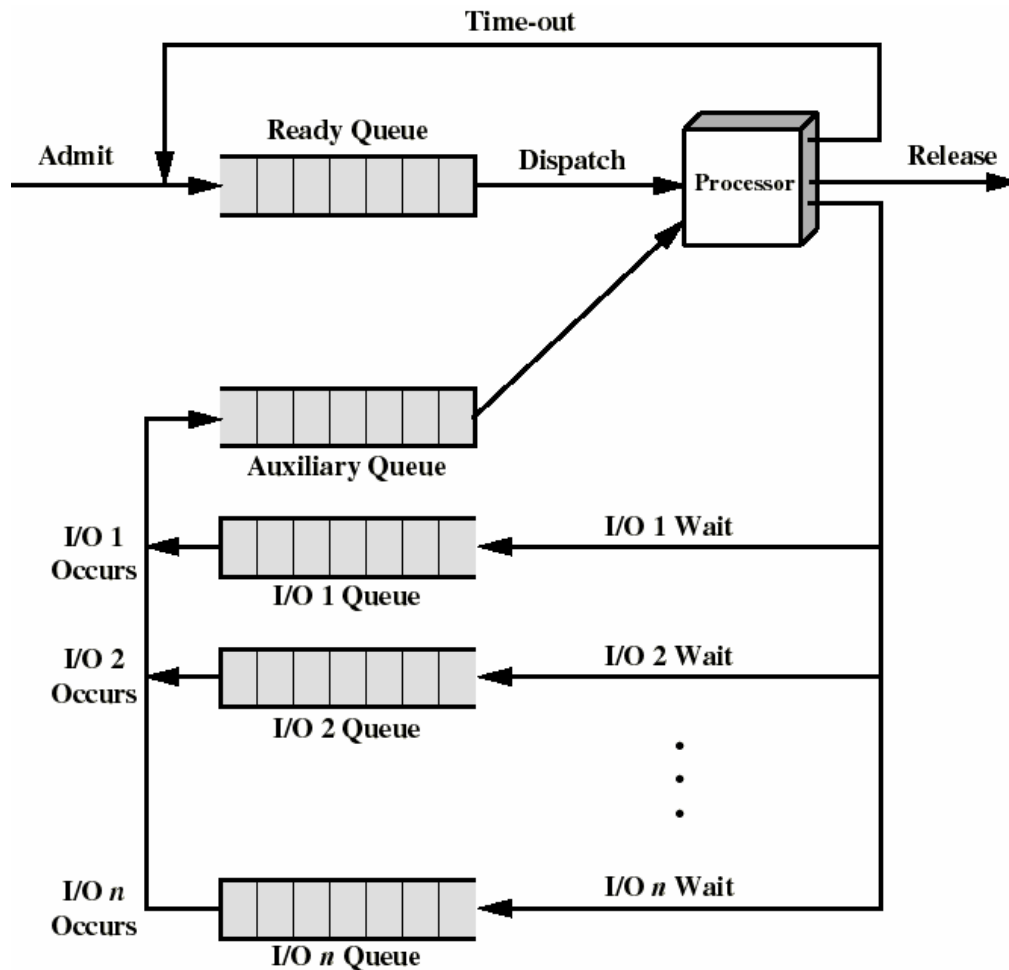
9.6.4. Αλγόριθμος Round Robin

Ο αλγόριθμος δρομολόγησης Round Robin (RR) δίνει στις διεργασίες ένα σταθερό ποσό χρόνου της CPU και αναφέρεται ως κβάντο χρόνου (time quantum), time slice, slot. Η σειρά των διεργασιών είναι συνήθως FIFO και η συνάρτηση επιλογής: ίδια με τον FCFS. Ωστόσο, λόγω της προεκχώρησης, μια διεργασία επιτρέπεται να εκτελείται μέχρι να συμπληρωθεί το κβάντο χρόνου (συνήθως 10 έως 100 ms) και στη συνέχεια δημιουργείται μια διακοπή ρολογιού και η εκτελούμενη διεργασία τίθεται στην ουρά των έτοιμων διεργασιών. Ο συγκεκριμένος αλγόριθμος και παραλλαγές του χρησιμοποιούνται συχνά σε περιβάλλοντα καταμερισμού χρόνου.

Ο αλγόριθμος RR ευνοεί τις προοριζόμενες στην CPU διεργασίες αφού μια προοριζόμενη για I/O διεργασία χρησιμοποιεί την CPU για χρονικό διάστημα μικρότερο του κβάντου χρόνου και στη συνέχεια αναστέλλεται περιμένοντας για I/O. Αντίθετα, μια προοριζόμενη για την CPU διεργασία εκτελείται για όλο το κβάντο χρόνου και τίθεται μετά στην ουρά των έτοιμων διεργασιών, μπροστά από τις διεργασίες που έχουν ανασταλεί.

Για ιδεατή λύση για τον αλγόριθμο RR είναι όταν μια I/O ολοκληρώνεται, η ανασταλμένη διεργασία να μετακινείται σε μια βοηθητική ουρά που προτιμάται έναντι της βασικής ουράς των έτοιμων διεργασιών. Μια διεργασία που αποστέλλεται από την

βοηθητική ουρά εκτελείται όχι περισσότερο από το βασικό κβάντο χρόνου μείον το συνολικό χρόνο που δαπανήθηκε για εκτέλεση



Σχήμα 9.3. Διάγραμμα ουράς για ιδεατό Round Robin

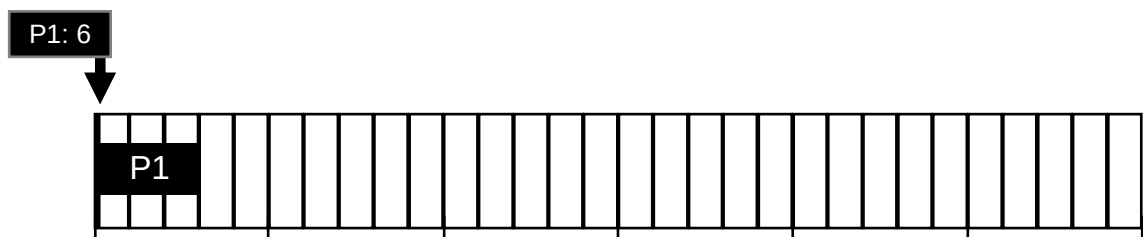
Πλεονεκτήματα του αλγορίθμου RR είναι η απλότητα, οι χαμηλές επιβαρύνσεις του (overhead) και το γεγονός ότι λειτουργεί αποτελεσματικά για αλληλεπιδραστικά συστήματα. Αντίθετα, αν το κβάντο χρόνου είναι πολύ μικρό, δαπανάται πολύς χρόνος για εναλλαγή πλαισίων και αν είναι πολύ μεγάλο προσεγγίζει τον αλγόριθμο FCFS. Τυπικές τιμές για το κβάντο είναι μεταξύ 10 και 100 msec, σύμφωνα με το γενικό κανόνα ότι η επιλογή του πρέπει να είναι τέτοια έτσι ώστε η μεγάλη πλειοψηφία (80-90%) των διεργασιών να ολοκληρώνουν τον καταιγισμό τους σε ένα κβάντο και ελαφρώς μεγαλύτερο από το χρόνο που απαιτεί μια τυπική αλληλεπίδραση. Πέρα από το μέγεθος, άλλες επιλογές για το κβάντο χρόνου αφορούν στο αν θα είναι σταθερό ή μεταβλητό καθώς και αν θα είναι το ίδιο για όλες τις διεργασίες ή διαφορετικό.

Παράδειγμα αλγορίθμου RR, με κβάντο ίσο με 3 χρονικές μονάδες

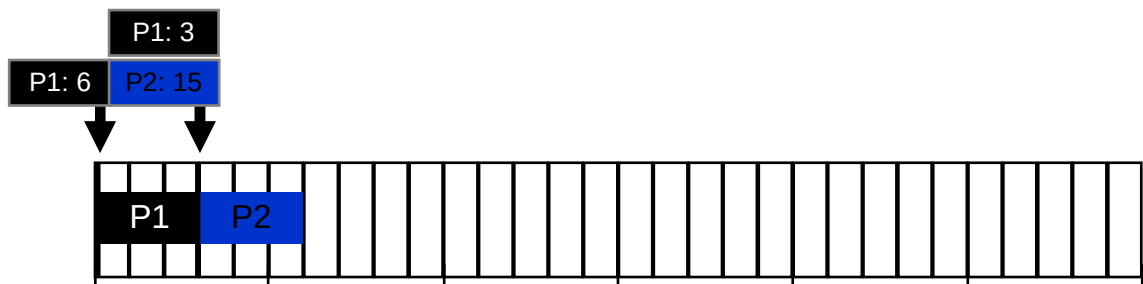
Στοιχεία εκτέλεσης διεργασιών

Process #	Arrival Time	Burst Length	Priority
P1	0	6	1
P2	3	15	1
P3	9	3	1
P4	14	4	1
P5	17	2	1

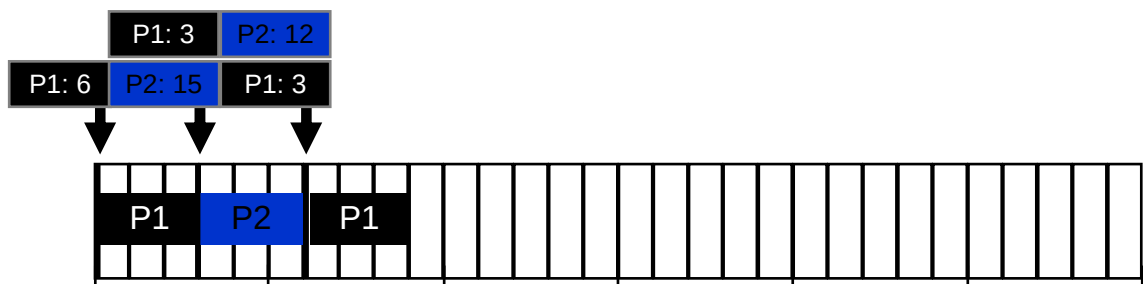
Βήμα 1



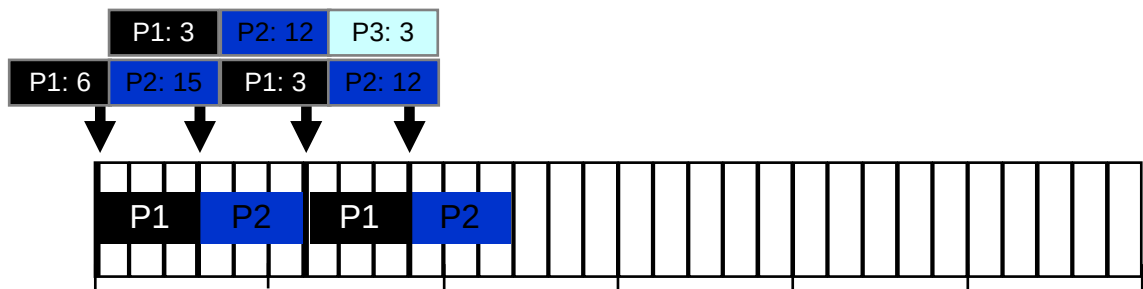
Βήμα 2



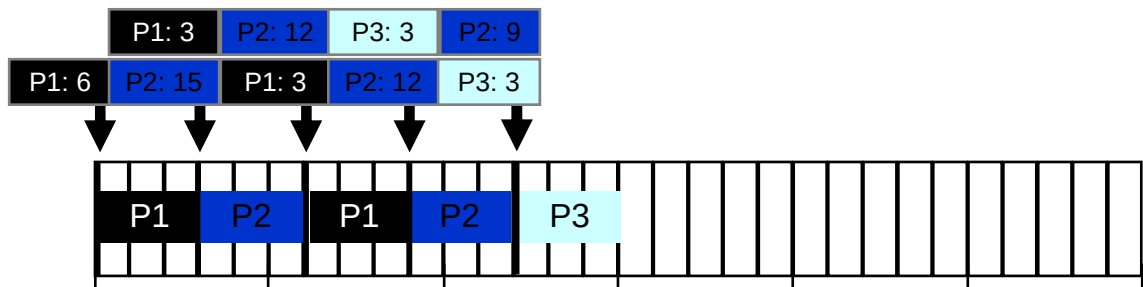
Βήμα 3



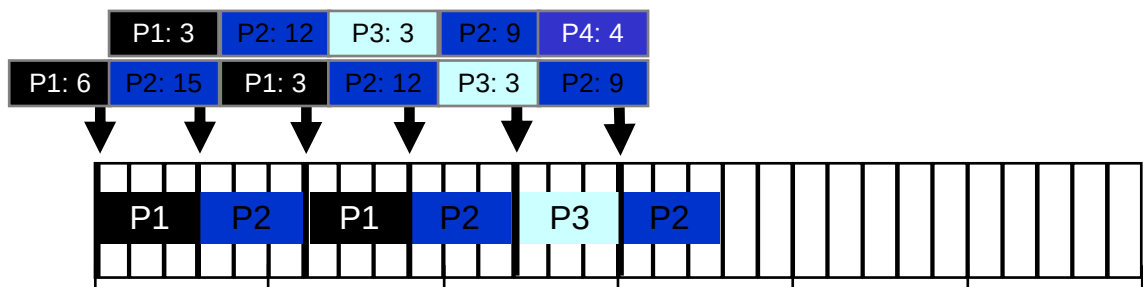
Βήμα 4



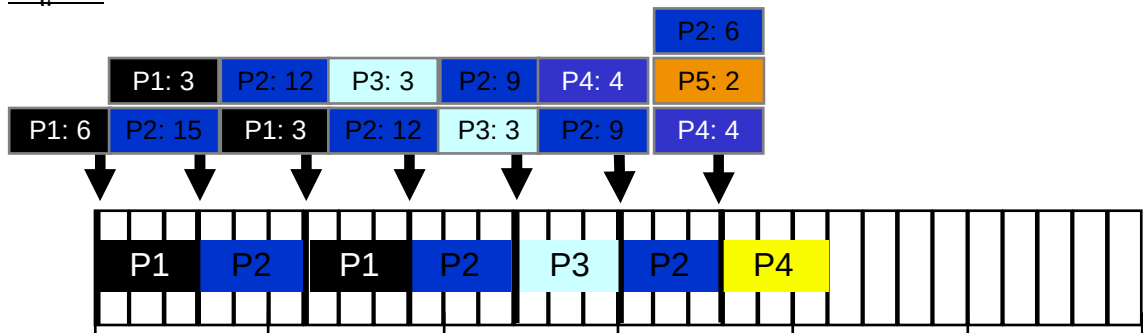
Βήμα 5



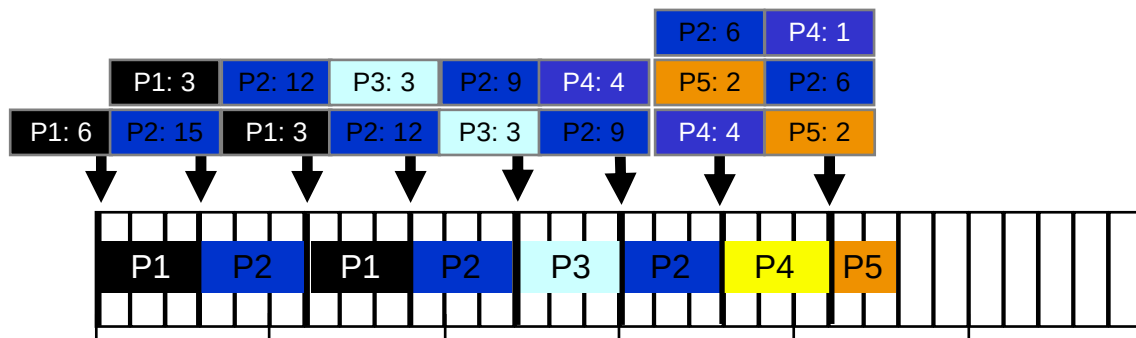
Βήμα 6



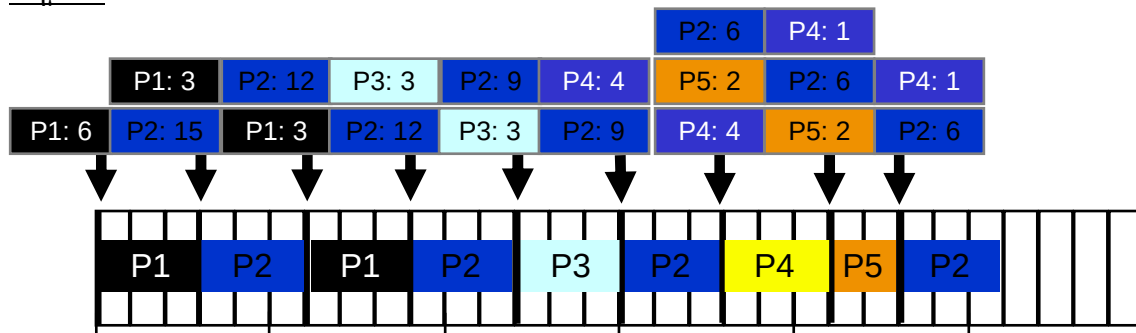
Βήμα 7



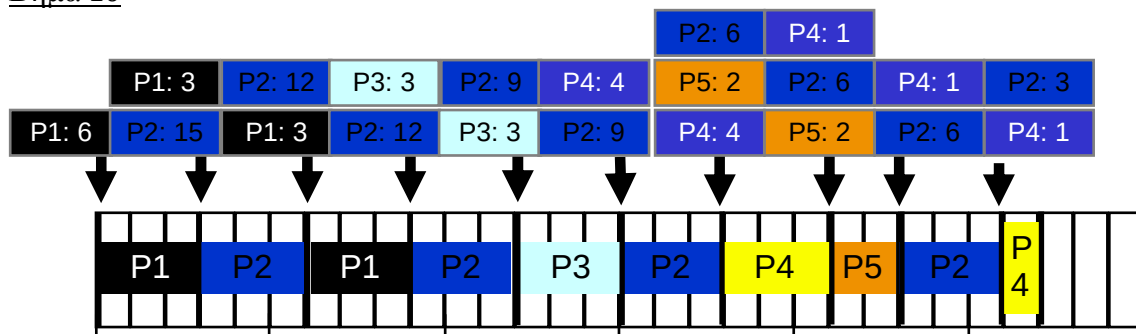
Βήμα 8



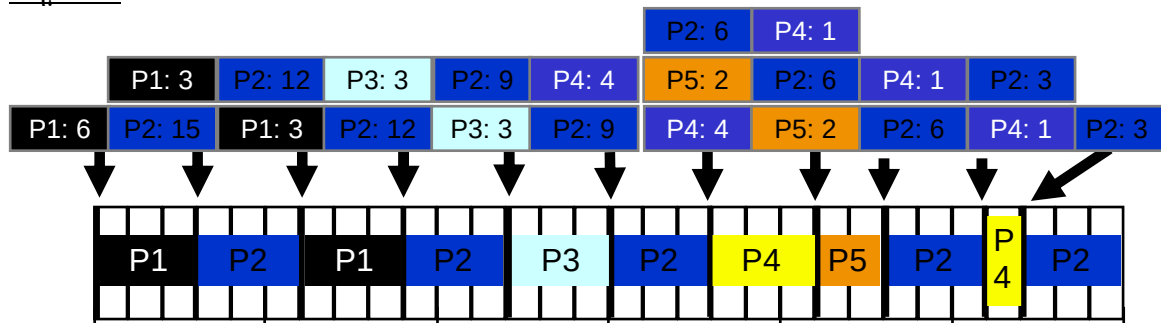
Βήμα 9



Βήμα 10



Βήμα 11



Τελικά αποτελέσματα

Process #	Waiting Time	Response Time	Turnaround Time	#of Context Switches
P1	0	0	9	2
P2	$(9-6) + (15-12) + (23-18) + (27-26) = 12$	0	$(30 - 3) = 27$	5
P3	$(12-9) = 3$	$(12-9) = 3$	$(15-9) = 6$	1
P4	$(18-14) + (26-21) = 9$	$(18-14) = 4$	$(27-14) = 13$	2
P5	$(21-17) = 4$	$(21-17) = 4$	$(23-17) = 6$	1
Average	$28/5 = 5.6$	$11/5 = 2.2$	$52/5 = 10.4$	$11/5 = 2.2$

9.7. Σύγκριση αλγορίθμων δρομολόγησης

Στην ερώτηση ποιος αλγόριθμος δρομολόγησης είναι ο καλύτερος, δεν μπορεί να δοθεί μια μονοσήμαντη απάντηση αφού αυτή εξαρτάται από διάφορους παράγοντες όπως είναι το φορτίο του υπολογιστικού συστήματος, το οποίο είναι ισχυρά μεταβαλλόμενο, την υποστήριξη υλικού για το δρομολογητή, το σχετικό βάρος των κριτηρίων απόδοσης (χρόνος απόκρισης, χρήση της, ρυθμοαπόδοση...) αλλά και τη μέθοδο αποτίμησης που χρησιμοποιείται με τη κάθε μια να έχει τους δικούς της περιορισμούς.

Παράδειγμα 1

Θεωρίστε το σύνολο διεργασιών του παρακάτω πίνακα, στο οποίο το μήκος των χρόνων εκτέλεσης (ή καταιγισμού επεξεργαστή) είναι σε milliseconds. Υποθέστε ότι οι διεργασίες έχουν φθάσει με τη σειρά P1, P2, P3, P4, και P5, όλες τη χρονική στιγμή 0. Να σχεδιαστεί το διάγραμμα εκτέλεσης για καθεμία από τις πολιτικές δρομολόγησης: FCFS, SJF, RR και να υπολογιστεί για κάθε διεργασία ο χρόνος επιστροφής και ο χρόνος αναμονής, καθώς και οι μέσες τιμές τους.

Διεργασία	Χρόνος εκτέλεσης
P1	10
P2	1
P3	2
P4	1
P5	5

Λύση

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
FCFS	P1	P1	P1	P1	P1	P1	P1	P1	P1	P1	P2	P3	P3	P4	P5	P5	P5	P5	P5
SJF	P2	P4	P3	P3	P5	P5	P5	P5	P5	P1	P1	P1	P1	P1	P1	P1	P1	P1	P1
RR	P1	P2	P3	P4	P5	P1	P3	P5	P1	P5	P1	P5	P1	P5	P1	P1	P1	P1	P1

Χρόνος επιστροφής = χρόνος αναμονής + χρόνος εκτέλεσης

	FCFS	SJF	RR
P1	10	19	19
P2	11	1	2
P3	13	4	7
P4	14	2	4
P5	19	9	14
M.O.	13.4	7.0	9.2

Χρόνος αναμονής = χρόνος επιστροφής – χρόνος εκτέλεσης στον επεξεργαστή

	FCFS	SJF	RR
P1	0	9	9
P2	10	0	1
P3	11	2	5
P4	13	1	3
P5	14	4	9
M.O.	9.6	3.2	5.4

Παράδειγμα 2

Να σχεδιάσετε το διάγραμμα εκτέλεσης για καθένα από τους παρακάτω αλγόριθμους δρομολόγησης: FCFS, RR με κβάντο= 2 χρονικές μονάδες, SJF χωρίς και με προεκχώρηση, και να υπολογίσετε τον μέσο χρόνο επιστροφής για τον κάθε αλγόριθμο.

Διεργασία	Χρόνος άφιξης	Χρόνος εκτέλεσης
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2

Αλγόριθμος FCFS

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
FCFS	P1	P1	P1	P2	P2	P2	P2	P2	P2	P3	P3	P3	P3	P4	P4	P4	P4	P4	P5	P5

Μέσος χρόνος επιστροφής = $(3 + 9 + 13 + 18 + 20) / 5 = 63 / 5 = 12.6$ χρονικές μονάδες

Αλγόριθμος RR, με κβάντο χρόνου = 2 χρονικές μονάδες

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
RR	P1	P1	P2	P2	P3	P3	P4	P4	P5	P5	P1	P2	P2	P3	P3	P4	P4	P2	P2	P4

Μέσος χρόνος επιστροφής = $(11 + 19 + 15 + 20 + 10) / 5 = 75 / 5 = 15$ χρονικές μονάδες

Αλγόριθμος SJF χωρίς προεκχώρηση

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
SJF	P5	P5	P1	P1	P1	P3	P3	P3	P3	P4	P4	P4	P4	P4	P2	P2	P2	P2	P2	P2

Μέσος χρόνος επιστροφής = $(2 + 5 + 9 + 14 + 20) / 5 = 50 / 5 = 10$ χρονικές μονάδες

Αλγόριθμος SJF με προεκχώρηση (SRTF)

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
SRTF	P1	P1	P1	P2	P3	P3	P3	P3	P5	P5	P2	P2	P2	P2	P2	P4	P4	P4	P4	P4

Μέσος χρόνος επιστροφής = $((3-0) + (15-2) + (8-4) + (20-6) + (10-8)) / 5 = 7.2$ χρονικές μονάδες

Ερωτήσεις Επανάληψης

1. Αναφέρετε και περιγράψτε τις κατηγορίες χρονοπρογραμματισμού.
2. Τι είναι η βραχυπρόθεσμη δρομολόγηση, πότε καλείται και ποια τα κριτήριά της;
3. Τι είναι η μεσοπρόθεσμη και η μακροπρόθεσμη δρομολόγηση;
4. Ποια η διαφορά μεταξύ δρομολόγησης με προεκχώρηση και δρομολόγησης χωρίς προεκχώρηση;
5. Περιγράψτε τους αλγορίθμους δρομολόγησης: FCFS, RR, SJF, SRTF.

Ασκήσεις

1. Ρυθμοαπόδοση είναι
 - α) η συχνή χρήση του επεξεργαστή από κάθε διεργασία
 - β) το ποσοστό του χρόνου κατά το οποίο ο επεξεργαστής χρησιμοποιείται
 - γ) ο αριθμός των εργασιών που ολοκληρώνονται σε μια χρονική περίοδο
 - δ) ο χρόνος εκτέλεσης μιας διεργασίας, από την αρχή ως τον τερματισμό της
2. Χρόνος απόκρισης είναι
 - α) ο χρόνος από τότε που μια απαίτηση υποβάλλεται μέχρι τη στιγμή που παράγεται το πρώτο αποτέλεσμα
 - β) ο χρόνος κατά τον οποίο μια διεργασία βρίσκεται σε μια ουρά αναμονής
 - γ) ο χρόνος εκτέλεσης μιας διεργασίας, από την αρχή ως τον τερματισμό της
 - δ) ο χρόνο που σπαταλάται για την εναλλαγή διεργασιών
3. Η μεσοπρόθεσμη δρομολόγηση
 - α) επιλέγει ποια διαθέσιμη διεργασία θα εκτελεστεί στον επεξεργαστή
 - β) καθορίζει ποια διεργασία θα γίνει αποδεκτή για επεξεργασία από το σύστημα
 - γ) πραγματοποιεί την εναλλαγή των διεργασιών στον επεξεργαστή
 - δ) αποφασίζει ποια διεργασία θα μεταφερθεί στην κύρια μνήμη
4. Η βραχυπρόθεσμη δρομολόγηση δεν ενεργοποιείται όταν συμβεί
 - α) διακοπή ρολογιού
 - β) διακοπή εισόδου-εξόδου
 - γ) δέσμευση μνήμης από το πρόγραμμα
 - δ) κλήση συστήματος
5. Στην περίπτωση της δρομολόγησης με προεκχώρηση
 - α) μια εκτελούμενη διεργασία εκτελείται συνεχώς μέχρι να τερματιστεί
 - β) μια εκτελούμενη διεργασία μπορεί να διακοπεί από το λειτουργικό σύστημα
 - γ) μια διεργασία σε εκτελούμενη κατάσταση, εκτελείται συνεχώς μέχρι να ανασταλεί από μόνη της περιμένοντας για την ολοκλήρωση
 - δ) επιλέγεται η διεργασία με την μεγαλύτερη προτεραιότητα
6. Στους αλγόριθμους δρομολόγησης με προεκχώρηση, η εκτέλεση μιας διεργασίας δεν εξαρτάται από:
 - α) το αν η διεργασία είναι αλληλεπιδραστική
 - β) την προτεραιότητα που έχει η διεργασία
 - γ) την συχνότητα ρολογιού του συστήματος
 - δ) τον χρόνο που έχει εκτελεστεί η διεργασία
7. Οι αλγόριθμοι δρομολόγησης με προεκχώρηση:

- α) εμποδίζουν τη μονοπώληση του επεξεργαστή
- β) έχουν απλή και εύκολη υλοποίηση
- γ) είναι ακατάλληλοι για πολυχρηστικά συστήματα
- δ) μπορούν να οδηγήσουν σε συνθήκες ανταγωνισμού

8. Ποιοι από τους παρακάτω είναι αλγόριθμοι δρομολόγησης με προεκχώρηση;

- α) Οι αλγόριθμοι FCFS και RR
- β) Οι αλγόριθμοι RR και SJF
- γ) Οι αλγόριθμοι RR και SRTF
- δ) Οι αλγόριθμοι SJF και SRTF

9. Θεωρήστε πέντε διεργασίες με χρόνους άφιξης και εκτέλεσης όπως φαίνονται στον παρακάτω πίνακα:

Διεργασία	Χρόνος άφιξης	Χρόνος εκτέλεσης
P1	0	3
P2	2	6
P3	4	4
P4	6	5
P5	8	2

Αν χρησιμοποιηθεί ο αλγόριθμος δρομολόγησης SJF, τότε η διεργασία που θα τελειώσει τελευταία θα είναι:

- α) η διεργασία P2
- β) η διεργασία P3
- γ) η διεργασία P4
- δ) η διεργασία P5

10. Θεωρήστε τις πέντε διεργασίες της προηγούμενης άσκησης (9). Αν χρησιμοποιηθεί ο αλγόριθμος δρομολόγησης RR, με κβάντο 1 χρονική μονάδα, τότε η διεργασία που θα τελειώσει τελευταία θα είναι:

- α) η διεργασία P2
- β) η διεργασία P3
- γ) η διεργασία P4
- δ) η διεργασία P5

Κεφάλαιο 10 – Το Λειτουργικό Σύστημα Unix

1. Εισαγωγή
2. Λογαριασμοί χρηστών
3. Είσοδος στο σύστημα
4. Το σύστημα αρχείων
5. Ονόματα διαδρομών
6. Η εντολή ls
7. Μετακίνηση στο σύστημα αρχείων του UNIX
8. Διαχείριση αρχείων
9. Χαρακτηριστικά αρχείων (File attributes)
10. Ανακατεύθυνση εισόδου / εξόδου (Input / Output Redirection)
11. Πολυπρογραμματισμός στο UNIX

10.1 Εισαγωγή

Το UNIX είναι ένα λειτουργικό σύστημα που χρησιμοποιείται από το 1969. Οι σύγχρονες διανομές UNIX διαθέτουν γραφική διεπαφή χρήστη (Graphical User Interface – GUI). Συνήθως χρησιμοποιείται μια διεπαφή γραμμής εντολών (command line interface) που απαιτεί από τους χρήστες να πληκτρολογούν τις εντολές για οτιδήποτε θέλουν να κάνουν. Το UNIX είναι ένα πανίσχυρο Λ.Σ. και διαθέτει πλήθος εντολών για οτιδήποτε. Κάθε χρήστης ενός συστήματος UNIX πρέπει να γνωρίζει το λογισμικό που χρησιμοποιεί ο server, τη λειτουργία των εντολών και τον τρόπο χρήσης τους, καθώς και το πώς και που θα βρει πληροφορίες για τις εντολές. Η πρόσβαση σε ένα σύστημα UNIX γίνεται συνήθως μέσω ενός προγράμματος απομακρυσμένου πελάτη αφού ο κεντρικός υπολογιστής του συστήματος (server) βρίσκεται σε απομακρυσμένη θέση σε σχέση με το τερματικό (terminal) που χρησιμοποιούμε.

10.2 Λογαριασμοί χρηστών

Η πρόσβαση σε ένα σύστημα Unix απαιτεί την ύπαρξη ενός λογαριασμού χρήστη (user account). Ο λογαριασμός χρήστη στ Unix περιλαμβάνει το όνομα χρήστη και το συνθηματικό του (username & password), τους αριθμούς χρήστη και ομάδας (userid & groupid), τον κατάλογο του λογαριασμού (home directory) και τον φλοιό εκτέλεσης εντολών (shell).

Το όνομα χρήστη (username) είναι τυπικά μια ακολουθία αλφαριθμητικών χαρακτήρων με μήκος όχι μεγαλύτερο από 8. Το username ταυτοποιεί αρχικά τα χαρακτηριστικά του λογαριασμού, ενώ συνήθως χρησιμοποιείται ως email address. Επίσης, το όνομα του βασικού καταλόγου του λογαριασμού συσχετίζεται με το όνομα χρήστη.

Το συνθηματικό (password) είναι μια μυστική λέξη που τη γνωρίζει μόνον ο χρήστης. Όταν ο χρήστης εισάγει το password στο σύστημα το σύστημα το κρυπτογραφεί και το συγκρίνει με την αποθηκευμένη λέξη που αντιστοιχεί στο username. Το μήκος των passwords συνήθως δεν υπερβαίνει τους 8 χαρακτήρες σε μήκος και συνιστάται η συμπερίληψη αριθμών και ειδικών χαρακτήρων.

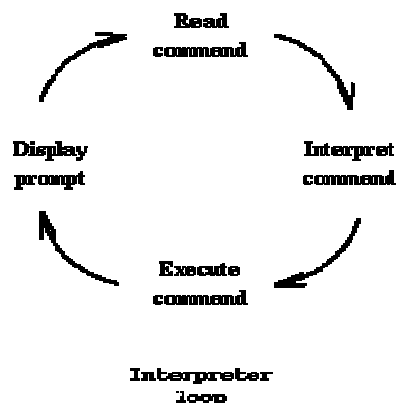
Ο αριθμός χρήστη (userid) είναι ένας ακέραιος αριθμός που ταυτοποιεί έναν λογαριασμό Unix. Κάθε userid είναι μοναδικό. Είναι ευκολότερο και περισσότερο αποτελεσματικό για το σύστημα να χρησιμοποιεί έναν αριθμό παρά ένα αλφαριθμητικό

ως username, χωρίς να απαιτείται από το χρήστη να γνωρίζει τον αριθμό λογαριασμού του.

Το Unix περιλαμβάνει ακόμα την έννοια της ομάδας (group) των χρηστών. Ένα Unix group μπορεί να διαμοιράζεται αρχεία και ενεργές διεργασίες. Κάθε λογαριασμός αντιστοιχείται σε ένα πρωτεύον (primary) group. Ο αριθμός ομάδας (groupid) είναι ένας αριθμός που αντιστοιχεί στο primary group. Ένας απλός λογαριασμός χρήστη μπορεί να ανήκει σε πολλά groups αλλά έχει μόνον ένα primary group.

Ο βασικός κατάλογος λογαριασμού (home directory) είναι μια περιοχή του συστήματος αρχείων όπου αποθηκεύονται τα αρχεία κάθε λογαριασμού. Ένας κατάλογος (directory) είναι ότι και ένας φάκελος (folder) στα Windows. Πολλές εντολές και εφαρμογές του Unix χρησιμοποιούν το home directory του λογαριασμού ως κατάλογο εργασίας καθώς και ως χώρο αναζήτησης των αρχείων προσαρμογής του αντίστοιχου λογαριασμού (customization files).

Ο φλοιός (Shell) είναι ένα πρόγραμμα Unix που παρέχει μια αλληλεπιδραστική συνεργασία (interactive session) μεταξύ χρήστη και συστήματος – είναι συνήθως μια διεπαφή χρήστη κατάστασης κειμένου (text-based user interface). Κατά τη σύνδεση σε ένα σύστημα Unix (login) το πρόγραμμα που αρχικά αλληλεπιδρά με το χρήστη είναι ο φλοιός. Υπάρχουν αρκετά διαθέσιμα και δημοφιλή προγράμματα φλοιού. Ο φλοιός εκτελεί επαναληπτικά τις τέσσερις εργασίες που απεικονίζονται στο σχήμα 10.1. Παραδείγματα φλοιών αποτελούν οι Bourne (sh), Korn (ksh), C (csh), Bourne Again (bash) κ.α. Σε όλους τους φλοιούς ο συνδυασμός των πλήκτρων Ctrl-C ακυρώνει (διακόπτει) την εκτέλεση μια εντολής.



Σχήμα 10.1 Επαναληπτικός τρόπος λειτουργίας του φλοιού

10.3 Είσοδος στο σύστημα

Η πρόσβαση σε ένα σύστημα Unix μπορεί να γίνει μέσω ενός τερματικού του συστήματος ή μέσω πρόσβασης διαμέσου του δικτύου χρησιμοποιώντας σύνδεση πελάτη telnet, SSH, SecureCRT ή άλλων εργαλείων απομακρυσμένης πρόσβασης. Ένα χαρακτηριστικό παράδειγμα τέτοιου λογισμικού είναι το SSH Secure Shell v.3.2.9, που διατίθεται ελεύθερο για μη εμπορική χρήση.

Κατά την διαδικασία εισόδου, το σύστημα ζητά αρχικά το όνομα του χρήστη και το συνθηματικό, κάνοντας διάκριση μεταξύ κεφαλαίων και μικρών χαρακτήρων. Μετά την επιτυχή σύνδεση εκκινεί το πρόγραμμα φλοιού και εμφανίζεται η ένδειξη αναμονής (prompt). Κατά την εκκίνηση του φλοιού γίνεται αναζήτηση στο home directory για τα αρχεία προσαρμογής του χρήστη (customization files). Ο χρήστης μπορεί να αλλάξει το shell prompt και μια ομάδα ρυθμίσεων δημιουργώντας νέα αρχεία προσαρμογής

Κάθε διεργασία, δηλαδή ένα στιγμιότυπο ενός προγράμματος που εκτελείται, έχει, στο Unix, μια αναφορά στον τρέχοντα κατάλογο εργασίας (current working directory). Ο φλοιός (που είναι μια διεργασία) ξεκινά θέτοντας ως current working directory το home directory του χρήστη. Ο φλοιός εμφανίζει την ένδειξη αναμονής (prompt – συνήθως είναι το \$) και αναμένει το χρήστη να πληκτρολογήσει μια εντολή. Ο φλοιός μπορεί να διερμηνεύσει δύο τύπους εντολών: α) εσωτερικές εντολές φλοιού (shell internals commands), τις οποίες ο φλοιός διαχειρίζεται άμεσα, και β) εξωτερικά προγράμματα (external programs), με το φλοιό να εκτελεί αυτά τα προγράμματα.

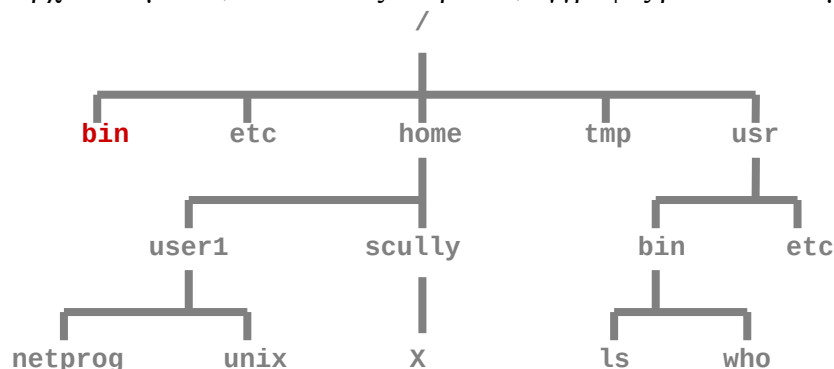
Μερικά παραδείγματα εντολών που εκτελούνται μέσω του φλοιού του Unix, είναι τα ακόλουθα:

- Η εντολή **ls**, που εμφανίζει μια λίστα των ονομάτων αρχείων (παρόμοια με την εντολή **dir** στο DOS).
- Η εντολή **who**, η οποία εμφανίζει μια λίστα των χρηστών που βρίσκονται τώρα στο σύστημα.
- Η εντολή **date**, που δείχνει την τρέχουσα ώρα και την ημερομηνία.
- Η εντολή **pwd**, η οποία εμφανίζει το τρέχον directory (working directory).

10.4 Το σύστημα αρχείων

Το σύστημα αρχείων αποτελεί μια λογική μέθοδος για την οργάνωση και την αποθήκευση μεγάλης ποσότητας πληροφοριών. Παρέχει ευκολία στη διαχείριση των αρχείων που αποτελούν τη βασική μονάδα αποθήκευσης δεδομένων. Υπάρχουν τέσσερις τύποι αρχείων και είναι τα κανονικά αρχεία, τα οποία αποθηκεύουν πληροφορίες, οι κατάλογοι (directories), που διατηρούν άλλα αρχεία και αναδρομικά άλλους καταλόγους, τα ειδικά αρχεία που αντιπροσωπεύουν τις φυσικές συσκευές όπως εκτυπωτές, τερματικά κλπ, καθώς επίσης και οι διασωληνώσεις (pipes), που πρόκειται για προσωρινά αρχεία τα οποία χρησιμοποιούνται για τη σύνδεση εντολών.

Κάθε αρχείο, ως η βασική μονάδα αποθήκευσης, διαθέτει ένα όνομα το οποίο μπορούν να περιέχει οποιουδήποτε χαρακτήρες, μπορεί να είναι σημαντικά μεγάλο και το ακριβές μήκος εξαρτάται από την έκδοση-διανομή του λειτουργικού συστήματος. Επίσης, κάθε αρχείο μπορεί να περιέχει μη επεξεργασμένα δεδομένα, αφού το Unix δεν επιβάλλει κάποια δομή στα αρχεία με αποτέλεσμα τα αρχεία να μπορούν να περιέχουν οποιοδήποτε ακολουθία από bytes. Ορισμένα προγράμματα διερμηνεύουν τα περιεχόμενα ενός αρχείου σαν να υπάρχει κάποια ειδική δομή, όπως συμβαίνει με αρχεία κειμένου, ακολουθίες ακεραίων, εγγραφές βάσεων δεδομένων κλπ.



Σχήμα 10.2 Παράδειγμα συστήματος αρχείων στο Unix

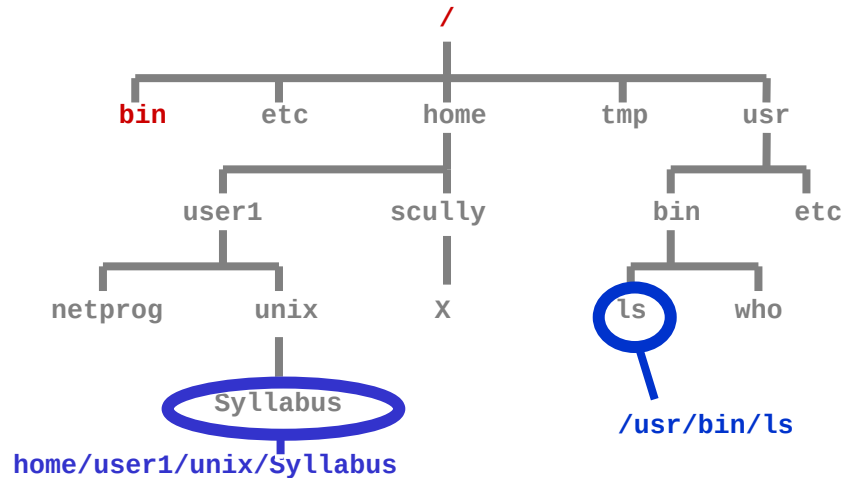
Ένας κατάλογος είναι ουσιαστικά μια ειδική μορφή αρχείου που χρησιμοποιείται από το Unix για να κρατά πληροφορίες σχετικές με άλλα αρχεία. Όπως είναι

κατανοητό, κάθε αρχείο στο ίδιο directory πρέπει να έχει ένα μοναδικό όνομα αλλά αρχεία σε διαφορετικούς καταλόγους μπορούν να έχουν ίδιο όνομα.

Το σύστημα αρχείων είναι ένα ιεραρχικό σύστημα οργάνωσης αρχείων και καταλόγων. Το κορυφαίο επίπεδο στην ιεραρχική δομή ονομάζεται ρίζα -"root" και διατηρεί όλα τα αρχεία και τους καταλόγους. Το όνομα του root directory είναι /. Εκτός από τη ρίζα, άλλοι βασικοί κατάλογοι του συστήματος αρχείων του Unix είναι ο κατάλογος **/bin**, που περιέχει δυαδικά εκτελέσιμα αρχεία, ο **/dev** με ειδικά αρχεία συσκευών, ο **/etc** με αρχεία διαχείρισης του συστήματος, ο **/home** που περιέχει τους βασικούς καταλόγους των λογαριασμών χρηστών, ο **/tmp** για αποθήκευση προσωρινών αρχείων και τέλος ο κατάλογος **/usr** που χρησιμοποιείται για ειδικά αρχεία χρηστών ή καταλόγους λογαριασμών.

10.5 Ονόματα διαδρομών (Pathnames)

Το όνομα διαδρομής (pathname) ενός αρχείου περιλαμβάνει το όνομα του αρχείου και το όνομα του καταλόγου που διατηρεί το αρχείο, και αναδρομικά το όνομα του καταλόγου που προηγούμενου καταλόγου μέχρι τη ρίζα του συστήματος αρχείων. Το όνομα διαδρομής κάθε αρχείου στο σύστημα αρχείων του Unix είναι μοναδικό. Η δημιουργία ενός ονόματος διαδρομής ξεκινά από τη ρίζα (δηλ. με "/"), στη συνέχεια ακολουθείται προς τα κάτω η ιεραρχική διαδρομή, περιλαμβάνοντας κάθε όνομα καταλόγου, και τελειώνει με το όνομα του αρχείου. Ταυτόχρονα, ανάμεσα σε κάθε όνομα καταλόγου θέτουμε ένα "/". Κάθε εκτελούμενο πρόγραμμα έχει έναν τρέχοντα κατάλογο (current directory) και όλα τα ονόματα αρχείων που σχετίζονται με το πρόγραμμα αυτό είναι ρητά συνδεδεμένα με το όνομα του καταλόγου αυτού, εκτός και αν ξεκινούν με /.

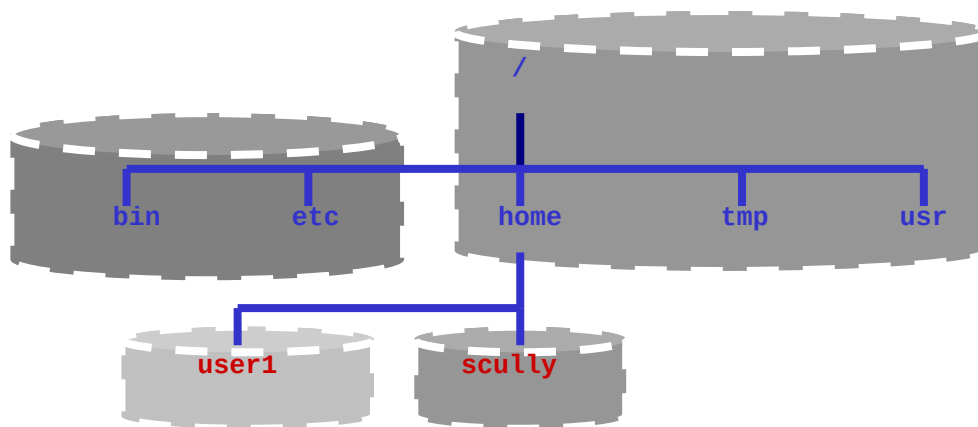


Σχήμα 10.3 Παραδείγματα ονομάτων διαδρομών

Τα ονόματα διαδρομών διακρίνονται σε απόλυτα και σχετικά. Τα ονόματα διαδρομών που περιγράφηκαν προηγουμένως ξεκινούν από τη ρίζα. Αυτά τα pathnames ονομάζονται απόλυτα ονόματα διαδρομών (absolute pathnames). Ωστόσο μπορούμε να αναφερθούμε στο όνομα διαδρομής ενός αρχείου σχετικά με έναν κατάλογο. Αν βρισκόμαστε στο directory **/users/hollid2**, το σχετικό pathname του αρχείου **Syllabus** είναι: **unix/Syllabus**. Οι περισσότερες εντολές του UNIX χρησιμοποιούν διαδρομές αρχείων. Ο προσδιορισμός αρχείων γίνεται συνήθως με χρήση σχετικών ονομάτων διαδρομών.

Κάθε αρχείο και κατάλογος στο σύστημα αρχείων μπορεί να οριστεί με ένα «πλήρες όνομα διαδρομής» (η διαδρομή από τη ρίζα μέχρι το αρχείο), π.χ. **/home/user1/email/f1**. Στην περίπτωση που έχουμε σχετικό όνομα διαδρομής, η θέση συσχετίζεται με τον τρέχοντα κατάλογο εργασίας (working directory) ή με το γονικό κατάλογο που βρίσκεται ένα επίπεδο πιο ψηλά στην ιεραρχία του συστήματος αρχείων. Έτσι λοιπόν, αν ο τρέχων κατάλογος είναι ο **/home/user1** τότε το σχετικό όνομα διαδρομής για το αρχείο **f1** είναι το **email/f1**.

Το ιεραρχικό σύστημα αρχείων μπορεί να περιλαμβάνεται σε πολλούς δίσκους, ενώ μερικοί κατάλογοι μπορούν να βρίσκονται σε άλλους υπολογιστές. Ειδικές περιπτώσεις καταλόγων για μια διεργασία είναι ο τρέχων και ο γονικός κατάλογος: υπάρχει ένα ειδικό σχετικό όνομα διαδρομής για τον τρέχοντα κατάλογο και είναι η μία τελεία (**.**), ένα ειδικό σχετικό όνομα διαδρομής για το γονικό κατάλογο και είναι οι δύο συνεχόμενες τελείες (**..**), ενώ το σύμβολο **~** αναφέρεται στο βασικό κατάλογο λογαριασμού χρήστη.



Σχήμα 10.4 Επέκταση του συστήματος αρχείων σε διαφορετικούς δίσκους

10.6. Η εντολή ls

Η εντολή **ls** εμφανίζει τα ονόματα ορισμένων αρχείων. Αν δοθεί το όνομα ενός καταλόγου ως παράμετρος της γραμμής εντολών (command line parameter) θα εμφανιστούν όλα τα ονόματα αρχείων που βρίσκονται στον κατάλογο με το συγκεκριμένο όνομα.

Παραδείγματα

ls	Εμφανίζει τα αρχεία στον τρέχοντα κατάλογο εργασίας
ls /	Εμφανίζει τα αρχεία που βρίσκονται στη ρίζα
ls .	Εμφανίζει τα αρχεία στον τρέχοντα κατάλογο εργασίας
ls ~	Εμφανίζει τα αρχεία του γονικού καταλόγου
ls /etc	Εμφανίζει τα αρχεία του καταλόγου /etc

Μπορούμε να τροποποιήσουμε τη μορφοποίηση της εξόδου της εντολής **ls** με μια επιλογή στη γραμμή εντολών. Για να χρησιμοποιηθεί μια παράμετρος γραμμής εντολών πρέπει να προηγείται της επιλογής το σύμβολο μείον: π.χ., **ls -a** ή **ls -l**. Μπορούν να χρησιμοποιηθούν δύο ή περισσότερες επιλογές ταυτόχρονα π.χ.: **ls -al**. Η εντολή **ls** υποστηρίζει πλήθος επιλογών, όπως οι ακόλουθες:

l	Εμφανίζει λεπτομέρειες των αρχείων, π.χ. χρόνους, ιδιοκτήτη και δικαιώματα χρήσης
a	Εμφανίζει τα κρυφά αρχεία, δηλαδή εκείνα που το όνομά τους ξεκινάει με τελεία, μαζί με τα κανονικά αρχεία
F	Εμφανίζει τους τύπους των αρχείων
R	Εμφανίζει όλα τα περιεχόμενα ενός καταλόγου και τα περιεχόμενα όλων των υποκαταλόγων του (subdirectories) αναδρομικά.

Λαμβάνοντας υπόψη όσα έχουμε αναφέρει μέχρι στιγμής, η γενική μορφή της εντολής **ls** είναι : **ls [options] [names]**. Οι αγκύλες που περιβάλλουν τις επιλογές και τα ονόματα στη γενική μορφή της εντολής **ls** σημαίνουν κάτι που είναι προαιρετικό. Με παρόμοιο τρόπο σχηματίζονται και πλήθος άλλες εντολές. Ορισμένες εντολές ωστόσο απαιτούν παραμέτρους. Οι επιλογές (options) πρέπει πάντοτε να προηγούνται, ενώ μπορούν να αναμειχθούν οποιεσδήποτε επιλογές με οποιαδήποτε ονόματα, π.χ. **ls -al /usr/bin**. Μπορούν να δοθούν περισσότερα ονόματα στην εντολή **ls**, π.χ. **ls /usr /etc** αλλά και **ls -l /usr/bin /tmp /etc**.

Με την εντολή **ls**, όπως όμως και με άλλες εντολές που χρησιμοποιούν ονόματα καταλόγων και αρχείων, υπάρχουν διαθέσιμοι κάποιοι ειδικοί χαρακτήρες για τον προσδιορισμό των ονομάτων. Ο ειδικός χαρακτήρας ***** χρησιμοποιείται ως wildcard και αντιπροσωπεύει οποιοδήποτε πλήθος χαρακτήρων. Για παράδειγμα, η εντολή **ls p*** θα εμφανίσει λίστα όλων των αρχείων στον τρέχοντα κατάλογο που ξεκινούν με το γράμμα **p**. Ο ειδικός χαρακτήρας **?** χρησιμοποιείται ως ένα wildcard για έναν μόνο χαρακτήρα. Για παράδειγμα, η εντολή **ls jun??.dat** θα εμφανίσει λίστα όλων των αρχείων που ξεκινούν από τους χαρακτήρες **jun**, τελειώνουν με **.dat** και έχουν ενδιάμεσα οποιουσδήποτε 2 χαρακτήρες. Ο χαρακτήρας **[** χρησιμοποιείται για να καθορίσει επακριβώς ένα σύνολο χαρακτήρων. Για παράδειγμα η εντολή **rm prog[2-4p-r].c** θα διαγράψει, εφόσον υπάρχουν τα αντίστοιχα αρχεία, **prog2.c**, **prog3.c**, **prog4.c**, **progr.c**, **progr.c**. Τέλος, ο χαρακτήρας **~** χρησιμοποιείται για να αντιπροσωπεύει το πλήρες όνομα διαδρομής του home directory. Για παράδειγμα, **ls ~user1** θα εμφανίσει τα περιεχόμενα του home directory του χρήστη **user1**, το πλήρες όνομα του οποίου μπορεί να είναι **/home/students/user1**.

10.7. Μετακίνηση στο σύστημα αρχείων του UNIX

Η εντολή **cd** (change directory) αλλάζει τον τρέχοντα κατάλογο εργασίας (current working directory). Η γενική μορφή της εντολής είναι: **cd [directory_name]**. Η χρήση της εντολής **cd** χωρίς παραμέτρους αλλάζει τον τρέχοντα κατάλογο με το home directory του χρήστη. Μπορούμε επίσης να δώσουμε ως παράμετρο στην **cd** ένα σχετικό ή απόλυτο όνομα διαδρομής (relative ή absolute pathname), π.χ. **cd /usr** ή **cd ...**. Ιδιαίτερα χρήσιμη κατά τη μετακίνηση στο σύστημα αρχείων είναι η εντολή **pwd**, η οποία επιστρέφει το απόλυτο όνομα διαδρομής του τρέχοντος καταλόγου εργασίας.

Παραδείγματα

cd ..	Πηγαίνει στο γονικό κατάλογο (ένα επίπεδο πάνω)
cd /	Πηγαίνει στη ρίζα
cd ~	Πηγαίνει στο βασικό κατάλογο λογαριασμού χρήστη
cd ~user1	Πηγαίνει στο βασικό κατάλογο του λογαριασμού χρήστη user1
cd /etc	Πηγαίνει στο κατάλογο etc που βρίσκεται στη ρίζα
cd ../sub	Πηγαίνει στο κατάλογο sub που βρίσκεται στο γονικό κατάλογο

10.8 Διαχείριση αρχείων

Η αντιγραφή αρχείων πραγματοποιείται με την εντολή **cp**, της οποίας η γενική μορφή είναι η εξής: **cp [options] source dest**, όπου **source** είναι το όνομα του αρχείου που θέλουμε να αντιγράψουμε και **dest** είναι το όνομα του νέου αρχείου. Τόσο το **source** όσο και το **dest** μπορούν να αναφέρονται σχετικά ή απόλυτα.

Αν το όνομα **dest** είναι ένας κατάλογος (directory), η εντολή **cp** θα τοποθετήσει ένα αντίγραφο του **source** στον κατάλογο, π.χ. με την εντολή **cp [options] source destdir**. Το όνομα αρχείου θα είναι το ίδιο με το όνομα του αρχείου **source**. Αν οριστούν περισσότερα από ένα ονόματα, η **cp** υποθέτει τη χρησιμοποίηση της μορφής: **cp [options] source... destdir**. Στην περίπτωση αυτή η **cp** θα αντιγράψει πολλά αρχεία στο **destdir**. Επομένως, το όνομα **source** στην γενική μορφή της εντολής σημαίνει τουλάχιστον ένα όνομα και άρα μπορεί να είναι περισσότερα από ένα.

Η διαγραφή (removing) αρχείων πραγματοποιείται με την εντολή **rm**, σύμφωνα με τη γενική μορφή: **rm [options] names...** Το όνομα της εντολής (**rm**) αντιστοιχεί στη λέξη "remove". Μπορούν επίσης να διαγραφούν πολλά αρχεία ταυτόχρονα, π.χ.
rm file1 /tmp/file2 /home/user1/file3.

10.9 Χαρακτηριστικά αρχείων (File attributes)

Κάθε αρχείο στο UNIX έχει ορισμένα χαρακτηριστικά (attributes), τα οποία είναι: α) οι χρόνοι προσπέλασης που φανερώνουν πότε ένα αρχείο δημιουργήθηκε, τροποποιήθηκε ή διαβάστηκε (προσπελάστηκε) για τελευταία φορά, β) το μέγεθος του, γ) οι ιδιοκτήτες - κάτοχοι του αρχείου (χρήστης και ομάδα), και δ) οι άδειες πρόσβασης (Permissions).

Τα χαρακτηριστικά ενός αρχείου εμφανίζονται χρησιμοποιώντας ειδικές παραμέτρους στην εντολή **ls**. Για παράδειγμα, η εντολή **ls -l** δείχνει πότε το αρχείο τροποποιήθηκε για τελευταία φορά, η εντολή **ls -lc** δείχνει πότε δημιουργήθηκε το αρχείο και η εντολή **ls -ul** πότε το αρχείο διαβάστηκε (προσπελάστηκε) για τελευταία φορά.

Κάθε αρχείο κατέχεται από ή ανήκει σε ένα χρήστη. Το όνομα του χρήστη που είναι κάτοχος του αρχείου μπορεί να βρεθεί με την παράμετρο "-l" της εντολής **ls**. Η ίδια εντολή φανερώνει επίσης την ομάδα (UNIX group) στην οποία ανήκει ένα αρχείο.

Κάθε αρχείο έχει επίσης ένα σύνολο αδειών πρόσβασης (permissions) που ελέγχουν ποιος μπορεί να χειρίζεται το αρχείο. Υπάρχουν τρία είδη αδειών πρόσβασης: ανάγνωσης (read και σε συντομογραφία **r**), εγγραφής (write ή **w**), και εκτέλεσης (execute ή **x**). Επίσης υπάρχουν διαφορετικές άδειες πρόσβασης για τον ιδιοκτήτη του αρχείου, την ομάδα (group) και οποιονδήποτε άλλο. Έτσι λοιπόν, κάθε αρχείο έχει ένα μοναδικό ιδιοκτήτη, μια συσχέτιση με ένα μοναδικό group και ένα σύνολο αδειών πρόσβασης που συσχετίζονται με αυτό. Οι άδειες πρόσβασης αποτελούνται από μια συμβολοσειρά με δέκα χαρακτήρες. Ο πρώτος χαρακτήρας δείχνει τον τύπο του αρχείου: είναι **d**, εφόσον πρόκειται για κατάλογο και **-** αν πρόκειται για απλό αρχείο. Το υπόλοιπο τμήμα της συμβολοσειράς αντιστοιχεί σε τρεις τριάδες χαρακτήρων και προσδιορίζει τις άδειες πρόσβασης για τον ιδιοκτήτη (**u**), την ομάδα (**g**) και όλους τους υπόλοιπους (**o**). Αν μια συγκεκριμένη άδεια (**r** ή **w** ή **x**) έχει τεθεί τότε στην αντίστοιχη θέση εμφανίζεται το σύμβολό της, διαφορετικά στη θέση υπάρχει ο χαρακτήρας **-**.

Για ένα αρχείο, οι άδειες πρόσβασης ελέγχουν τι μπορεί να γίνει με τα περιεχόμενα του αρχείου. Για ένα directory, οι άδειες πρόσβασης ελέγχουν αν ένα αρχείο του directory μπορεί να εμφανιστεί στη λίστα, να αναζητηθεί να αλλάξει όνομα ή να διαγραφεί.

Άδεια	Για ένα αρχείο	Για έναν κατάλογο
r (read)	Εμφάνιση ή εκτύπωση των περιεχομένων	Τα περιεχόμενα εμφανίζονται σε λίστα αλλά δεν μπορεί να γίνει αναζήτηση σε αυτά. Τα r και x συνήθως χρησιμοποιούνται μαζί
w (write)	Αλλαγή ή διαγραφή των περιεχομένων	Αρχεία μπορούν να προστεθούν ή να διαγραφούν
x (execute)	Το αρχείο τρέχει όπως ένα πρόγραμμα.	Το Directory μπορεί να ανιχνευθεί και να γίνει περιπλάνηση (cd) σε αυτό.

Πίνακας 10.1 Άδειες πρόσβασης για αρχεία και καταλόγους

Η αλλαγή των αδειών πρόσβασης που συσχετίζονται με ένα αρχείου ή κατάλογο πραγματοποιείται με την εντολή **chmod**. Υπάρχει ένα πλήθος μορφών για την chmod, η απλούστερη είναι: **chmod mode file**. Το mode έχει την παρακάτω μορφή: **[ugoa][+|=][rwx]**, όπου u=user, g=group, o=other, a=all. Επίσης το σύμβολο + σημαίνει προσθήκη άδειας, το σύμβολο - σημαίνει αφαίρεση άδειας και τέλος το σύμβολο = σημαίνει ανάθεση άδειας.

Κάθε σύστημα Unix έχει ένα τουλάχιστον αριθμό χρήστη με ξεχωριστή σημασία και αποτελεί τον διαχειριστή ή υπερχρήστη του συστήματος. Αυτός ο χρήστης αναφέρεται συνήθως ως **root** και έχει τη δυνατότητα πρόσβασης σε όλα τα αρχεία και τους καταλόγους, ανεξάρτητα των δικαιωμάτων και των αδειών πρόσβασης. Οι άδειες πρόσβασης ενός αρχείου ή καταλόγου μπορούν να τροποποιηθούν μόνο από τον ιδιοκτήτη τους και από τον διαχειριστή του συστήματος (root).

Παραδείγματα

- **\$ chmod a=r-x file**
- **\$ chmod u=rw- file**
- **\$ chmod ugo+r file**
- **\$ chmod go-w file**

Εκτός από την παραπάνω μέθοδο των χαρακτήρων, η αλλαγή των αδειών πρόσβασης μπορεί να πραγματοποιηθεί και με αριθμητική μέθοδο, με χρήση οκταδικών αριθμών. Στην περίπτωση αυτή, η ύπαρξη ενός συμβόλου άδειας σε μια θέση των αδειών πρόσβασης αντιστοιχεί στην μονάδα του δυαδικού συστήματος ενώ η απουσία του στο μηδέν. Οι οκταδικοί αριθμοί προκύπτουν μετατρέποντας τις τριάδες δυαδικών αριθμών που προκύπτουν στους αντίστοιχους οκταδικούς. Επομένως, για μια δεδομένη τριάδα ισχύει ότι η άδεια ανάγνωσης αντιστοιχεί στον οκταδικό αριθμό 4 ($r=100=4$), η άδεια εγγραφής στον αριθμό 2 ($w=010=2$), η άδεια εκτέλεσης στο 1 ($x=001=1$), ενώ η απουσία αδειών στο 0 ($---=000=0$). Επιπλέον, οι αριθμοί μπορούν να προστίθενται, δίνοντας τον συνολικό οκταδικό αριθμό μιας τριάδας, π.χ. η άδεια ανάγνωσης-εγγραφής-εκτέλεσης ισούται με 7 ($rw x=111=(4+2+1)=7$), η άδεια ανάγνωσης-εγγραφής με 6 ($rw=110=(4+2+0)=6$), ενώ η άδεια ανάγνωσης-εκτέλεσης με 5 ($r-x=101=(4+0+1)=5$).

Παραδείγματα

- **\$ chmod 744 file**
- **\$ chmod 600 file**

Παράδειγμα εντολής chmod

\$ ls -al file

```

rwxrwx--x 1 user1 students ... file
$ chmod g-wx staff file
$ ls -al file
-rwxrw---- 1 user1 staff
$ chmod u-r file
$ ls -al file
ls: .: Permission denied

```

10.9 Άλλες εντολές

date	Δείχνει την τρέχουσα ημερομηνία και ώρα
mkdir <i>dirname</i>	δημιουργία καταλόγου (make directory)
rmdir <i>dirname</i>	Διαγραφή καταλόγου (remove directory)
touch <i>filename</i>	Αλλάζει το timestamp του αρχείου, δημιουργεί ένα νέο κενό αρχείο διαφορετικά
mv <i>filename1 filename2</i>	Μεταφορά – μετονομασία αρχείου
cat <i>filename</i>	Συνδέει αρχεία και τα εμφανίζει στην οθόνη του τερματικού.
file <i>filename</i>	Δείχνει το είδος των περιεχομένων ενός αρχείου
who	Δείχνει ποιο χρήστες είναι συνδεδεμένοι στο σύστημα
ps	Δείχνει τις διεργασίες που εκτελούνται στο σύστημα
df	Δείχνει την κατάσταση του συστήματος αρχείων (ελεύθερος χώρος, δίσκοι, κλπ)
man <i>command</i>	Δείχνει πληροφορίες σχετικές με τις εντολές

Πίνακας 10.2 Συχνά χρησιμοποιούμενες εντολές

10.10 Ανακατεύθυνση εισόδου / εξόδου (Input / Output Redirection)

Συνήθως τα προγράμματα διαβάζουν από το πληκτρολόγιο και γράφουν τα αποτελέσματα και τα μηνύματα λάθους στην οθόνη. Κάθε εκτελούμενο πρόγραμμα ανοίγει τρία αρχεία: standard input, standard output, and standard error που συσχετίζονται με το πληκτρολόγιο, την οθόνη και την οθόνη αντίστοιχα. Η είσοδος μπορεί να γίνει απευθείας από ένα αρχείο χρησιμοποιώντας τα σύμβολα "<" και "<<". Αντίστοιχα, η έξοδος μπορεί να κατευθυνθεί σε ένα αρχείο χρησιμοποιώντας τα σύμβολα ">" και ">>".



Σχήμα 10.5 Τυπική εντολή Unix

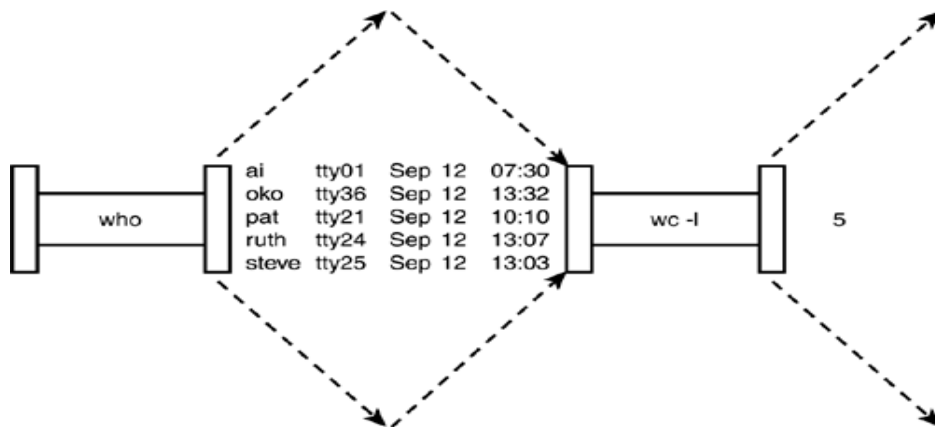
Η διασωλήνωση (Piping) είναι μια τεχνική που κατευθύνει την έξοδο μιας εντολής στην είσοδο μιας άλλης. Το σύμβολο διασωλήνωσης που χρησιμοποιείται μεταξύ των εντολών είναι το "|". Για παράδειγμα, η εντολή **ls /etc | more** ισοδυναμεί με τις ακόλουθες τρεις εντολές:

```

$ ls /etc > list.temp
$ more list.temp

```

\$ rm list.temp



Σχήμα 10.6 Διαδικασία σωλήνωσης για την εντολή : `who | wc -l`

more filename	Εμφανίζει τα περιεχόμενα ενός αρχείου, μια σελίδα ή γραμμή κάθε φορά
head filename	εμφανίζει τις 10 πρώτες γραμμές ενός αρχείου
tail filename	εμφανίζει τις 10 τελευταίες γραμμές ενός αρχείου
sort filename	ταξινομεί τις γραμμές ενός αρχείου κατά αλφαβητική σειρά
wc filename	μετρά γραμμές, λέξεις και χαρακτήρες ενός αρχείου

Πίνακας 10.3 Εντολές που λειτουργούν ως φίλτρα

10.11 Πολυπρογραμματισμός στο UNIX

Το UNIX είναι ένα πολυχρηστικό και πολυδιεργασιακό λειτουργικό σύστημα. Περισσότεροι από ένας χρήστες μπορούν να εκτελούν προγράμματα την ίδια χρονική στιγμή και περισσότερες από μια διεργασίες μπορούν να εκτελούνται την ίδια χρονική στιγμή. Στο Unix, κάθε πρόγραμμα ξεκινά ως μια διεργασία, που όπως έχουμε δει είναι ένα πρόγραμμα σε εκτέλεση. Μπορούν να υπάρχουν πολλές διεργασίες που εκτελούν το ίδιο πρόγραμμα (π.χ. έναν editor). Για κάθε χρήστη που αλληλεπιδρά με το σύστημα υπάρχει μόνο μια διεργασία βρίσκεται στο προσκήνιο (foreground), αλλά μπορούν να υπάρχουν αρκετές διεργασίες στο παρασκήνιο (background). Ο καταμερισμός χρόνου του UNIX επιτρέπει πολλές διεργασίες να εκτελούνται «εικονικά» την ίδια στιγμή. Κάθε διεργασία έχει έναν αριθμό ταυτότητας διεργασίας (process identification number – PID).

Κατά την εκτέλεσή της, μια διεργασία προσκηνίου κλειδώνει το τερματικό μέχρι να ολοκληρωθεί, μη επιτρέποντας την εκκίνηση άλλων εντολών από το χρήστη. Αντίθετα, μία ή περισσότερες μπορούν να εκτελούνται την ίδια χρονική στιγμή στο παρασκήνιο, αφού κατά την εκκίνησή τους αποδεσμεύουν το τερματικό. Η ακύρωση μιας διεργασίας που εκτελείται στο προσκήνιο πραγματοποιείται από το πληκτρολόγιο με το συνδυασμό **Ctrl-c**. Μια διεργασία παρασκηνίου μπορεί να τερματιστεί από το χρήστη με την εντολή **kill**, δίνοντας ως όρισμα τον αριθμό (PID) της διεργασίας, δηλαδή **kill <pid>**.

Η εντολή χρήστη για την εμφάνιση των διεργασιών είναι η **ps**. Εκτός από τα στοιχεία των διεργασιών, μεταξύ των οποίων και το PID τους, η εντολή παρέχει ακόμα πληροφορίες όπως το τρέχον ποσοστό χρησιμοποίησης του επεξεργαστή από τις διεργασίες, το ποσοστό μνήμης, ο συνολικός χρόνος επεξεργαστή και άλλα στατιστικά. Τα αποτελέσματά της δεν είναι πραγματικού χρόνου αλλά η εντολή παίρνει μια εικόνα

του συστήματος. Όταν η εντολή **ps** εκτελεστεί χωρίς ορίσματα εμφανίζει τις τρέχουσες διεργασίες του χρήστη. Χρησιμοποιώντας τις παραμέτρους γραμμής εντολών **ps -aef**, η εντολή εμφανίζει όλες τις διεργασίες που εκτελούνται στο σύστημα.

Ερωτήσεις Επανάληψης

1. Τι είναι ο φλοιός (shell);
2. Τι είναι το σύστημα αρχείων και ποιοι οι βασικοί τύπου αρχείων;
3. Σε τι διαφέρουν τα απόλυτα ονόματα διαδρομών από τα σχετικά;
4. Ποια είναι τα χαρακτηριστικά (attributes) ενός αρχείου στο Unix;
5. Τι είναι η διασωλήνωση (pipring);

Ασκήσεις

1. Δεν περιλαμβάνεται στο λογαριασμό χρήστη στο Unix:
 - α) ο κατάλογος του λογαριασμού
 - β) το όνομα χρήστη
 - γ) ο συνολικός αριθμός αρχείων του χρήστη
 - δ) το συνθηματικό
2. Ο αριθμός χρήστη είναι
 - α) ένας ακέραιος αριθμός
 - β) μια ακολουθία αλφαριθμητικών χαρακτήρων
 - γ) ένα ειδικό αρχείο
 - δ) μια μυστική λέξη που τη γνωρίζει μόνον ο χρήστης
3. Ο φλοιός είναι
 - α) μια περιοχή του συστήματος αρχείων
 - β) μια ειδική εντολή του Unix
 - γ) ο λογαριασμός του διαχειριστή του συστήματος
 - δ) ένα πρόγραμμα
4. Η εντολή **pwd** εμφανίζει
 - α) μια λίστα των ονομάτων αρχείων
 - β) τον τρέχοντα κατάλογο
 - γ) μια λίστα των χρηστών που βρίσκονται τώρα στο σύστημα
 - δ) την τρέχουσα ώρα και την ημερομηνία.
5. Τα ειδικά αρχεία
 - α) αντιπροσωπεύουν τις φυσικές συσκευές
 - β) αποθηκεύουν πληροφορίες,
 - γ) διατηρούν άλλα αρχεία
 - δ) είναι προσωρινά αρχεία τα οποία χρησιμοποιούνται για τη σύνδεση εντολών
6. Ο κατάλογος **/etc** του Unix περιέχει
 - α) δυαδικά εκτελέσιμα αρχεία
 - β) ειδικά αρχεία συσκευών
 - γ) αρχεία διαχείρισης του συστήματος
 - δ) προσωρινά αρχεία

7. Η εντολή **ls -l** εμφανίζει
- α) όλα τα αρχεία αναδρομικά
 - β) τους τύπους των αρχείων
 - γ) τα κρυφά αρχεία
 - δ) τις λεπτομέρειες των αρχείων
8. Ποια από τις παρακάτω εντολές εμφανίζει λίστα μόνο τα αρχεία που ξεκινούν με **tei**, έχουν κατάληξη **txt** και έχουν ενδιάμεσα οποιουσδήποτε 2 χαρακτήρες.
- α) `ls t*.*`
 - β) `ls tei??.txt`
 - γ) `ls [tei]*.txt`
 - δ) `ls [tei]??.txt`
9. Η άδεια πρόσβασης **rw-rwxr-x** αντιστοιχεί σε αριθμητική μορφή
- α) 675
 - β) 475
 - γ) 641
 - δ) 744
10. Η εντολή **tail <όνομα αρχείου>**
- α) εμφανίζει τις 10 πρώτες γραμμές ενός αρχείου
 - β) εμφανίζει τα περιεχόμενα ενός αρχείου, μια σελίδα ή γραμμή κάθε φορά
 - γ) μετρά γραμμές, λέξεις και χαρακτήρες ενός αρχείου
 - δ) εμφανίζει τις 10 τελευταίες γραμμές ενός αρχείου

Βιβλιογραφία

1. Γ. Παπακωνσταντίνου, Ν. Μπελαλής και Π. Τσανάκας, «Λειτουργικά συστήματα Ι», Εκδόσεις Συμμετρία, Αθήνα 1999.
2. Π. Σπυράκης, «Λειτουργικά Συστήματα Ι», Ελληνικό Ανοικτό Πανεπιστήμιο (Ε.Α.Π.), Θεματική Ενότητα ΠΛΗ 11, Τόμος Β'.
3. B.W. Kernighan and R. Pike, «Το περιβάλλον Προγραμματισμού UNIX», Κλειδάριθμος, 1989.
4. J. D. Peek, G. Torino and J. Strang, «Learning UNIX Operating System», 5th Edition, O'Reilly & Associates, 2002.
5. J.L. Peterson and A. Silberschatz, «Operating Systems Concepts», World Student Series Edition, Addison–Wesley Inc., Reading, Massachusetts, 1985.
6. A. Silberschatz, P. Galvin and G. Gagne, «Operating System Concepts», 6th Edition, John Wiley & Sons, 2001.
7. W. Stallings, «Λειτουργικά Συστήματα – Αρχές Σχεδίασης», 4η Έκδοση, Εκδόσεις Τζιόλα, Θεσσαλονίκη 2003.
8. A.S. Tanenbaum, «Σύγχρονα Λειτουργικά Συστήματα» (Τόμος Α'), Εκδόσεις Παπασωτηρίου, Αθήνα 1999.

Λύσεις Ασκήσεων

Κεφάλαιο 1: 1.γ, 2.α, 3.β, 4.δ, 5.γ, 6.β, 7.α, 8.β, 9.δ, 10.γ

Κεφάλαιο 2: 1.β, 2.α, 3.γ, 4.δ, 5.β, 6.δ, 7.β, 8.α, 9.α, 10.β

Κεφάλαιο 3: 1.δ, 2.β, 3.α, 4.β, 5.γ, 6.δ, 7.γ, 8.β, 9.α, 10.γ

Κεφάλαιο 4: 1.β, 2.β, 3.δ, 4.γ, 5.α, 6.β, 7.γ, 8.α, 9.δ, 10.γ

Κεφάλαιο 5: 1.α, 2.β, 3.β, 4.δ, 5.α, 6.δ, 7.γ, 8.β, 9.δ, 10.β

Κεφάλαιο 6: 1.γ, 2.α, 3.β, 4.β, 5.γ, 6.δ, 7.β, 8.α, 9.δ, 10.β

Κεφάλαιο 7: 1.γ, 2.β, 3.δ, 4.α, 5.β, 6.δ, 7.δ, 8.α, 9.γ, 10.α

Κεφάλαιο 8: 1.γ, 2.α, 3.δ, 4.γ, 5.β, 6.β, 7.α, 8.β, 9.β, 10.γ

Κεφάλαιο 9: 1.γ, 2.α, 3.δ, 4.γ, 5.β, 6.γ, 7.α, 8.γ, 9.γ, 10.γ

Κεφάλαιο 10: 1.γ, 2.α, 3.δ, 4.β, 5.α, 6.γ, 7.δ, 8.β, 9.α, 10.δ

